```
19          print(c.radius, "\t\t", c.getArea())
20          c.radius = c.radius + 1
21          times = times - 1
22
23  main() # Call the main function
```

```
Radius                  Area

1                       3.141592653589793
2                       12.566370614359172
3                       29.274333882308138
4                       50.26548245743669
5                       79.53981633974483

Radius is 6
n is 5
```

程序清单 7-1 中定义了 Circle 类。程序传递一个 Circle 对象 myCircle 和一个 int 对象 n 去调用 printAreas(myCircle,n)（第 9 行），它打印一个半径分别为 1、2、3、4 和 5 所对应的面积的列表，如样本输出所示。

当你将一个对象传递给函数，就是将这个对象的引用传递给函数。但是，传递不可变对象和可变对象之间还有更重要的区别。

- 像数字或字符串这样的不可变对象参数，函数外的对象的原始值并没有被改变。
- 像圆这样的可变对象参数，如果对象的内容在函数内被改变，则对象的原始值被改变。

在第 20 行，Circle 对象 c 的 radius 属性增加 1。c.radius+1 创建了一个新的 int 对象，并将它赋值给 c.radius。myCircle 和 c 都指向同一个对象。当 printAreas 函数完成后，c.radius 是 6。所以，由第 12 行可看出，myCircle.radius 的输出结果是 6。

在第 21 行，times-1 创建一个新的 int 对象，它被赋值给 times。在函数 printAreas 之外，n 还是 5。所以，在第 13 行，n 的输出还是 5。

检查点

7.11　给出下面程序的输出结果：

```
class Count:
    def __init__(self, count = 0):
        self.count = count

def main():
    c = Count()
    times = 0
    for i in range(100):
        increment(c, times)

    print("count is", c.count)
    print("times is", times)

def increment(c, times):
    c.count += 1
    times += 1

main() # Call the main function
```

7.12　给出下面程序的输出结果：

```
class Count:
    def __init__(self, count = 0):
        self.count = count
```

```
def main():
    c = Count()
    n = 1
    m(c, n)

    print("count is", c.count)
    print("n is", n)

def m(c, n):
    c = Count(5)
    n = 3

main() # Call the main function
```

7.5　隐藏数据域

关键点：使数据域私有来保护数据，让类更易于维护。

你可以通过对象的实例变量直接访问数据域。例如：下面的代码，让你通过 c.radius 访问圆的半径，它是合法的：

```
>>> c = Circle(5)
>>> c.radius = 5.4  # Access instance variable directly
>>> print(c.radius) # Access instance variable directly
5.4
>>>
```

但是，直接访问对象的数据域不是一个好方法，原因有两个。

- 首先是因为数据可能会被篡改。例如：TV 类中的 channel 取值在 1 和 120 之间，但是，它也可能被错误地设置为一个不合法的值（例如：tv1.channel=125）。
- 其次是因为类会变得难以维护并且易于出错。假设你想修改 Circle 类以保证确保在其他程序用过这个类后半径是非负值。你就不仅仅需要更改 Circle 类，还得更改使用它的程序，因为客户端可能直接修改半径（例如：myCircle.radius=-5）。

为避免直接修改数据域，就不要让客户端直接访问数据域。这被称为*数据隐藏*，并可以通过定义*私有数据域*实现。在 Python 语言中，私有数据域是以两个下划线开始来定义的。你也可以以两个下划线开始来定义*私有方法*。

私有数据域和方法可以在类内部被访问，但它们不能在类外被访问。为了让客户端访问数据域，就要提供一个 get 方法返回它的值。为了使数据域可以被更改，就要提供一个 set 方法去设置一个新值。

通俗地讲，get 方法是指*获取器*（或*访问器*），set 方法是指*设置器*（或*修改器*）。

一个 get 方法有下面的方法头：

```
def getPropertyName(self):
```

如果返回类型是布尔型，那么习惯上 get 方法被如下定义：

```
def isPropertyName(self):
```

一个 set 方法有下面的方法头：

```
def setPropertyName(self, propertyValue):
```

程序清单 7-6 通过在属性名前加两个下划线（第 6 行）将 radius 属性定义为私有的来修

改程序清单 7-1 中的 Circle 类。

程序清单 7-6 CircleWithPrivateRadius.py

```
1  import math
2
3  class Circle:
4      # Construct a circle object
5      def __init__(self, radius = 1):
6          self.__radius = radius
7
8      def getRadius(self):
9          return self.__radius
10
11     def getPerimeter(self):
12         return 2 * self.__radius * math.pi
13
14     def getArea(self):
15         return self.__radius * self.__radius * math.pi
```

radius 属性在新 Circle 类中不能被直接访问。但是，你可以使用 getRadius() 方法读取它。例如：

```
1  >>> from CircleWithPrivateRadius import Circle
2  >>> c = Circle(5)
3  >>> c.__radius
4  AttributeError: no attribute '__radius'
5  >>> c.getRadius()
6  5
7  >>>
```

第一行导入 Circle 类，这个类是在程序清单 7-6 中的 CircleWithPrivateRadius 模块中定义的。第二行创建了一个 Circle 对象。第三行试图访问属性 __radius。这会导致一个错误，因为 __radius 是私有的。然而，你可以使用 getRadius() 方法返回 radius（第 5 行）。

提示：如果类是被设计来给其他程序使用的，为了防止数据被篡改并使类易于维护，就将数据域定义为私有的。如果这个类只是在程序内部使用，那就没必要隐藏数据域。

注意：使用两个下划线开头来命名私有数据域和方法，但不要以一个以上的下划线结尾。在 Python 语言中，以两个下划线开头同时以两个下划线结尾的名字具有特殊的含义。例如：__radius 是一个私有数据域，但是 __radius__ 并不是私有数据域。

检查点

7.13　运行下面的程序时会出现什么问题？如何修改它？

```
class A:
    def __init__(self, i):
        self.__i = i

def main():
    a = A(5)
    print(a.__i)

main() # Call the main function
```

7.14　下面的代码正确吗？如果正确，它的输出是什么？

```
1  def main():
2      a = A()
```

```
    a.print()

class A:
    def __init__(self, newS = "Welcome"):
        self.__s = newS

    def print(self):
        print(self.__s)

main() # Call the main function
```

7.15　下面的代码正确吗？如果不正确，修改这个错误。

```
class A:
    def __init__(self, on):
        self.__on = not on

def main():
    a = A(False)
    print(a.on)

main() # Call the main function
```

7.16　数据隐藏的优点是什么？在 Python 中如何实现它？

7.17　如何定义一个私有方法？

7.6　类的抽象与封装

关键点：类的抽象是将类的实现和类的使用分离的概念。类的实现的细节对用户而言是不可见的。这就是类的封装。

软件开发中有许多不同层次的抽象。在第 6 章，你已经学习了函数抽象以及在逐步求精中使用它。类的抽象是指将类的实现和类的使用分离开。类的创建者描述类的功能，让客户端知道如何使用这个类。类是方法以及对这些方法要完成动作的描述的一个集合，它被用来作为给客户端的类合约。

如图 7-8 所示，类的用户并不需要知道类是如何实现的。实现的细节被封装并对用户隐藏。这就被称为类的封装。本质上讲，封装将数据和方法整合到一个单一的对象中并对用户隐藏数据域和方法的实现。例如：你可以创建一个 Circle 对象，在不知道面积是如何被计算出来的情况下获取圆的面积。因此，类也被称为抽象数据类型（ADT）。

图 7-8　类抽象将类的实现从类的使用中分离出来

类的抽象和封装是同一个硬币的两面。许多现实生活的例子阐释了类抽象的概念。例如，考虑建造一个计算机系统。你的个人计算机有许多组件，CPU、存储器、磁盘、主板、风扇等。每个组件都可以被看作是一个具有属性和方法的对象。为了让这些组件一起工作，你只需要知道如何使用每个组件，以及它们之间如何相互作用。你不需要知道每个组件内部是如何工作的。内部实现都是被封装的，而且是对你隐藏的。你甚至可以在不知道每个组件是如何实现的情况下组装一台计算机出来。

计算机系统精确模拟映射面向对象方法。每个组件可以看作是组件类的一个对象。例

如：你可能已经有一个定义电脑中使用的风扇的类，它有像大小、速度等的属性，也有像启动和停止这样的方法。一个特定的风扇就是这个类的一个带有具体属性值的对象。

考虑将获取一笔贷款作为另外一个例子。一笔特定的贷款可以看作是这个 Loan 类的一个对象。利率、贷款额以及贷期周期都是它的数据属性，而计算月支付额及年支付额是它的方法。当你买车时，根据你的贷款利率、贷款额以及贷款周期实例化贷款这个类，并创建一个特定的贷款对象。然后，你可以使用这些方法计算出你贷款的月支付额和总支付额。作为 Loan 类的使用者，你不需要知道这些方法是如何实现的。

程序清单 2-8 给出一个计算贷款支付额的程序。目前编写的这个程序是不能被其他程序所重用的。解决这个问题的一种方式是定义计算月支付额和总支付额的函数。但是，这个解决方案具有局限性。假设你想让贷款与贷款人相关联。除了使用对象，没有更好的方法将贷款与贷款人捆绑在一起。传统的面向过程的程序设计范型是动作驱动的；数据是独立于动作的。面向对象程序设计范型的重点在对象上，所以，动作是依照对象中的数据而定义的。要将贷款人和贷款关联，你可以定义一个含有贷款人及和它有关的贷款属性作为数据域。贷款对象就会包含数据以及操作和处理数据的动作，贷款数据和动作都集合在一个对象里。图 7-9 给出 Loan 类的 UML 类图。注意：UML 类图中的横线（–）表示类的私有数据域或方法。

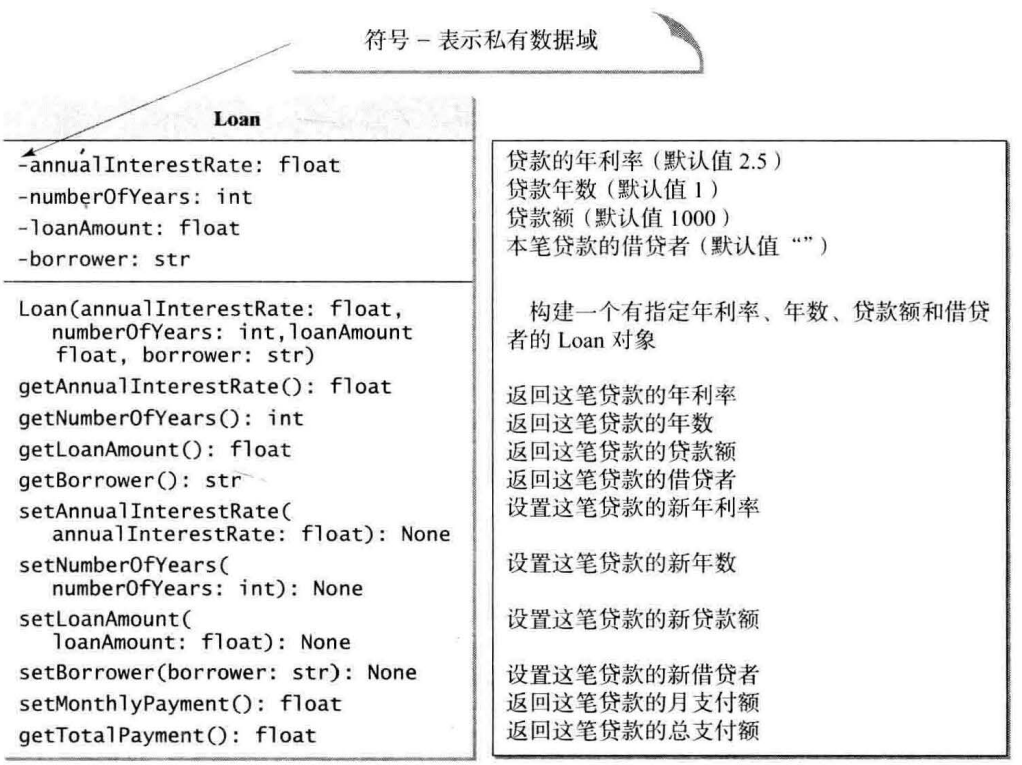


图 7-9　Loan 类的 UML 图是对贷款属性和动作的建模

图 7-9 中的 UML 类图就像是 Loan 类的合约。也就是说，用户可以在不知道类是如何实现的情况下使用这个类。假定这个 Loan 类是可用的。我们开始编写一个测试程序来使用程序清单 7-7 中的 Loan 类。

程序清单 7-7 TestLoanClass.py

```
1  from Loan import Loan
2
3  def main():
4      # Enter yearly interest rate
5      annualInterestRate = eval(input
6          ("Enter yearly interest rate, for example, 7.25: "))
7
8      # Enter number of years
9      numberOfYears = eval(input(
10         "Enter number of years as an integer: "))
11
12     # Enter loan amount
13     loanAmount = eval(input(
14         "Enter loan amount, for example, 120000.95: "))
15
16     # Enter a borrower
17     borrower = input("Enter a borrower's name: ")
18
19     # Create a Loan object
20     loan = Loan(annualInterestRate, numberOfYears,
21         loanAmount, borrower)
22
23     # Display loan date, monthly payment, and total payment
24     print("The loan is for", loan.getBorrower() )
25     print("The monthly payment is",
26         format(loan.getMonthlyPayment() , ".2f"))
27     print("The total payment is",
28         format(loan.getTotalPayment() , ".2f"))
29
30 main() # Call the main function
```

```
Enter yearly interest rate, for example, 7.25: 2.5 Enter
Enter number of years as an integer: 5 Enter
Enter loan amount, for example, 120000.95: 1000 Enter
Enter a borrower's name: John Jones Enter
The loan is for John Jones
The monthly payment is 17.75
The total payment is 1064.84
```

main 函数：①读取利率、支付周期（以年为单位）和贷款额；②创建 Loan 对象；③使用 Loan 类的实例方法获取月支付额（第 26 行）和总支付额（第 28 行）。

Loan 类的实现如程序清单 7-8 所示。

程序清单 7-8 Loan.py

```
1  class Loan :
2      def __init__(self, annualInterestRate = 2.5,
3          numberOfYears = 1, loanAmount = 1000, borrower = " "):
4          self.__annualInterestRate = annualInterestRate
5          self.__numberOfYears = numberOfYears
6          self.__loanAmount = loanAmount
7          self.__borrower = borrower
8
9      def getAnnualInterestRate(self):
10         return self.__annualInterestRate
11
12     def getNumberOfYears(self):
13         return self.__numberOfYears
14
15     def getLoanAmount(self):
```

```
16          return self.__loanAmount
17
18      def getBorrower(self):
19          return self.__borrower
20
21      def setAnnualInterestRate(self, annualInterestRate):
22          self.__annualInterestRate = annualInterestRate
23
24      def setNumberOfYears(self, numberOfYears):
25          self.__numberOfYears = numberOfYears
26
27      def setLoanAmount(self, loanAmount):
28          self.__loanAmount = loanAmount
29
30      def setBorrower(self, borrower):
31          self.__borrower = borrower
32
33      def getMonthlyPayment(self):
34          monthlyInterestRate = self.__annualInterestRate / 1200
35          monthlyPayment = \
36            self.__loanAmount * monthlyInterestRate / (1 - (1 /
37            (1 + monthlyInterestRate) ** (self.__numberOfYears * 12)))
38          return monthlyPayment
39
40      def getTotalPayment(self):
41          totalPayment = self.getMonthlyPayment() * \
42              self.__numberOfYears * 12
43          return totalPayment
```

因为数据域 annualInterestRate、numberOfYears、loanAmount 以及 borrower 都被定义为私有的（以两个下划线开头），不能在类之外由客户端访问它们。

从类开发者的角度来看，类是为许多不同用户使用而设计的。为了满足更广泛的应用需求，类必须为用户提供各种途径用方法对类进行定制。

重要的教学提示：Loan 类的 UML 图如图 7-9 所示。你应该首先编写一个测试程序，它可以在你不知道 Loan 类是如何实现的情况下使用 Loan 类。这有三点益处：

- 它表明开发类和使用类是两件不同的任务。
- 它是在不打乱课本连续性的情况让你跳过某些类的复杂实现。
- 如果你熟悉如何使用类，那么学习如何实现类就会更容易。

从现在以后，对所有的类开发实例，首先使用该类创建一个对象，并尽量使用类的方法，然后再把你的注意力放在它的实现上。

7.7　面向对象的思考

关键点：面向过程范型程序设计的重点在设计函数上。而面向对象范型将数据和方法一起合并到对象中。使用面向对象范型的软件设计的重点是在对象和对象上的操作。

本书的目的是在学习面向对象程序设计之前教授解决问题的方法和基本程序设计技巧。本节展示面向过程和面向对象程序设计的区别。你将会看到面向对象程序设计的益处并学会如何高效地使用它。我们将用面向对象的方法优化第 4 章介绍的 BMI 问题的解决方法。在这个优化过程中，你会清楚地了解到面向过程和面向对象程序设计的不同，看到使用对象和类开发可复用代码的好处。

程序清单 4-6 给出计算身体质量指数的程序。代码本身不能被其他程序所重用。为了使它可重用，定义一个独立的函数来计算身体质量指数，如下所示：

```
def getBMI(weight, height):
```

这个函数在计算指定体重和身高的身体质量指数时很有用。但是它有局限性。假设你需要将身高、体重和个人的名字、生日相关联。你可以创建不同的变量来存储这些值，但是这些值并不是紧耦合的。最理想的耦合它们的方法是创建包含它们的对象。因为这些值都被绑定到一个单独的对象上的，所以它们应该被存储于数据域中。你可以定义一个名为 BMI 的类，如图 7-10 所示。

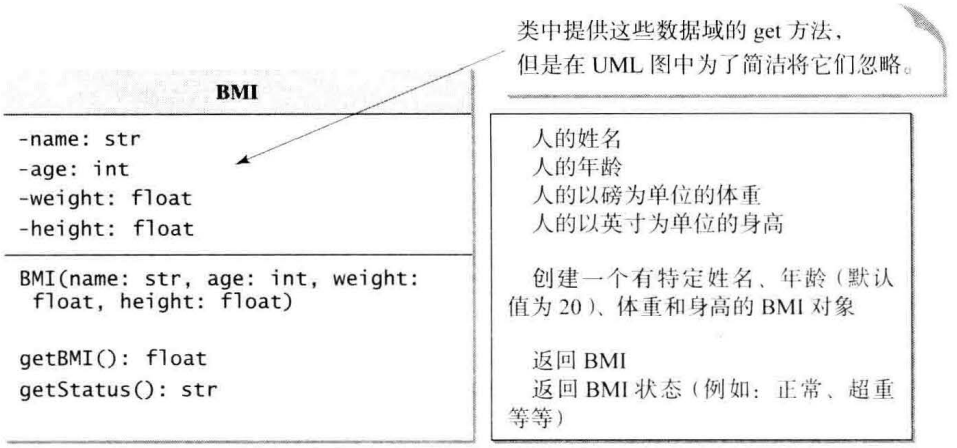


图 7-10　BMI 类封装 BMI 数据和方法

假设 BMI 类是可用的。程序清单 7-9 是一个使用这个类的测试程序。

程序清单 7-9　UseBMIClass.py

```
1  from BMI import BMI
2
3  def main():
4      bmi1 = BMI("John Doe", 18, 145, 70)
5      print("The BMI for", bmi1.getName(), "is",
6          bmi1.getBMI(), bmi1.getStatus())
7
8      bmi2 = BMI("Peter King", 50, 215, 70)
9      print("The BMI for", bmi2.getName(), "is",
10         bmi2.getBMI(), bmi2.getStatus())
11
12 main() # Call the main function
```

```
The BMI for John Doe is 20.81 Normal
The BMI for Peter King is 30.85 Obese
```

第 4 行为 John Doe 创建一个对象 bmi1，第 8 行为 Peter King 创建一个对象 bmi2。你可以使用方法 getName()、getBMI() 和 getStatus() 返回 BMI 对象中的 BMI 信息（第 5 行和第 9 行）。

BMI 类可以如程序清单 7-10 所示来实现。

程序清单 7-10　BMI.py

```
1  class BMI:
2      def __init__(self, name, age, weight, height):
3          self.__name = name
4          self.__age = age
```

```
5           self.__weight = weight
6           self.__height = height
7
8       def getBMI(self):
9           KILOGRAMS_PER_POUND = 0.45359237
10          METERS_PER_INCH = 0.0254
11          bmi = self.__weight * KILOGRAMS_PER_POUND /  \
12             ((self.__height * METERS_PER_INCH) * \
13             (self.__height * METERS_PER_INCH))
14          return round(bmi * 100) / 100
15
16      def getStatus(self):
17          bmi = self.getBMI()
18          if bmi < 18.5:
19              return "Underweight"
20          elif bmi < 25:
21              return "Normal"
22          elif bmi < 30:
23              return "Overweight"
24          else:
25              return "Obese"
26
27      def getName(self):
28          return self.__name
29
30      def getAge(self):
31          return self.__age
32
33      def getWeight(self):
34          return self.__weight
35
36      def getHeight(self):
37          return self.__height
```

使用体重和身高计算 BMI 的数学公式在第 4.9 节中已经给出。方法 getBMI() 返回 BMI。因为身高和体重都是对象中的数据域，所以方法 getBMI() 可以使用这些属性计算对象的 BMI。

方法 getStatus() 返回一条解释 BMI 的字符串。解释也在第 4.9 节中给出。

这个例子演示了面向对象范型相对于面向过程范型的优势。面向对象方法结合了面向过程范型和一个将数据和操作集合在对象中的附加维度。

在面向过程程序设计中，数据和操作是分离的，这种方法论需要将数据发送给方法。面向对象程序设计将数据以及和它们相关的操作一起放在一个对象中。这种方法解决了许多继承自面向过程中的问题。面向对象程序设计方法组织程序的方式在某种程度上反映了现实世界，现实世界中的所有对象都是既和属性相关联又和动作相关联。使用对象可以提高软件复用性，同时可以使程序易于开发和维护。Python 中的程序设计涉及在对象方面的思考；Python 程序可被视为相互作用的对象的集合。

检查点

7.18 描述面向过程和面向对象范型的不同之处。

关键术语

abstract data type (ADT)（抽象数据类型）
accessor (getter)（访问器（获取器））
actions（动作）
anonymous object（匿名对象）
attributes（属性）
behavior（行为）

class（类）
class abstraction（类抽象）
class encapsulation（类封装）
class's contract（类的合约）
client（客户端）
constructor（构造方法）
data fields（数据域）
data hiding（数据隐藏）
dot operator (.)（圆点运算符（.））
identity（实体）
initializer（实例）
instance（实例方法）
instance variable（实例变量）
instantiation（实例化）
mutator (setter)（修改器（设置器））
object-oriented programming (OOP)（面向对象程序设计（OOP））
private data fields（私有数据域）
private method（私有方法）
property（属性）
state（状态）
Unified Modeling Language (UML)（统一建模语言（UML））

本章总结

1. 类是一种对象的模板、蓝图、合约和数据类型。它定义了对象的属性，并提供用于初始化对象的初始化程序和操作这些属性的方法。
2. 初始化程序总是以 __init__ 命名。每个方法的第一个参数包括类中的初始化程序，它指向调用这个方法的对象。按照惯例，这个参数以 self 命名。
3. 对象是类的一个实例。你使用构造方法来创建一个对象，使用圆点运算符（.）通过引用变量来访问对象的成员。
4. 实例变量或方法属于类的一个实例。它的使用和每个独立的实例相关联。
5. 类中的数据域应该被隐藏以避免被更改并使类易于维护。
6. 你可以提供 get 方法或 set 方法使客户端可以查看或更改数据。通俗地讲，get 方法被称为获取器（或访问器），而 set 方法被称为设置器（或修改器）。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

第 7.2 ～ 7.3 节

7.1 （Rectangle 类）按照第 7.2 节中 Circle 类的例子，设计一个名为 Rectangle 类来表示矩形。这个类包括：

- 两个名为 width 和 height 的数据域。
- 构造方法创建一个指定 width 和 heightd 的矩形。将 1 和 2 分别作为 width 和 height 的默认值。
- 一个名为 getArea() 的方法来返回这个矩形的面积。
- 一个名为 getPerimeter() 的方法返回周长。

绘制该类的 UML 类图，然后实现这个类。编写一个测试程序创建两个 Rectangle 对象，一个宽为 4 高为 40，而另一个宽为 3.5 高为 35.7。按照这个顺序显示每个矩形的宽、高、面积和周长。

第 7.4 ～ 7.7 节

7.2 （Stock 类）设计一个名为 Stock 的类来表示一个公司的股票，它包括：

- 一个名为 symbol 的私有字符串数据域表示股票的符号。
- 一个名为 name 的私有字符串数据域表示股票的名字。

- 一个名为 previousClosingPrice 的私有浮点数据域存储前一天的股票价。
- 一个名为 currentPrice 的私有浮点数据域存储当前的股票价。
- 一个构造方法创建一支具有特定的符号、名字、之前价和当前价的股票。
- 一个返回股票名字的 get 方法。
- 一个返回股票符号的 get 方法。
- 获取 / 设置股票之前价的 get 和 set 方法。
- 获取 / 设置股票当前价的 get 和 set 方法。
- 一个名为 getChangePercent() 的方法返回从 previousClosingPrice 到 currentPrice 所改变的百分比。

绘制这个类的 UML 类图，然后实现这个类。编写一个测试程序，创建一个 Stock 对象，它的符号是 INTC，它的名字是 Intel Corporation，前一天的结束价是 20.5，新的当前价是 20.35，并且显示价格改变的百分比。

7.3 （Account 类）设计一个名为 Account 的类，它包括：

- 账户的一个名为 id 的私有 int 数据域。
- 账户的一个名为 balance 的私有浮点数据域。
- 一个名为 annualInterestRate 的私有浮点数据域存储当前利率。
- 一个构造方法创建具有特定 id（默认值 0）、初始额（默认值 100）以及年利率（默认值 0）。
- id、balance 和 annualInterestRate 的访问器和修改器。
- 一个名为 getMonthlyInterestRate() 的方法返回月利率。
- 一个名为 getMonthlyInterest() 的方法返回月利息。
- 一个名为 withdraw 的方法从账户取出特指定数额。
- 一个名为 deposit 的方法向账户存入指定数额。

绘制这个类的 UML 类图，然后实现这个类（提示：方法 getMonthlyInterest() 返回每月利息额，而不是返回利率。使用这个公式计算月利息：balance*monthlyInterestRate。monthlyInterestRate 是 annualInterestRate/12。注意：annualInterestRate 是一个百分数（如 4.5%））。你需要将它除以 100。

编写测试程序创建一个 Account 对象，这个账户的 id 是 1122，账户额是 20 000 美元而年利率是 4.5%。使用 withdraw 方法取 2500 美元，使用 deposit 方法存 3000 美元，并打印 id、金额、月利率和月利息。

7.4 （Fan 类）设计一个名为 Fan 的类表示一个风扇。这个类包括：

- 三个名为 SLOW、MEDIUM 和 FAST 的常量，它们的值分别是 1、2 和 3 以表示风扇速度。
- 一个名为 speed 的私有整型数据域表明风扇的速度。
- 一个名为 on 的私有布尔数据域表明风扇是否是打开状态（默认值是 False）。
- 一个名为 radius 的私有浮点数据域表明风扇的半径。
- 一个名为 color 的私有字符串数据域表明风扇的颜色。
- 四个数据域的访问器和修改器。
- 一个构造方法创建一个具有特定速度（默认值为 SLOW）、半径（默认值为 5）、颜色（默认值为 blue）以及是否打开（默认值为 False）。

绘制这个类的 UML 类图，然后实现这个类。编写测试程序创建两个 Fan 对象。对第一个对象，赋值最大速度、半径为 10、颜色为 yellow，打开它。对第二个对象，赋值中速、半径为 5、颜色为 blue，关闭它。显示每个对象的 speed、radius、color 和 on 属性。

*7.5 （几何：正 n 边形）一个正 n 边形的边都有同样的长度，所有的角都有同样的度数（即多边形是等边等角的）。设计一个名为 RegularPolygon 的类，包括：

- 一个名为 n 的私有整型数据域定义多边形的边数。
- 一个名为 side 的私有浮点数据域存储边的长度。

- 一个名为 x 的私有浮点数据域定义多边形中心的 x 轴坐标值，默认值为 0。
- 一个名为 y 的私有浮点数据域定义多边形中心的 y 轴坐标值，默认值为 0。
- 一个构造方法创建一个具有指定边数 n（默认值是 3）、边长（默认值是 1）、x（默认值是 0）和 y（默认值是 0）的正多边形。
- 所有数据域的访问器和修改器。
- 方法 getPerimeter() 返回多边形的周长。
- 方法 getArea() 返回多边形的面积。计算一个正多边形面积的公式是 $Area = \frac{n \times s^2}{4 \times \tan\left(\frac{\pi}{n}\right)}$

绘制这个类的 UML 图，然后实现这个类。编写测试程序创建三个 RegularPolygon 对象，这三个对象是分别使用 RegularPolygon()、RegularPolygon(6,4) 和 RegularPolygon(10,4,5.6,7.8) 来创建的。对于每个对象，显示它的周长和面积。

*7.6 （代数：平方根）设计一个名为 QuadraticEquation 类来计算方程式 $ax^2+bx+c=0$ 的平方根。该类包括：

- 私有数据域 a、b 和 c 表示三个系数。
- 以 a、b 和 c 为参数的构造方法。
- a、b 和 c 各自的 get 方法。
- 名为 getDescriminant() 的方法返回判别式，即 b^2-4ac
- 名为 getRoot1() 和 getRoot2() 的方法是使用下面这些公式返回方程式的两个根：

$$r_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a} \text{ 和 } r_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

这些方法都是只在判别式非负时才有用。如果判别式是负数，则让这些方法返回 0。

绘制这个类的 UML 图，然后实现这个类。编写一个测试程序提示用户输入 a、b 和 c 的值，然后显示基于这个判别式的结果。如果判别式为正，显示两个根。如果判别式为 0，显示一个根。否则，显示“该方程式无根”。参见编程题 4.1 的示例运行。

*7.7 （代数：2×2 线性方程式）设计一个名为 LinearEquation 的类，它是 2×2 的线性方程式：

$$\begin{aligned} ax + by &= e \\ cx + dy &= f \end{aligned} \qquad x = \frac{ed - bf}{ad - bc} \qquad y = \frac{af - ec}{ad - bc}$$

这个类包括：

- 具有 get 方法的私有数据域 a、b、c、d、e 和 f。
- 一个参数为 a、b、c、d、e 和 f 的构造方法。
- a、b、c、d、e 和 f 各自的 get 方法。
- 一个名为 isSolvable() 的方法，如果 $ad-bc$ 不为零则返回 true。
- 方法 getX() 和 getY() 返回这个方程的解。

绘制这个类的 UML 类图，然后实现这个类。编写测试程序提示用户输入 a、b、c、d、e 和 f 的值，然后显示结果。如果 $ad-bc$ 为零，那么显示“这个方程式无解”。参见编程题 4.3 的示例运行。

*7.8 （跑表）设计一个名为 StopWatch 的类。该类包括：

- 具有 get 方法的私有数据域 startTime 和 endTime。
- 使用当前时间初始化 startTime 的构造方法。
- 一个名为 start() 的方法将 startTime 重置为当前时间。
- 一个名为 stop() 的方法将 endTime 设置为当前时间。
- 一个名为 getElapsedTime() 的方法返回秒表流逝的毫秒数。

绘制该类的 UML 类图，然后实现这个类。编写测试程序测量将数值从 1 增加到 1 000 000

的执行时间。

**7.9 （几何：交叉线）假设有两条线段相交。第一条线段的两个端点是 (x1,y1) 和 (x2,y2)，第二条线段的两个端点是 (x3,y3) 和 (x4,y4)。编写程序提示用户输入这四个端点，然后显示它们的交点（提示：使用编程题 7.7 中的 LinearEquation 类）。

```
Enter the endpoints of the first line segment: 2.0, 2.0, 0, 0 Enter
Enter the endpoints of the second line segment: 0, 2.0, 2.0, 0 Enter
The intersecting point is: (1.0, 1.0)
```

*7.10 （Time 类）设计一个名为 Time 的类。该类包括：

- 表示时间的私有数据域 hour、minute 和 second。
- 一个使用当前时间初始化 hour、minute 和 second 来构造一个 Time 对象的构造方法。
- 数据域 hour、minute 和 second 各自的 get 方法。
- 一个名为 setTime(elapseTime) 的方法使用流逝的秒数为对象设置一个新时间。例如：如果流逝的时间是 555 550 秒，那么小时数是 10，分钟数是 19 而秒数是 12。

绘制这个类的 UML 类图，然后实现这个类。编写一个测试程序创建一个 Time 对象，然后显示它的小时数、分钟数和秒数。然后，你的程序提示用户输入流逝的时间，将它的流逝时间设置在 Time 对象中，然后显示它的小时数、分钟数和秒数。下面是一个示例运行：

```
Current time is 12:41:6
Enter the elapsed time: 55550505 Enter
The hour:minute:second for the elapsed time is 22:41:45
```

（提示：初始化程序将从流逝的时间中提取出小时数、分钟数和秒数。当前的流逝时间可以通过使用 time.time() 来获取，如程序清单 2-7 所示。）

第 8 章

Introduction to Programming Using Python

更多字符串和特殊方法

学习目标

- 学习如何创建字符串（第 8.2.1 节）。
- 使用 len、min 和 max 函数获取一个字符串的长度、串中的最大和最小的字符（第 8.2.2 节）。
- 使用下标运算符（[]）访问字符串中的元素（第 8.2.3 节）。
- 使用截取运算符 str[start:end] 从较长的字符串中得到一个子串（第 8.2.4 节）。
- 使用 + 运算符连接两个字符串，通过 * 运算符复制一个字符串（第 8.2.5 节）。
- 使用 in 和 not in 运算符判断一个字符串是否包含在另一个字符串内（第 8.2.6 节）。
- 使用比较运算符（==、！ =、<、<=、>、>=）对字符串进行比较（第 8.2.7 节）。
- 使用 for 循环迭代字符串中的字符（第 8.2.8 节）。
- 使用方法 isalnum、isalpha、isdigit、isidentifier、islower、isupper 和 isspace 来测试字符串（第 8.2.9 节）。
- 使用方法 endswith、startswith、find、rfind 和 count 搜索子串（第 8.2.10 节）。
- 使用方法 capitalize、lower、upper、title、swapcase 和 replace 转换字符串（第 8.2.11 节）。
- 使用方法 lstrip、rstrip 和 strip 从一个字符串的左侧或右侧删除空格（第 8.2.12 节）。
- 使用方法 center、ljust、rjust 和 format 格式化字符串（第 8.2.13 节）。
- 在应用程序（CheckPalindrome、HexToDecimalConversion）的开发过程中应用字符串（第 8.3 ～ 8.4 节）。
- 为运算符定义特殊的方法（第 8.5 节）。
- 设计 Rational 类表示有理数（第 8.6 节）。

8.1 引言

关键点：本章将重点放在类的设计上，它使用 Python 中的 str 类为例并探索 Python 中特殊方法的作用。

前一章介绍了关于类和对象的一些重要概念。我们已经学习了如何定义一个类，以及如何创建和使用对象。str 类不仅在表示字符串方面很有用，而且它也是一个很好的类设计的例子。这个类在第 3 章中已经介绍过。本章我们将更深入地讨论 str 类。

在 Python 语言中特殊方法起着非常重要的作用。这里也会介绍一些特殊方法和运算符重载，以及使用特殊方法设计类。

8.2 str 类

关键点：一个 str 对象是不可变的；这也就是说，一旦创建了这个字符串，那么它的内容是不可变的。

在第 7 章中，我们已经学会如何定义类 Loan 和 BMI 以及如何从这些类创建对象，并且将会经常使用这些来自 Python 库的类来开发程序。这里介绍 Python 的 str 类。

字符串是计算机科学的基础，而处理字符串是程序设计中的常见任务。字符串是 str 类的对象。到目前为止，你已经在输入和输出中使用过字符串。input 函数从键盘返回一个字符串，而 print 函数在显示器上显示一个字符串。

8.2.1　创建字符串

你可以使用构造函数构建字符串，如下所示：

```
s1 = str() # Create an empty string object
s2 = str("Welcome") # Create a string object for Welcome
```

Python 提供了一个简单语法，它通过使用字符串值创建一个字符串。例如：

```
s1 = "" # Same as s1 = str()
s2 = "Welcome" # Same as s2 = str("Welcome")
```

一个字符串对象是不可变的：一旦创建一个字符串对象出来，那么它的内容就不会再改变。为了优化性能，Python 使用一个对象来表示具有相同内容的字符串。如图 8-1 所示，s1 和 s2 都指向同一个字符串对象，它们都有着相同的 id 数。

```
>>> s1 = "Welcome"
>>> s2 = "Welcome"
>>> id(s1)
505408902
>>> id(s2)
505408902
>>>
```

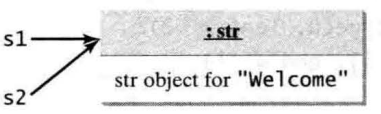


图 8-1　具有相同内容的字符串实际上是同一个对象

这个动作对 Python 库中的所有不可变对象都是真的。例如：int 是一种不可变类。两个具有相同值的 int 对象实际上是共享了相同的对象，如图 8-2 所示。

```
>>> x = 10
>>> y = 10
>>> id(x)
35747680
>>> id(y)
35747680
>>>
```

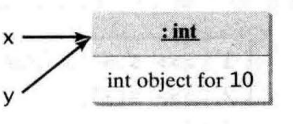


图 8-2　所有不可变的具有相同内容的对象都被存储在同一个对象中

8.2.2　处理字符串的函数

Python 的一些内置函数可以和字符串一起使用。你可以使用 len 函数来返回一个字符串中的字符个数，而 max 和 min 函数（第 3 章已经介绍过）返回字符串中的最大和最小字符。下面是一些例子：

```
1  >>> s = "Welcome"
2  >>> len(s)
3  7
4  >>> max(s)
5  'o'
6  >>> min(s)
7  'W'
8  >>>
```

因为字符串 s 有 7 个字符，len(s) 返回 7（第 3 行）。注意：小写字母的 ASCII 码值要高于小写字母的 ASCII 码值，所以，max(s) 返回 o（第 5 行），而 min(s) 返回 W（第 7 行）。

下面是另一个例子：

```
s = input("Enter a string: ")
if len(s) % 2 == 0:
    print(s, "contains an even number of characters")
else:
    print(s, "contains an odd number of characters")
```

如果运行代码时你输入 computer，它会显示

```
computer contains an even number of characters
```

8.2.3　下标运算符 []

一个字符串是一个字符序列。可以使用下面的语法通过下标运算符访问字符串中的一个字符：

```
s[index]
```

下标是基于 0 的；也就是说，它们的范围从 0 到 len(s)−1，如图 8-3 所示。

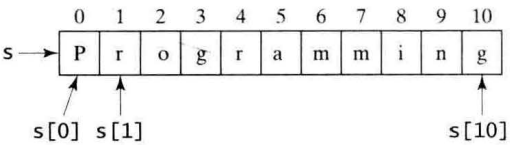


图 8-3　字符串中的字符可以通过下标运算符访问

例如：

```
>>> s = "Welcome"
>>> for i in range(0, len(s), 2):
...     print(s[i], end = '')
Wloe
>>>
```

在 for 循环中，i 是 0、2、4 和 6。所以，显示 s[0]、s[2]、s[4] 和 s[6]。

Python 也允许负数作为下标，它表示相对于字符串末尾字符的位置。确切的位置是通过负数下标和字符串长度相加来获取。例如：

```
1  >>> s = "Welcome"
2  >>> s[-1]
3  'e'
4  >>> s[-2]
5  'm'
6  >>>
```

在第 2 行中，s[−1] 和 s[−1+len（s）] 一样，它们都是字符串的最后一个字符。在第 4 行中，s[−2] 和 s[−2+len(s)] 一样，它们都是字符串的倒数第二个字符。

注意：由于字符串是不可变的，所以你不能改变它们的内容。例如，下面的代码是非法的。

```
s[2] = 'A'
```

8.2.4　截取运算符 [start：end]

截取运算符通过使用语法 s[start:end] 返回字符串其中的一段。这一段就是从下标 start 到下标 end−1 的一个子串。例如：

```
1  >>> s = "Welcome"
2  >>> s[1 : 4]
3  'elc'
```

s[1:4] 返回从下标 1 到下标 3 的子串。

起始下标或结束下标都可以被忽略。在这种情况下，默认的起始下标是 0，而结束下标是最后一个下标。例如：

```
1  >>> s = "Welcome"
2  >>> s[ : 6]
3  'Welcom'
4  >>> s[4 : ]
5  'ome'
6  >>> s[1 : -1]
7  'elcom'
8  >>>
```

在第 2 行中，s[:6] 和 s[0:6] 是一样的，都返回的是从下标 0 到下标 5 的子串。在第 4 行中，s[4:] 和 s[4:7] 是一样的，都返回的是从下标 4 到下标 6 的子串。你也可以在截取字符串的过程中使用负下标。例如，在第 6 行中，s[1:−1] 和 s[1:−1+len(s)] 是一样的。

注意：如果截取操作 s[i:j] 中的下标（i 或 j）是负数，那么就用 len(s)+index 来替换下标。如果 j>len(s)，那么 j 就会被设置成 len(s)。如果 i>=j，那么截取的子串就会成为空串。

8.2.5　连接运算符 + 和复制运算符 *

你可以使用连接运算符 + 组合或连接两个字符串。你也可以使用复制运算符 * 来连接相同的字符串多次。下面是一些例子：

```
 1  >>> s1 = "Welcome"
 2  >>> s2 = "Python"
 3  >>> s3 = s1 + " to " + s2
 4  >>> s3
 5  'Welcome to Python'
 6  >>> s4 = 3 * s1
 7  >>> s4
 8  'WelcomeWelcomeWelcome'
 9  >>> s5 = s1 * 3
10  >>> s5
11  'WelcomeWelcomeWelcome'
12  >>>
```

注意：3*s1 和 s1*3 具有相同的效果（第 6 ～ 11 行）。

8.2.6　in 和 not in 运算符

你可以使用 in 和 not in 操作来测试一个字符串是否在另一个字符串中。下面是一些例子：

```
>>> s1 = "Welcome"
>>> "come" in s1
True
>>> "come" not in s1
False
>>>
```

下面是另一个例子：

```
s = input("Enter a string: ")
if "Python" in s:
    print("Python", "is in", s)
else:
    print("Python", "is not in", s)
```

如果你在运行这个程序的时候将字符串“Welcome to Python”作为输入，这个程序就应该显示：

```
python is in Welcome to Python.
```

8.2.7　比较字符串

你可以使用比较运算符来对字符串进行比较（第 4.2 节中已经介绍过 ==、!=、>、<、>= 和 <=）。Python 是通过比较字符串中对应的字符进行比较的，比较是通过计算字符的数值代码实现的。例如，a 比 A 大，因为 a 的数值代码比 A 的数值代码大。参见附录 B 中 ASCII 码字符集，找出字符的数值码。

假设你需要比较字符串 s1("Jane") 和 s2("Jake")。首先，比较 s1 和 s2 中的首字符（J 和 J）。因为它们是一样的，所以比较它们的第二个字符（a 和 a）。因为它们也是一样的，所以比较它们的第三个字符（n 和 k）。因为 n 的 ASCII 码值要大于 k 的，所以 s1 是大于 s2 的。

下面是一些例子：

```
>>> "green" == "glow"
False
>>> "green" != "glow"
True
>>> "green" > "glow"
True
>>> "green" >= "glow"
True
>>> "green" < "glow"
False
>>> "green" <= "glow"
False
>>> "ab" <= "abc"
True
>>>
```

下面是另一个例子：

```
1  s1 = input("Enter the first string: ")
2  s2 = input("Enter the second string: ")
3  if s2 < s1:
4      s1, s2 = s2, s1
5
6  print("The two strings are in this order:", s1, s2)
```

如果你运行这个程序的时候输入“Peter”，然后输入“John”，那么 s1 是 Peter，s2 是 John（第 1 ～ 2 行）。因为 s2<s1 为 True（第 3 行），所以它们在第 4 行进行交换。因此，程序在第 6 行显示以下消息。

```
The two strings are in this order: John Peter
```

8.2.8　迭代字符串

一个字符串是可迭代的。这意味着你可以使用一个 for 循环来顺序遍历字符串中的所有字符。例如，下面的代码显示了字符串 s 中的所有字符：

```
for ch in s:
    print(ch)
```

你可以将这段代码读作“对于 s 中的每个字符 ch，都打印 ch”。

这个 for 循环没有使用下标来访问字符。但是，如果你想以不同顺序遍历字符，那么你仍然必须使用下标。例如，下面的代码显示了字符串奇数位置的字符：

```
for i in range(0, len(s), 2):
    print(s[i])
```

这段代码使用变量 i 作为字符串 s 的下标。i 的初始值是 0，然后，在它达到或超过 len(s) 之前每次增加 2。对于每个 i 值，都打印 s[i]。

8.2.9　测试字符串

str 类有许多有用的方法。图 8-4 中的方法测试字符串中的字符。

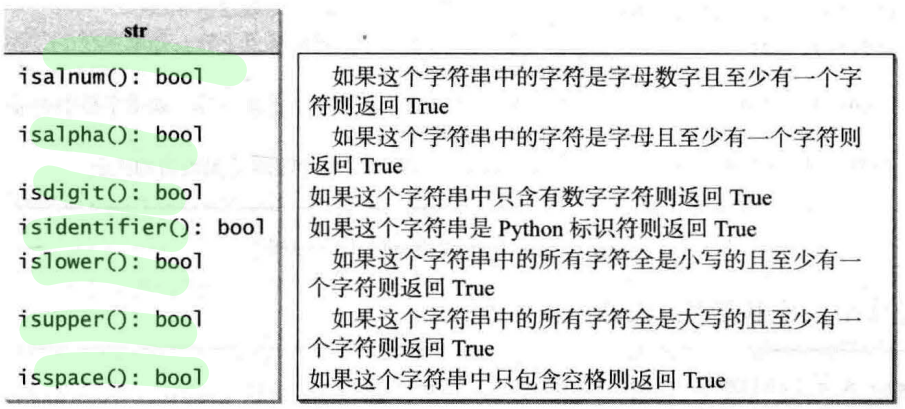


图 8-4　str 类包含测试它的字符的一些方法

下面是一些使用字符串测试方法的例子：

```
>>> s = "welcome to python"
>>> s.isalnum()
False
>>> "Welcome".isalpha()
True
>>> "2012".isdigit()
True
>>> "first Number".isidentifier()
False
>>> s.islower()
True
>>> s.isupper()
False
>>> s.isspace()
False
>>>
```

s.isalnum() 返回 False（第 2 行），因为 s 字符串中包含空格，它既不是字母也不是数字。Welcome 包含的都是字母（第 4 行），所以 "Welcome".isalpha() 返回 True。由于 2012 包含的都是数字，所以 "2012".isdigit() 返回 True（第 6 行）。而且，因为 first Number 中包含一个空格，它并不是一个标识符，所以 "first Number".isidentifier() 返回 False（第 8 行）。

下面是另一个例子：

```
s = "2011"
if s.isdigit():
    print(s, "is a numeric string")
```

这段代码显示：

```
2011 is a numeric string
```

8.2.10　搜索子串

你可以使用图 8-5 中的方法来搜索字符串中的子串。

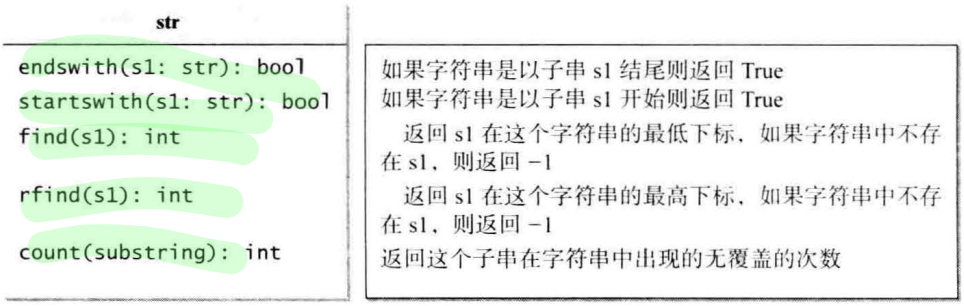


图 8-5　str 类包含搜索子串的方法

下面是使用字符串搜索方法的一些例子：

```
 1  >>> s = "welcome to python"
 2  >>> s.endswith("thon")
 3  True
 4  >>> s.startswith("good")
 5  False
 6  >>> s.find("come")
 7  3
 8  >>> s.find("become")
 9  -1
10  >>> s.rfind("o")
11  15
12  >>> s.count("o")
13  3
14  >>>
```

由于可以在字符串 s 的下标 3 的位置上找到 come，所以 s.find("come") 返回 3（第 7 行）。因为子串 o 从右至左第一个出现的位置是在下标 17 处，所以 s.rfind("o") 返回 17（第 11 行）。在第 8 行，s.find("become") 返回 −1，因为 become 并不在字符串 s 中。在第 12 行，s.count("o") 返回 3，因为 o 在字符串 s 中出现了 3 次。

下面是另一个例子：

```
s = input("Enter a string: ")
if s.startswith("comp"):
    print(s, "begins with comp")
if s.endswith("er"):
    print(s, "ends with er")

print('e', "appears", s.count('e'), "time in", s)
```

如果你运行这段代码时输入“computer”，它会显示：

```
computer begins with comp
computer ends with er
e appears 1 time in computer
```

8.2.11　转换字符串

你可以使用图 8-6 中的方法来复制字符串。这些方法可以让你控制字符串复制过程中字母的大小写，或是整体地替换字符串。

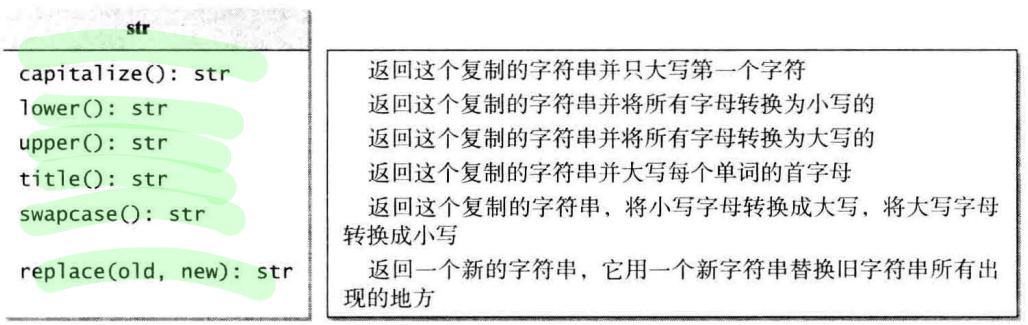


图 8-6　str 类中包含转换字符串中大小写的方法以及使用一个字符串替换另一个字符串的方法

capitalize() 方法返回大写首字母的字符串。lower() 和 upper() 方法返回所有字母都变成小写或大写的字符串。title() 方法返回每个单词首字母大写的字符串。swapCase() 方法返回将小写字母转换为大写而将大写字母转换为小写的字符串。replace(old,new）方法返回新子串 new 替换旧子串 old 的字符串。下面是一些使用这些方法的例子：

```
>>> s = "welcome to python"
>>> s1 = s.capitalize()
>>> s1
'Welcome to python'
>>> s2 = s.title()
>>> s2
'Welcome To Python'
>>> s = "New England"
>>> s3 = s.lower()
>>> s3
'new england'
>>> s4 = s.upper()
>>> s4
'NEW ENGLAND'
>>> s5 = s.swapcase()
```

```
>>> s5
'nEW eNGLAND'
>>> s6 = s.replace("England", "Haven")
>>> s6
'New Haven'
>>> s
'New England'
>>>
```

注意：如同之前所述，字符串是不可变的。str 类中没有方法能改变字符串的内容，这些方法都是创建了新的字符串。如同上述代码所展示的那样，在应用完方法 s.lower()、s.upper()、s.swapcase() 和 s.replace("England","Haven") 之后，s 仍然是 New England（第 21 ～ 22 行）。

8.2.12　删除字符串中的空格

你可以使用图 8-7 中的方法从字符串的前端、末端或者两端来删除字符串中的空格。回顾一下，字符 ''、\t、\f、\r 和 \n 都被称作空白字符（参见第 3.5 节）。

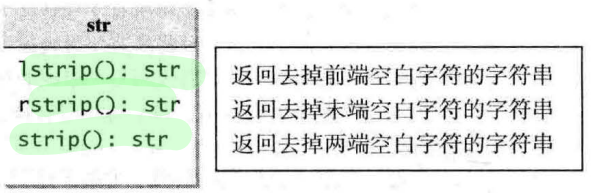


图 8-7　str 类包含去掉前端和末端空白字符的方法

下面是一些使用字符串删除方法的例子：

```
>>> s = "  Welcome to Python\t"
>>> s1 = s.lstrip()
>>> s1
'Welcome to Python\t'
>>> s2 = s.rstrip()
>>> s2
'  Welcome to Python'
>>> s3 = s.strip()
>>> s3
'Welcome to Python'
>>>
```

在第 2 行，s.lstrip() 去掉字符串 s 左端的空白字符。在第 5 行，s.rstrip() 去掉字符串 s 右端的空白字符。在第 8 行，s.strip() 去掉 s 左右两端的空白字符。

注意：删除空白字符的方法只能去掉字符串前后两端的空白字符，并不能删除被非空白字符包围的空白字符。

提示：在输入字符串上应用 strip() 方法来确保删除输入的末尾任何不需要的字符，这是很好的经验。

8.2.13　格式化字符串

你可以使用图 8-8 中的方法返回一个格式化的字符串。

str	
center(width): str	返回在给定宽度域上居中的字符串副本
ljust(width): str	返回在给定宽度域上左对齐的字符串文本
rjust(width): str	返回在给定宽度域上右对齐的字符串文本
format(items): str	格式化一个字符串

图 8-8　str 类中包含这些格式化方法

下面是使用 center、ljust 和 rjust 方法的一些例子：

```
 1  >>> s = "Welcome"
 2  >>> s1 = s.center(11)
 3  >>> s1
 4  '  Welcome  '
 5  >>> s2 = s.ljust(11)
 6  >>> s2
 7  'Welcome    '
 8  >>> s3 = s.rjust(11)
 9  >>> s3
10  '    Welcome'
11  >>>
```

在第 2 行，s.center(11) 将字符串 s 放在占位 11 个字符的字符串中央。在第 5 行，s.ljust(11) 将字符串 s 放在占位 11 个字符的字符串左端。在第 8 行，s.rjust(11) 将字符串 s 放在占位 11 个字符的字符串右端。

第 3.6 节介绍了 format 函数来格式化数字或字符串。str 类也有 format 方法，在补充材料 II.C 中介绍，这与 format 函数是非常相似的。

检查点

8.1　假设给定如下 s1、s2、s3 和 s4 四个字符串：

```
s1 = "Welcome to Python"
s2 = s1
s3 = "Welcome to Python"
s4 = "to"
```

下面表达式的结果是什么？

```
a. s1 == s2
b. s2.count('o')
c. id(s1) == id(s2)
d. id(s1) == id(s3)
e. s1 <= s4
f. s2 >= s4
g. s1 != s4
h. s1.upper()
i. s1.find(s4)
j. s1[4]
k. s1[4 : 8]
l. 4 * s4
m. len(s1)
n. max(s1)
o. min(s1)
p. s1[-4]
q. s1.lower()
r. s1.rfind('o')
s. s1.startswith("o")
t. s1.endswith("o")
u. s1.isalpha()
v. s1 + s1
```

8.2　假设 s1 和 s2 是两个字符串，下面哪个语句或表达式是错误的？

```
s1 = "programming 101"
s2 = "programming is fun"
s3 = s1 + s2
s3 = s1 - s2
s1 == s2
s1 >= s2
i = len(s1)
c = s1[0]
t = s1[ : 5]
t = s1[5 : ]
```

8.3　下面代码输出内容是什么？

```
s1 = "Welcome to Python"
s2 = s1.replace("o","abc")
print(s1)
print(s2)
```

8.4　假设 s1 是 "Welcome" 而 s2 是 "welcome"，编写下面语句的代码。

(a) 检查 s1 和 s2 是否相等，并且将结果赋值给布尔变量 isEqual。

(b) 检查 s1 和 s2 是否相等，忽略大小写，并且将结果赋值给布尔变量 isEqual。

(c) 检查 s1 是否有前缀 AAA，并且将结果赋值给布尔变量 b。

(d) 检查 s1 是否有后缀 AAA，并且将结果赋值给布尔变量 b。

(e) 将 s1 的长度赋值给变量 x。

(f) 将 s1 的首字符赋值给变量 x。

(g) 将 s1 和 s2 组合在一起创建一个新字符串 s3。

(h) 创建一个从下标 1 开始的子串 s1。

(i) 创建一个从下标 1 到下标 4 的子串 s1。

(j) 创建一个将 s1 中的字母转换成小写的新字符串 s3。

(k) 创建一个将 s1 中的字母转换成大写的新字符串 s3。

(l) 创建一个删除字符串 s1 两端空白字符的新字符串 s3。

(m) 用 E 替换字符串 s1 中的 e。

(n) 将字母 e 在字符串 s1 中第一次出现的下标赋值给变量 x。

(o) 将字符串 abc 在字符串中最后一次出现的下标赋值给变量 x。

8.5　字符串对象中的方法能够改变字符串的内容吗？

8.6　假设字符串 s 是一个空串，那么 len(s) 是多少？

8.7　怎么判断一个字符是大写的还是小写的？

8.8　怎么判断一个字符是字母数字的？

8.3　实例研究：校验回文串

关键点：本节给出一个如何检验某个字符串是否是回文串的程序。

如果一个字符串从前往后和从后往前读时是一样的，那么就称这个字符串是回文串。例如："mom"、"dad" 和 "noon" 都是回文串。

编写一段程序，提示用户输入一个字符串，然后输出该字符串是否是回文串。一种解决方案就是让程序检测字符串中的首字符与末尾字符是否相同。如果一样，那么程序就会检测第二个字符是否与倒数第二个字符是否相同。这个过程持续进行，直到有字符不匹配或检测完所有字符才会停止，如果字符串有奇数个字符则不比较中间的字符。

为了实现这个想法，使用两个变量，即 low 和 high 来表示字符串 s 的起始和结束位置的两个字符，如程序清单 8-1 所示（第 13 和第 16 行）。初始状态下，low 是 0 而 high 是 len（s）-1。如果在这两个位置的字符相同，那么 low 增加 1 而 high 减去 1（第 22 ～ 23 行）。这个过程持续直到（low>=high）或出现一个不匹配。

程序清单 8-1 CheckPalindrome.py

```
1  def main():
2      # Prompt the user to enter a string
3      s = input("Enter a string: ").strip()
4
5      if isPalindrome(s):
6          print(s, "is a palindrome")
7      else:
8          print(s, " is not a palindrome")
9
10 # Check if a string is a palindrome
11 def isPalindrome(s):
12     # The index of the first character in the string
13     low = 0
14
15     # The index of the last character in the string
16     high = len(s) - 1
17
18     while low < high:
19         if s[low] != s[high]:
20             return False # Not a palindrome
21
22         low += 1
23         high -= 1
24
25     return True # The string is a palindrome
26
27 main() # Call the main function
```

```
Enter a string: noon ↵Enter
noon is a palindrome
```

```
Enter a string: moon ↵Enter
moon is not a palindrome
```

这段程序提示用户将字符串输入到 s 中（第 3 行），它使用 strip() 方法去掉任何起始端和结尾端的空白字符，然后调用 isPalindrome(s) 来判断 s 是否是回文串。

8.4　实例研究：将十六进制数转换为十进制数

关键点：本节给出一个将十六进制数转换为十进制数的程序。

第 6.8 节中阐述了一段将十进制数转化为十六进制形式的程序。那么如何将一个十六进制数转换为十进制数呢？

给定一个十六进制数 $h_n h_{n-1} h_{n-2} \cdots h_2 h_1 h_0$，和它等值的十进制数是

$$h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \cdots + h_2 \times 16^2 + h_1 \times 16^1 + h_0 \times 16^0$$

例如：十六进制数 AB8C 是

$$10 \times 16^3 + 11 \times 16^2 + 8 \times 16^1 + 12 \times 16^0 = 43\,916$$

我们的程序提醒用户输入一个十六进制数作为字符串，然后使用下面的函数将它转换为

十进制数：

```
def hexToDecimal(hex):
```

暴力破解方法是将每个十六进制数转换为十进制数，就是将第 i 位的十六进制数乘以 16 的 i 次方，然后将所有项加在一起就会获得与十六进制数等值的十进制数。

注意：

$$h_n \times 16^n + h_{n-1} \times 16^{n-1} + h_{n-2} \times 16^{n-2} + \cdots + h_1 \times 16^1 + h_0 \times 16^0$$
$$= (\cdots((h_n \times 16 + h_{n-1}) \times 16 + h_{n-2}) \times 16 + \cdots + h_1) \times 16 + h_0$$

这种方法被称作霍纳算法，下面就是将一个十六进制字符串转换成一个十进制数的代码：

```
decimalValue = 0
for i in range(len(hex)):
    hexChar = hex[i]
    decimalValue = decimalValue * 16 + hexCharToDecimal(hexChar)
```

下面是对十六进制数 AB8C 转换的算法跟踪。

	i	hexChar	hexCharToDecimal (hexChar)	DecimalValue
在循环之前				0
在第 1 次迭代之后	0	A	10	10
在第 2 次迭代之后	1	B	11	10 * 16 + 11
在第 3 次迭代之后	2	8	8	(10 * 16 + 11) * 16 + 8
在第 4 次迭代之后	3	C	12	((10 * 16 + 11) * 16 + 8) * 16 + 12

程序清单 8-2 给出完整的程序。

程序清单 8-2 HexToDecimalConversion.py

```
def main():
    # Prompt the user to enter a hex number
    hex = input("Enter a hex number: ").strip()

    decimal = hexToDecimal(hex.upper())
    if decimal == None:
        print("Incorrect hex number")
    else:
        print("The decimal value for hex number",
            hex, "is", decimal)

def hexToDecimal(hex):
    decimalValue = 0
    for i in range(len(hex)):
        ch = hex[i]
        if 'A' <= ch <= 'F' or '0' <= ch <= '9':
            decimalValue = decimalValue * 16 + \
                hexCharToDecimal(ch)
        else:
            return None

    return decimalValue

def hexCharToDecimal(ch):
    if 'A' <= ch <= 'F':
        return 10 + ord(ch) - ord('A')
    else:
        return ord(ch) - ord('0')

main() # Call the main function
```

```
Enter a hex number: AB8C  Enter
The decimal value for hex number AB8C is 43916
```

```
Enter a hex number: af71  Enter
The decimal value for hex number af71 is 44913
```

```
Enter a hex number: ax71  Enter
Incorrect hex number
```

这个程序从控制台读取一个字符串（第 3 行），然后调用函数 hexToDecimal 将一个十六进制字符串转换为一个十进制数（第 5 行）。输入的字符既可以是小写的也可以是大写的，程序在调用 hexToDecimal 函数之前会把它们都转换为大写的。

在第 12 到 22 行定义的函数 hexToDecimal 返回一个整数。字符串长度是在第 14 行调用 len(hex) 来决定的。如果输入是不正确的十六进制数，那么这个函数返回 None（第 20 行）。

hexCharToDecimal 函数在第 24 到 28 行定义，返回一个表示十六进制字符的十进制数值。字符可能是小写的也可能是大写的。调用 hex.upper() 将字符转换为大写的。当调用 hexCharToDecimal(ch) 时，字符 ch 已经是大写的了。如果 ch 是一个 A 到 F 之间的字母，那么程序会返回一个十进制值 10+ord(ch)−ord('A')（第 26 行）。如果 ch 是一个数字，那么程序会返回一个十进制值 ord(ch)−ord('0')（第 28 行）。

8.5　运算符重载和特殊方法

关键点：Python 允许为运算符和函数定义特殊的方法来实现常用的操作。Python 使用一种独特方式来命名这些方法以辨别它们的关联性。

在之前的章节中，我们已经学会了如何使用完成字符串操作的运算符。可以使用运算符 + 来结合两个字符串，而运算符 * 可以结合同一字符串多次，关系运算符（==、!=、<、<=、>、>=）用来比较两个字符串，而下标运算符 [] 用来访问一个字符。例如：

```
1  s1 = "Washington"
2  s2 = "California"
3  print("The first character in s1 is", s1[0])
4  print("s1 + s2 is", s1 + s2)
5  print("s1 < s2?", s1 < s2)
```

这些运算符实际上都是在 str 类中定义的方法。为运算符定义方法被称作运算符重载。运算符重载允许程序员使用内嵌的运算符为用户定义方法。表 8-1 罗列出运算符和方法之间的映射关系。当你命名这些方法时，名字前后要加两个下划线以便于 Python 辨识它们的关联性。例如：为了将运算符 + 作为方法来使用，你应当定义一个名为 __add__ 的方法。注意：这些方法并不是私有的，因为它们除了两个起始下划线还有两个结尾下划线。回顾一下，类中的初始化程序被命名为 __init__，这是一个初始化对象的特殊方法。

例如，你可以使用如下方法改写之前的代码：

```
1  s1 = "Washington"
2  s2 = "California"
3  print("The first character in s1 is", s1.__getitem__(0))
4  print("s1 + s2 is", s1.__add__(s2))
5  print("s1 < s2?",  s1.__lt__(s2))
```

表 8-1　运算符重载：运算符和特殊方法

运算符	方法	描述	运算符	方法	描述
+	__add__(self, other)	加法	!=	__ne__(self, other)	不等于
*	__mul__(self, other)	乘法	>	__gt__(self, other)	大于
-	__sub__(self, other)	减法	>=	__ge__(self, other)	大于等于
/	__truediv__(self, other)	除法	[index]	__getitem__(self, index)	下标运算符
%	__mod__(self, other)	求余	in	__contains__(self, value)	检查其成员资格
<	__lt__(self, other)	小于	len	__len__(self)	元素个数
<=	__le__(self, other)	小于等于	str	__str__(self)	字符串表示
==	__eq__(self, other)	等于			

s1.__getitem__(0) 与 s1[0] 相同，s1.__add__(s2) 与 s1+s2 相同，而 s1.__lt__(s2) 与 s1<s2 相同。现在，你可以看出重载运算符的优势。重载运算符可以极大地简化程序，让程序更易读、易维护。

Python 支持 in 运算符，它可以用来判断一个字符是否在另一个字符串中或者一个元素是否是一个容器中的成员。对应的方法被命名为 __contains__(self,e)。你可以使用方法 __contain__ 或使用 in 运算符来判断一个字符是否在一个字符串内，代码如下所示。

```
1  s1 = "Washington"
2  print("Is W in s1?", 'W' in s1)
3  print("Is W in s1?", s1.__contains__('W'))
```

W in s1 和 s1.__contains__('w') 一样。

如果一个类定义了 __len__(self) 方法，那么 Python 允许你使用方便的语法将调用方法作为函数调用。例如：__len__ 方法被定义在 str 类中，该方法返回字符串的字符个数。你既可以使用 __len__ 方法也可以使用 len 函数来获取字符串中的字符个数，代码如下所述：

```
1  s1 = "Washington"
2  print("The length of s1 is", len(s1))
3  print("The length of s1 is", s1.__len__())
```

len(s1) 和 s1.__len__() 是相同的。

许多特殊的运算符都被定义为 Python 的内置类型，例如：int 和 float。假设 i 是 3 而 j 是 4。i.__add__(j) 和 i+j 是相同的，而 i.__sub__(j) 和 i-j 是相同的。

注意：你可以传递一个对象去调用 print(x)。这等价于调用 print(x.__str__()) 或 print(str(x))。

注意：比较运算符（==、!=、<、<=、> 和 >=）也可以通过使用方法 __cmp__(self,other) 来实现。如果 self<other，那么该方法返回负整数。如果 self==other，那么该方法返回 0。如果 self>other，那么该方法返回正整数。对于两个对象 a 和 b，如果 __lt__ 可用的话，那么 a<b 就调用 a.__lt__(b)。如果不行的话就调用 __cmp__ 方法来决定顺序。

检查点

8.9　什么是运算符重载？

8.10　运算符 +、-、*、/、%、==、!=、<、<=、> 和 >= 对应的特殊方法是什么？

8.6　实例研究：Rational 类

关键点：本节给出如何设计一个有理数类 Rational 来表示和处理有理数。

一个有理数在形式上有分子和分母 a/b，其中 a 为分子而 b 为分母。例如：1/3、3/4 和 10/4 都是有理数。

一个有理数分母不能为零，但是分子可以为零。每一个整数 i 都等价于有理数 i/1。有理数用来做具体计算的时候是有小数部分的，例如，1/3 是 0.333 33…。这个数使用 float 数据类型表示，而且它是不能精确表示的。为了获得精确的结果，我们必须使用有理数。

Python 为我们提供了整数和浮点数的数据类型，但是没有有理数数据类型。本节给出如何设计一个有理数类。

一个有理数可以使用两个数据域来表示：numerator 和 denominator。你可以创建一个指定分子和分母的有理数或分子为 0 而分母为 1 的默认有理数。你可以加、减、乘、除以及比较两个有理数。你也可以将有理数转换为整数、浮点数或字符串。Rational 类的 UML 类图在图 8-9 中给出。

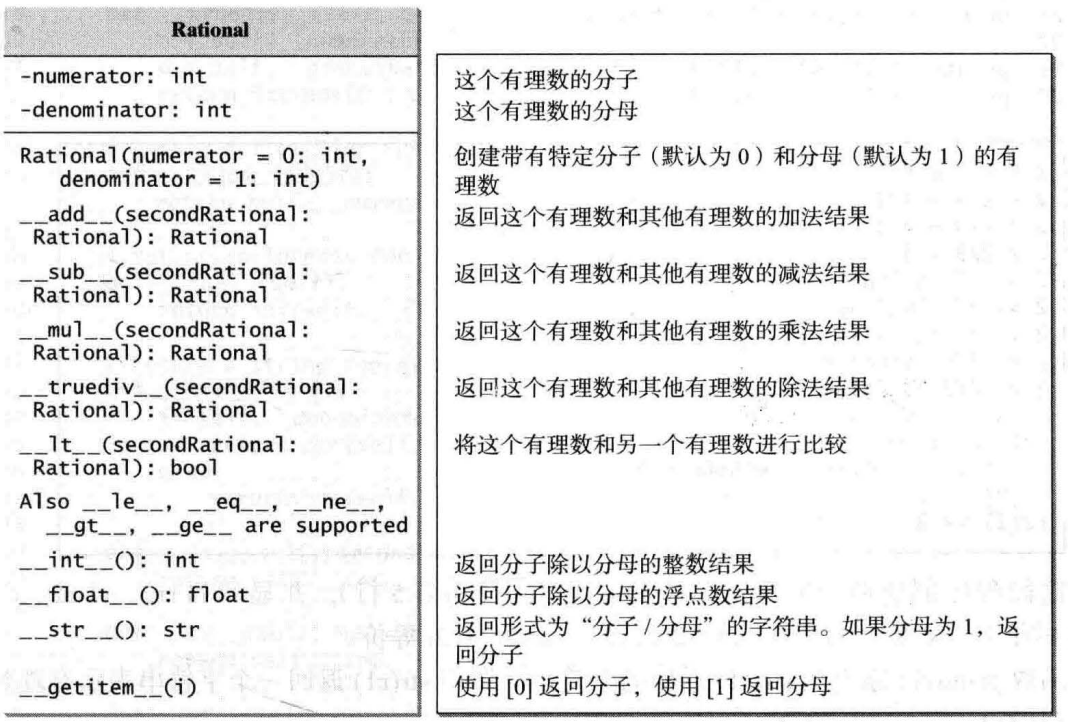


图 8-9　包括属性、初始化程序和方法的 Rational 类的 UML 类图

这里有许多等价的有理数，例如，1/3=2/6=3/9=4/12。为了简单起见，我们使用 1/3 表示所有等价于 1/3 的有理数。由于 1/3 的分子和分母除了 1 之外没有任何公约数，所以 1/3 可以被称为最简形式。

为了将有理数化简至最简形式，你需要找到分子和分母绝对值的最大公约数（GCD），然后将分子分母同除以这个最大公约数。可以使用程序清单 5-8 中建议的计算两个整数 n 和 d 最大公约数的函数。Rational 对象中的分子和分母都被化简为最简形式。

通常，我们首先编写一个测试程序来创建 Rational 对象测试 Rational 类中的函数。程序清单 8-3 是一个测试程序。

程序清单 8-3　TestRationalClass.py

```
1  import Rational
2
```

```
3  # Create and initialize two rational numbers r1 and r2.
4  r1 = Rational.Rational(4, 2)
5  r2 = Rational.Rational(2, 3)
6
7  # Display results
8  print(r1, "+", r2, "=", r1 + r2)
9  print(r1, "-", r2, "=", r1 - r2)
10 print(r1, "*", r2, "=", r1 * r2)
11 print(r1, "/", r2, "=", r1 / r2)
12
13 print(r1, ">", r2, "is", r1 > r2)
14 print(r1, ">=", r2, "is", r1 >= r2)
15 print(r1, "<", r2, "is", r1 < r2)
16 print(r1, "<=", r2, "is", r1 <= r2)
17 print(r1, "==", r2, "is", r1 == r2)
18 print(r1, "!=", r2, "is", r1 != r2)
19
20 print("int(r2) is", int(r2))
21 print("float(r2) is", float(r2))
22
23 print("r2[0] is", r2[0])
24 print("r2[1] is", r2[1])
```

```
2 + 2/3 = 8/3
2 - 2/3 = 4/3
2 * 2/3 = 4/3
2 / 2/3 = 3
2 > 2/3 is True
2 >= 2/3 is True
2 < 2/3 is False
2 <= 2/3 is False
2 == 2/3 is False
2 != 2/3 is True
int(r2) is 0
float(r2) is 0.6666666666666666
r2[0] is 2
r2[1] is 3
```

这段程序创建两个有理数，r1 和 r2（第 4 行和第 5 行），并显示 r1+r2、r1-r2、r1*r2、r1/r2 的结果（第 8 ～ 11 行）。r1+r2 和 r1.__add__(r2) 等价。

函数 print(r1) 输出从 str(r1) 返回的字符串。调用 str(r1) 返回一个字符串表示有理数 r1，它与调用 r1.__str__() 等价。

调用函数 int(r2)（第 20 行）返回一个有理数 r2 的整数，它与调用 r2.__init__() 等价。

调用函数 float(r2)(第 21 行）返回一个有理数 r2 的浮点数，它与调用 r2.__float__() 等价。

调用 r2[0]（第 23 行）与调用 r2.__getitem__(0) 等价，它返回 r2 的分子。

Rational 类的实现在程序清单 8-4 中。

程序清单 8-4　Rational.py

```
1  class Rational:
2      def __init__(self, numerator = 1, denominator = 0):
3          divisor = gcd(numerator, denominator)
4          self.__numerator = (1 if denominator > 0 else -1) \
5              * int(numerator / divisor)
6          self.__denominator = int(abs(denominator) / divisor)
7
8      # Add a rational number to this rational number
9      def __add__(self, secondRational):
10         n = self.__numerator * secondRational[1] + \
```

```
            self.__denominator * secondRational[0]
        d = self.__denominator * secondRational[1]
        return Rational(n, d)

    # Subtract a rational number from this rational number
    def __sub__(self, secondRational):
        n = self.__numerator * secondRational[1] - \
            self.__denominator * secondRational[0]
        d = self.__denominator * secondRational[1]
        return Rational(n, d)

    # Multiply a rational number by this rational number
    def __mul__(self, secondRational):
        n = self.__numerator * secondRational[0]
        d = self.__denominator * secondRational[1]
        return Rational(n, d)

    # Divide a rational number by this rational number
    def __truediv__(self, secondRational):
        n = self.__numerator * secondRational[1]
        d = self.__denominator * secondRational[0]
        return Rational(n, d)

    # Return a float for the rational number
    def __float__(self):
        return self.__numerator / self.__denominator

    # Return an integer for the rational number
    def __int__(self):
        return int(self.__float__())

    # Return a string representation
    def __str__(self):
        if self.__denominator == 1:
            return str(self.__numerator)
        else:
            return str(self.__numerator) + "/", self.__denominator)

    def __lt__(self, secondRational):
        return self.__cmp__(secondRational) < 0

    def __le__(self, secondRational):
        return self.__cmp__(secondRational) <= 0

    def __gt__(self, secondRational):
        return self.__cmp__(secondRational) > 0

    def __ge__(self, secondRational):
        return self.__cmp__(secondRational) >= 0

    # Compare two numbers
    def __cmp__(self, secondRational):
        temp = self.__sub__(secondRational)
        if temp[0] > 0:
            return 1
        elif temp[0] < 0:
            return -1
        else:
            return 0

    # Return numerator and denominator using an index operator
    def __getitem__(self, index):
        if index == 0:
            return self.__numerator
```

```
75          else:
76              return self.__denominator
77
78  def gcd(n, d):
79      n1 = abs(n)
80      n2 = abs(d)
81      gcd = 1
82
83      k = 1
84      while k <= n1 and k <= n2:
85          if n1 % k == 0 and n2 % k == 0:
86              gcd = k
87          k += 1
88
89      return gcd
```

有理数被封装在 Rational 对象中。有理数内部都是以最简形式表示的（第 4 ～ 6 行），而分子决定其符号（第 4 行）。分母永远都是正的（第 6 行）。数据域 numerator 和 denominator 被定义为具有两个前导下划线的私有数据域。

gcd() 并不是 Rational 类中的成员方法，而是定义在 Rational 模块（Rational.py）中的函数（第 78 ～ 89 行）。

两个 Rational 对象之间可以相互进行加、减、乘和除的操作。这些方法都返回一个新 Rational 对象（第 19 ～ 32 行）。注意：secondRational[0] 是指 secondRational 的分子，而 secondRational[1] 是指 secondRational 的分母。下标运算符的使用是由 __getitem__ 方法来支持的（第 72 ～ 76 行），它将会根据下标返回有理数的分子和分母。

__cmp__(secondRational) 方法（第 62 ～ 69 行）将这个有理数和另一个有理数进行比较。它首先用这个有理数减去第二个有理数，并将结果保存在 temp 中（第 63 行）。方法将根据 temp 的分子值是小于、等于还是大于 0 分别返回 −1、0 或 1。

比较方法 __lt__、__le__、__gt__ 和 __ge__ 都是通过使用 __cmp__ 方法来实现的（第 49 ～ 59 行）。注意：方法 __ne__、__eq__ 并不是被显式实现的，如果 __cmp__ 方法是可用的，那么 Python 会隐式实现它们。

你已经使用了 str、int、float 函数将对象转换为 str、int 或 float。方法 __str__()、__int__() 和 __float__() 在 Rational 类中实现（第 35 ～ 47 行），从 Rational 对象返回 str 对象、int 对象和 float 对象。

检查点

8.11　如果将 Rational.py 中第 63 行换成如下代码，程序能否成功运行？

```
temp = self - secondRational
```

8.12　如果将程序中第 43 ～ 47 行的 __str__ 方法替换成如下代码，程序能否成功运行？

```
def __str__(self):
    if self.__denominator == 1:
        return str(self[0])
    else:
        return str(self[0]) + "/" + str(self[1])
```

关键术语

concatenation operator（连接运算符）
index operator（下标运算符）
iterable（可迭代的）
operator overloading（运算符重载）

repetition operator（复制运算符）　　slicing operator（截取运算符）

本章总结

1. 字符串对象是不可变的，不可以改变它的内容。
2. 可以使用 Python 函数 len、min 和 max 来返回字符串的长度、最大元素和最小元素。
3. 可以使用下标运算符 [] 来指向字符串中的一个单独的字符。
4. 可以使用连接运算符 + 来连接两个字符串，使用复制运算符 * 来复制一个字符串多次，使用截取运算符 [：] 来获取子串，而使用运算符 in 和 not in 来判断一个字符是否在一个字符串中。
5. 使用比较运算符（==、!=、<、<=、> 和 >=）比较两个字符串。
6. 使用 for 循环迭代字符串中的所有字符。
7. 可以在字符串对象上使用像 endswith、startswitch、isalpha、islower、isupper、lower、upper、find、count、replace 和 strip 这样的函数。
8. 可以为重载操作定义特殊方法。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

第 8.2 ～ 8.4 节

*8.1 （检测 SSN）编写一个程序，提示用户按照格式 ddd-dd-dddd 输入一个社会安全号码，其中 d 是数字。如果是正确的社会安全号码，那么程序输出“Valid SSN”，否则输出“Invalid SSN”。

**8.2 （检测子串）你可以使用 str 类中的 find 方法检测一个字符串是否是另一个字符串的子串。编写出你自己的函数实现 find。编写一个程序，提示用户输入两个字符串，然后检测第一个字符串是否是第二个字符串的子串。

**8.3 （检测密码）一些网站会给密码强加一些规则。编写函数检测一个字符串是否是一个合法的密码。假设密码规则如下述：

- 密码必须至少有 8 个字符。
- 密码只能包含英文字母和数字。
- 密码应该至少包含两个数字。

编写程序提示用户输入一个密码，如果遵循了规则就显示“valid password”，否则，就显示“invalid password”。

8.4 （特定字符的出现次数）使用下面的函数头，编写一个函数找出字符串中某个特定字符的出现次数。

```
def count(s, ch):
```

str 类有 count 方法。不使用 count 方法实现你的方法。例如：count("welcome", 'e') 返回 2。编写一个测试程序，提示用户输入一个字符串，然后再输入一个字符，显示该字符在字符串中出现的次数。

**8.5 （特定字符串的出现次数）使用下面的函数头编写一个函数，统计一个特定的不重叠的字符串 s2 在另一个字符串 s1 中的出现次数。

```
def count(s1, s2):
```

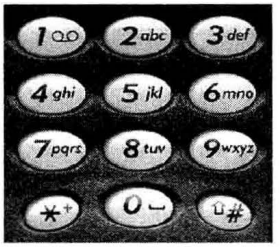
例如：count("system error, syntax error","error") 返回 2。编写一个测试程序，提示用户输入两个字符串，并返回第二个字符串在第一个字符串中的出现次数。

*8.6（统计字符串中的字母个数）使用下面的函数头编写函数统计一个字符串中的字母出现次数。

```
def countLetters(s):
```

编写一个程序提示用户输入一个字符串然后显示字符串中的字母数。

*8.7（手机键盘）手机的国际标准字母 / 数字对应键盘如下所示。



编写函数，假设有一个大写字母，返回对应的数字，如下所示。

```
def getNumber(uppercaseLetter):
```

编写一个测试程序，提示用户输入一个电话号码作为一个字符串。输入数字可能包含字母。程序将字母（大写或小写）转换为数字，保留其他剩余字符不变。下面是程序的示例运行。

```
Enter a string: 1-800-Flowers Enter
1-800-3569377
```

```
Enter a string: 1800flowers Enter
18003569377
```

*8.8（二进制转换为十进制）编写函数将二进制数作为字符串转换为一个十进制整数。使用下面的函数头。

```
def binaryToDecimal(binaryString):
```

例如：二进制字符串 10001 是 17（$1 \times 2^4+0 \times 2^3+0 \times 2^2+0 \times 2+1=17$）。所以，binaryToDecimal（“10001”）返回 17。

编写一个测试程序，提示用户输入一个二进制字符串，然后显示对应的十进制整数值。

**8.9（二进制转换为十六进制）编写函数将二进制数转换为一个十六进制数。函数头如下所示。

```
def binaryToHex(binaryValue):
```

编写一个测试程序，提示用户输入一个二进制数，然后显示对应的十六进制数。

**8.10（十进制转换为二进制）编写一个函数将一个十进制数转换为一个二进制数。使用的函数头如下所示。

```
def decimalToBinary(value):
```

编写一个测试程序，提示用户输入一个十进制整数值，然后显示对应的十六进制数。

第 8.5 节

*8.11（反向字符串）编写一个函数反向一个字符串。这个函数头是：

```
def reverse(s):
```

编写一个测试程序，提示用户输入一个字符串，调用 reverse 函数，然后显示被反向的字符串。

*8.12（生物信息：找出基因）生物学家使用字母 A、C、T 和 G 构成的字符串建模一个基因组。一个基因是基因组的一个子串，它从三元组 ATG 后开始并在三元组 TAG、TAA 或 TGA 之前结束。此外，基因字符串的长度是 3 的倍数，而且基因不包含三元组 ATG、TAG、TAA 和 TGA。编写

程序提示用户输入一个基因组，然后显示基因组里的所有基因。如果在输入序列中没有找到基因，那么程序显示“no gene is found”。下面是示例运行。

```
Enter a genome string: TTATGTTTTAAGGATGGGGCGTTAGTT ↵Enter
TTT
GGGCGT
```

```
Enter a genome string: TGTGTGTATAT ↵Enter
no gene is found
```

*8.13 （最长公共前缀）编写一个方法返回两个字符串最长的公共前缀。例如：“distance”和“disinfection”的最长公共前缀是“dis”。这个方法头是：

```
def prefix(s1, s2)
```

如果两个字符串没有公共的前缀，那么这个方法返回一个空串。

编写一个 main 方法，提示用户输入两个字符串，然后显示它们的公共前缀。

**8.14 （金融：信用卡号合法性）使用字符串输入作为一个信用卡号来改写编程题 6.29。

**8.15 （商业：检测 ISBN-10）一个 ISBN-10（国际标准书号）包括 10 个数：$d_1d_2d_3d_4d_5d_6d_7d_8d_9d_{10}$。最后一个数：$d_{10}$ 是一个校验数，它是使用下面的公式从其他 9 个数计算而来。

$$(d_1\times 1+d_2\times 2+d_3\times 3+d_4\times 4+d_5\times 5+d_6\times 6+d_7\times 7+d_8\times 8+d_9\times 9)\%11$$

依据 ISBN 的规范，如果校验数是 10，那么最后一个数用 X 表示。编写程序提示用户将前 9 个数字作为一个字符串输入，然后显示 10 位的 ISBN（包括前面的零）。你的程序应该将输入作为一个字符串输入。下面是示例运行。

```
Enter the first 9 digits of an ISBN-10 as a string:
  013601267  ↵Enter
The ISBN-10 number is 0136012671
```

```
Enter the first 9 digits of an ISBN-10 as a string:
  013031997  ↵Enter
The ISBN-10 number is 013031997X
```

**8.16 （商业：检测 ISBN-13）ISBN-13 是辨认书籍的新标准。它使用 13 个数：$d_1d_2d_3d_4d_5d_6d_7d_8d_9d_{10}d_{11}d_{12}d_{13}$。最后一位 d_{13} 是一个校验数，它是使用下面的公式从其他几位计算而来的。

$$10-(d_1+3d_2+d_3+3d_4+d_5+3d_6+d_7+3d_8+d_9+3d_{10}+d_{11}+3d_{12})\%10$$

如果校验数是 10，就用 0 替换它。你的程序应该将它作为一个字符串读取输入。下面是示例运行。

```
Enter the first 12 digits of an ISBN-13 as a string:
  978013213080  ↵Enter
The ISBN-13 number is 9780132130806
```

```
Enter the first 12 digits of an ISBN-13 as a string:
  978013213079  ↵Enter
The ISBN-13 number is 9780132130790
```

第 8.6 节

**8.17 （Point 类）设计一个名为 Point 的类来表示一个带 x 坐标和 y 坐标的点。这个类包括：

- 表示坐标的两个私有数据域 x 和 y 以及它们的 get 方法。

- 一个构建指定坐标轴在默认点 (0,0) 的点的构造方法。
- 一个名为 distance 的方法返回从 Point 类型的一个点到另一个 Point 类型的点之间的距离。
- 一个名为 isNearBy(p1) 方法，如果点 p1 紧邻这个点就返回 true。如果两点的距离小于 5 则表示两点距离很近。
- 实现 __str__ 方法返回形式为 (x,y) 的字符串。

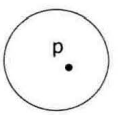
绘制这个类的 UML 图，然后实现这个类。编写一个测试程序，提示用户输入两个点，显示两个点之间的距离，然后表明它们是否离得很近。下面是示例运行。

```
Enter two points x1, y1, x2, y2: 2.1, 2.3, 19.1, 19.2 Enter
The distance between the two points is 23.97
The two points are not near each other
```

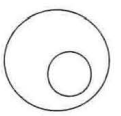
```
Enter two points x1, y1, x2, y2: 2.1, 2.3, 2.3, 4.2 Enter
The distance between the two points is 1.91
The two points are near each other
```

*8.18 （几何：Circle2D 类）定义 Circle2D 类包括：

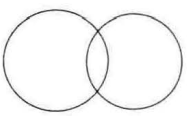
- 两个指定圆中心位置的名为 x 和 y 的私有浮点数据域，以及它们相应的 get/set 方法。
- 私有数据域 radius 以及它们相应的 get/set 方法。
- 一个创建了一个指定 x、y 和 radius 的圆的构造方法。默认值都是 0。
- 方法 getArea() 返回圆的面积。
- 方法 getPerimeter() 返回圆的周长。
- 方法 containsPoint(x,y)，如果指定的点 (x,y) 在圆内，则返回 True（参见图 8-10a）。
- 方法 contain(circle2D)，如果指定的圆在这个圆内，则返回 True（参见图 8-10b）。
- 方法 overlaps(circle2D)，如果指定的圆和这个圆有重叠，则返回 True（参见图 8-10c）。
- 实现 __contains__(another) 方法，如果这个圆包含在另一个圆内，则返回 True。
- 实现 __cmp__、__lt__、__le__、__eq__、__ne__、__gt__、__ge__ 方法，基于圆的半径来比较两个圆的大小。



a）点在圆内



b）圆在另一个圆内



c）一个圆和另一个圆重叠

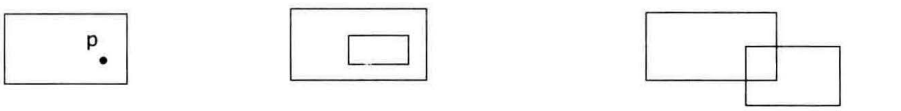
图　8-10

绘制这个类的 UML 类图，然后实现这个类。编写一个测试程序，提示用户输入两个圆的 x、y 坐标以及半径，创建两个 Circle2D 对象 c1 和 c2，显示它们的面积和周长，并显示 c1.containsPoint(c2.getX(),c2.getY())、c1.contains(c2) 以及 c1.overlaps(c2) 的结果。下面是一个示例运行。

```
Enter x1, y1, radius1: 5, 5.5, 10 Enter
Enter x2, y2, radius2: 9, 1.3, 10 Enter
Area for c1 is 314.1592653589793
Perimeter for c1 is 62.83185307179586
Area for c2 is 314.1592653589793
Perimeter for c2 is 62.83185307179586
c1 contains the center of c2? True
c1 contains c2? False
c1 overlaps c2? True
```

*8.19 （几何：Rectangle2D 类）定义 Rectangle2D 类包括：

- 两个名为 x 和 y 的浮点数据域，它们指定矩形的中心位置，以及 x 和 y 的 get/set 方法。（假设矩形的边平行于 x 轴和 y 轴。）
- 数据域 width 和 height，以及它们对应的 get/set 方法。
- 一个创建一个带指定 x、y、width 和 height 的矩形的构造方法，默认值都是 0。
- 方法 getArea() 返回矩形的面积。
- 方法 getPerimeter() 返回矩形的周长。
- 如果指定点 (x,y) 在矩形内，则方法 containsPoint(x,y) 返回 True（参见图 8-11a）。
- 如果指定矩形在这个矩形内，则方法 contain(Rectangle2D) 返回 True（参见图 8-11b）。
- 如果指定的矩形和这个矩形有重叠，则方法 overlaps(Rectangle2D) 返回 True（参见图 8-11c）。
- 如果这个矩形包含在另一个矩形内，则实现 __contains__(another) 方法返回 True。
- 实现 __cmp__、__lt__、__le__、__eq__、__ne__、__gt__、__ge__ 方法，基于矩形的面积来比较两个矩形的大小。



a）点在矩形内　　b）矩形在另一个矩形内　　c）一个矩形和另一个矩形重叠

图 8-11

绘制这个类的 UML 类图，然后实现这个类。编写一个测试程序，提示用户输入带 x、y 坐标，宽度和高度的两个矩形，创建两个 Rectangle2D 对象 r1 和 r2，显示它们的面积和周长，并显示 r1.containsPoint(r2.getX(),r2.getY())、r1.contains(r2) 以及 r1.overlaps(r2) 的结果。下面是一个示例运行。

```
Enter x1, y1, width1, height1: 9, 1.3, 10, 35.3 Enter
Enter x2, y2, width2, height2: 1.3, 4.3, 4, 5.3 Enter
Area for r1 is 353.0
Perimeter for r1 is 90.6
Area for r2 is 21.2
Perimeter for r2 is 18.6
r1 contains the center of r2? False
r1 contains r2? False
r1 overlaps r2? False
```

8.20 （使用 Rational 类）编写一个程序使用 Rational 类计算下面的求和数列。

$$\frac{1}{2}+\frac{1}{3}+\frac{3}{4}+\ldots+\frac{8}{9}+\frac{9}{10}$$

8.21 （数学：Complex 类）Python 有一个用于完成复数的算术运算的 complex 类。在这道题中，你将会设计和实现你自己的 Complex 类。注意：Python 中的 complex 是以小写命名的，而我们自定制的 Complex 类以大写 C 来命名。

复数是一个形式为 a+bi 的数，其中 a 和 b 都是实数，而 i 是 $\sqrt{-1}$。数字 a 和 b 分别被称为复数的实部和虚部。可以使用下面的公式来实现复数的加、减、乘、除。

$$(a+bi)+(c+di)=(a+c)+(b+d)i$$
$$a+bi-(c+di)=(a-c)+(b-d)i$$
$$(a+bi)*(c+di)=(ac-bd)+(bc+ad)i$$
$$(a+bi)/(c+di)=(ac+bd)/(c^2+d^2)+(bc-ad)i/(c^2+d^2)$$

也可以使用下面的公式求复数的绝对值。

$$|a+bi| = \sqrt{a^2+b^2}$$

（复数可以被解释为平面上的一个点，(a,b) 的值可作为点的坐标。复数的绝对值对应从原点到该点的距离，如图 8-12 所示。）

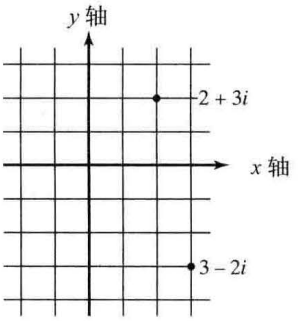


图 8-12　点 (2,3) 可以被写作一个复数 (2+3i) 而点 (3,−2) 可以被写作 (3−2i)

设计一个名为 Complex 的类，表示复数和实现复数运算的方法 __add__、__sub__、__mul__、__truediv__ 和 __abs__，并重载方法 __str__ 返回一个表示复数的字符串。方法 __str__ 将 (a+bi) 作为字符串返回。如果 b 是 0，则它仅返回 a。

提供一个构造方法 Complex(a,b) 来创建一个复数 a+bi，a 和 b 的默认值为 0。同时提供 getRealPart() 和 getImaginaryPart() 方法分别返回复数的实部和虚部。

编写一个测试程序提示用户输入两个复数，然后显示它们加、减、乘、除的结果。下面是一个示例运行：

```
Enter the first complex number: 3.5, 6.5 ↵Enter
Enter the second complex number: -3.5, 1 ↵Enter
(3.5 + 6.5i) + (-3.5 + 1i) = (0.0 + 7.5i)
(3.5 + 6.5i) - (-3.5 + 1i) = (7.0 + 5.5i)
(3.5 + 6.5i) * (-3.5 + 1i) = (-18.75 - 19.25i)
(3.5 + 6.5i) / (-3.5 + 1i) = (-0.43396226415 - 1.98113207547i)
|(3.5 + 6.5i)| = 4.47213595499958
```

第 9 章
Introduction to Programming Using Python

使用 Tkinter 进行 GUI 程序设计

学习目标

- 使用 Tkinter 创建一个简单的 GUI 应用程序（第 9.2 节）。
- 使用绑定到小构件命令选项的回调函数来处理事件（第 9.3 节）。
- 使用标签、输入域、按钮、复选按钮、单选按钮、消息和文本创建图形用户界面（第 9.4 节）。
- 在画布上绘制线段、矩形、椭圆、多边形和圆弧并显示文本字符串（第 9.5 节）。
- 使用几何管理器在容器中布局小构件（第 9.6 节）。
- 使用网格管理器在网格中布局小构件（第 9.6.1 节）。
- 使用包管理器将小构件一个挨一个地放置或一个叠加一个地放置（第 9.6.2 节）。
- 使用位置管理器将小构件放置在绝对位置上（第 9.6.3 节）。
- 使用容器来划分小构件以获得期望的布局（第 9.7 节）。
- 在小构件中使用图片（第 9.8 节）。
- 创建包含菜单的应用程序（第 9.9 节）。
- 创建包含弹出菜单的应用程序（第 9.10 节）。
- 绑定一个小构件的鼠标和按键事件以回调一个函数来处理事件（第 9.11 节）。
- 开发动画（第 9.12 节）。
- 使用滚动条扫描一个文本小构件的内容（第 9.13 节）。
- 使用标准对话框显示消息并接收使用者的输入（第 9.14 节）。

9.1 引言

关键点：Tkinter 能开发 GUI 程序，而且它也是一个极好的学习面向对象程序设计的教学工具。

在 Python 中有许多 GUI 模块可以用来开发 GUI 程序。我们已经使用过 Turtle 模块绘制几何图形。Turtle 是非常容易使用的，并且它是一个为初学者介绍程序设计基础的有效的教学工具。然而，我们不能使用 Turtle 来创建图形用户界面。本章介绍的 Tkinter 能开发 GUI 项目，它不仅是创建 GUI 项目的有用工具，并且也是一个学习面向对象程序设计的有价值的工具。

注意：Tkinter（发音为 T-K-Inter）是“Tk interface”的缩写。Tk 是一个可以被许多基于 Windows、Mac、UNIX 的程序设计语言用来开发 GUI 程序的 GUI 库。Tkinter 为 Python 开发者提供一个使用 Tk GUI 库的接口，而且它事实上也是用 Python 开发 GUI 程序的标准。

9.2 开始使用 Tkinter

关键点：Tkinter 模块包含创建各种 GUI 的类。Tk 类创建一个放置 GUI 小构件的窗口（即可视化组件）。

程序清单 9-1 使用了一个简单的例子介绍 Tkinter。

程序清单 9-1 SimpleGUI.py

```
1  from tkinter import *  # Import all definitions from tkinter
2
3  window = Tk()  # Create a window
4  label = Label(window, text = "Welcome to Python")  # Create a label
5  button = Button(window, text = "Click Me")  # Create a button
6  label.pack()  # Place the label in the window
7  button.pack()  # Place the button in the window
8
9  window.mainloop()  # Create an event loop
```

当运行这个程序时，Tkinter 的窗口中就会出现一个标签和一个按钮，如图 9-1 所示。

图 9-1　程序清单 9-1 中创建的标签和按钮

任何时候在 Tkinter 中创建一个基于 GUI 程序时，都需要导入 Tkinter 模块（第 1 行）并且要使用 Tk 类创建一个窗口（第 3 行）。回顾第 1 行里的星号（*），它表示将 Tkinter 模块中的所有类、函数和常量的定义导入到程序中。Tk() 创建了一个窗口实例。Label 和 Button 是创建标签和按钮的 Python Tkinter 小构件类。小构件类的第一个参数总是父容器（即小构件将要放置的容器）。语句（第 4 行））

```
label = Label(window, text = "Welcome to Python")
```

创建一个带文本“Welcome to Python”的标签，将它包含在窗口内。

语句（第 6 行）

```
label.pack()
```

使用一个包管理器将 label 放在容器中。在这个例子中，包管理器将小构件一行一行地放在窗口中。更多关于包管理器的内容将在第 9.6.2 节介绍。从现在开始，可以在不知道包管理器的全部细节的情况下使用它。

Tkinter GUI 程序设计是事件驱动的。在显示用户界面之后，程序等待用户进行交互，例如：单击鼠标和敲击键盘。这是在下面语句中指定的（第 9 行）

```
window.mainloop()
```

这条语句创建了一个事件循环。这个事件循环持续处理事件直到关闭主窗口，如图 9-2 所示。

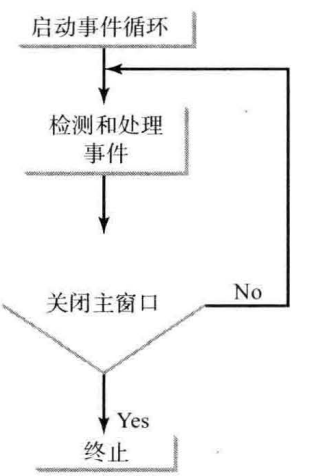


图 9-2　Tkinter GUI 程序在连续循环中侦听和处理事件

检查点

9.1　Turtle 和 Tkinter 适合做什么？

9.2　如何创建窗口？

9.3　window.mainloop() 的作用是什么？

9.3　处理事件

关键点：一个 Tkinter 小构件可以与一个函数绑定，当事件发生时被调用。

Button 小构件类是一个阐明事件驱动程序设计基础的很好方式，所以我们在接下来的例

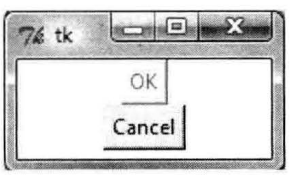
子中会使用它。

当用户单击一个按钮时，程序就应该处理这个事件。可以通过定义一个处理函数并且将这个函数与这个按钮绑定来实现该功能，如程序清单 9-2 所示。

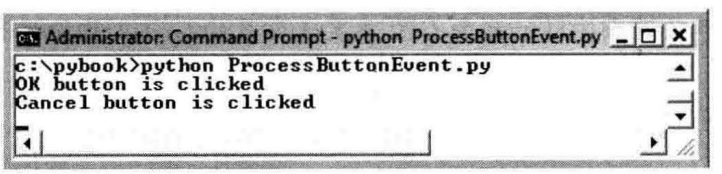
程序清单 9-2 ProcessButtonEvent.py

```
1  from tkinter import *  # Import all definitions from tkinter
2
3  def processOK():
4      print("OK button is clicked")
5
6  def processCancel():
7      print("Cancel button is clicked")
8
9  window = Tk() # Create a window
10 btOK = Button(window, text = "OK", fg = "red", command = processOK)
11 btCancel = Button(window, text = "Cancel", bg = "yellow",
12                    command = processCancel)
13 btOK.pack() # Place the OK button in the window
14 btCancel.pack() # Place the Cancel button in the window
15
16 window.mainloop() # Create an event loop
```

当运行这个程序时，就会出现两个按钮，如图 9-3a 所示。可以观察到事件正在被处理，可以在图 9-3b 中的命令窗口中看到它们的相关信息。



a）程序清单 9-2 在窗口中显示两个按钮



b）命令窗口中观察到事件正在被处理

图　9-3

这个程序定义了函数 processOK 和 processCancel（第 3 ～ 7 行）。当创建这些按钮时，这些函数被绑定到按钮。这些函数被称作*回调函数*，或者被称为*处理器*。下面的语句（第 10 行）

```
btOK = Button(window, text = "OK", fg = "red", command = processOK)
```

将“OK”按钮绑定到 processOK 函数，当按钮被单击时，这个函数将被调用。fg 选项指定按钮的前景色，而 bg 选项指定按钮的背景色。默认情况下，对所有的小构件而言，fg 是黑色的，而 bg 是灰色的。

也可以通过将所有函数放在一个类中来编写这个程序，如程序清单 9-3 所示。

程序清单 9-3 ProcessButtonEventAlternativeCode.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  class ProcessButtonEvent:
4      def __init__(self):
5          window = Tk() # Create a window
6          btOK = Button(window, text = "OK", fg = "red",
7                        command = self.processOK )
8          btCancel = Button(window, text = "Cancel", bg = "yellow",
9                            command = self.processCancel)
10         btOK.pack() # Place the OK button in the window
```

```
11          btCancel.pack() # Place the Cancel button in the window
12
13          window.mainloop() # Create an event loop
14
15      def processOK(self):
16          print("OK button is clicked")
17
18      def processCancel(self):
19          print("Cancel button is clicked")
20
21  ProcessButtonEvent() # Create an object to invoke __init__ method
```

程序在__init__方法中定义一个创建GUI的类（第4行）。现在，函数processOK和processCancel都是类中的实例方法，所以它们被self.processOK(第7行）和self.processCancel（第9行）调用。

定义一个类来创建GUI和处理GUI事件有两个优点。首先，可以将来重复使用这个类。其次，将所有函数定义为方法可以让它们访问类中的实例数据域。

检查点

9.4　当从一个小构件类创建一个小构件对象时，第一个参数应该是什么？

9.5　小构件的command选项的作用是什么？

9.4　小构件类

关键点：Tkinter的GUI类定义常见的GUI小构件，例如按钮、标签、单选按钮、复选按钮、输入域、画布和其他小构件。

表9-1描述了Tkinter提供的核心小构件类。

表9-1　Tkinter小构件类

小构件类	描述
Button	一个用来执行一条命令的简单按钮
Canvas	结构化的图形，用于绘制图形、创建图形编辑器以及实现自定制的小构件类
Checkbutton	单击复选按钮在值之间切换
Entry	一个文本输入域，也被称为文本域或文本框
Frame	包含其他小构件的一个容器小构件
Label	显示文本或图像
Menu	用来实现下拉和弹出菜单的菜单栏
Menubutton	用来实现下拉菜单的菜单按钮
Message	显示文本，类似于标签小构件，但能自动将文本放在给定的宽度或宽高比内
Radiobutton	单击单选按钮设置变量为那个值，同时清除所有和同一个变量相关联的其他单选按钮
Text	格式化的文本显示，允许用不同的风格和属性显示和编辑文本，也支持内嵌的图片和窗口

从这些类中建立小构件对象有许多选项。第一个参数总是父容器。当构建一个小构件对象时，可以指定前景色、背景色、字体和光标风格。

为了指定某种颜色，可以使用色彩名称（例如：红、黄、绿、蓝、白、黑、紫）或者通过使用字符串#RRGGBB显式指定红、绿和蓝（RGB）的颜色比例，这里的RR、GG和BB分别是红、绿、蓝值的十六进制表示。

可以指定字符串的字体，包括字体名、大小和风格。下面是一些例子。

```
Times 10 bold
Helvetica 10 bold italic
```

```
CourierNew 20 bold italic
Courier 20 bold italic overstrike underline
```

默认情况下，标签或按钮上的文本是居中的。使用命名常量 LEFT、CENTER 或者 RIGHT 的 justify 选项可以改变它的基准线。（注意：正如第 2.6 节讨论的，命名常量全部是大写的。）也可以通过插入新行字符 \n 来分隔文本行，从而多行显示文本。

可以通过为 cursor 选项指定 arrow（默认值）、circle、cross、plus 或其他图形的字符串值来指定鼠标光标的特定风格。

当构建一个小构件时，可以在构造方法中指定它的属性，例如 fg、bg、font、cursor、text 和 command。稍后，在程序中可以使用下面的语法改变小构件的属性。

```
widgetName["propertyName"] = newPropertyValue
```

例如：下面的代码创建了一个按钮，它的 text 属性改为 Hide，bg 属性改为 red，而 fg 属性改为 #AB84F9。#AB84F9 是一种以 RRGGBB 方式指定的颜色。

```
btShowOrHide = Button(window, text = "Show", bg = "white")
btShowOrHide["text"] = "Hide"
btShowOrHide["bg"] = "red"
btShowOrHide["fg"] = "#AB84F9" # Change fg color to #AB84F9
btShowOrHide["cursor"] = "plus" # Change mouse cursor to plus
btShowOrHide["justify"] = LEFT # Set justify to LEFT
```

每一个类都有相当多的方法。关于这些类的完整信息超出了本书的范围。www.pythonware.com/library/tkinter 是关于 Tkinter 的一个很好的参考资源。本章只是提供了一些如何使用这些小构件的例子。

程序清单 9-4 是一个使用 Frame、Button、Checkbutton、Radiobutton、Label、Entry（也称为文本域）、Message 和 Text（也称为文本区）的程序实例。

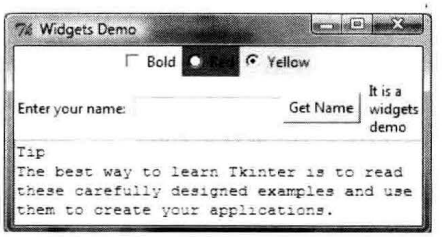
程序清单 9-4 WidgetsDemo.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  class WidgetsDemo:
4      def __init__(self):
5          window = Tk() # Create a window
6          window.title("Widgets Demo") # Set a title
7
8          # Add a check button, and a radio button to frame1
9          frame1 = Frame(window) # Create and add a frame to window
10         frame1.pack()
11         self.v1 = IntVar()
12         cbtBold = Checkbutton(frame1, text = "Bold",
13             variable = self.v1, command = self.processCheckbutton)
14         self.v2 = IntVar()
15         rbRed = Radiobutton(frame1, text = "Red", bg = "red",
16                 variable = self.v2, value = 1,
17                 command = self.processRadiobutton)
18         rbYellow = Radiobutton(frame1, text = "Yellow",
19                 bg = "yellow", variable = self.v2, value = 2,
20                 command = self.processRadiobutton)
21         cbtBold.grid(row = 1, column = 1)
22         rbRed.grid(row = 1, column = 2)
23         rbYellow.grid(row = 1, column = 3)
24
25         # Add a label, an entry, a button, and a message to frame1
26         frame2 = Frame(window) # Create and add a frame to window
27         frame2.pack()
```

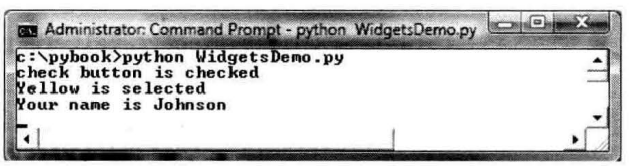
```
28          label = Label(frame2, text = "Enter your name: ")
29          self.name = StringVar()
30          entryName = Entry(frame2, textvariable = self.name)
31          btGetName = Button(frame2, text = "Get Name",
32              command = self.processButton)
33          message = Message(frame2, text = "It is a widgets demo")
34          label.grid(row = 1, column = 1)
35          entryName.grid(row = 1, column = 2)
36          btGetName.grid(row = 1, column = 3)
37          message.grid(row = 1, column = 4)
38
39          # Add text
40          text = Text(window) # Create and add text to the window
41          text.pack()
42          text.insert(END,
43              "Tip\nThe best way to learn Tkinter is to read ")
44          text.insert(END,
45              "these carefully designed examples and use them ")
46          text.insert(END, "to create your applications.")
47
48          window.mainloop() # Create an event loop
49
50      def processCheckbutton(self):
51          print("check button is "
52              + ("checked " if self.v1.get() == 1 else "unchecked"))
53
54      def processRadiobutton(self):
55          print(("Red" if self.v2.get() == 1 else "Yellow")
56              + " is selected " )
57
58      def processButton(self):
59          print("Your name is " + self.name.get())
60
61  WidgetsDemo() # Create GUI
```

当运行这个程序时，显示这些小构件，如图 9-4a 所示。当单击“Bold”按钮时，选择“Yellow”单选按钮，并且输入“Johnson”，就可以观察到事件正在被处理，并且在命令行窗口中看到它们的相关信息，如图 9-4b 所示。

这个程序创建了窗口（第 5 行），并调用 title 方法设置标题（第 6 行）。使用 Frame 类创建一个名为 framel 的框架，并且将窗口作为这个框架的父容器（第 9 行）。这个框架被用作第 12 行创建的复选按钮以及第 15 行和第 18 行创建的两个单选按钮的父容器。



a）在用户界面中显示小构件



b）观察到事件正在被处理

图　9-4

可以使用输入域（文本域）输入值。这个值必须是 IntVar 对象、DoubleVar 对象或 StringVar 对象以分别表示整数、浮点数或字符串。IntVar、DoubleVar 和 StringVar 都在 Tkinter 模块中定义。

程序创建一个复选按钮，并将它与变量 v1 相关联。v1 是一个 IntVar 的实例（第 11

行）。如果选中复选按钮，那么 v1 的值被设置为 1，否则 v1 的值为 0。当单击复选按钮时，Python 调用 processCheckbutton 方法（第 13 行）。

然后，程序创建一个单选按钮，并将它与一个 IntVar 变量 v2 相关联。如果选中“Red”单选按钮，那么 v2 被设置为 1，而如果选中“Yellow”单选按钮，那么 v2 被设置为 2。也可以在建立单选按钮时定义任意一个整数值或者字符串值。当单击这两个按钮中的任意一个时，就会调用方法 processRadiobutton。

网格几何管理器用来将复选按钮、单选按钮放在 frame1 中。这三个小构件被分别放在同一行的第 1、第 2 和第 3 列（第 21 ～ 23 行）。

程序创建了另一个框架 frame2（第 26 行）来放置一个标签、一个输入域、一个按钮和一个消息小构件。就像 frame1，frame2 也放在窗口内。

创建一个输入域，并将它与 StringVar 类型的变量 name 相关联，以将值存储在输入域中（第 29 行）。当单击按钮 Get Name 时，方法 processButton 显示输入域中的值（第 59 行）。除了能自动调整将单词进行多行显示外，Message 小构件很像一个标签。

网格几何管理器用来将小构件放在 frame2 中。这些小构件被分别放在同一行的第 1、第 2、第 3 和第 4 列（第 34 ～ 37 行）。

程序创建一个 Text 小构件（第 40 行）来显示和编辑文本。它被放置在窗口中（第 41 行）。可以使用 insert 方法将文本插入到小构件中。选项 END 表明文本被插入到当前内容的结尾。

程序清单 9-5 是一个改变标签颜色、字体和内容的程序，如图 9-5 所示。

程序清单 9-5 ChangeLabelDemo.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  class ChangeLabelDemo:
4      def __init__(self):
5          window = Tk() # Create a window
6          window.title("Change Label Demo") # Set a title
7
8          # Add a label to frame1
9          frame1 = Frame(window) # Create and add a frame to window
10         frame1.pack()
11         self.lbl = Label(frame1, text = "Programming is fun")
12         self.lbl.pack()
13
14         # Add a label, entry, button, two radio buttons to frame2
15         frame2 = Frame(window) # Create and add a frame to window
16         frame2.pack()
17         label = Label(frame2, text = "Enter text: ")
18         self.msg = StringVar()
19         entry = Entry(frame2, textvariable = self.msg)
20         btChangeText = Button(frame2, text = "Change Text",
21             command = self.processButton)
22         self.v1 = StringVar()
23         rbRed = Radiobutton(frame2, text = "Red", bg = "red",
24             variable = self.v1, value = 'R',
25             command = self.processRadiobutton)
26         rbYellow = Radiobutton(frame2, text = "Yellow",
27             bg = "yellow", variable = self.v1, value = 'Y',
28             command = self.processRadiobutton)
29
30         label.grid(row = 1, column = 1)
31         entry.grid(row = 1, column = 2)
```

```
32          btChangeText.grid(row = 1, column = 3)
33          rbRed.grid(row = 1, column = 4)
34          rbYellow.grid(row = 1, column = 5)
35
36          window.mainloop() # Create an event loop
37
38      def processRadiobutton(self):
39          if self.v1.get() == 'R':
40              self.lbl["fg"] = "red"
41          elif self.v1.get() == 'Y':
42              self.lbl["fg"] = "yellow"
43
44      def processButton(self):
45          self.lbl["text"] = self.msg.get() # New text for the label
46
47  ChangeLabelDemo() # Create GUI
```

当选择单选按钮时，标签的前景色会发生变化。如果在文本域中输入新文本并且单击“Change Text”按钮，新文本就会出现在标签中。

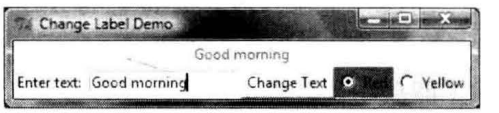


图 9-5　程序动态地改变标签的 text 和 fg 属性

程序创建了一个窗口（第 5 行）并且调用它的 title 方法设置标题（第 6 行）。Frame 类用来创建一个名为 frame1 的框架，窗口是它的父容器（第 9 行）。这个框架作为在第 11 行创建的标签的父容器。因为标签是类的一个数据域，所以它可以在回调函数中被引用。

程序建立了另一个框架 frame2（第 15 行）来放置标签、输入域、按钮和两个单选按钮。如同 frame1，frame2 也放在窗口中。

创建了一个输入域，并且将它与 StringVar 类型的变量 msg 相关联以存储输入域中的值（第 19 行）。当单击“Change Text”按钮时，processButton 方法使用输入域中的文本为 frame1 中的标签设置一个新文本输入域（第 45 行）。

创建两个单选按钮并且将它与 SringVar 变量 v2 相关联。如果选择“Red”单选按钮，那么 v2 就被设置为 R，如果选择“Yellow”单选按钮，那么 v2 就被设置为 Y。当用户单击两个按钮之一时，Python 就会调用 processRadiobutton 方法来改变 frame1 中标签的前景色（第 38 ～ 42 行）。

检查点

9.6　如何创建一个文本为“welcome”、前景色为白色以及背景色为黑色的标签？

9.7　如何创建一个文本为“OK”、前景色为白色、背景色为红色、并带有回调函数 processOK 的按钮？

9.8　如何创建一个文本为“apple”、前景色为白色、背景色为红色、与变量 v1 相关联，并带有回调函数 processApple 的复选按钮？

9.9　如何创建一个文本为“senior”、前景色为白色、背景色为红色、与变量 v1 相关联，并带有回调函数 processSenior 的单选按钮？

9.10　如何创建一个前景色为白色、背景色为红色、与变量 v1 相关联的输入域？

9.11　如何创建文本为“programming is fun”、前景色为白色、背景色为红色的消息？

9.12　LEFT、CENTER 和 RIGHT 是定义在 Tkinter 模块中的命名常量。使用 print 语句显示由 LEFT、CENTER 和 RIGHT 定义的值。

9.5　画布

关键点：可以使用 Canvas 小构件来显示图形。

可以使用方法 create_rectangle、create_oval、create_arc、create_polygon 或 create_line 分别在画布上绘制出矩形、椭圆、圆弧、多边形或者线段。

程序清单 9-6 显示如何使用 Canvas 小构件。程序显示了一个矩形、一个椭圆、一段圆弧、一个多边形、一条线段和一个文本字符串。这些对象都由按钮控制，如图 9-6 所示。

程序清单 9-6　CanvasDemo.py

```
from tkinter import * # Import all definitions from tkinter

class CanvasDemo:
    def __init__(self):
        window = Tk() # Create a window
        window.title("Canvas Demo") # Set title

        # Place canvas in the window
        self.canvas = Canvas(window, width = 200, height = 100,
            bg = "white")
        self.canvas.pack()

        # Place buttons in frame
        frame = Frame(window)
        frame.pack()
        btRectangle = Button(frame, text = "Rectangle",
            command = self.displayRect)
        btOval = Button(frame, text = "Oval",
            command = self.displayOval)
        btArc = Button(frame, text = "Arc",
            command = self.displayArc)
        btPolygon = Button(frame, text = "Polygon",
            command = self.displayPolygon)
        btLine = Button(frame, text = "Line",
            command = self.displayLine)
        btString = Button(frame, text = "String",
            command = self.displayString)
        btClear = Button(frame, text = "Clear",
            command = self.clearCanvas)
        btRectangle.grid(row = 1, column = 1)
        btOval.grid(row = 1, column = 2)
        btArc.grid(row = 1, column = 3)
        btPolygon.grid(row = 1, column = 4)
        btLine.grid(row = 1, column = 5)
        btString.grid(row = 1, column = 6)
        btClear.grid(row = 1, column = 7)

        window.mainloop() # Create an event loop

    # Display a rectangle
    def displayRect(self):
        self.canvas.create_rectangle(10, 10, 190, 90, tags = "rect")

    # Display an oval
    def displayOval(self):
        self.canvas.create_oval(10, 10, 190, 90, fill = "red",
            tags = "oval")

    # Display an arc
    def displayArc(self):
        self.canvas.create_arc(10, 10, 190, 90, start = 0,
```

```
52              extent = 90, width = 8, fill = "red", tags = "arc")
53
54      # Display a polygon
55      def displayPolygon(self):
56          self.canvas.create_polygon(10, 10, 190, 90, 30, 50,
57              tags = "polygon")
58
59      # Display a line
60      def displayLine(self):
61          self.canvas.create_line(10, 10, 190, 90, fill = "red",
62              tags = "line")
63          self.canvas.create_line(10, 90, 190, 10, width = 9,
64              arrow = "last", activefill = "blue", tags = "line")
65
66      # Display a string
67      def displayString(self):
68          self.canvas.create_text(60, 40, text = "Hi, I am a string",
69             font = "Times 10 bold underline", tags = "string")
70
71      # Clear drawings
72      def clearCanvas(self):
73          self.canvas.delete("rect", "oval", "arc", "polygon",
74              "line", "string")
75
76  CanvasDemo() # Create GUI
```

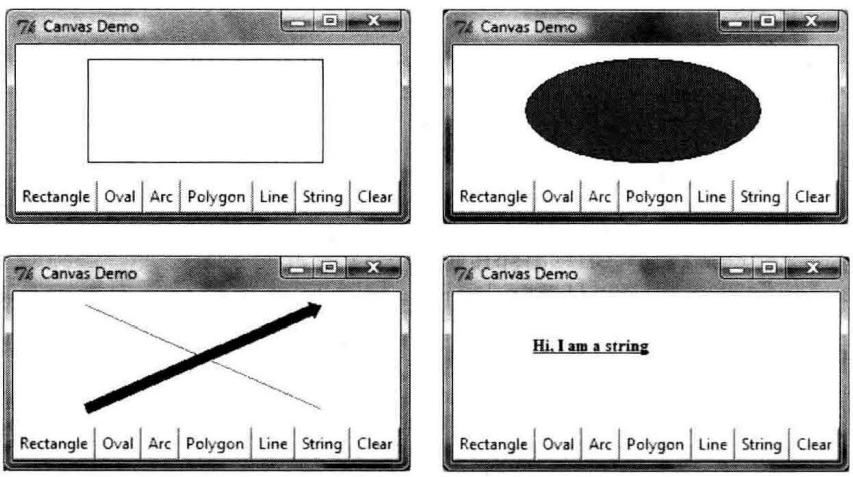


图 9-6　在画布上绘制几何图形和字符串

程序创建一个窗口（第 5 行）并且设置它的标题（第 6 行）。在窗口中创建一个 Canvas 小构件，它的宽度为 200 像素、高度为 100 像素且背景色为 white（第 9 ～ 10 行）。

创建七个分别被标记为“Rectangular”、“Oval”、“Arc”、“Polygon”、“Line”、“String”和“Clear”的按钮（第 16 ～ 29 行）。网格管理器将按钮放在框架中的同一行（第 30 ～ 36 行）。

为了绘制图形，需要说明小构件在哪里绘制。每一个小构件有自己的以左上角为原点（0,0）的坐标系。x 坐标向右增加，而 y 坐标向下增加。注意：Tkinter 的坐标系与传统的坐标系不同，如图 9-7 所示。

方法 create_rectangle、create_oval、create_arc、create_polygon 和 creare_line（第 42、46、51、56 和 61 行）分别用来绘制矩形、椭圆、圆弧、多边形以及线段，如图 9-8 所示。

方法 create_text 用来绘制文本字符串（第 68 行）。注意：create_text(x ,y，text）是指显示文本的水平方向和垂直方向的中心位于 (x, y) 处。

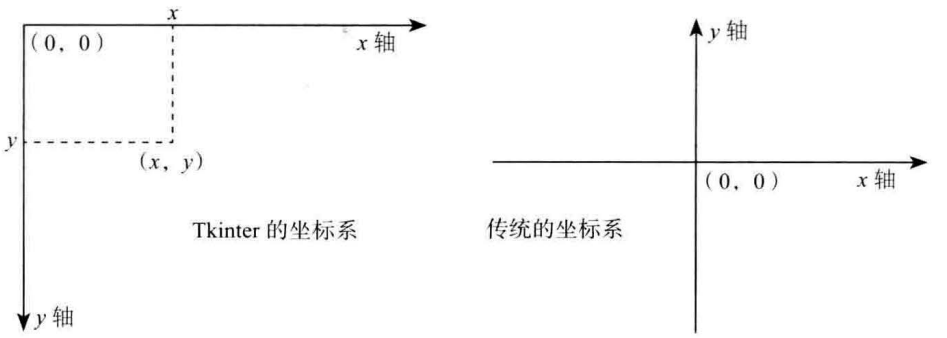


图 9-7　Tkinter 坐标系是以像素为单位的，原点 (0,0) 在左上角

所有的绘图方法都使用 tags 参数标识所绘图形。这些 tags 用在 delete 方法中以从画布上清除图形（第 73 ～ 74 行）。

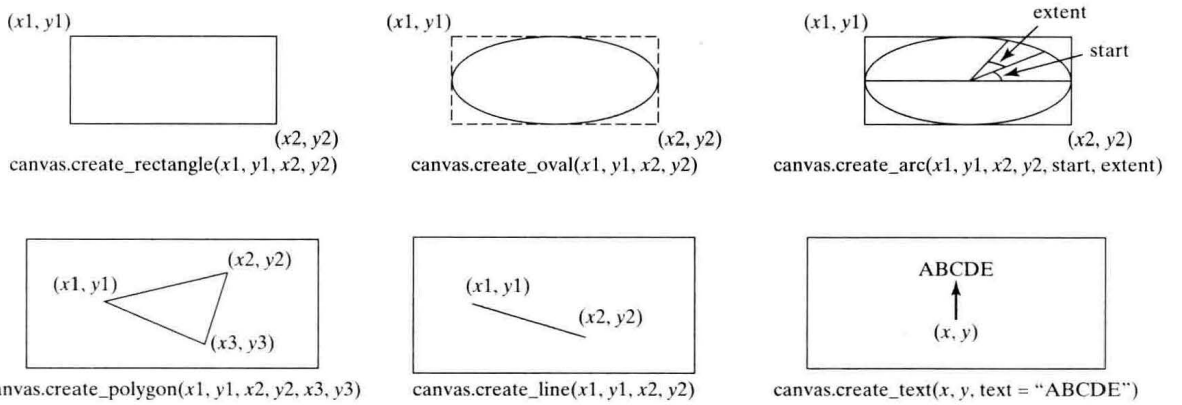


图 9-8　Canvas 类包含绘制图形的方法

参数 width 用来指定以像素为单位的笔的粗细以绘制图形（第 52 和 63 行）。

参数 arrow 可以和 create_line 一起使用来绘制一条有箭头的线段（第 64 行）。当参数值为 first、end 或 both 时，箭头分别出现在线段的开始、结尾和两端。

当鼠标经过图形时，参数 activefill 会改变图形颜色（第 64 行）。

检查点

9.13　编写代码绘制从点 (34,50) 到 (50, 90) 的一条线段。

9.14　编写代码绘制以点 (70,70) 为中心、宽度为 100 和高度为 100 的矩形，填充矩形为红色。

9.15　编写代码绘制以点 (70,70) 为中心、宽度为 200 和高度为 100 的椭圆，填充椭圆为红色。

9.16　编写代码绘制以点 (10,10) 为左上角、点 (80,80) 为外接矩形画一个圆弧，它的起始角度为 30 度，张开角度为 45 度。

9.17　编写代码绘制一个顶点在 (10,10)、(15,30)、(140,10) 和 (10,100) 的多边形，填充多边形为红色。

9.18　如何使用大尺寸笔绘制图形？

9.19　如何绘制一个带箭头的线段？

9.20　如何绘制一个图形当鼠标经过它时改变颜色？

9.6　几何管理器

关键点：Tkinter 使用几何管理器将小构件放入容器中。

Tkinter 支持三种几何管理器：网格管理器、包管理器和位置管理器。我们已经使用过网格管理器和包管理器，本节将描述这些管理器，并介绍一些附加的特性。

提示：由于每个管理器都有自己放置小构件的风格，所以最好不要对同一容器中的小构件们使用多个管理器。可以使用框架作为子容器以获取期望的布局。

9.6.1 网格管理器

网格管理器将小构件放在容器中一个不可见网格的每个单元内。可以将小构件放在某个特定的行和列内，也可以使用 rowspan 和 columnspan 参数将小构件放在多行和多列中。程序清单 9-7 使用网格管理器对一组小构件进行布局，如图 9-9 所示。

程序清单 9-7 GridManagerDemo.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  class GridManagerDemo:
4      window = Tk() # Create a window
5      window.title("Grid Manager Demo") # Set title
6
7      message = Message(window, text =
8          "This Message widget occupies three rows and two columns")
9      message.grid(row = 1, column = 1, rowspan = 3, columnspan = 2)
10     Label(window, text = "First Name:").grid(row = 1, column = 3)
11     Entry(window).grid(row = 1, column = 4, padx = 5, pady = 5)
12     Label(window, text = "Last Name:").grid(row = 2, column = 3)
13     Entry(window).grid(row = 2, column = 4)
14     Button(window, text = "Get Name").grid(row = 3,
15         padx = 5, pady = 5, column = 4, sticky = E)
16
17     window.mainloop() # Create an event loop
18
19 GridManagerDemo() # Create GUI
```

小构件 Message 被放置在第 1 行第 1 列，将它扩展为 3 行 2 列（第 9 行）。按钮“Get Name”使用 sticky = E 选项（第 15 行）设置在单元格的东边，这样它和 Entry 小构件右对齐在同一列。选项 sticky 定义如果最终的单元格比小构件本身大时如何扩展小构件。选项 sticky 可以是命名常量 S、N、E 和 W，或者 NW、NE、SW 和 SE 的任意结合。

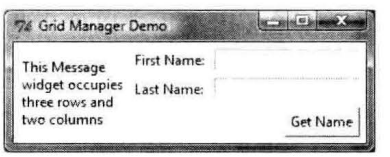


图 9-9　使用网格管理器放置这些小构件

选项 padx 和 pady 填充单元格中水平方向和垂直方向上的可选空间（第 11 和 15 行）。你也可以使用 ipadx 和 ipady 选项填充小构件边界里水平方向和垂直方向上的可选空间。

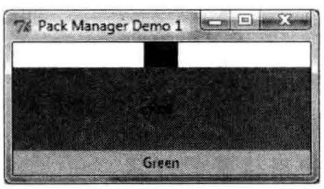
9.6.2 包管理器

包管理器将小构件依次地一个放置在另一个的顶部或将它们一个挨着一个地放置。你也可以使用 fill 选项使一个小构件充满它的整个容器。

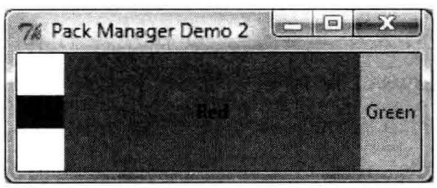
程序清单 9-8 显示三个标签，如图 9-10a 所示。这三个标签一个个地放置在顶部。红色标签使用选项 fill，它的值为 BOTH，而 expand 的值为 1。Fill 选项使用命名常量 X、Y 或者 BOTH 填充水平、垂直或者两个方向的空间。选项 expand 告诉包管理器分配额外的空间给小构件框。如果父小构件比容纳所有打包小构件的所需空间都大，那么额外的空间将被分配给小构件们，它们的 expand 选项被设置为非零值。

程序清单 9-8 PackManagerDemo.py

```
from tkinter import * # Import all definitions from tkinter

class PackManagerDemo:
    def __init__(self):
        window = Tk() # Create a window
        window.title("Pack Manager Demo 1") # Set title

        Label(window, text = "Blue", bg = "blue").pack()
        Label(window, text = "Red", bg = "red").pack(
            fill = BOTH, expand = 1)
        Label(window, text = "Green", bg = "green").pack(
            fill = BOTH)

        window.mainloop() # Create an event loop

PackManagerDemo() # Create GUI
```



a）包管理器使用 fill 选项填充容器



b）将小构件一个挨着一个放置

图　9-10

程序清单 9-9 显示如图 9-10b 所示的三个标签。使用 side 选项将这三个标签一个挨着一个放在一起。选项 side 可以是 LEFT、RIGH、TOP 或 BOTTOM。默认情况下，它被设置为 TOP。

程序清单 9-9 PackManagerDemoWithSide.py

```
from tkinter import * # Import all definitions from tkinter

class PackManagerDemoWithSide:
    window = Tk() # Create a window
    window.title("Pack Manager Demo 2") # Set title

    Label(window, text = "Blue", bg = "blue").pack(side = LEFT)
    Label(window, text = "Red", bg = "red").pack(
        side = LEFT, fill = BOTH, expand = 1)
    Label(window, text = "Green", bg = "green").pack(
        side = LEFT, fill = BOTH)

    window.mainloop() # Create an event loop

PackManagerDemoWithSide() # Create GUI
```

9.6.3 位置管理器

位置管理器将小构件放在绝对位置上。程序清单 9-10 显示如图 9-11 所示的三个标签。

程序清单 9-10 PlaceManagerDemo.py

```
from tkinter import * # Import all definitions from tkinter

class PlaceManagerDemo:
    def __init__(self):
```

```
5          window = Tk() # Create a window
6          window.title("Place Manager Demo") # Set title
7
8          Label(window, text = "Blue", bg = "blue").place(
9              x = 20, y = 20)
10         Label(window, text = "Red", bg = "red").place(
11             x = 50, y = 50)
12         Label(window, text = "Green", bg = "green").place(
13             x = 80, y = 80)
14
15         window.mainloop() # Create an event loop
16
17 PlaceManagerDemo() # Create GUI
```

蓝色标签的左上角在 (20,20) 处。所有三个标签都使用位置管理器放置。

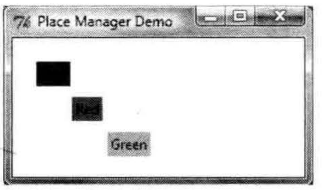


图 9-11　位置管理器将小构件放在绝对位置

注意：位置管理器不能兼容所有计算机。如果你在分辨率为 1024×768 的 Windows 上运行程序，那么布局的大小正合适。当程序运行在分辨率高一些的 Windows 上时，组件会显得非常小并且更拥挤。当程序运行在分辨率低一些的 Windows 上时，它们不能整体显示。由于这些不兼容因素，我们应该尽可能避免使用位置管理器。

检查点

9.21　如果对按钮使用包管理器编写下面的代码，那么错在哪儿？

```
button.pack(LEFT)
```

9.22　如果需要填充小构件之间的空间，应该使用哪个几何管理器？

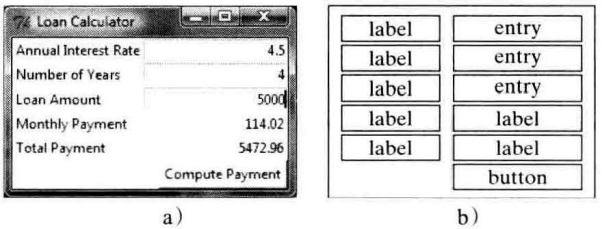
9.23　为什么应该避免使用位置管理器？

9.24　X、Y、BOTH、S、N、E 和 W，或者 NW、NE、SW 和 SE 是定义在 Tkinter 模块中的命名常量。使用 print 语句来显示定义在这些常量中的值。

9.7　实例研究：贷款计算器

关键点：本节提供了一个使用 GUI 小构件、几何布局管理器以及事件的例子。

程序清单 2-8 开发了一个基于控制台的计算贷款的程序。这里开发了一个计算贷款支付额的 GUI 应用程序，如图 9-12a 所示。



a)　　b)

图 9-12　程序计算贷款额并提供图形用户化界面

开发一个 GUI 应用程序涉及设计用户界面和编写处理事件的代码。下面是编写程序的主要步骤：

1）绘制如图 9-12b 所示的轮廓来设计用户界面 (UI)。UI 包括标签、文本输入框和按钮。

可以使用网格管理器将它们放在窗口中。

2）处理事件。当单击按钮时，程序就会调用一个回调函数从文本输入框获得用户输入的利率、年数以及贷款额，然后计算每月支付额和总支付额，并且在标签中显示值。

程序清单 9-11 给出完整的程序。

程序清单 9-11 LoanCalculator.py

```
from tkinter import * # Import all definitions from tkinter

class LoanCalculator:
    def __init__(self):
        window = Tk() # Create a window
        window.title("Loan Calculator") # Set title

        Label(window, text = "Annual Interest Rate").grid(row = 1,
            column = 1, sticky = W)
        Label(window, text = "Number of Years").grid(row = 2,
            column = 1, sticky = W)
        Label(window, text = "Loan Amount").grid(row = 3,
            column = 1, sticky = W)
        Label(window, text = "Monthly Payment").grid(row = 4,
            column = 1, sticky = W)
        Label(window, text = "Total Payment").grid(row = 5,
            column = 1, sticky = W)

        self.annualInterestRateVar = StringVar()
        Entry(window, textvariable = self.annualInterestRateVar,
            justify = RIGHT).grid(row = 1, column = 2)
        self.numberOfYearsVar = StringVar()
        Entry(window, textvariable = self.numberOfYearsVar,
            justify = RIGHT).grid(row = 2, column = 2)
        self.loanAmountVar = StringVar()
        Entry(window, textvariable = self.loanAmountVar,
            justify = RIGHT).grid(row = 3, column = 2)

        self.monthlyPaymentVar = StringVar()
        lblMonthlyPayment = Label(window, textvariable =
            self.monthlyPaymentVar).grid(row = 4, column = 2,
                sticky = E)
        self.totalPaymentVar = StringVar()
        lblTotalPayment = Label(window, textvariable =
            self.totalPaymentVar).grid(row = 5,
                column = 2, sticky = E)
        btComputePayment = Button(window, text = "Compute Payment",
            command = self.computePayment).grid(
                row = 6, column = 2, sticky = E)

        window.mainloop() # Create an event loop

    def computePayment(self):
        monthlyPayment = self.getMonthlyPayment(
            float(self.loanAmountVar.get()),
            float(self.annualInterestRateVar.get()) / 1200,
            int(self.numberOfYearsVar.get()))
        self.monthlyPaymentVar.set(format(monthlyPayment, "10.2f"))
        totalPayment = float(self.monthlyPaymentVar.get()) * 12 \
            * int(self.numberOfYearsVar.get())
        self.totalPaymentVar.set(format(totalPayment, "10.2f"))

    def getMonthlyPayment(self,
            loanAmount, monthlyInterestRate, numberOfYears):
        monthlyPayment = loanAmount * monthlyInterestRate / (1
```

```
56              - 1 / (1 + monthlyInterestRate) ** (numberOfYears * 12))
57          return monthlyPayment;
58
59  LoanCalculator()  # Create GUI
```

程序使用网格管理器在窗口中创建具有标签、输入域和按钮的用户界面（第 8 ～ 39 行）。按钮的命令选项被设置到 computePayment 方法（第 38 行）。当单击“Compute Payment”按钮时，调用该方法获得用户输入的年利率、贷款年数和贷款额以计算月支付额和总支付额（第 43 ～ 51 行）。

9.8　显示图像

关键点：可以向标签、按钮、复选按钮或单选按钮添加图像。

使用如下的 PhotoImage 类创建图像：

```
photo = PhotoImage(file = imagefilename)
```

图像文件必须是 GIF 格式。可以使用转换工具将其他格式的图像转换为 GIF 格式。

程序清单 9-12 显示如何将图像加入标签、按钮、复选按钮和单选按钮。你也可以使用 create_image 方法在画布上显示一副图像，如图 9-13 所示。

程序清单 9-12　ImageDemo.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  class ImageDemo:
4      def __init__(self):
5          window = Tk() # Create a window
6          window.title("Image Demo") # Set title
7
8          # Create PhotoImage objects
9          caImage = PhotoImage(file = "image/ca.gif")
10         chinaImage = PhotoImage(file = "image/china.gif")
11         leftImage = PhotoImage(file = "image/left.gif")
12         rightImage = PhotoImage(file = "image/right.gif")
13         usImage = PhotoImage(file = "image/usIcon.gif")
14         ukImage = PhotoImage(file = "image/ukIcon.gif")
15         crossImage = PhotoImage(file = "image/x.gif")
16         circleImage = PhotoImage(file = "image/o.gif")
17
18         # frame1 to contain label and canvas
19         frame1 = Frame(window)
20         frame1.pack()
21         Label(frame1, image = caImage).pack(side = LEFT)
22         canvas = Canvas(frame1)
23         canvas.create_image(90, 50, image = chinaImage)
24         canvas["width"] = 200
25         canvas["height"] = 100
26         canvas.pack(side = LEFT)
27
28         # frame2 contains buttons, check buttons, and radio buttons
29         frame2 = Frame(window)
30         frame2.pack()
31         Button(frame2, image = leftImage).pack(side = LEFT)
32         Button(frame2, image = rightImage).pack(side = LEFT)
33         Checkbutton(frame2, image = usImage).pack(side = LEFT)
34         Checkbutton(frame2, image = ukImage).pack(side = LEFT)
35         Radiobutton(frame2, image = crossImage).pack(side = LEFT)
36         Radiobutton(frame2, image = circleImage).pack(side = LEFT)
37
```

```
38          window.mainloop() # Create an event loop
39
40  ImageDemo() # Create GUI
```

程序将图像文件放在当前程序所在目录的图像文件夹中，然后在第 9 ～ 16 行为几个图像创建 PhotoImage 对象。这些对象都用在小构件中。图像是 Label、Button、Checkbutton 和 RadioButton 中的属性（第 21 行和第 31 ～ 36 行）。图像不是 Canvas 的属性，但你可以使用 create_image 方法在画布上绘制图像（第 23 行）。事实上，一块画布上可以显示多张图像。

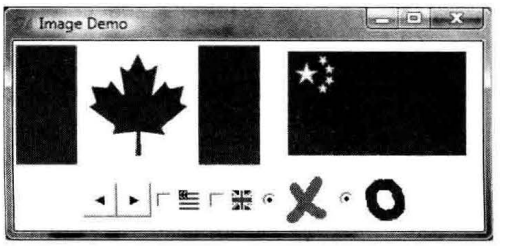


图 9-13　程序显示带图像的小构件

检查点

9.25　Python 支持什么图像格式？

9.26　使用下面的语句创建 PhotoImage 的错在哪？

```
image = PhotoImage("image/us.gif")
```

9.27　如何创建显示存储路径为 c:\pybook\image\canada.gif 的图像的按钮？

9.9　菜单

关键点：可以使用 Tkinter 创建菜单、弹出菜单以及工具栏。

Tkinter 提供了一个建立图形用户界面的全面解决方案。本节介绍菜单、弹出菜单和工具栏。

菜单可以使选择更方便，并广泛应用在 Windows 中。你可以使用 Menu 类创建菜单栏和菜单，然后使用 add_command 方法给菜单添加条目。

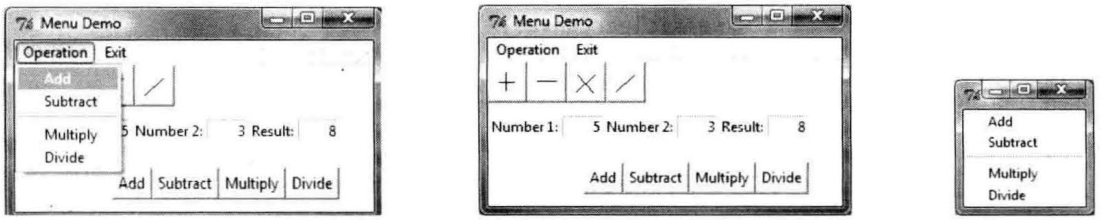
程序清单 9-13 给出如何创建如图 9-14 所示的菜单。

程序清单 9-13　MenuDemo.py

```
1  from tkinter import *
2
3  class MenuDemo:
4      def __init__(self):
5          window = Tk()
6          window.title("Menu Demo")
7
8          # Create a menu bar
9          menubar = Menu(window)
10         window.config(menu = menubar) # Display the menu bar
11
12         # Create a pull-down menu, and add it to the menu bar
13         operationMenu = Menu(menubar, tearoff = 0)
14         menubar.add_cascade(label = "Operation", menu = operationMenu)
15         operationMenu.add_command(label = "Add",
16             command = self.add)
17         operationMenu.add_command(label = "Subtract",
18             command = self.subtract)
19         operationMenu.add_separator()
20         operationMenu.add_command(label = "Multiply",
21             command = self.multiply)
22         operationMenu.add_command(label = "Divide",
23             command = self.divide)
```

```
24
25          # Create more pull-down menus
26          exitmenu = Menu(menubar, tearoff = 0)
27          menubar.add_cascade(label = "Exit", menu = exitmenu)
28          exitmenu.add_command(label = "Quit", command = window.quit)
29
30          # Add a tool bar frame
31          frame0 = Frame(window) # Create and add a frame to window
32          frame0.grid(row = 1, column = 1, sticky = W)
33
34          # Create images
35          plusImage = PhotoImage(file = "image/plus.gif")
36          minusImage = PhotoImage(file = "image/minus.gif")
37          timesImage = PhotoImage(file = "image/times.gif")
38          divideImage = PhotoImage(file = "image/divide.gif")
39
40          Button(frame0, image = plusImage, command =
41              self.add).grid(row = 1, column = 1, sticky = W)
42          Button(frame0, image = minusImage,
43              command = self.subtract).grid(row = 1, column = 2)
44          Button(frame0, image = timesImage,
45              command = self.multiply).grid(row = 1, column = 3)
46          Button(frame0, image = divideImage,
47              command = self.divide).grid(row = 1, column = 4)
48
49          # Add labels and entries to frame1
50          frame1 = Frame(window)
51          frame1.grid(row = 2, column = 1, pady = 10)
52          Label(frame1, text = "Number 1:").pack(side = LEFT)
53          self.v1 = StringVar()
54          Entry(frame1, width = 5, textvariable = self.v1,
55                justify = RIGHT).pack(side = LEFT)
56          Label(frame1, text = "Number 2:").pack(side = LEFT)
57          self.v2 = StringVar()
58          Entry(frame1, width = 5, textvariable = self.v2,
59                justify = RIGHT).pack(side = LEFT)
60          Label(frame1, text = "Result:").pack(side = LEFT)
61          self.v3 = StringVar()
62          Entry(frame1, width = 5, textvariable = self.v3,
63                justify = RIGHT).pack(side = LEFT)
64
65          # Add buttons to frame2
66          frame2 = Frame(window) # Create and add a frame to window
67          frame2.grid(row = 3, column = 1, pady = 10, sticky = E)
68          Button(frame2, text = "Add", command = self.add).pack(
69              side = LEFT)
70          Button(frame2, text = "Subtract",
71                 command = self.subtract).pack(side = LEFT)
72          Button(frame2, text = "Multiply",
73                 command = self.multiply).pack(side = LEFT)
74          Button(frame2, text = "Divide",
75                 command = self.divide).pack(side = LEFT)
76
77          mainloop()
78
79      def add(self):
80          self.v3.set(eval(self.v1.get()) + eval(self.v2.get()))
81
82      def subtract(self):
83          self.v3.set(eval(self.v1.get()) - eval(self.v2.get()))
84
85      def multiply(self):
86          self.v3.set(eval(self.v1.get()) * eval(self.v2.get()))
87
```

```
88      def divide(self):
89          self.v3.set(eval(self.v1.get()) / eval(self.v2.get()))
90
91  MenuDemo() # Create GUI
```



a)　　b)　　c)

图 9-14　程序使用菜单命令、工具栏按钮和按钮完成算术运算

程序在第 9 行创建了一个菜单栏，然后菜单栏被添加到窗口中。为了显示菜单，使用 config 方法将菜单栏添加到容器（第 10 行）。为了在菜单栏内创建一个菜单，将菜单栏作为父容器（第 13 行），然后调用菜单栏的 add_cascade 方法来设置菜单标签（第 14 行）。你可以使用 add_command 方法将条目添加到菜单（第 15 ～ 23 行）。注意：tearoff 被设置为 0，它表明菜单不能移出窗口。如果没有设置这个选项，菜单就会从窗口中移走，如图 9-14c 所示。

程序创建另一个名为 Exit 的菜单（第 26 ～ 27 行），并且将 Quit 菜单项添加给它（第 28 行）。

程序创建一个名为 frame0 的框架（第 31 ～ 32 行）并且使用它来容纳工具栏按钮。工具栏按钮是使用 PhotoImage 类创建的，它们都是带图像的按钮（第 35 ～ 38 行）。当单击工具栏按钮时，每一个按钮对应的命令指明要调用的回调函数。

程序创建一个名为 frame1 的框架（第 50 ～ 51 行），并用它来容纳标签和数字域。变量 v1、v2 和 v3 绑定到这些域（第 53、57、61 行）。

程序创建一个名为 frame2 的框架（第 66 ～ 67 行），用它来容纳 4 个分别实现加、减、乘和除的按钮。Add 按钮、Add 菜单项和 Add 工具栏按钮都有同样的回调函数 add（第 79 ～ 80 行），当单击其中任意一个（按钮、菜单项或菜单栏按钮）时，都会调用这个函数。

9.10　弹出菜单

关键点：弹出菜单，也称为上下文菜单，就像一般的菜单，但是不同的是它没有菜单栏而且能浮现在屏幕任何一个地方。

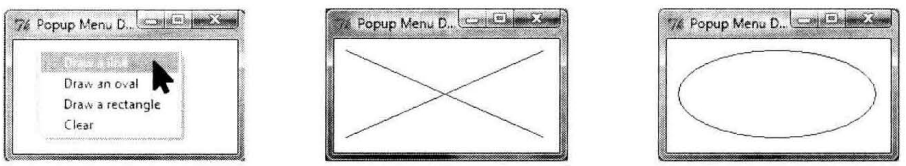
创建弹出菜单与创建一般菜单类似。首先，创建一个 Menu 的实例，然后向它添加条目。最后，将一个小构件和一个事件绑定以弹出菜单。

程序清单 9-14 中的例子使用弹出菜单命令来选择要显示在画布上的图形，如图 9-15 所示。

程序清单 9-14　PopupMenuDemo.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  class PopupMenuDemo:
4      def __init__(self):
5          window = Tk() # Create a window
6          window.title("Popup Menu Demo") # Set title
7
8          # Create a popup menu
9          self.menu = Menu(window, tearoff = 0)
10         self.menu.add_command(label = "Draw a line",
```

```
11              command = self.displayLine)
12          self.menu.add_command(label = "Draw an oval",
13              command = self.displayOval)
14          self.menu.add_command(label = "Draw a rectangle",
15              command = self.displayRect)
16          self.menu.add_command(label = "Clear",
17              command = self.clearCanvas)
18
19          # Place canvas in window
20          self.canvas = Canvas(window, width = 200,
21              height = 100, bg = "white")
22          self.canvas.pack()
23
24          # Bind popup to canvas
25          self.canvas.bind("<Button-3>", self.popup)
26
27          window.mainloop() # Create an event loop
28
29      # Display a rectangle
30      def displayRect(self):
31          self.canvas.create_rectangle(10, 10, 190, 90, tags = "rect")
32
33      # Display an oval
34      def displayOval(self):
35          self.canvas.create_oval(10, 10, 190, 90, tags = "oval")
36
37      # Display two lines
38      def displayLine(self):
39          self.canvas.create_line(10, 10, 190, 90, tags = "line")
40          self.canvas.create_line(10, 90, 190, 10, tags = "line")
41
42      # Clear drawings
43      def clearCanvas(self):
44          self.canvas.delete("rect", "oval", "line")
45
46      def popup(self, event):
47          self.menu.post(event.x_root, event.y_root)
48
49  PopupMenuDemo() # Create GUI
```



a)　b)　c)

图 9-15　当单击画布时程序显示一个弹出菜单

程序创建了一个菜单来容纳菜单项（第 9 ～ 17 行）。创建画布显示图形。菜单项使用回调函数指导画布绘制图形。

通常，通过指向一个小构件并单击鼠标右键就会显示弹出菜单。程序在 canvas 上将右击鼠标按钮和弹出回调函数绑定（第 25 行）。当你右击鼠标按钮时，就会调用 popup 回调函数，它会在鼠标被单击的地方显示菜单。

检查点

9.28　使用什么方法显示菜单栏？

9.29　如何显示弹出菜单？

9.11　鼠标、按键事件和绑定

关键点：可以使用 bind 方法将鼠标和按键事件绑定到一个小构件。

前面的例子利用下面的语法使用小构件的 bind 方法将鼠标事件与回调处理器绑定：

```
widget.bind(event, handler)
```

如果一个匹配的事件发生，那就调用处理器。在前面的例子中，事件是 <Button-3> 而处理器函数是 popup。这个事件是一个标准的 Tkinter 对象，当一个事件发生时会自动创建它。每一个处理器都将一个事件作为它的参数。下面的例子使用事件作为参数来定义处理器：

```
def popup(event):
    menu.post(event.x_root, event.y_root)
```

event 对象都有许多和事件相关的描述事件的特性。例如：对一个鼠标事件，event 对象使用 x、y 属性捕获鼠标当前以像素为单位的位置。

表 9-2 罗列出一些常用事件，而表 9-3 罗列出一些事件属性。

程序清单 9-15 中的程序处理鼠标和按键事件。它显示如图 9-16a 所示的窗口。鼠标和按键事件被处理，并在命令行窗口中显示处理信息，如图 9-16b 所示。

表 9-2　事件

事件	描述
<Bi-Motion>	当鼠标左键被按住在小构件且移动鼠标时事件发生
<Button-i>	Button-1、Button-2、Button-3 表明左键、中间键和右键，当在小构件上单击鼠标左键时，Tkinter 会自动抓到鼠标指针的位置，ButtonPressed-i 是 Button-i 的代名词
<ButtonReleased-i>	当释放鼠标左键时事件发生
<Double-Button-i>	当双击鼠标左键时事件发生
<Enter>	当鼠标光标进入小构件时事件发生
<Key>	当单击一个键时事件发生
<Leave>	当鼠标光标离开小构件时事件发生
<Return>	当单击“Enter”键时事件发生，可以将键盘上的任意键（像“A”、“B”、“Up”、“Down”、“Left”、“Right”）和一个事件绑定
<Shift+A>	当单击“Shift+A”键时事件发生，可以将 Alt、Shift 和 Control 和其他键组合
<Triple-Button-i>	当三次单击鼠标左键时事件发生

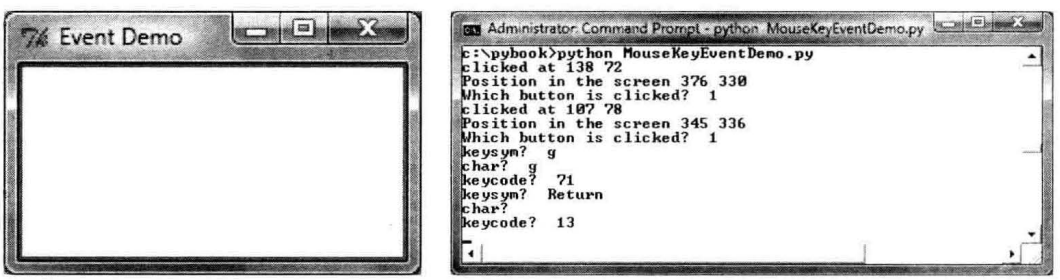
表 9-3　事件属性

事件属性	描述
char	从键盘输入的和按键事件相关的字符
keycode	从键盘输入的和按键事件相关的键的键代码（即统一码）
keysym	从键盘输入的和按键事件相关的键的键符号（即字符）
num	按键数字（1、2、3）表明按下的是哪个鼠标键
widget	触发这个事件的小构件对象
x 和 y	当前鼠标在小构件中以像素为单位的位置
x_root 和 y_root	当前鼠标相对于屏幕左上角的以像素为单位的位置

程序清单 9-15　MouseKeyEventDemo.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  class MouseKeyEventDemo:
```

```
4       def __init__(self):
5           window = Tk() # Create a window
6           window.title("Event Demo") # Set a title
7           canvas = Canvas(window, bg = "white", width = 200, height = 100)
8           canvas.pack()
9
10          # Bind with <Button-1> event
11          canvas.bind("<Button-1>", self.processMouseEvent)
12
13          # Bind with <Key> event
14          canvas.bind("<Key>", self.processKeyEvent)
15          canvas.focus_set()
16
17          window.mainloop() # Create an event loop
18
19      def processMouseEvent(self, event):
20          print("clicked at", event.x, event.y)
21          print("Position in the screen", event.x_root, event.y_root)
22          print("Which button is clicked? ", event.num)
23
24      def processKeyEvent(self, event):
25          print("keysym? ", event.keysym)
26          print("char? ", event.char)
27          print("keycode? ", event.keycode)
28
29  MouseKeyEventDemo() # Create GUI
```



a)　　b)

图 9-16　程序处理鼠标和按键事件

程序创建画布（第 7 行），并将画布上的鼠标事件 <Button-1> 和 processMouseEvent 回调函数（第 11 行）绑定。画布上什么也没画，所以它是空白的，如图 9-16a 所示。当鼠标单击画布时，就会创建一个事件。processMouseEvent 被调用以处理在画布上（第 20 行）、在屏幕上（第 21 行）显示鼠标指针位置以及哪个鼠标按钮被单击（第 22 行）的事件。

Canvas 小构件也是其他按键事件的源。程序在画布上将按键事件和回调函数 processKeyEvent 进行绑定（第 14 行）并且在画布上设置焦点，以便于从键盘上获取输入（第 15 行）。

程序清单 9-16 在画布上显示了一个圆。圆的半径随鼠标左击而增加，随着右击而减少，如图 9-17 所示。

程序清单 9-16　EnlargeShrinkCircle.py

```
1   from tkinter import * # Import all definitions from tkinter
2
3   class EnlargeShrinkCircle:
4       def __init__(self):
5           self.radius = 50
6
7           window = Tk() # Create a window
```

```
 8          window.title("Control Circle Demo") # Set a title
 9          self.canvas = Canvas(window, bg = "white",
10              width = 200, height = 200)
11          self.canvas.pack()
12          self.canvas.create_oval(
13              100 - self.radius, 100 - self.radius,
14              100 + self.radius, 100 + self.radius, tags = "oval")
15
16          # Bind canvas with mouse events
17          self.canvas.bind("<Button-1>", self.increaseCircle)
18          self.canvas.bind("<Button-3>", self.decreaseCircle)
19
20          window.mainloop() # Create an event loop
21
22      def increaseCircle(self, event):
23          self.canvas.delete("oval")
24          if self.radius < 100:
25              self.radius += 2
26          self.canvas.create_oval(
27              100 - self.radius, 100 - self.radius,
28              100 + self.radius, 100 + self.radius, tags = "oval")
29
30      def decreaseCircle(self, event):
31          self.canvas.delete("oval")
32          if self.radius > 2:
33              self.radius -= 2
34          self.canvas.create_oval(
35              100 - self.radius, 100 - self.radius,
36              100 + self.radius, 100 + self.radius, tags = "oval")
37
38  EnlargeShrinkCircle() # Create GUI
```

程序创建了一张画布（第 9 行），并且在画布上显示一个初始半径为 50 的圆（第 5 行和第 12 ～ 14 行）。画布将鼠标事件 <Button-1> 和处理器 increaseCircle 进行绑定（第 17 行），将鼠标事件 <Button-3> 和处理器 decreaseCircle 进行绑定（第 18 行）。当左击鼠标时，调用 increaseCircle 函数以增加半径（第 24 ～ 25 行）然后重新显示这个圆（第 26 ～ 28 行）。当右击鼠标时，调用 decreaseCircle 函数以减少半径（第 32 ～ 33 行），然后重新显示这个圆（第 34 ～ 36 行）。

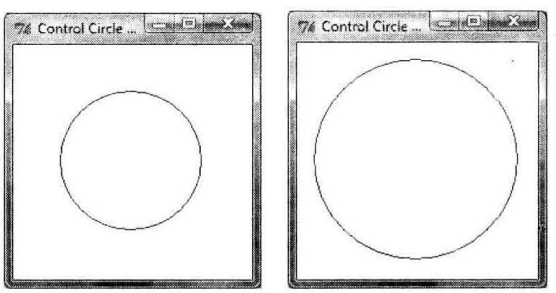


图 9-17　通过鼠标事件控制圆的大小

检查点

9.30　如何在画布上将鼠标单击事件和回调函数 p 绑定？

9.31　当右击鼠标按钮时，移动鼠标的事件是什么？

9.32　单击鼠标两次的事件是什么？

9.33　单击鼠标中间键三次的事件是什么？

9.34　什么参数会自动传递给事件处理函数？

9.35　如何从事件对象中获取当前鼠标位置？

9.36　如何从事件对象中获取关键字符？

9.12　动画

关键点：通过显示一系列的图画来创建动画。

Canvas 类也可以被用来开发动画。可以在画布上显示图片和文本，并使用 move(tags, dx,dy) 方法移动图片，如果 dx 是正值，则图片右移 dx 个像素，而如果 dy 是正值，则图片下移 dy 个像素。如果 dx 或 dy 是负值，图片则向左移或向上移。

程序清单 9-17 中的程序持续地从左向右重复显示一条移动的信息，如图 9-18 所示。

程序清单 9-17 AnimationDemo.py

```
1  from tkinter import * # Import all definitions from tkinter
2
3  class AnimationDemo:
4      def __init__(self):
5          window = Tk() # Create a window
6          window.title("Animation Demo") # Set a title
7
8          width = 250 # Width of the canvas
9          canvas = Canvas(window, bg = "white",
10             width = 250, height = 50)
11         canvas.pack()
12
13         x = 0 # Starting x position
14         canvas.create_text(x, 30,
15             text = "Message moving?", tags = "text")
16
17         dx = 3
18         while True:
19             canvas.move("text", dx, 0) # Move text dx unit
20             canvas.after(100) # Sleep for 100 milliseconds
21             canvas.update() # Update canvas
22             if x < width:
23                 x += dx  # Get the current position for string
24             else:
25                 x = 0 # Reset string position to the beginning
26                 canvas.delete("text")
27                 # Redraw text at the beginning
28                 canvas.create_text(x, 30, text = "Message moving?",
29                     tags = "text")
30
31         window.mainloop() # Create an event loop
32
33 AnimationDemo() # Create GUI
```

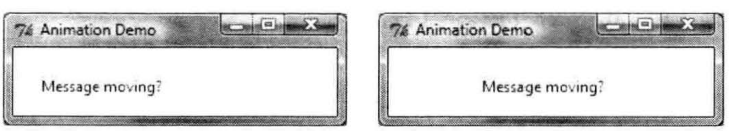


图 9-18　程序以动画显示一条信息

程序创建了一张画布（第 9 行）并且在画布的指定初始位置显示文本（第 13 ～ 15 行）。动画实际上就是下面循环中的 3 条语句完成的（第 19 ～ 21 行）：

```
canvas.move("text", dx, 0) # Move text dx unit
canvas.after(100) # Sleep for 100 milliseconds
canvas.update() # Update canvas
```

调用 canvas.move 可以使位置的 x 坐标右移 dx 个单位 (第 19 行）。调用 canvas.after(100) 函数会使程序暂停 100 毫秒（第 20 行）。调用 canvas.update() 会重新显示画布（第 21 行）。

可以添加工具来控制动画的速度、停止动画以及重新启动动画。程序清单 9-18 通过添加了控制动画的四个按钮改写程序清单 9-17 中的程序，如图 9-19 所示。

程序清单 9-18 ControlAnimation.py

```
from tkinter import * # Import all definitions from tkinter

class ControlAnimation:
    def __init__(self):
        window = Tk() # Create a window
        window.title("Control Animation Demo") # Set a title

        self.width = 250 # Width of self.canvas
        self.canvas = Canvas(window, bg = "white",
            width = self.width, height = 50)
        self.canvas.pack()

        frame = Frame(window)
        frame.pack()
        btStop = Button(frame, text = "Stop", command = self.stop)
        btStop.pack(side = LEFT)
        btResume = Button(frame, text = "Resume",
            command = self.resume)
        btResume.pack(side = LEFT)
        btFaster = Button(frame, text = "Faster",
            command = self.faster)
        btFaster.pack(side = LEFT)
        btSlower = Button(frame, text = "Slower",
            command = self.slower)
        btSlower.pack(side = LEFT)

        self.x = 0 # Starting x position
        self.sleepTime = 100 # Set a sleep time
        self.canvas.create_text(self.x, 30,
            text = "Message moving?", tags = "text")

        self.dx = 3
        self.isStopped = False
        self.animate()

        window.mainloop() # Create an event loop

    def stop(self): # Stop animation
        self.isStopped = True

    def resume(self): # Resume animation
        self.isStopped = False
        self.animate()

    def faster(self): # Speed up the animation
        if self.sleepTime > 5:
            self.sleepTime -= 20

    def slower(self): # Slow down the animation
        self.sleepTime += 20

    def animate(self): # Move the message
        while not self.isStopped:
            self.canvas.move("text", self.dx, 0) # Move text
            self.canvas.after(self.sleepTime) # Sleep
            self.canvas.update() # Update canvas
            if self.x < self.width:
                self.x += self.dx  # Set new position
            else:
                self.x = 0 # Reset string position to beginning
                self.canvas.delete("text")
                # Redraw text at the beginning
```

```
63                  self.canvas.create_text(self.x, 30,
64                      text = "Message moving?", tags = "text")
65
66  ControlAnimation() # Create GUI
```

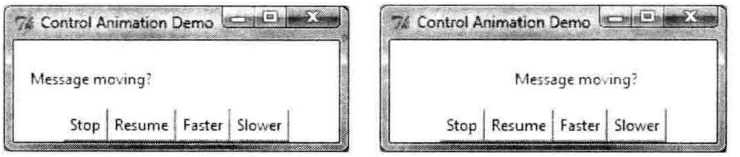


图 9-19　程序使用按钮来控制动画

程序调用 animate() 函数来启动动画（第 34 行）。变量 isStopped 决定了动画是否继续移动。初始状态下，它被设置为 Fasle（第 33 行）。当它的值为假时，animate 方法中的循环会被不断执行（第 53 ～ 64 行）。

单击按钮“Stop”、“Resume”、“Faster”或“Slower”来停止、重新开始、加速或者放慢动画。当单击“Stop”按钮时，就会调用 stop 函数来设置 isStopped 为 True（第 39 行）。这会导致动画循环终止（第 53 行）。当单击“Resume”按钮时，就会调用 resume 函数来设置 isStopped 为 False(第 42 行），并重启动画（第 43 行）。

变量 sleepTime 控制动画的速度，初始状态下，它被设置为 100 毫秒（第 28 行）。当单击“Faster”按钮时，就会调用 faster 方法将 sleepTime 减少 20(第 47 行）。当单击“Slower”按钮时，就会调用 slower 函数将 sleepTime 值增加 20（第 50 行）。

检查点

9.37　可以使用哪个方法使程序处在休眠状态？

9.38　可以使用哪个方法更新画面？

9.13　滚动条

关键点：Scrollbar 小构件可以在水平方向或者垂直方向展开 Text、Canvas 或者 Listbox 小构件里的内容。

程序清单 9-19 给出一个在 Text 小构件中展开的例子，如图 9-20 所示。

程序清单 9-19　ScrollText.py

```
1   from tkinter import * # Import all definitions from tkinter
2
3   class ScrollText:
4       def __init__(self):
5           window = Tk() # Create a window
6           window.title("Scroll Text Demo") # Set title
7
8           frame1 = Frame(window)
9           frame1.pack()
10          scrollbar = Scrollbar(frame1)
11          scrollbar.pack(side = RIGHT, fill = Y)
12          text = Text(frame1, width = 40, height = 10, wrap = WORD,
13                      yscrollcommand = scrollbar.set)
14          text.pack()
15          scrollbar.config(command = text.yview)
16
17          window.mainloop() # Create an event loop
```

```
18
19  ScrollText() # Create GUI
```

程序创建了一个 Scrollbar（第 10 行），然后将它放在了文本的右端（第 11 行）。滚动条被绑定到 Text 小构件（第 15 行），这样，Text 小构件的内容就被滚动显示出来。

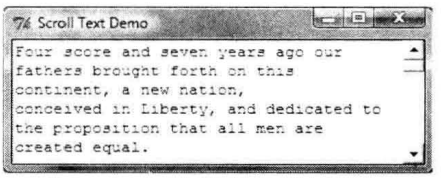


图 9-20　可以使用滚动条（最右端）查看当前 Text 小构件中不可见的文本

检查点

9.39　什么小构件可以与滚动条一起使用？

9.40　如何将滚动条和一个视图关联？

9.14　标准对话框

关键点：可以使用标准对话框显示消息框或者提示用户输入数字和字符串。

最后，让我们来看看 Tkinter 标准对话框（通常被简称为对话框）。程序清单 9-20 给出了使用这些对话框的例子。程序的一个示例运行如图 9-21 所示。

程序清单 9-20　DialogDemo.py

```
1  import tkinter.messagebox
2  import tkinter.simpledialog
3  import tkinter.colorchooser
4
5  tkinter.messagebox.showinfo("showinfo", "This is an info msg")
6
7  tkinter.messagebox.showwarning("showwarning", "This is a warning")
8
9  tkinter.messagebox.showerror("showerror", "This is an error")
10
11 isYes = tkinter.messagebox.askyesno("askyesno", "Continue?")
12 print(isYes)
13
14 isOK = tkinter.messagebox.askokcancel("askokcancel", "OK?")
15 print(isOK)
16
17 isYesNoCancel = tkinter.messagebox.askyesnocancel(
18     "askyesnocancel", "Yes, No, Cancel?")
19 print(isYesNoCancel)
20
21 name = tkinter.simpledialog.askstring(
22     "askstring", "Enter your name")
23 print(name)
24
25 age = tkinter.simpledialog.askinteger(
26     "askinteger", "Enter your age")
27 print(age)
28
29 weight = tkinter.simpledialog.askfloat(
30     "askfloat", "Enter your weight")
31 print(weight)
```

程序调用函数 showinfo、showwarning 和 showerror 来显示一条消息（第 5 行）、一个警告（第 7 行）和一个错误（第 9 行）。这些函数都被定义在 tkinter.messagebox 模块中（第 1 行）。

函数 askyesno 在对话框中显示“Yes”和“No”按钮（第 11 行）。如果单击“Yes”按钮，则函数返回值为 True，而如果单击“No”按钮，则函数返回值为 False。

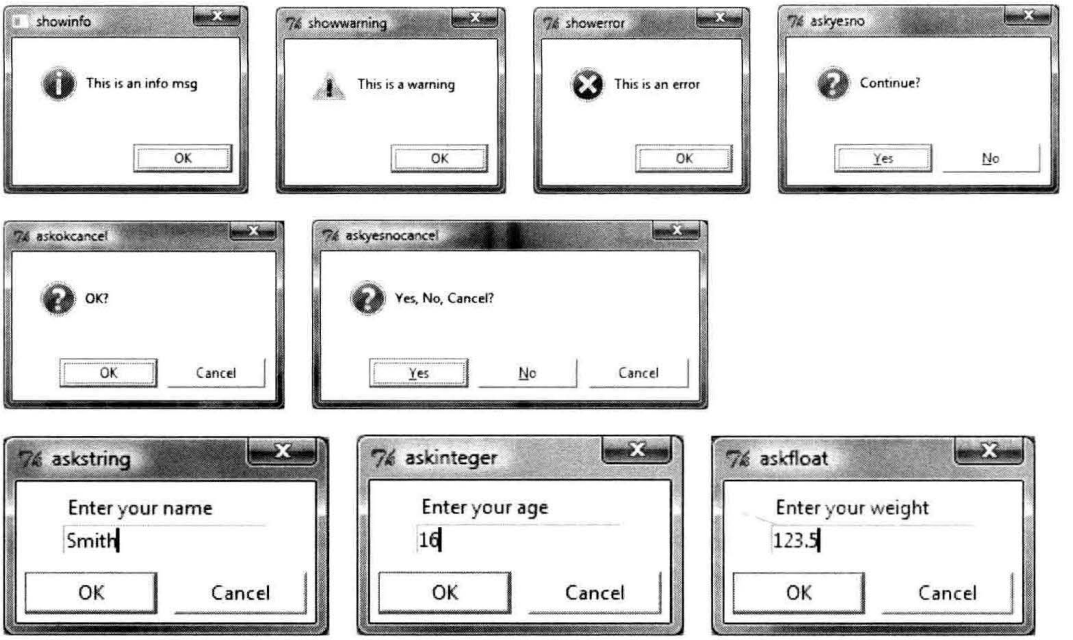


图 9-21　可以使用标准对话框来显示消息框并接受输入

函数 askokcancel 在对话框中显示“OK”和“Cancel”按钮（第 14 行）。如果单击“OK”按钮，则函数的返回值为 True，而如果单击“Cancel”按钮，则函数的返回值为 False。

函数 askyesnocancel 在对话框中显示“Yes”、“No”和“Cancel”按钮（第 17 行）。如果单击“Yes”按钮，则函数的返回值为 True，而如果单击“NO”按钮，则函数返回值为 False，而如果单击“Cancel”按钮，则函数返回值为 None。

函数 askstring（第 21 行）会在单击“OK”按钮时返回对话框中输入的字符串，而单击“Cancel”按钮时，则返回 None。

函数 askinteger（第 25 行）会在单击“OK”按钮时返回对话框中输入的整数，而单击“Cancel”按钮时，则返回 None。

函数 askfloat（第 29 行）会在单击“OK”按钮时返回对话框中输入的浮点数，而单击“Cancel”按钮时，则返回 None。

所有的对话框都是模态窗口，它意味着一旦对话框消失，程序将不会继续。

检查点

9.41　编写一条语句在消息会话框中显示“Welcome to Python”。

9.42　使用对话框编写语句提示用户输入一个整数、一个浮点数和一个字符串。

关键术语

callback function（回调函数）
geometry manager（几何管理器）
grid manager（网格管理器）
handler（处理器）
pack manager（包管理器）
parent container（父容器）
place manager（位置管理器）
widget class（小构件类）

本章总结

1. 为了使用 Tkinter 开发一个 GUI 应用，首先要使用 Tk 类创建一个窗口，然后创建一些小构件并将它们放在窗口里。每个 widget 类的第一个参数必须是父容器。
2. 为了将一个小构件放在容器中，必须具体指明它的几何管理器。
3. Tkinter 支持三个几何管理器：包、网格和位置。包管理器将小构件一个挨一个放置或一个在另一个顶部地放置。网格管理器将小构件放在网格中。位置管理器将小构件放在绝对位置。
4. 许多小构件有将事件与回调函数绑定的命令选项。当事件发生时，就会调用回调函数。
5. Canvas 小构件可以被用来绘制直线、矩形、椭圆、圆弧和多边形，并且显示图像和文本字符串。
6. 许多小构件中都可以使用图片，例如：标签、按钮、复选按钮、单选按钮和画布。
7. 可以使用 Menu 类创建菜单栏、菜单项和弹出菜单。
8. 可以将一个小构件的鼠标和按键事件绑定到一个回调函数。
9. 可以使用画布来开发动画。
10. 可以使用标准对话框显示消息和接收输入。

测试题

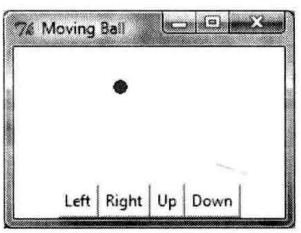
本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

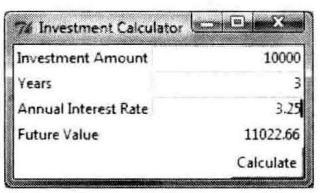
注意：整本书编程题中使用的图像图标可以从 www.cs.armstrong.edu/liang/py/book.zip 中的 image 文件夹中获取。

第 9.2 ～ 9.8 节

*9.1 （移动小球）编写程序移动面板上的小球。你应该定义一个面板类来显示小球，并提供将小球向左、向右、向上和向下移动的方法，如图 9-22a 所示。检测边界防止小球完全地从视野中消失。



a）单击按钮来移动小球



b）输入投资额、年数和年利率来获取未来值

图　9-22

*9.2 （创建一个投资值计算器）编写程序计算在给定利息、指定年数的情况下投资的未来值。这个计算公式如下所示。

$$futureValue = investmentAmount * (1 + monthlyInterestRate)^{years * 12}$$

使用文本域输入投资额、年份和利率。当用户单击“Calculate”按钮时，在文本域中显示未来的投资值，如图 9-22b 所示。

*9.3 （选择几何图形）编写程序绘制一个矩形或椭圆，如图 9-23 所示。用户从单选按钮选择一个图形，然后选择一个复选按钮指定是否填充图形。

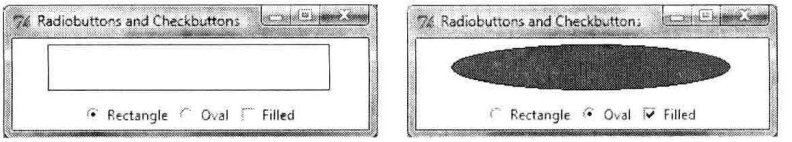


图 9-23　当选择一种图形类型时程序会显示一个矩形或椭圆以及是否给它们填充颜色

*9.4（显示矩形）编写一个程序显示 20 个矩形，如图 9-24 所示。

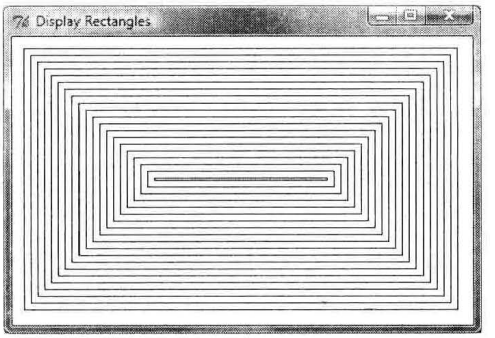
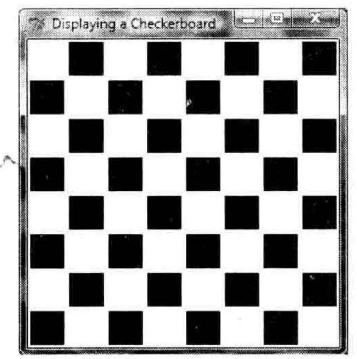
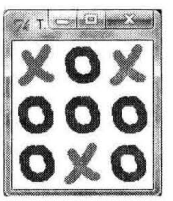


图 9-24　程序显式 20 个矩形

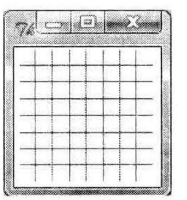
9.5（游戏：显示棋盘）编写程序显示一个棋盘，每个白色格子和黑色格子都是一个背景为黑色或白色的画布，如图 9-25a 所示。



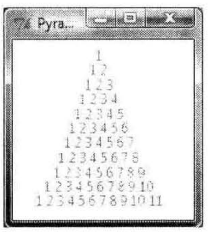
a）程序显示一个棋盘



b）程序显示一个井字盘



c）程序显示一个网格



d）程序显示一个三角形状的数字

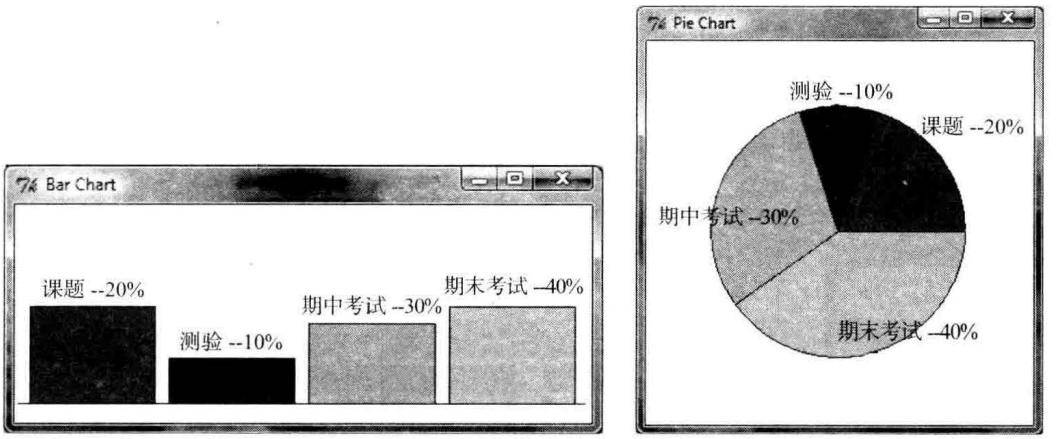
图　9-25

9.6（游戏：显示一个井字盘）编写一个程序显示 9 个标签。每一个标签可以显示一个表示 X 的图像图标或一个表示 O 的图像图标，如图 9-25b 所示。显示什么是随机决定的。使用函数 random.randint(0,1) 生成一个整数 0 或 1，它对应到显示叉号图像（X）图标或非图像（O）图标。叉号图像和非图像在文件 x.gif 和 o.gif 中。

9.7（显示一个 8 × 8 的网格）编写程序显示一个 8 × 8 的网格，如图 9-25c 所示。垂直线用红色，而水平线用蓝色。

**9.8（显示三角形状的数字）编写程序显示三角形状的数字，如图 9-25d 所示。显示的行数根据窗口的大小做相应的调整。

**9.9（显示一个条状图）编写程序，使用条状图显示课题、测验、期中考试和期末考试的成绩占总成绩的百分比，如图 9-26a 所示。假设课题成绩占总成绩的 20%，使用红色显示；测验成绩占总成绩的 10%，使用蓝色显示；期中成绩占 30%，使用绿色显示；期末成绩占 40%，使用橙色显示。



a）程序显示一个条状图　　b）程序显示一个饼状图

图　9-26

**9.10 （显示一个饼状图）编写程序，使用一个饼状图显示课题、测验、期中考试和期末考试成绩占总成绩的百分比，如图 9-26b 所示。假定课题成绩占总成绩的 20%，使用红色显示；测验成绩占总成绩的 10%，使用蓝色显示；期中成绩占 30%，使用绿色显示；期末成绩占 40%，使用橙色显示。

**9.11 （显示时钟）编写程序，显示时钟并显示当前时间，如图 9-27a 所示。为了获取当前时间，使用附录 II.B 中的 datetime 类。

a）程序显示当前时间的时钟　　b）～c）程序在显示两条消息之间交替变换

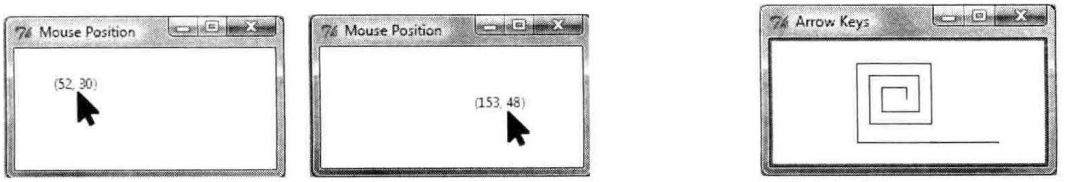
图　9-27

第 9.9 ～ 9.14 节

**9.12 （交替变换两条消息）编写程序，当单击鼠标时，就会在画布上交替显示两条消息“Programming is fun”和“It is fun to program”，如图 9-27b、9-27c 所示。

*9.13 （显示鼠标位置）编写两个程序：一个程序在单击鼠标时显示鼠标的位置（参见图 9-28a、9-28b），而另一个程序在按住鼠标时显示位置，松开鼠标时停止显示。

*9.14 （使用箭头键绘制线段）编写程序使用箭头键绘制线段。线段从框架的中心开始，当单击 Right 箭头键、Up 箭头键、Left 箭头键或者 Down 箭头键时，分别向东、北、西或南方向绘制线段，如图 9-28c 所示。



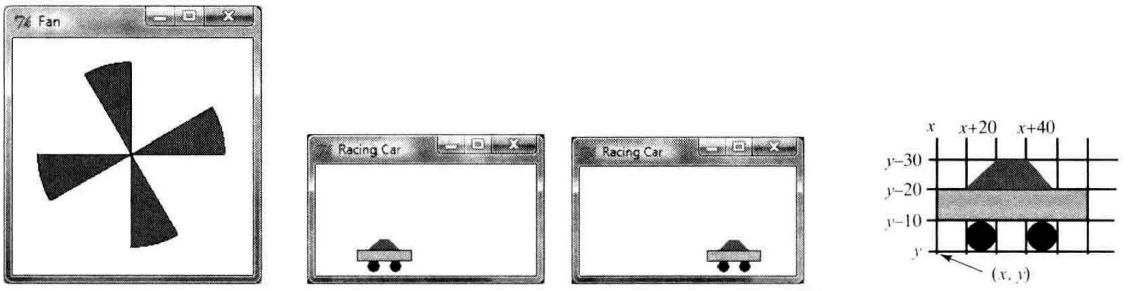
a）～b）程序显示当单击鼠标时鼠标光标的位置　　c）程序在单击 Right、Up、Left 或者 Down 箭头键时绘制一条线段

图　9-28

**9.15 （显示一台静态风扇）编写程序显示一台静态风扇，如图 9-29a 所示。

**9.16 （显示一台转动的风扇）编写程序显示一台转动的风扇，如图 9-29a 所示。

**9.17 （赛车）编写程序模拟赛车跑动，如图 9-29（b ～ d）所示。赛车从左向右移动。当赛车到达右端时，汽车从左端重新启动，然后不断重复相同的过程。让用户通过按住 Up 和 Down 箭头键分别对赛车加速和减速。

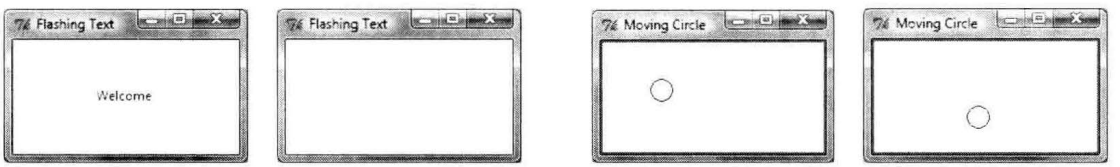


a）程序显示一台风扇　b）～ c）一辆移动的赛车　d）你可以使用一个新基点重新绘制一辆赛车

图 9 -29

*9.18 （显示闪烁的文本）编写程序显示闪烁的文本“ Welcome”，如图 9-30（a ～ b）所示。（提示：为了使文本闪烁，需要在画布上重复绘制它或者交替删除它。使用 Bool 变量来控制交替变换过程。）

*9.19 （使用箭头移动圆）编写程序使用箭头键分别向上、向下、向左或向右移动圆，如图 9-30（c ～ d）所示。



a）～ b）程序显示带文本的闪烁标签　c）～ d）当按下箭头键时圆被移动

图 9-30

**9.20 （几何：在圆内吗？）编写程序绘制一个以点 (100，60) 为圆心、半径为 50 的固定圆。无论何时单击左键移动鼠标，都会显示鼠标指针是否是在圆内的消息，如图 9-31 所示。

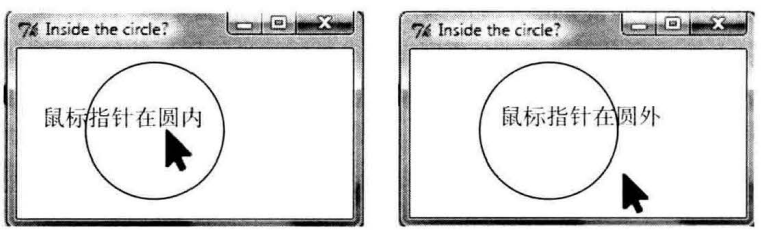


图 9-31　检测鼠标指针是否在圆内

**9.21 （几何：在矩形内吗？）编写程序绘制一个以点 (100,60) 为中心、宽为 100、高为 40 的固定矩形。无论何时单击左键移动鼠标，都会显示鼠标指针是否在矩形内的消息，如图 9-32 所示。为了判定指针是否在矩形内，使用编程题 8.19 中的 Rectangle2D 类。

**9.22 （几何：钟摆）编写程序动态演示钟摆的摆动，如图 9-33 所示。按 Up 箭头键加速，按 Down 箭头键减速。按 S 键停止动画，按 R 键重新开始。

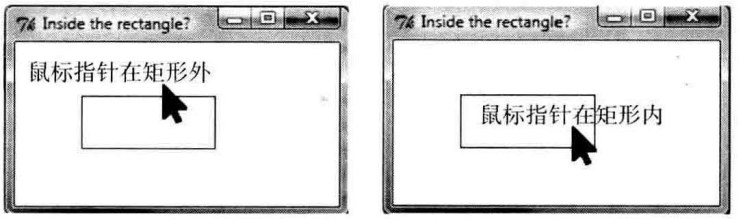


图 9-32　检测鼠标指针是否在矩形内

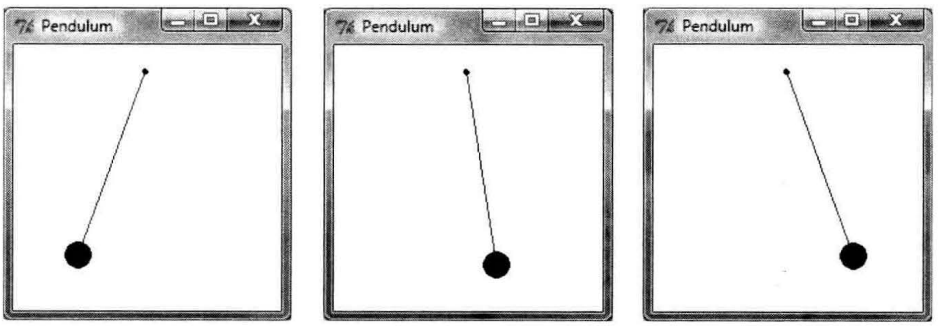


图 9-33　程序动态演示钟摆的摆动

*9.23 （按钮和单选按钮）编写程序使用单选按钮选择文本的背景色，如图 9-34 所示。变量色彩是红色、黄色、灰色和绿色。程序使用按钮“<=”和“=>”将文本向左或向右移动。

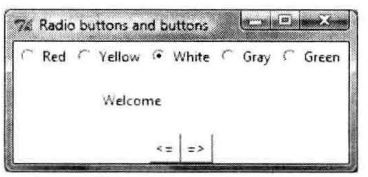


图 9-34　按钮“<=”和“=>”移动面板上的消息，而且你也可以设置消息的背景色

9.24 （显示圆）编写程序，单击左键显示一个新的较大的圆，单击右键可以去掉最大的圆，如图 9-35 所示。

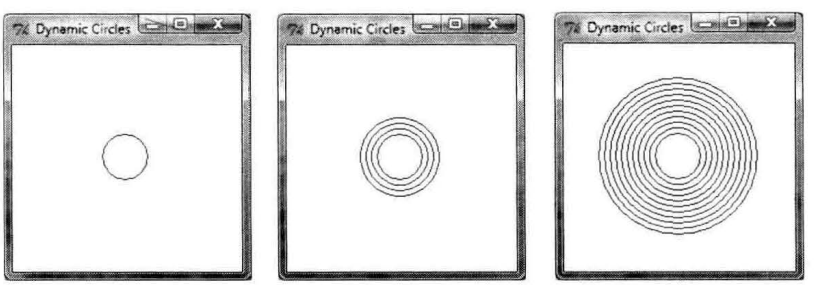
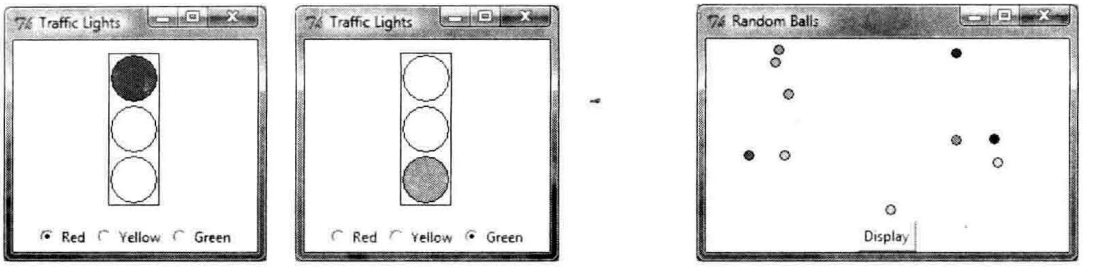


图 9-35　程序在单击鼠标左 / 右键时添加 / 删除圆

**9.25 （交通灯）编写程序模拟交通灯。程序让用户从三盏灯：红灯、黄灯或绿灯中选择一盏。当选择一个单选按钮时，灯就被打开，并且一次只能亮一个灯（参见图 9-36（a-b）。当程序启动时，没有灯亮。

*9.26 （显示颜色随机的球）编写程序显示 10 个颜色随机的球，并且将它们放在随机位置上，如图 9-36c 所示。



a）～b）单选按钮让用户选择交通灯的颜色　　c）10 个颜色随机的球显示在随机位置

图　9-36

*9.27　（使用不同利率比较贷款）改写编程题 5.23 来创建如图 9-37 所示的用户界面。你的程序应该让用户在文本域中输入贷款额和以年为单位的贷款期，并且在文本域中显示利率从 5% 开始到 8%，每次递增 1/8 的情况下的每月支付额和总支付额。

**9.28　（几何：显示角度）编写程序让用户拖拽三角形的顶点并且动态显示角度，如图 9-38a 所示。当鼠标移动接近一个顶点时，鼠标光标会变为十字形。计算角 A、B 和 C（参见图 9-38b）的公式在程序清单 3-2 中已给出。

提示：使用 Point 类来表示一个点，如编程题 8.17 所描述。初始状态时，创建 3 个随机位置的点。当鼠标被移近一个点时，光标就会被改变为十字型（+）并且将点重置到鼠标所在位置。无论何时移动一个点，都要重新显示三角形和角度。

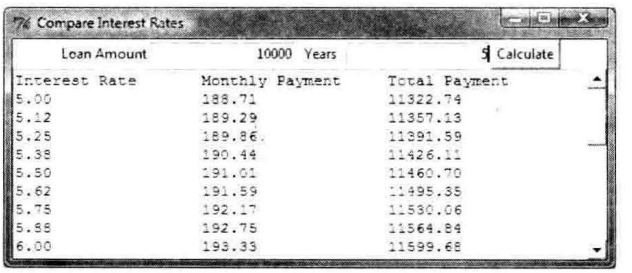
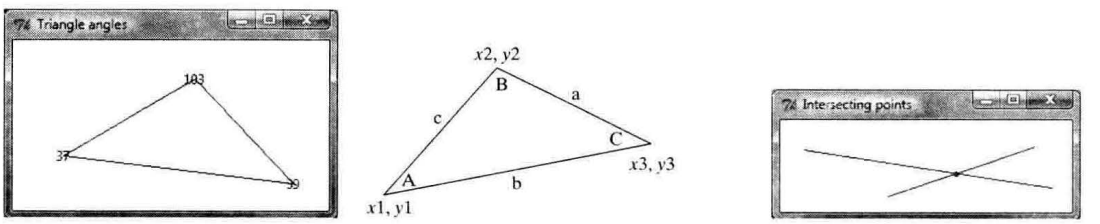


图 9-37　程序显示在不同利率下给定贷款的每月支付额和总支付额的表格



a）～b）程序可以让用户拖曳三角形的顶点并动态显示角度　　c）程序可以让用户拖曳三角形的顶点并且动态显示线段和交点

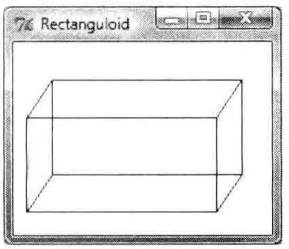
图　9-38

**9.29　（几何：交点）编写程序显示两条直线段以及它们的端点和交点。初始状态时，线段 1 的端点为 (20,20) 和 (56,130)，而线段 2 的端点为 (100,20) 和 (16,130)。用户可以使用鼠标拖动一个点并且动态显示交点，如图 9-38c 所示。（提示：参见编程题 4.25 寻找两条无界线的交点。编程题 9.28 的提示也适用于本题。）

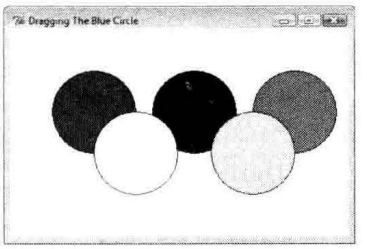
9.30　（显示一个长方体）编写程序显示一个长方体，如图 9-39a 所示。

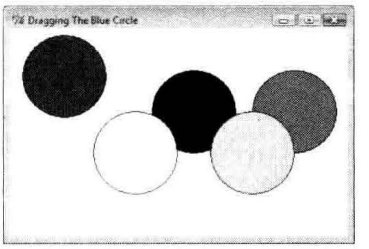
9.31　（显示 5 个填充的圆）编写程序显示 5 个填充的圆，如图 9-39b 所示。让用户可以用鼠标拖动蓝

色的圆，如图 9-39c 所示。



a）程序显示长方体





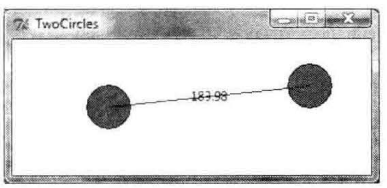
b）～c）使用鼠标拖动蓝色圆

图　9-39

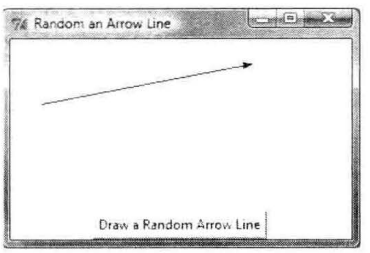
*9.32　（两个可移动顶点以及它们的距离）编写程序显示两个圆，圆心分别为 (20,20) 和 (120,50)，半径均为 20，使用一条线段连接两个圆，如图 9-40a 所示。圆之间的距离显示在线上。用户可以拖动一个圆。当拖动发生时，圆与线被移动并且更新圆之间的距离。你的程序不能让两个圆太接近。至少应该保持两个圆的中心在 70 个像素的距离。

**9.33　（绘制带箭头的线）编写程序，当单击“Draw a Random Arrow Line”按钮时随机绘制一条带箭头的线，如图 9-40b 所示。

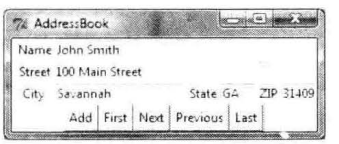
**9.34　（地址簿）编写程序，创建一个用户界面显示地址，如图 9-40c 所示。



a）用户可以拖动圆而程序可以重新显示距离



b）程序随机绘制一条带箭头的线



c）程序创建用户界面显示地址

图　9-40

第 10 章

Introduction to Programming Using Python

列　　表

学习目标

- 描述为什么列表在程序设计中很有用（第 10.1 节）。
- 学习如何创建列表（第 10.2.1 节）。
- 探究列表的常用操作（第 10.2.2 节）。
- 对列表使用 len、min、max、sum 和 random.shuffle 函数（第 10.2.3 节）。
- 使用下标变量访问列表元素（第 10.2.4 节）。
- 使用截取运算符 [start:end] 从一个较长的列表中获取子列表（第 10.2.5 节）。
- 在列表中使用 +（联接）、*（重复）和 in/not in 操作（第 10.2.6 节）。
- 使用 for 循环遍历一个列表中的元素（第 10.2.7 节）。
- 使用比较运算符比较两个列表的内容（第 10.2.8 节）。
- 使用列表解析来创建列表（第 10.2.9 节）。
- 调用列表的 append、count、extend、index、insert、pop、remove、reverse 和 sort 方法（第 10.2.10 节）。
- 使用 str 的 split 方法将一个字符串分成一个列表（第 10.2.11 节）。
- 从控制台中读取数据到列表（第 10.2.12 节）。
- 在应用程序开发中使用列表（第 10.3 ～ 10.5 节）。
- 将一个列表的内容复制到另外一个列表（第 10.6 节）。
- 开发和调用包含有列表参数且返回列表的函数（第 10.7 ～ 10.9 节）。
- 使用线性查找算法（第 10.10.1 节）或者二分查找算法（第 10.10.2 节）来查找元素。
- 使用选择排序法对列表进行排序（第 10.11.1 节）。
- 使用插入排序法对列表进行排序（第 10.11.2 节）。
- 使用列表开发一个弹球动画（第 10.12 节）。

10.1　引言

关键点：一个列表可以存储任意大小的数据集合。

程序一般都需要存储大量的数值。假设，举个例子，需要读取 100 个数字，计算出它们的平均值，然后找出多少个数字是高于这个平均值的。程序首先读取 100 个数字并计算它们的平均值，然后把每个数字和平均值进行比较来确定它是否超过了平均值。为了完成这个任务，这些数字都必须存储在变量内。为了这样做，你必须创建 100 个变量并且重复编写几乎同样的一段代码 100 次。显然，编写一个这样的程序是不切实际的。因此，这个问题该怎么解决？

我们需要一个高效、条理的方式。Python 提供了一种被称为列表的数据类型，它可以存储一个有序的元素集合。在例子中，可以把 100 个数字存储在一个列表中并且通过一个单独

的列表变量来访问它们。这个解决方案可能看起来如程序清单 10-1 所示。

程序清单 10-1 DataAnalysis.py

```
1  NUMBER_OF_ELEMENTS = 5 # For simplicity, use 5 instead of 100
2  numbers = [] # Create an empty list
3  sum = 0
4
5  for i in range(NUMBER_OF_ELEMENTS):
6      value = eval(input("Enter a new number: "))
7      numbers.append(value)
8      sum += value
9
10 average = sum / NUMBER_OF_ELEMENTS
11
12 count = 0 # The number of elements above average
13 for i in range(NUMBER_OF_ELEMENTS):
14     if numbers[i] > average:
15         count += 1
16
17 print("Average is", average)
18 print("Number of elements above the average is", count)
```

```
Enter a new number: 1 Enter
Enter a new number: 2 Enter
Enter a new number: 3 Enter
Enter a new number: 4 Enter
Enter a new number: 5 Enter
Average is 3.0
Number of elements above the average is 2
```

这个程序首先创建一个空列表（第 2 行）。它重复读取数字（第 6 行）并将其追加给列表（第 7 行），随后将它累加给 sum（第 8 行）。程序在第 10 行获取了 average。随后，列表中的每一个数字与平均值进行比较以统计数字值大于平均值的个数（第 12 ～ 15 行）。

注意：在很多其他程序设计语言中，也许会用到一个称作数组的数据类型来存储一个数据序列。数组有固定的大小。Python 列表的大小是可变的。它可以根据需求增加或缩小。

10.2 列表基础

关键点：列表是一个用 list 类定义的序列，它包括了创建、操作和处理列表的方法。列表中的元素可以通过下标来访问。

10.2.1 创建列表

list 类定义了列表。为了创建一个列表，可以使用 list 的构造方法，如下所示：

```
list1 = list() # Create an empty list
list2 = list([2, 3, 4]) # Create a list with elements 2, 3, 4
list3 = list(["red", "green", "blue"]) # Create a list with strings
list4 = list(range(3, 6)) # Create a list with elements 3, 4, 5
list5 = list("abcd") # Create a list with characters a, b, c, d
```

也可以使用下面这个更简单一些的语法来创建列表：

```
list1 = [] # Same as list()
list2 = [2, 3, 4] # Same as list([2, 3, 4])
list3 = ["red", "green"] # Same as list(["red", "green"])
```

列表中的元素用逗号分隔并且由一对中括号（[]）括住。

注意：一个列表既可以包含同样类型的元素也可以包括不同类型的元素。例如，下面的列表也是可以的：

```
list4 = [2, "three", 4]
```

10.2.2　列表是一种序列类型

Python 中的字符串和列表都是序列类型。一个字符串是一个字符序列，而一个列表则是任何元素的序列。序列的常用操作被总结在表 10-1 中。字符串的这些操作已经在第 8 章介绍过。对字符串的序列操作同样适用于列表。第 10.2.3 节到第 10.2.8 节给出在列表上运用这些操作符的例子。

表 10-1　序列 s 的常用操作

操作	描述
x in s	如果元素 x 在序列 s 中则返回 true
x not in s	如果元素 x 不在序列 s 中则返回 true
s1+s2	连接两个序列 s1 和 s2
s*n,n*s	n 个序列 s 的连接
s[i]	序列 s 的第 i 个元素
s[i:j]	序列 s 从下标 i 到 j-1 的片段
len(s)	序列 s 的长度，即 s 中的元素个数
min(s)	序列 s 中的最小元素
max(s)	序列 s 中的最大元素
sum(s)	序列 s 中所有元素之和
for loop	在 for 循环中从左到右反转元素
<、<=、>、>=、=、！=	比较两个序列

10.2.3　列表使用的函数

一些 Python 内嵌函数可以和列表一起使用。可以使用 len 函数返回列表的元素个数，使用 max/min 函数返回列表中的最大值元素和最小值元素，而 sum 函数返回列表中所有元素之和。还可以使用 random 模块中的 shuffle 函数随意排列列表中的元素。下面是一些例子：

```
>>> list1 = [2, 3, 4, 1, 32]
>>> len(list1)
5
>>> max(list1)
32
>>> min(list1)
1
>>> sum(list1)
42
>>> import random
>>> random.shuffle(list1) # Shuffle the elements in list1
>>> list1
[4, 1, 2, 32, 3]
>>>
```

调用 random.shuffle（list1）(第 11 行）随意排列 list1 中的元素。

10.2.4　下标运算符 []

一个列表中的元素都可以使用下面的语法通过下标操作符访问：

```
myList[index]
```

列表下标是基于 0 的，也就是说，下标的范围从 0 到 len(myList)-1，如图 10-1 中阐述：

myList[index] 可以像变量一样使用，所以它也被称为下标变量。例如：下面的代码将 myList[0] 与 myList[1] 中的值相加并赋给 myList[2]。

```
myList[2] = myList[0] + myList[1]
```

下面的循环将 0 赋值给 myList[0]、将 1 赋值给 myList[1]、…、将 9 赋值给 myList[9]：

```
for i in range(len(myList)):
    myList[i] = i

    myList = [5.6, 4.5, 3.3, 13.2, 4.0, 34.33, 34.0, 45.45, 99.993, 11123]
```

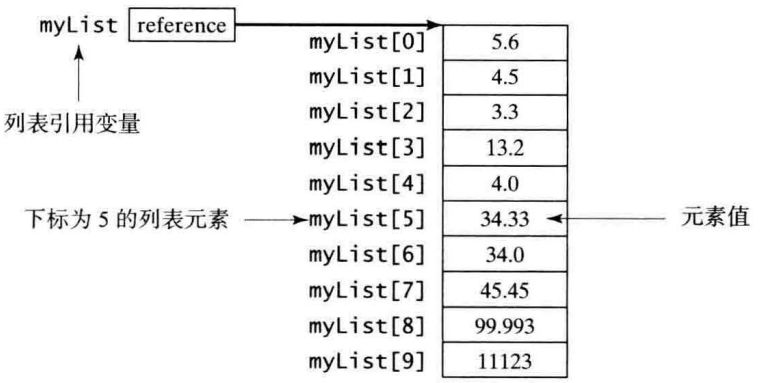


图 10-1　列表 myList 有 10 个下标从 0 到 9 的元素

警告：越界访问列表是一个常见的程序设计错误，它会导致一个运行时的“IndexError”。为了避免这种错误，要确保没有使用超出 len(myList)-1 的下标。

程序员经常会错误地使用下标 1 来引用列表的第一个元素，但其实它应该是 0。这称作“差一错误”。在循环中应该使用“<”的地方使用“<=”也是常见错误。例如，下面的循环是错误的：

```
i = 0
while i <= len(myList):
    print(myList[i])
    i += 1
```

应该用“<”替换“<=”。

Python 也允许使用负数作为下标来引用相对于列表末端的位置。将列表长度和负数下标相加就可以得到实际的位置。例如：

```
1  >>> list1 = [2, 3, 5, 2, 33, 21]
2  >>> list1[-1]
3  21
4  >>> list1[-3]
5  2
6  >>>
```

在第 2 行，list1[−1] 和 list[−1+len(list1)] 一样，它们都给出列表的最后一个元素。在第 4 行，list1[−3] 和 list1[−3+len(list1)] 一样，它们都给出列表的倒数第三个元素。

10.2.5 列表截取 [start:end]

下标运算符允许选择一个指定下标位置上的元素。而截取操作使用语法 list[start:end] 返回列表的一个片段。这个片段是下标从 start 到 end−1 的元素构成的一个子列表。下面是一些例子：

```
1  >>> list1 = [2, 3, 5, 7, 9, 1]
2  >>> list1[2 : 4]
3  [5, 7]
4  >>>
```

起始下标和结尾下标是可以省略的。在这种情况下，起始下标为 0 而结尾下标是最后一个下标。例如：

```
1  >>> list1 = [2, 3, 5, 2, 33, 21]
2  >>> list1[ : 2]
3  [2, 3]
4  >>> list1[3 : ]
5  [2, 33, 21]
6  >>>
```

注意：list1[:2] 和 list1[0:2](第二行) 是一样的，而 lisr1[3:] 和 list1[3:len(list1)](第 4 行) 是一样的。

可以在截取过程中使用负数下标。例如：

```
1  >>> list1 = [2, 3, 5, 2, 33, 21]
2  >>> list1[1 : -3]
3  [3, 5]
4  >>> list1[-4 : -2]
5  [5, 2]
6  >>>
```

在第 2 行，list1[1:−3] 和 list1[1:−3+len(list1)] 一样。在第 4 行，list1[−4:−2] 和 list1[−4+len (list1):−2+len(list1)] 一样。

注意：如果 start>=end，那么 list[start:end] 将返回一个空表。如果 end 指定了一个超出列表结尾的位置，那么 Python 会将使用列表长度替代 end。

10.2.6 +、* 和 in/not in 运算符

可以使用连接运算符（+）来组合两个列表，使用复制运算符（*）复制列表中的元素。下面是一些例子：

```
 1  >>> list1 = [2, 3]
 2  >>> list2 = [1, 9]
 3  >>> list3 = list1 + list2
 4  >>> list3
 5  [2, 3, 1, 9]
 6  >>>
 7  >>> list4 = 3 * list1
 8  >>> list4
 9  [2, 3, 2, 3, 2, 3]
10  >>>
```

通过连接 list1 和 list2 就会得到一个新列表（第 3 行）。第 7 行将 list1 复制三次以创建一个新列表。注意：3*list1 和 list1*3 相同。

可以使用 in 或者 not in 运算符来判断一个元素是否在列表中。例如：

```
>>> list1 = [2, 3, 5, 2, 33, 21]
>>> 2 in list1
True
>>> 2 not in list1
False
>>>
```

10.2.7 使用 for 循环遍历元素

Python 列表中的元素是可迭代的。Python 支持一种便利的 for 循环，它可以让你在不使

用下标变量的情况下顺序遍历列表。例如，下面的代码显示列表 mylist 中的所有元素。

```
for u in myList:
    print(u)
```

可以这样读代码："对于 mylist 中的每个元素 u，输出它。"

如果希望以不同的顺序遍历列表或者改变列表中的元素，那么仍然必须使用下标变量。例如，下面的代码显示奇数位置上的元素。

```
for i in range(0, len(myList), 2):
    print(myList[i])
```

10.2.8 比较列表

可以使用比较运算符（>、>=、<、<=、==、!=）对列表进行比较。为了进行比较，两个列表必须包含同样类型的元素。比较使用的是字典顺序：首先比较前两个元素，如果它们不同就决定了比较的结果；如果它们相同，那就继续比较接下来两个元素，一直重复这个过程，直到比较完所有的元素。下面是一些示例。

```
>>> list1 = ["green", "red", "blue"]
>>> list2 = ["red", "blue", "green"]
>>> list2 == list1
False
>>> list2 != list1
True
>>> list2 >= list1
True
>>> list2 > list1
True
>>> list2 < list1
False
>>> list2 <= list1
False
>>>
```

10.2.9 列表解析

列表解析提供了一种创建顺序元素列表的简洁方式。一个列表解析由多个方括号组成，方括号内包含后跟一个 for 子句的表达式，之后是 0 或多个 for 或 if 子句。列表解析可以产生一个由表达式求值结果组成的列表。这里是一些例子。

```
1  >>> list1 = [x for x in range(5)] # Returns a list of 0, 1, 2, 3, 4
2  >>> list1
3  [0, 1, 2, 3, 4]
4  >>>
5  >>> list2 = [0.5 * x for x in list1]
6  >>> list2
7  [0.0, 0.5, 1.0, 1.5, 2.0]
8  >>>
9  >>> list3 = [x for x in list2 if x < 1.5]
10 >>> list3
11 [0.0, 0.5, 1.0]
12 >>>
```

在第 1 行，使用一个 for 子句表达式创建 list1。list1 中的数字是 0、1、2、3 和 4。list2 中的每一个数字都是 list1 中对应数字的一半（第 5 行）。在第 9 行，list3 包含了 list2 中那些

值小于 1.5 的数字。

10.2.10 列表方法

一旦列表被创建，可以使用 list 类的方法（如图 10-2 所示）来操作列表。

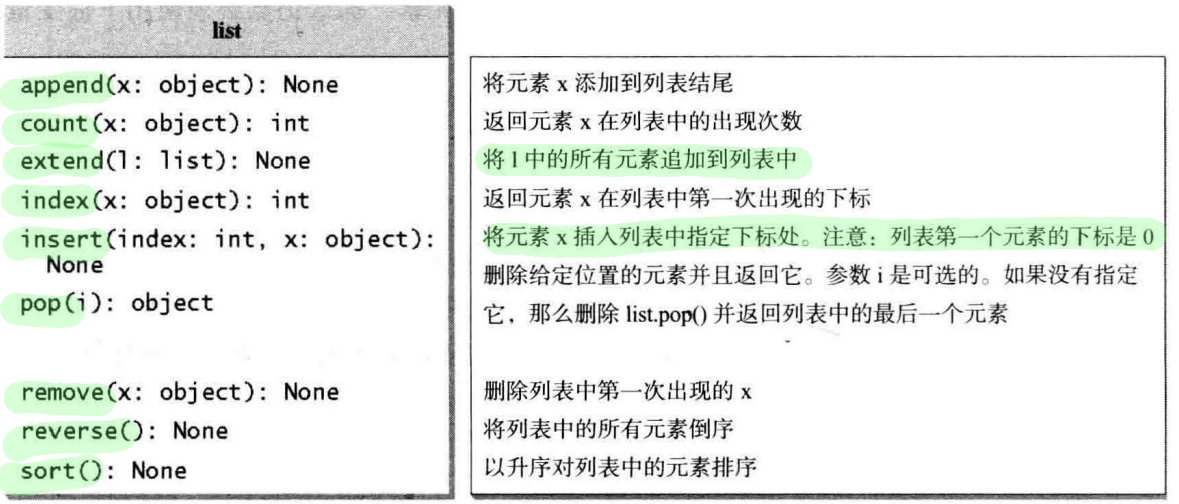


图 10-2　list 类包含操作列表的方法

下面是一些使用 append、count、extend、index 和 insert 方法的例子。

```
>>> list1 = [2, 3, 4, 1, 32, 4]
>>> list1.append(19)
>>> list1
[2, 3, 4, 1, 32, 4, 19]
>>> list1.count(4) # Return the count for number 4
2
>>> list2 = [99, 54]
>>> list1.extend(list2)
>>> list1
[2, 3, 4, 1, 32, 4, 19, 99, 54]
>>> list1.index(4) # Return the index of number 4
2
>>> list1.insert(1, 25) # Insert 25 at position index 1
>>> list1
[2, 25, 3, 4, 1, 32, 4, 19, 99, 54]
>>>
```

第 2 行将 19 追加到列表中，而第 5 行返回了元素 4 在列表中的出现次数。调用 list1.extend()（第 8 行）将 list2 追加到 list1。第 11 行返回列表中元素 4 的下标，而第 13 行将 25 插入到列表中下标 1 的位置上。

下面是一些使用 insert、pop、remove、reverse 和 sort 方法的例子。

```
>>> list1 = [2, 25, 3, 4, 1, 32, 4, 19, 99, 54]
>>> list1.pop(2)
3
>>> list1
[2, 25, 4, 1, 32, 4, 19, 99, 54]
>>> list1.pop()
54
```

```
 8  >>> list1
 9  [2, 25, 4, 1, 32, 4, 19, 99]
10  >>> list1.remove(32) # Remove number 32
11  >>> list1
12  [2, 25, 4, 1, 4, 19, 99]
13  >>> list1.reverse() # Reverse the list
14  >>> list1
15  [99, 19, 4, 1, 4, 25, 2]
16  >>> list1.sort() # Sort the list
17  >>> list1
18  [1, 2, 4, 4, 19, 25, 99]
19  >>>
```

第 2 行将下标 2 的元素从列表中移除。调用 list1.pop()（第 6 行）返回和移除 list1 的最后一个元素。第 10 行从 list1 中移除元素 23，第 13 行倒置列表中的元素，而第 15 行对列表中的元素进行升序排列。

10.2.11 将字符串分成列表

str 类包括了 split 方法，它对于将字符串中的条目分成列表是非常有用的。例如，下面的语句：

```
items = "Jane John Peter Susan".split()
```

就会将字符串“Jane John Peter Susan”分离成列表 ['Jane', 'John', 'Peter', 'Susan']。在这种情况下，字符串中的条目是被空格分隔的。可以使用一个非空格的限定符。例如，下面的语句：

```
items = "09/20/2012".split("/")
```

将字符串“09/20/2012”分成了列表 ['09', '20', '2012']。

注意：Python 支持正则表达式，它是一种使用模式来匹配和分隔字符串的最有效且最有力的特征。正则表达式对于初学者来讲是复杂的。因此，我们将在补充材料 II.A 正则表达式中涉及这些内容。

10.2.12 输入列表

可能经常需要编写代码从控制台将数据读入列表。可以在循环里每一行输入一个数据条目并将它追加到列表。例如：下面的代码将 10 个数字读入一个列表，每一行读一个数字。

```
lst = [] # Create a list
print("Enter 10 numbers: ")
for i in range(10):
    lst.append(eval(input()))
```

有时候在一行中以空格分隔数据会更加方便。可以使用字符串的 split 方法从一行输入中提取数据。例如：下面的代码从一行读取 10 个空格分隔的数给列表。

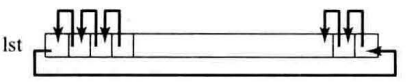
```
# Read numbers as a string from the console
s = input("Enter 10 numbers separated by spaces from one line: ")
items = s.split() # Extract items from the string
lst = [eval(x) for x in items] # Convert items to numbers
```

调用 input() 来读取一个字符串。使用 s.split() 来提取字符串 s 中被空格分隔的条目并返回列表中的条目。最后一行通过将条目转化成数字来创建一个数字列表。

10.2.13 对列表移位

有时候，需要将列表中的元素向左或向右移动。Python 并没有在 list 类中提供这样的方法，但是可以编写下面的函数来实现向左移。

```
def shift(lst):
    temp = lst[0] # Retain the first element

    # Shift elements left
    for i in range(1, len(lst)):
        lst[i - 1] = lst[i]

    # Move the first element to fill in the last position
    lst[len(lst) - 1] = temp
```



10.2.14 简化代码

列表可以大大简化某些任务的。例如：假设你希望通过给定的月份数字来得到月份的英文名。如果月份名被存储在一个列表中，那么给定月份的名字可以简单地通过下标访问。下面的代码提示用户输入月份数，然后显示它的月份名：

```
months = ["January", "February", "March", ..., "December"]
monthNumber = eval(input("Enter a month number (1 to 12): "))
print("The month is", months[monthNumber - 1])
```

如果不使用 months 列表，你就只能使用一个冗长的多重 if-else 语句来决定月份名，如下所示：

```
if monthNumber == 0:
    print("The month is January")
elif monthNumber == 1:
    print("The month is February")
...
else:
    print("The month is December")
```

检查点

10.1　如何创建一个空表以及有三个整数 1、32 和 2 的列表？

10.2　假设 lst = [30,1,12,14,10,0]，那么 lst 中有多少个元素？ lst 中的第一个元素的下标是什么？ lst 中的最后一个元素的下标是什么？ lst[2] 是什么？ lst[−2] 是什么？

10.3　假设 lst = [30,1,2,1,0]，在应用下面的每条语句之后列表变成了什么？假设每行代码都是独立的。

```
lst.append(40)
lst.insert(1, 43)
lst.extend([1, 43])
lst.remove(1)
lst.pop(1)
lst.pop()
lst.sort()
lst.reverse()
random.shuffle(lst)
```

10.4　假设 lst = [30,1,2,1,0]，下面每条语句的返回值是什么？

```
lst.index(1)
lst.count(1)
len(lst)
max(lst)
```

```
min(lst)
sum(lst)
```

10.5 假设 list1 = [30,1,2,1,0] 而 list2 = [1,21,13]，下面每条语句的返回值是什么？

```
list1 + list2
2 * list2
list2 * 2
list1[1 : 3]
list1[3]
```

10.6 假设 lst = [30,1,2,1,0]，下面每条语句的返回值是什么？

```
[x for x in list1 if x > 1]
[x for x in range(0, 10, 2)]
[x for x in range(10, 0, -2)]
```

10.7 假设 list1 = [30,1,2,1,0] 而 list2 = [1,21,13]，下面每条语句的返回值是什么？

```
list1 < list2
list1 <= list2
list1 == list2
list1 != list2
list1 > list2
list1 >= list2
```

10.8 指出下面语句是真还是假：

(a) 列表中的每个元素必须是相同类型。

(b) 在创建列表之后，它的大小是固定的。

(c) 列表可以有重复的元素。

(d) 列表中的元素可以通过下标运算符访问。

10.9 在执行完下面的代码行之后 list1 和 list2 是什么？

```
list1 = [1, 43]
list2 = list1
list1[0] = 22
```

10.10 在执行完下面的代码行之后 list1 和 list2 是什么？

```
list1 = [1, 43]
list2 = [x for x in list1]
list1[0] = 22
```

10.11 如何从字符串中获取一个列表？假设 s1 是 welcome。那么 s1.split('o') 是什么？

10.12 编写语句实现：

(a) 创建含 100 个布尔 False 值的列表。

(b) 给列表最后一个元素赋值 5.5。

(c) 显示前两个元素之和。

(d) 计算列表前五个元素的和。

(e) 找出列表的最小元素。

(f) 随机产生一个下标并显示列表中这个下标的元素。

10.13 当你的程序试图访问列表中非法下标元素会发生什么？

10.14 下面代码的输出是什么？

```
lst = [1, 2, 3, 4, 5, 6]

for i in range(1, 6):
```

```
        lst[i] = lst[i - 1]

    print(lst)
```

10.3　实例研究：乐透数

关键点：编写一个程序决定输入的数字是否涵盖了 1 到 99 之间所有的数。

“选 10 乐透”的每张彩票都有 10 个独特的范围在 1 到 99 之间的数字，假如你买了很多彩票并且希望它们能够涵盖 1 到 99 的所有数字。编写一个程序从一个文件读取彩票的数字并且判断是否涵盖所有数字。假定文件里最后一个数字是 0。假如文件包含如下这些数字：

```
80 3 87 62 30 90 10 21 46 27
12 40 83 9 39 88 95 59 20 37
80 40 87 67 31 90 11 24 56 77
11 48 51 42 8 74 1 41 36 53
52 82 16 72 19 70 44 56 29 33
54 64 99 14 23 22 94 79 55 2
60 86 34 4 31 63 84 89 7 78
43 93 97 45 25 38 28 26 85 49
47 65 57 67 73 69 32 71 24 66
92 98 96 77 6 75 17 61 58 13
35 81 18 15 5 68 91 50 76
0
```

你的程序应该显示：

```
The tickets cover all numbers
```

假如文件包含数字：

```
11 48 51 42 8 74 1 41 36 53
52 82 16 72 19 70 44 56 29 33
0
```

你的程序应该显示：

```
The tickets don't cover all numbers
```

如何标记一个数字是被涵盖的？你可以创建一个拥有 99 个布尔元素的列表。列表中的每一个元素被用来标记一个数字是否被涵盖。假如这个列表是 isCovered。初始状态下，每个元素都是 Flase，如图 10-3a 所示。每当读取一个数字时，它对应的元素被设置成 True。假定输入的数字是 1、2、3、99 和 0。当数字 1 被读取时，isCovered[0] 被设置成 True（参见图 10-3b）。当数字 2 被读取时，isCovered[2−1] 被设置成 True（参见图 10-3c）。当数字 3 被读取时，isCovered[3−1] 被设置成 True（参见图 10-3d）。当数字 99 被读取时，isCovered[98] 被设置成 True（参见图 10-3e）。

这个程序的算法可以如下描述：

```
for each number k read from the file,
    mark number k as covered by setting isCovered[k - 1] true

if every isCovered[i] is true:
    The tickets cover all numbers
else:
    The tickets don't cover all numbers
```

程序清单 10-2 给出完整的程序。

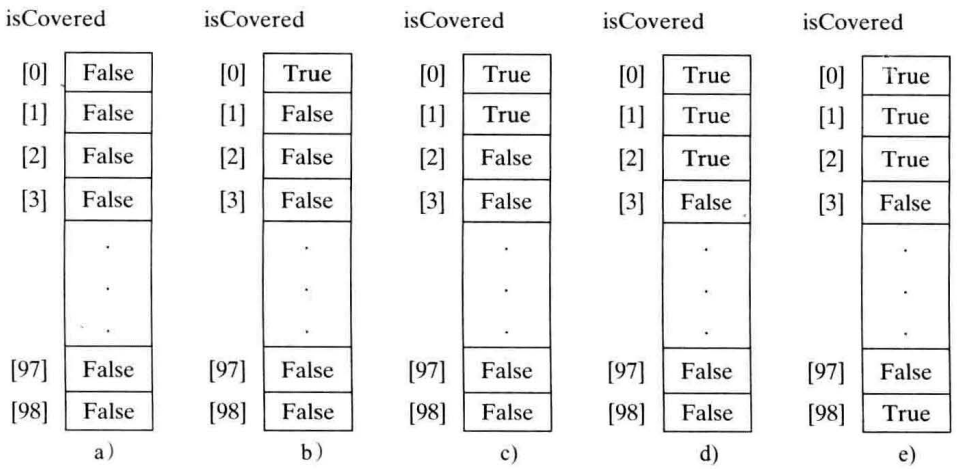


图 10-3 如果数字 i 出现在乐透彩票中，isCovered[i−1] 就被设置为真

程序清单 10-2 LottoNumbers.py

```
# Create a list of 99 Boolean elements with value False
isCovered = 99 * [False]
endOfInput = False
while not endOfInput:
    # Read numbers as a string from the console
    s = input("Enter a line of numbers separated by spaces: ")
    items = s.split() # Extract items from the string
    lst = [eval(x) for x in items] # Convert items to numbers

    for number in lst:
        if number == 0:
            endOfInput = True
        else:
            # Mark its corresponding element covered
            isCovered[number - 1] = True

# Check whether all numbers (1 to 99) are covered
allCovered = True # Assume all covered initially
for i in range(99):
    if not isCovered[i]:
        allCovered = False  # Find one number not covered
        break

# Display result
if allCovered:
    print("The tickets cover all numbers")
else:
    print("The tickets don't cover all numbers")
```

```
Enter a line of numbers separated by spaces: 2 5 6 5 4 3 Enter
Enter a line of numbers separated by spaces: 23 43 2 0 Enter
The tickets don't cover all numbers
```

```
Enter a line of numbers separated by spaces: 1 2 3 4 5 6 Enter
Enter a line of numbers separated by spaces: 7 8 9 10 11 Enter
...
The tickets cover all numbers
```

假如已经创建了一个名为 LottoNumbers.txt 的包含下面输入数据的文本文件：2 5 6 5 4

3 23 43 2 0

可以在命令行窗口使用下面的命令来运行程序。

```
python LottoNumbers.py < LottoNumbers.txt
```

这个程序创建了一个拥有 99 个初始化为 False 布尔值的列表（第 2 行）。它重复地读取一行数字（第 6 行），并从行中提取这些数字（第 7 ～ 8 行）。对每个数字，这个程序在循环中执行下面的操作：

- 如果这个数字为 0，则设置 endOfInput 为 True（第 12 行）。
- 如果这个数字不为 0，则设置 isCovered 相应的值为 True（第 15 行）。

如果这个数字是 0，则输入终止（第 4 行）。这个程序在 18 ～ 22 行判断是否涵盖所有的数字，然后在第 25 ～ 28 行显示结果。

10.4　实例研究：一副扑克牌

关键点：编写一个从 52 张扑克牌中随机抽取 4 张牌的程序。

所有的牌可以用一个名为 deck 的列表表示，列表填充的初始值从 0 到 51，如下所示。

```
deck = [x for x in range(52)]
```

或者也可以使用：

```
deck = list(range(52))
```

牌的数字 0 到 12、13 到 25、26 到 38 以及 39 到 51 分别代表 13 个黑桃、13 个红桃、13 个方块和 13 个梅花，如图 10-4 所示。cardNumber//13 决定这张牌属于哪个花色，而 cardNumber%13 决定这张牌的大小，如图 10-5 所示。在洗牌之后，从牌堆中选出前 4 张牌。并由程序显示出这四张牌。

程序清单 10-3 给出这个问题的解决方案。

程序清单 10-3　DeckOfCards.py

```
1  # Create a deck of cards
2  deck = [x for x in range(52)]
3
4  # Create suits and ranks lists
5  suits = ["Spades", "Hearts", "Diamonds", "Clubs"]
6  ranks = ["Ace", "2", "3", "4", "5", "6", "7", "8", "9",
7        "10", "Jack", "Queen", "King"]
8
9  # Shuffle the cards
10 import random
11 random.shuffle(deck)
12
13 # Display the first four cards
14 for i in range(4):
15     suit = suits[deck[i] // 13]
16     rank = ranks[deck[i] % 13]
17     print("Card number", deck[i], "is the", rank, "of", suit)
```

```
Card number 6 is the 7 of Spades
Card number 48 is the 10 of Clubs
Card number 11 is the Queen of Spades
Card number 24 is the Queen of Hearts
```

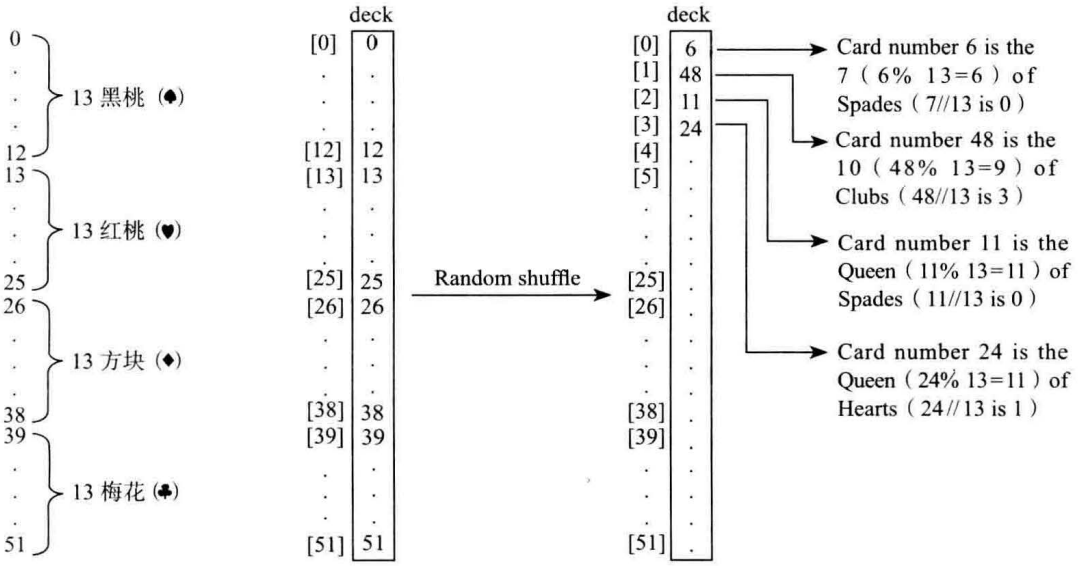


图 10-4　52 张牌被存储在一个名为 deck 的列表中

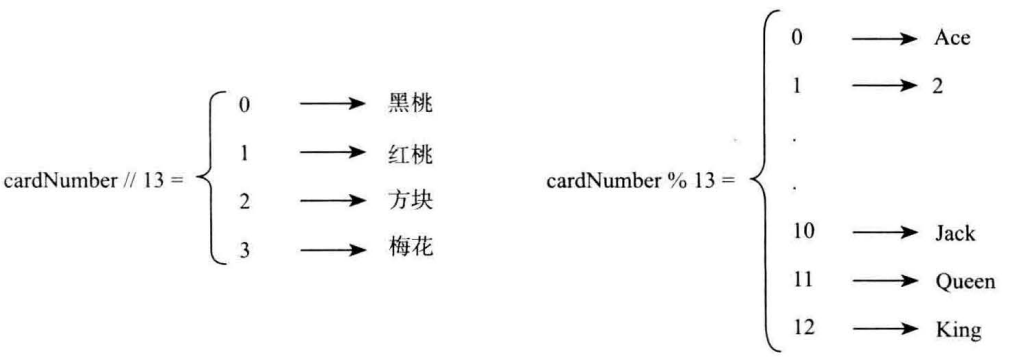


图 10-5　一个牌数确定一张牌

这个程序创建了一幅 52 张的牌（第 2 行），列表 suits 对应四种花色（第 5 行），而列表 ranks 对应一个花色的 13 张牌（第 6 ～ 7 行）。suits 和 ranks 中的元素类型是字符串。

deck 被初始化为从 0 到 51 的值。牌值 0 表示黑桃 A，1 表示黑桃 2，13 表示红桃 A，而 14 表示红桃 2。

第 10 ～ 11 行对这副牌进行随意洗牌。在洗牌之后，deck[i] 包含一个任意值，deck[i]//13 是 0、1、2 或者 3，它决定了花色（第 15 行）；deck[i]%13 是一个 0 到 12 之间的值，它决定了牌值（第 16 行）。

如果列表 suits 没有被定义，那么就必须通过使用一个冗长的 if 语句来判断，如下所示。

```
if deck[i] // 13 == 0:
    print("suit is Spades")
elif deck[i] // 13 == 1:
    print("suit is Hearts")
elif deck[i] // 13 == 2:
    print("suit is Diamonds")
else:
    print("suit is Clubs")
```

由于 suits = ["spades", "hearts", "diamonds", "clubs"] 定义了一个列表，所以 suits [deck//13] 给出 deck 的花色。使用列表大大简化了这个问题的解题程序。

10.5　扑克牌图形用户界面

关键点：本程序实现从 52 张扑克牌中随机抽取 4 张牌并显示这些牌。

这里给出一个图形用户界面程序，单击“Shuffle”按钮，在控制台上显示 4 张随机牌的图像，如图 10-6 所示。

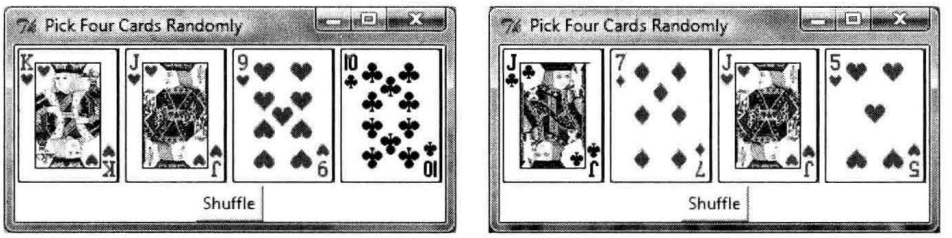


图 10-6　单击“Shuffle”按钮随机显示四张牌

在 Python 中，可以使用 Turtle 或 Tkinter 开发一个图形用户界面程序。Turtle 是一个介绍程序设计基础的很好的教学工具，但是它的能力局限在绘制线条、图形和文本字符串。对于开发复杂的图形用户界面项目，则应该用 Tkinter。从现在开始，我们将在图形用户界面示例中使用 Tkinter。程序清单 10-4 给出创建一个“Shuffle”按钮并随机显示四张牌的图形用户界面程序。

程序清单 10-4　DeckOfCardsGUI.py

```
from tkinter import * # Import all definitions from tkinter
import random

class DeckOfCardsGUI:
    def __init__(self):
        window = Tk() # Create a window
        window.title("Pick Four Cards Randomly") # Set title

        self.imageList = []  # Store images for cards
        for i in range(1, 53):
            self.imageList.append(PhotoImage(file = "image/card/"
                    + str(i) + ".gif"))

        frame = Frame(window) # Hold four labels for cards
        frame.pack()

        self.labelList = [] # A list of four labels
        for i in range(4):
            self.labelList.append(Label(frame,
                image = self.imageList[i]))
            self.labelList[i].pack(side = LEFT)

        Button(window, text = "Shuffle",
            command = self.shuffle).pack()

        window.mainloop() # Create an event loop

    # Choose four random cards
    def shuffle(self):
        random.shuffle(self.imageList)
        for i in range(4):
            self.labelList[i]["image"] = self.imageList[i]

DeckOfCardsGUI() # Create GUI
```

从存储在程序当前目录下的 image/card 文件夹的图像文件中创建 52 个图像（第 9 ～ 12 行）。这些文件被命名为 1.gif、2.gif、……、52.gif。这些图像被添加到 imagelist。每个图像都是 PhotoImage 类的一个实例。

程序创建了一个容纳四个标签的框架（第 14 ～ 15 行）。这四个标签被添加到 labellist(第 17 ～ 21 行)。

程序创建一个按钮（第 23 行）。当单击按钮时，就会调用“ shuffle”函数将图像列表随意打乱（第 30 行）并将列表中的前四个图像设置为标签（第 31 ～ 32 行）。

10.6 复制列表

关键点：为了将一个列表中的数据复制给另一个列表，必须将元素逐个地从源列表复制到目标列表。

经常需要在程序中复制一个列表或列表的一部分。在某些情况下，可能会尝试使用赋值语句（=），如下所示：

```
list2 = list1
```

但是，这条语句不会将 list1 引用的列表内容复制给 list2；事实上，它仅仅将 list1 引用值赋给 list2。在这条语句之后，list1 和 list2 都将指向同一个列表，如图 10-7 所示。list2 之前指向的列表将不再被引用，它就变成了垃圾（garbage）。list2 所占用的内存空间将被自动收集起来被 Python 编译器重新使用。

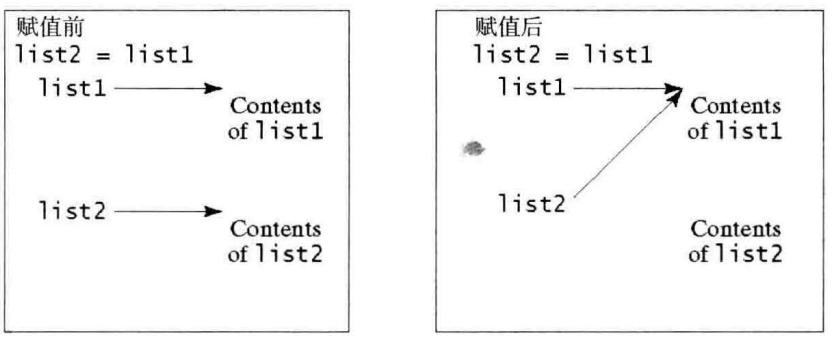


图 10-7 在赋值语句之前，list1 和 list2 指向各自的内存位置。在赋值之后，list1 的引用值被传递给 list2

下面是一个阐明概念的例子：

```
 1  >>> list1 = [1, 2]
 2  >>> list2 = [3, 4, 5]
 3  >>> id(list1)
 4  36207312
 5  >>> id(list2)
 6  36249848
 7  >>>
 8  >>> list2 = list1
 9  >>> id(list2)
10  36207312
11  >>>
```

两个列表被创建（第 1 ～ 2 行）并且每个列表都有不同 id 的独立对象（第 4 行和第 6 行）。在将 list1 赋值给 list2 之后，list2 和 list1 的 id 相同（第 10 行）。现在，list1 和 list2 指

向同一个对象。

为了将 list1 完全相同地复制给 list2，可以使用：

```
list2 = [x for x in list1]
```

或者简化为：

```
list2 = [] + list1
```

检查点

10.15　下面代码的输出是什么？

```
list1 = list(range(1, 10, 2))
list2 = list1
list1[0] = 111
print(list1)
print(list2)
```

10.16　下面代码的输出是什么？

```
list1 = list(range(1, 10, 2))
list2 = [] + list1
list1[0] = 111
print(list1)
print(list2)
```

10.7　将列表传递给函数

关键点：当列表被传递给函数时，由于列表是一个可变对象，所以列表的内容可能会在函数调用后改变。

因为列表是一个对象，所以将列表传递给函数就像给函数传递一个对象。例如：下面的函数显示列表中的元素。

```
def printList(lst):
    for element in lst:
        print(element)
```

可以通过传递列表来调用它。例如：下面的语句调用 printlist 函数显示 3、1、2、6、4 和 2。

```
printList([3, 1, 2, 6, 4, 2])
```

注意：前面的语句创建了一个列表，然后把它传递给函数。这里没有显示指向列表的引用变量。这样的列表被称作匿名列表。

因为列表是可变对象，所以列表的内容可能会在函数内改变。例如，采用程序清单 10-5 的代码。

程序清单 10-5　PassListArgument.py

```
1  def main():
2      x = 1 # x is an int variable
3      y = [1, 2, 3] # y is a list
4
5      m(x, y) # Invoke m with arguments x and y
6
7      print("x is", x)
8      print("y[0] is", y[0])
9
10 def m(number, numbers):
```

```
11      number = 1001 # Assign a new value to number
12      numbers[0] = 5555 # Assign a new value to numbers[0]
13
14  main() # Call the main function
```

```
x is 1
y[0] is 5555
```

在这个示例中，你可以看到在 m 被调用后（第 5 行），x 保持为 1，但 y[0] 被改变为 5555。这是因为 y 和 numbers 都指向同一个列表对象。当 m（x，y）被调用时，x 和 y 的引用值被传递给 number 和 numbers。由于 y 包含指向列表的引用值，现在，numbers 包含的就是指向同一列表的相同引用值。由于 number 是不可变的，所以在一个函数里改变它会创建一个新实例，而函数外的原始实例并没有被改变。所以，在函数外面 x 仍然是 1。

另一个需要我们解决的问题是将列表作为一个默认参数。考虑程序清单 10-6 中的代码。

程序清单 10-6 DefaultListArgument.py

```
1  def add(x, lst = []):
2      if x not in lst:
3          lst.append(x)
4
5      return lst
6
7  def main():
8      list1 = add(1)
9      print(list1)
10
11     list2 = add(2)
12     print(list2)
13
14     list3 = add(3, [11, 12, 13, 14])
15     print(list3)
16
17     list4 = add(4)
18     print(list4)
19
20 main()
```

```
[1]
[1, 2]
[11, 12, 13, 14, 3]
[1, 2, 4]
```

如果 x 不在列表中，那么函数 add 将 x 追加给列表 lst（第 1 ～ 5 行）。当函数第一次执行时（第 8 行），参数 lst 的默认值 [] 被创建。这个默认值只会被创建一次。add(1) 将 1 加到 lst。

当函数被再次调用时（第 11 行），lst 是 [1] 而不是 []，因为 lst 只被创建一次。在 add(2) 被执行时，lst 就变成 [1,2]。

在第 14 行，给出列表参数 [11,12,13,14]，并且将这个列表传递给 lst。

在第 17 行，默认列表参数被使用。因为默认列表现在是 [1,2]，所以在调用 add(4) 之后，默认列表变成 [1,2,4]。

如果想要默认列表在每次函数调用时都是 []，可以像程序清单 10-7 那样修改函数。

程序清单 10-7 DefaultNoneListArgument.py

```
1  def add(x, lst = None):
```

```
2       if lst == None:
3           lst = []
4       if x not in lst:
5           lst.append(x)
6
7       return lst
8
9   def main():
10      list1 = add(1)
11      print(list1)
12
13      list2 = add(2)
14      print(list2)
15
16      list3 = add(3, [11, 12, 13, 14])
17      print(list3)
18
19      list4 = add(4)
20      print(list4)
21
22  main()
```

```
[1]
[2]
[11, 12, 13, 14, 3]
[4]
```

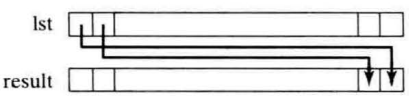
每次 add 函数被调用且没有列表参数时，这里都会创建一个新的空列表（第 3 行）。如果调用函数时已给出列表参数，那就不使用默认列表。

10.8 从函数返回一个列表

关键点：当函数返回一个列表时，就会返回这个列表的引用值。

在调用函数时可以传递列表参数。函数也可以返回列表。例如：下面的函数返回了一个列表，它是另一个列表倒置的结果。

```
1   def reverse(lst):
2       result = []
3
4       for element in lst:
5           result.insert(0, element)
6
7       return result
```



第 2 行创建一个新列表 result。第 4 ～ 5 行将名为 lst 的列表中的元素复制给名为 result 的列表。第 7 行返回这个列表。例如：下面的语句返回元素为 6、5、4、3、2 和 1 的新列表 list2。

```
list1 = [1, 2, 3, 4, 5, 6]
list2 = reverse(list1)
```

注意：list 类有 reverse() 方法，可以调用它来倒置一个列表。

检查点

10.17　真还是假？当传递一个列表给函数时，会创建一个新列表传递给函数。

10.18　给出下面两段程序的输出：

```
def main():
    number = 0
    numbers = [10]

    m(number, numbers)

    print("number is", number,
        "and numbers[0] is",
        numbers[0])

def m(x, y):
    x = 3
    y[0] = 3

main()
```

a)

```
def main():
    lst = [1, 2, 3, 4, 5]
    reverse(lst)
    for value in lst:
        print(value, end = ' ')

def reverse(lst):
    newLst = len(lst) * [0]

    for i in range(len(lst)):
        newLst[i] = lst[len(lst) - 1 - i]

    lst = newLst

main()
```

b)

10.19 给出下面两段程序的输出：

```
def main():
    list1 = m(1)
    print(list1)
    list2 = m(1)
    print(list2)

def m(x, lst = [1, 1, 2, 3]):
    if x in lst:
        lst.remove(x)
    return lst

main()
```

a)

```
def main():
    list1 = m(1)
    print(list1)
    list2 = m(1)
    print(list2)

def m(x, lst = None):
    if lst == None:
        lst = [1, 1, 2, 3]

    if x in lst:
        lst.remove(x)
    return lst

main()
```

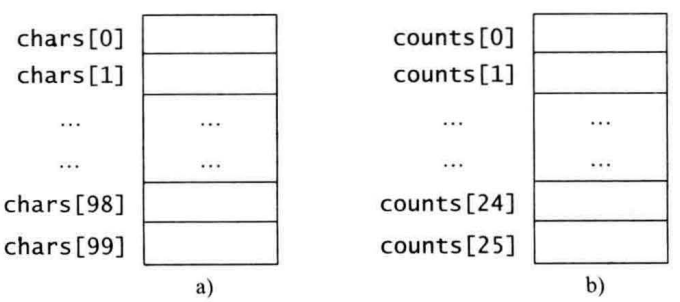
b)

10.9 实例研究：统计每个字母的出现次数

关键点：本节的程序实现统计 100 个字母中每个字母的出现次数。

程序清单 10-8 给出统计每个字母在一个字符列表里出现次数的程序，这个程序如下所示。

1）随机生成 100 个小写字母并且把它们赋值给一个名为 chars 的字符列表，如图 10-8a 所示。可以使用程序清单 6-11 里的 randomcharacter 模块中的 getRandomLowerCaseLetter() 函数获取一个随机字母。



a) b)

图 10-8 列表 chars 存储了 100 个字符，而 counts 列表存储了 26 个计数器，每一个都记录了一个字母的出现次数

2）统计列表中每个字母的出现次数。为了做到这样，创建一个有 26 个 int 值的名为 counts 的列表，每一个都对应一个字母的出现次数，如图 10-8b 所示。也就是说，counts[0] 是字母 a 的出现次数，count[1] 是字母 b 的出现次数，依此类推。

程序清单 10-8 CountLettersInList.py

```
import RandomCharacter # Defined in Listing 6.11

def main():
    # Create a list of characters
    chars = createList()

    # Display the list
    print("The lowercase letters are:")
    displayList(chars)

    # Count the occurrences of each letter
    counts = countLetters(chars)

    # Display counts
    print("The occurrences of each letter are:")
    displayCounts(counts)

# Create a list of characters
def createList():
    # Create an empty list
    chars = []

    # Create lowercase letters randomly and add them to the list
    for i in range(100):
        chars.append(RandomCharacter.getRandomLowerCaseLetter())

    # Return the list
    return chars

# Display the list of characters
def displayList(chars):
    # Display the characters in the list with 20 on each line
    for i in range(len(chars)):
        if (i + 1) % 20 == 0:
            print(chars[i])
        else:
            print(chars[i], end = ' ')

# Count the occurrences of each letter
def countLetters(chars):
    # Create a list of 26 integers with initial value 0
    counts = 26 * [0]

    # For each lowercase letter in the list, count it
    for i in range(len(chars)):
        counts[ord(chars[i]) - ord('a')] += 1

    return counts

# Display counts
def displayCounts(counts):
    for i in range(len(counts)):
        if (i + 1) % 10 == 0:
            print(counts[i], chr(i + ord('a')))
        else:
            print(counts[i], chr(i + ord('a')), end = ' ')

main() # Call the main function
```

```
The lowercase letters are:
e y l s r i b k j v j h a b z n w b t v
s c c k r d w a m p w v u n q a m p l o
a z g d e g f i n d x m z o u l o z j v
h w i w n t g x w c d o t x h y v z y z
q e a m f w p g u q t r e n n w f c r f
The occurrences of each letter are:
5 a 3 b 4 c 4 d 4 e 4 f 4 g 3 h 3 i 3 j
2 k 3 l 4 m 6 n 4 o 3 p 3 q 4 r 2 s 4 t
3 u 5 v 8 w 3 x 3 y 6 z
```

函数 createlist（第 19 ～ 28 行）生成一个 100 个随机小写字母的列表。第 5 行调用这个函数并将这个列表赋值给 chars。如果按如下方式改写代码，错在哪里？

```
chars = 100 * [' ']
chars = createList()
```

上述代码是想创建两个列表。第 1 行是想使用 100*[''] 创建一个列表。第 2 行是想通过调用 creatlist() 创建一个列表并将这个列表的引用值赋给 chars。第 1 行创建的列表可能会成为垃圾，因为它可能将不再被引用。Python 将在后台自动收集垃圾。程序将会正确编译和运行，但它可能会创建一个不必要的列表。

调用 getRandomLowerCaseLetter()（第 25 行）返回了一个随机小写字母。这个函数被定义在程序清单 6-11 中的 RandomCharacter 类中。

countLetters 函数（第 40 ～ 48 行）返回一个具有 26 个 int 值的列表，每个都存储了一个字母的出现次数。这个函数处理列表中的每个字母并对它的计数器增加 1。一个蛮力统计每个字母出现次数的方法可能如下所示。

```
for i in range(len(chars)):
    if chars[i] == 'a':
        counts[0] += 1
    elif chars[i] == 'b':
        counts[1] += 1
    ...
```

但是一个更好的解决方案在第 45 ～ 46 行给出。

```
for i in range(len(chars)):
    counts[ord(chars[i]) - ord('a')] += 1
```

如果这个字母(chars[i])是 a，那么相应的计数器是 counts[ord('a')−ord('a')]（即 counts[0]）。如果这个字母是 b，因为 b 的统一码比 a 的大 1，所以与之对应的计数器是 counts[ord('b')−ord('a')]（即 counts[1]）。如果字母是 z，因为 z 的统一码比 a 大 25，所以与之对应的计数器是 counts[ord('z')−ord('a')]（即 counts[25]）。

10.10 查找列表

关键点：如果一个列表是排好序的，那么要查找一个列表中的某个元素，二分查找比线性查找更高效。

查找是在列表中查找一个特定元素的方法。例如：判定某个分数是不是包含在一个分数列表里。list 类提供了 index 方法来查找并返回匹配列表中某个元素的下标。它也支持 in 和 not in 运算符以决定一个元素是否在列表中。

查找在计算机程序设计里是一个常见任务。许多算法都致力于查找。本节将讨论两种常

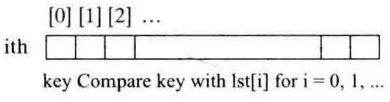
用的方法：线性查找和二分查找。

10.10.1 线性查找法

线性查找法顺序地将关键元素 key 和列表中的每一个元素进行比较。它连续这样做，直到这个关键字匹配列表中的某个元素，或者在没有找到匹配元素时已经查找完整个列表。如果找到一个匹配元素，那么线性查找将返回匹配元素在列表中的下标。如果没有匹配，那么查找返回 −1。程序清单 10-9 中的 linearSearch 函数可以解释这个方法。

程序清单 10-9 LinearSearch.py

```
1  # The function for finding a key in the list
2  def linearSearch(lst, key):
3      for i in range(len(lst)):
4          if key == lst[i]:
5              return i
6
7      return -1
```



为了更好地理解这个函数，使用下面的语句对程序进行跟踪。

```
lst = [1, 4, 4, 2, 5, -3, 6, 2]
i = linearSearch(lst, 4)  # Returns 1
j = linearSearch(lst, -4) # Returns -1
k = linearSearch(lst, -3) # Returns 5
```

线性查找函数将关键字和列表的每一个元素进行比较。这些元素可以是任意顺序。如果这个元素存在，那么算法在找到这个关键字之前需要平均检测列表的一半元素。因为线性查找的运行时间和列表中元素的数量成正比，所以对于大型列表而言，线性查找的效率是很低的。

10.10.2 二分查找法

二分查找是对列表值进行查找的另一种常用方法。想运用二分查找法，列表中的元素必须是事先排好序的。假设列表是升序排列的，那么二分查找法会首先将关键字和列表的中间元素进行比较，这时需要考虑下面三种情况：

- 如果关键字小于列表中间的元素，那么你只需要在列表的前半部分继续寻找关键字。
- 如果关键字等于列表中间的元素，那么查找因为找到一个匹配而结束。
- 如果关键字大于列表中间的元素，那么你只需要在列表的后半部分继续寻找关键字。

注意：毫无疑问，二分查找法每次比较之后都排除了一半的列表。有时排除一半的元素，有时排除一半加一个元素。假定这个列表有 n 个元素。为方便起见，假设 n 是 2 的幂。在第一次比较之后，$n/2$ 个元素被留下来进行下一步比较；在第二次比较之后，（$n/2$）/2 个元素被留下。在第 k 次比较之后，$n/2^k$ 个元素被留下进行下一步查找。当 $k = \log_2 n$ 时，列表中只剩下一个元素，只需要进行一次比较即可。因此，在用二分查找时，最坏情况下需要进行 $\log_2 n+1$ 次比较来在排序列表中找到那个元素。对于一个有着 1024（2^{10}）个元素的列表来说，最坏情况下二分查找只需要进行 11 次比较，而线性查找则需要进行 1023 次比较。

每一次比较之后列表中需要查找的部分就减少一半，分别用 low 和 high 来表示列表中当前要查找的第一个下标和最后一个下标。初始情况下，low 是 0，而 high 是 len(lst)−1。mid 表示中间元素的下标，因此 mid 是 (low+high)/2，图 10-9 给出如何利用二分查找在列表 [2,4,7,10,11,45,50,59,60,66,69,70,79] 中找到关键字 11。

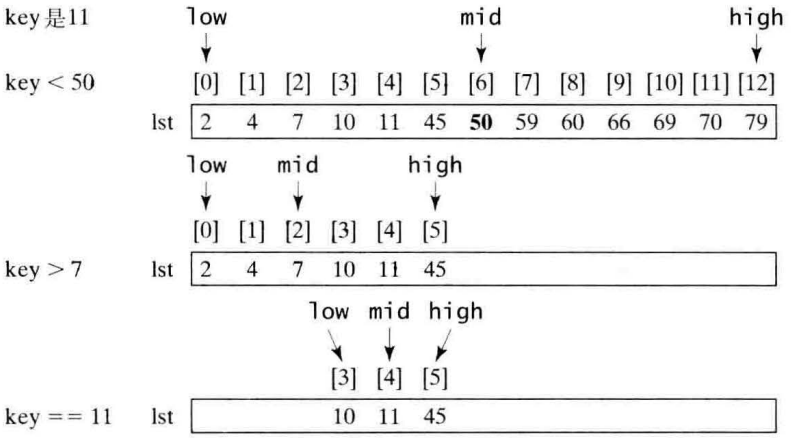


图 10-9 一个二分查找在每次比较之后将下一步要考虑的列表减少到一半

现在，你知道二分查找是如何工作的。下一个任务是如何用 Python 实现它。但是不要急于一下子就完全实现它。应该逐步开发，一次只做一步。如图 10-10a 所示，可以从查找的第一次迭代开始。它将关键字和列表的中间元素进行比较，这时 low 下标是 0 而 high 是 len(lst)−1。如果 key<lst[mid]，将 high 下标指向 mid−1；如果 key==lst[mid]，就找到了一个匹配对象，程序将返回 mid；如果 key>lst[mid]，将 low 下标指向 mid+1。

接下来，考虑添加一个循环来实现函数以完成重复查找，如图 10-10b 所示。当找到这个关键字，或者当 low>high 还没有找到，那么这个查找结束。

```
def binarySearch(lst, key):
    low = 0
    high = len(lst) - 1

    mid = (low + high) // 2
    if key < lst[mid]:
        high = mid - 1
    elif key == lst[mid]:
        return mid
    else:
        low = mid + 1
```

a) 版本 1

```
def binarySearch(lst, key):
    low = 0
    high = len(lst) - 1

    while high >= low:
        mid = (low + high) // 2
        if key < lst[mid]:
            high = mid - 1
        elif key == lst[mid]:
            return mid
        else:
            low = mid + 1

    return -1  # Not found
```

b) 版本 2

图 10-10 二分查找是逐步实现的

当没有找到关键字时，low 是关键字应该被插入以保证列表顺序的插入点。返回插入点要比返回 −1 更有用。这个函数必须返回一个负值来表示这个关键字不在列表中。能否简单地返回 −low？不可以，因为关键字小于 lst[0]。一个好的选择是如果关键字不在列表中则让函数返回 −low−1。返回 −low−1 不仅表示这个值不在列表中，也表示值应该被插入的地方。

完整的程序在程序清单 10-10 中给出。

程序清单 10-10 BinarySearch.py

```
1 # Use binary search to find the key in the list
```

```
2  def binarySearch(lst, key):
3      low = 0
4      high = len(lst) - 1
5
6      while high >= low:
7          mid = (low + high) // 2
8          if key < lst[mid]:
9              high = mid - 1
10         elif key == lst[mid]:
11             return mid
12         else:
13             low = mid + 1
14
15     return -low - 1 # Now high < low, key not found
```

如果匹配元素在列表中，则二分查找将返回它的下标（第 11 行），否则，它将返回 -low-1（第 15 行）。

如果我们把第 6 行的 (high>=low) 替换成 (high>low) 会怎样？这个查找将丢失一个可能的匹配元素。考虑到列表只有一个元素的情况：查找将漏掉这个元素。

如果列表中有重复的元素，那这个函数是否还能工作？可以，只要元素在列表中以升序排列，函数就返回其中一个匹配元素的下标，当然前提是该元素在列表中。

为了更好地理解这个函数，用下面的语句跟踪它并识别函数返回的 low 和 high。

```
lst = [2, 4, 7, 10, 11, 45, 50, 59, 60, 66, 69, 70, 79]
i = binarySearch(lst, 2) # Returns 0
j = binarySearch(lst, 11) # Returns 4
k = binarySearch(lst, 12) # Returns -6
l = binarySearch(lst, 1) # Returns -1
m = binarySearch(lst, 3) # Returns -2
```

下面的表格给出函数退出时 low 和 high 的值，也给出调用函数后返回的值。

函数	low	high	返回值
binarsySearch(1st, 2)	0	1	0
binarsySearch(1st, 11)	3	5	4
binarsySearch(1st, 12)	5	4	-6
binarsySearch(1st, 1)	0	-1	-1
binarsySearch(1st, 3)	1	0	-2

注意：线性查找法在一个小列表或未排序队列中查找元素时很有用，但是对大型列表而言效率很低，而二分查找法更高效，但是它们需要列表是提前排好序的。

10.11 排序列表

关键点：对列表元素进行排序的策略有很多种。选择排序和插入排序是两种常用方法。

就像查找一样，排序也是程序设计中的一个常见任务。类 list 提供了 sort 方法来对一个列表进行排序。

已经有很多排序算法被开发出来。下面介绍两种简单、直观的排序算法：选择排序和插入排序。通过使用这些算法，可以学会开发和实现其他算法的有价值的技巧。

10.11.1 选择排序

假设你希望对一个列表进行升序排列。选择排序会找到列表中的最小元素并将它和第一

个元素交换。然后找到剩余元素中值最小的元素并和剩余列表的第一个元素交换，依此类推，直到只剩一个元素。图 10-11 给出如何运用选择排序对列表 [2,9,5,4,8,1,6] 进行排序。

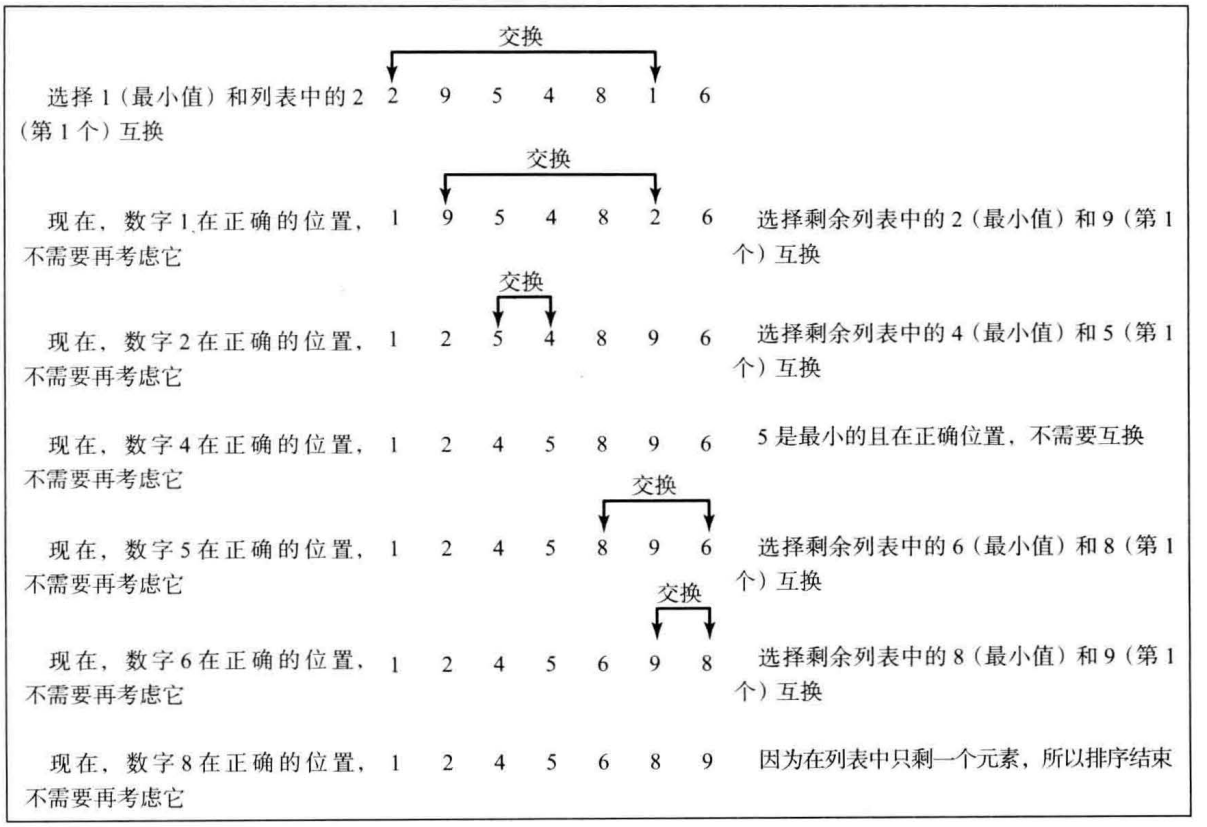


图 10-11 选择排序重复选择剩余列表中的最小元素，并将它和剩余列表中的第一个元素进行互换

第一次尝试开发一个完整的排序程序可能是比较困难的。编写一段代码完成第一轮迭代，它找到列表的最小元素之后和列表的第一个元素互换，然后观察第二轮迭代时有什么不同，接着是第三轮，依此类推。这样观察会让你能编写一个推广到所有迭代的循环。

解决方案可以如下描述：

```
for i in range(len(lst)-1):
    select the smallest element in lst[i : len(lst)]
    swap the smallest with lst[i], if necessary
    # lst[i] is in its correct position.
    # The next iteration applies to lst[i+1 : len(lst)]
```

程序清单 10-11 实现了这个解决方案。

程序清单 10-11 SelectionSort.py

```
1  # The function for sorting elements in ascending order
2  def selectionSort(lst):
3      for i in range(len(lst) - 1):
4          # Find the minimum in the lst[i : len(lst)]
5          currentMin = lst[i]
6          currentMinIndex = i
7
8          for j in range(i + 1, len(lst)):
9              if currentMin > lst[j]:
10                 currentMin = lst[j]
11                 currentMinIndex = j
```

```
12
13          # Swap lst[i] with lst[currentMinIndex] if necessary
14          if currentMinIndex != i:
15              lst[currentMinIndex] = lst[i]
16              lst[i] = currentMin
```

函数 selectionSort(lst) 对任意元素列表进行排序。这个函数通过嵌套的 for 循环来实现。最外层循环（循环变量 i）（第 3 行）的迭代是为了找到范围从 lst[i] 到 lst[len(lst)-1] 的列表的最小元素，并且将它和 lst[i] 交换。

变量 i 的初始值为 0，在外层循环每一次迭代之后，lst[i] 都在正确的位置。最终，所有的元素都被放在正确的位置；这样，整个列表就完成了排序。

为了更好地理解这个函数，使用下面的语句跟踪它：

```
lst = [1, 9, 4.5, 10.6, 5.7, -4.5]
selectionSort(lst)
```

10.11.2　插入排序

假如想升序排列一个列表。插入排序算法是通过重复地将一个新元素插入到一个已排好序的子列表中，直到整个列表排好序。图 10-12 给出如何利用插入排序对列表 [2,9,5,4,8,1,6] 进行排序。

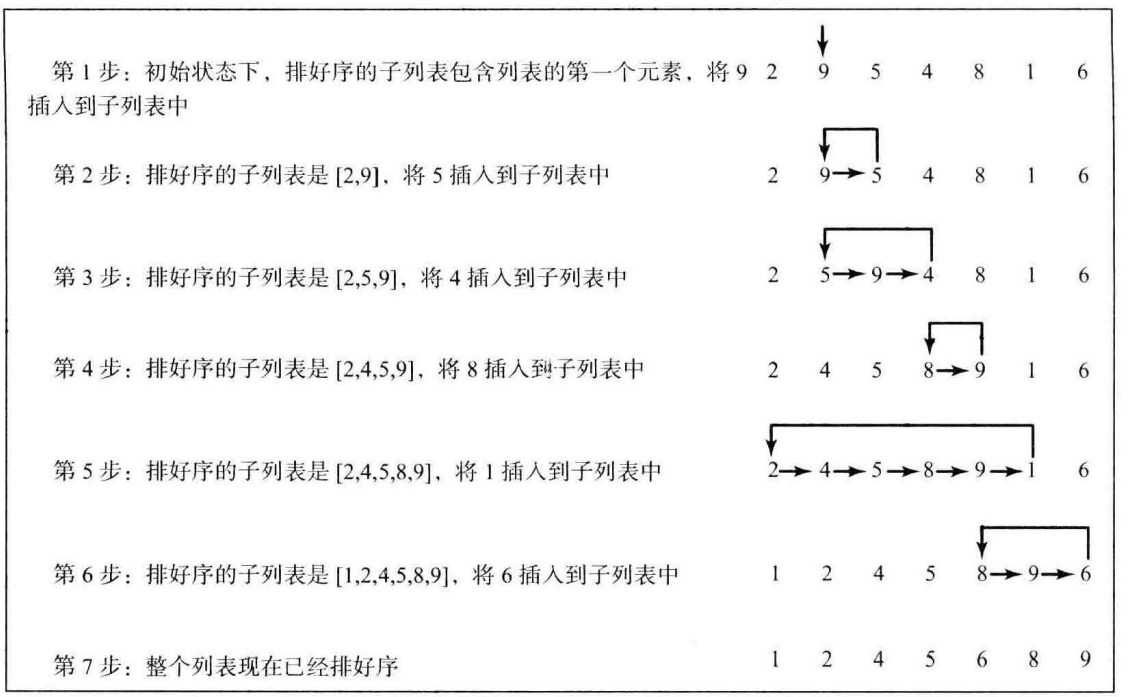


图 10-12　插入排序法将一个新元素重复插入到一个排好序的子列表

这个算法可以用如下代码描述：

```
for i in range(1, len(lst)):
    insert lst[i] into a sorted sublist lst[0 : i] so that
    lst[0..i+1] is sorted.
```

为了将 lst[i] 插入到 lst[0...i-1] 中，将 lst[i] 存在一个名为 currentElement 的临时变量。如果 lst[i−1]>currentElement，就将 lst[i-1] 移到 lst[i]；如果 lst[i−2]>currentElement，就将

lst[i−2] 移到 lst[i−1]；依此类推，直到 lst[i−k]<=currentElement 或者 k>i（我们传递有序列表中的第一个元素）。将 currentElement 赋值给 lst[i-k+1]。例如：在图 10-13 的第 3 步中，为了将 4 插入 [2,5,9]，因为 9>4，所以将 lst[2]（9）移到 lst[3]，又因为 5>4，所以将 lst[1]（5）移到 lst[2]。最后，将 currenteElement(4) 移到 lst[1]。

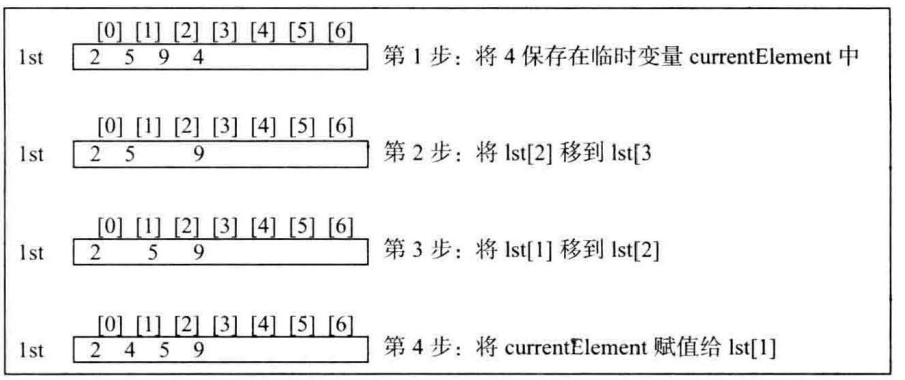


图 10-13 一个新元素被插入一个有序子列表

这个算法如程序清单 10-12 所示被扩展和实现。

程序清单 10-12 InsertionSort.py

```
1  # The function for sorting elements in ascending order
2  def insertionSort(lst):
3      for i in range(1, len(lst)):
4          # insert lst[i] into a sorted sublist lst[0 : i] so that
5          #   lst[0 : i+1] is sorted.
6          currentElement = lst[i]
7          k = i - 1
8          while k >= 0 and lst[k] > currentElement:
9              lst[k + 1] = lst[k]
10             k -= 1
11
12         # Insert the current element into lst[k + 1]
13         lst[k + 1] = currentElement
```

函数 insertionSort(lst) 可以对元素构成的任何列表进行排序。这个函数利用嵌套的 for 循环来实现。最外层循环（循环变量为 i）（第 3 行）是为了获得范围从 lst[0] 到 lst[1] 排好序的子列表而进行迭代的。内层循环（循环变量为 k）将 lst[i] 插入从 lst[0] 到 lst[i-1] 的子列表中。

为了更好地理解这个函数，使用下面的语句跟踪它。

```
lst = [1, 9, 4.5, 10.6, 5.7, -4.5]
insertionSort(lst)
```

检查点

10.20 使用图 10-8 作为范例给出如何应用二分查找法在列表 [2,4,7,10,11,45,50,59,60,66,69,70,79] 中搜索关键值 10 和 12。

10.21 如果二分函数返回 −4，那么这个关键值在列表中吗？如果你希望将这个关键值插入列表，那你应该将这个关键值插入到哪里？

10.22 使用图 10-10 作为范例给出如何应用选择排序法对 [3.4,5,3,3.5,2.2,1.9,2] 进行排序。

10.23 使用图 10-11 作为范例给出如何应用插入排序法对 [3.4,5,3,3.5,2.2,1.9,2] 进行排序。

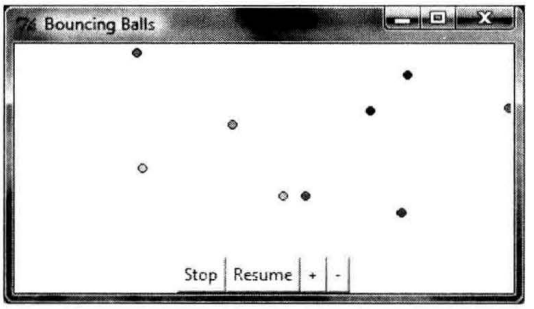
10.24 如何修改程序清单 10-11 中的 selectionSort 函数将元素降序排列？

10.25 如何修改程序清单 10-12 中的 insertionSort 函数将元素降序排列？

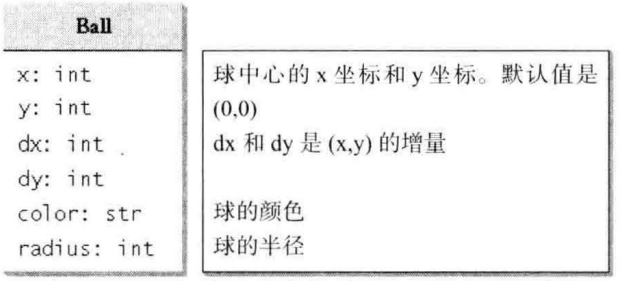
10.12　实例学习：弹球

关键点：本节的程序展示存在一个列表中的弹球。

现在，让我们把所学用于开发一个有趣的项目上。这里我们编写一个显示弹球的程序，如图 10-14a 所示。



a）程序通过控制按钮显示弹球



b）类 Ball 封装关于球的信息

图　10-14

这个程序允许用户通过“+”和“-”按钮来在画布上增加或减少一个球，还可以通过单击“Stop”和“Resume”按钮来停止球的移动或者重新开始移动。

每个球都有自己的中心位置（x，y）、radius、color 和相对于中心位置的下一个增量 dx 和 dy。可以通过定义一个类来封装所有的信息，如图 10-14b 所示。初始状态下，球的中心位置在（0,0）而 dx = 2 且 dy = 2。在这个动画中，球被移动到（x + dx，y + dy）。当球到达右边界时将 dx 改成 -2。当球到达底部边界时将 dy 改为 -2。当球到达左边界时将 dx 改为 2。当球到了顶部边界时将 dy 改成 2。这个程序通过当球碰到画布边界时改变 dx 和 dy 的值模拟了一个弹球。

当单击“+”按钮时，一个新的弹球就被创建了。如何将这个球存储到程序里呢？可以把所有的球都存在一个列表中。当单击“-”按钮时，列表中最后一个弹球就被清除了。

完整的程序在程序清单 10-13 中给出。

程序清单 10-13　BounceBalls.py

```
1  from tkinter import * # Import all definitions from tkinter
2  from random import randint
3
4  # Return a random color string in the form #RRGGBB
5  def getRandomColor():
6      color = "#"
7      for j in range(6):
8          color += toHexChar(randint(0, 15)) # Add a random digit
9      return color
10
11 # Convert an integer to a single hex digit in a character
12 def toHexChar(hexValue):
13     if 0 <= hexValue <= 9:
14         return chr(hexValue + ord('0'))
15     else:  # 10 <= hexValue <= 15
16         return chr(hexValue - 10 + ord('A'))
17
18 # Define a Ball class
19 class Ball:
20     def __init__(self):
21         self.x = 0 # Starting center position
```

```
22          self.y = 0
23          self.dx = 2 # Move right by default
24          self.dy = 2 # Move down by default
25          self.radius = 3 # The radius is fixed
26          self.color = getRandomColor() # Get random color
27
28  class BounceBalls:
29      def __init__(self):
30          self.ballList = [] # Create a list for balls
31
32          window = Tk() # Create a window
33          window.title("Bouncing Balls") # Set a title
34
35          self.width = 350 # Width of the self.canvas
36          self.height = 150 # Height of the self.canvas
37          self.canvas = Canvas(window, bg = "white",
38              width = self.width, height = self.height)
39          self.canvas.pack()
40
41          frame = Frame(window)
42          frame.pack()
43          btStop = Button(frame, text = "Stop", command = self.stop)
44          btStop.pack(side = LEFT)
45          btResume = Button(frame, text = "Resume",
46              command = self.resume)
47          btResume.pack(side = LEFT)
48          btAdd = Button(frame, text = "+", command = self.add)
49          btAdd.pack(side = LEFT)
50          btRemove = Button(frame, text = "-", command = self.remove)
51          btRemove.pack(side = LEFT)
52
53          self.sleepTime = 100 # Set a sleep time
54          self.isStopped = False
55          self.animate()
56
57          window.mainloop() # Create an event loop
58
59      def stop(self): # Stop animation
60          self.isStopped = True
61
62      def resume(self): # Resume animation
63          self.isStopped = False
64          self.animate()
65
66      def add(self): # Add a new ball
67          self.ballList.append(Ball())
68
69      def remove(self): # Remove the last ball
70          self.ballList.pop()
71
72      def animate(self): # Animate ball movements
73          while not self.isStopped:
74              self.canvas.after(self.sleepTime) # Sleep
75              self.canvas.update() # Update self.canvas
76              self.canvas.delete("ball")
77
78              for ball in self.ballList:
79                  self.redisplayBall(ball)
80
81      def redisplayBall(self, ball):
82          if ball.x > self.width or ball.x < 0:
83              ball.dx = -ball.dx
84
85          if ball.y > self.height or ball.y < 0:
```

```
86              ball.dy = -ball.dy
87
88          ball.x += ball.dx
89          ball.y += ball.dy
90          self.canvas.create_oval(ball.x - ball.radius,
91              ball.y - ball.radius, ball.x + ball.radius,
92              ball.y + ball.radius, fill = ball.color, tags = "ball")
93
94  BounceBalls() # Create GUI
```

程序为显示球创建了画布（第 35 ～ 39 行），创建了按钮“ Stop”、“ Resume”、“ +” 和“ –”（第 43 ～ 51 行），并且启动了动画（第 57 行）。

方法 animate 每 100 毫秒就会重画画布（第 72 ～ 79 行），它重新显示球列表中的每个球（第 78 ～ 79 行）。redisplayBall 方法在球碰到画布的任何界限时通过 dx 和 dy 改变方向（第 82 ～ 86 行），为球设置一个新的中心位置（第 88 ～ 89 行），然后在画布上重新显示这个球（第 90 ～ 92 行）。

当单击“ stop” 按钮时，就会调用 stop 方法将 isStopped 变量设置为 True（第 60 行) 同时停止动画（第 73 行）。当单击“ Resume” 按钮时，就会调用 resume 方法将 isStopped 变量设置为 False（第 63 行）同时让动画继续进行（第 73 行）。

当单击“ +” 按钮时，就会调用 add 方法在球列表里增加一个新球（第 67 行）。当单击“ –” 按钮时，就会调用 remove 方法将球列表中的最后一个球移除（第 70 行）。

在一个球被创建时（第 67 行），就会调用 Ball 的 _init_ 方法来创建和初始化属性 x、y、dx、dy、radius 和 color。颜色是一个字符串 #RRGGBB，这里的 R、G、B 都是一个十六进制的数字。每一个十六进制数字都是随机生成的（第 26 行）。toHexChar(hexValue) 方法返回了一个值在 0 到 15 之间的十六进制字符（第 12 ～ 16 行）。

关键术语

anonymous list（匿名列表）
binary searches（二分查找）
garbage collection（垃圾回收）
index（下标）
insertion sort（插入排序）
linear searches（线性查找）
selection sort（选择排序）

本章总结

1. 可以利用 Python 内置的 len、max、min 和 sum 函数返回一个列表的长度、列表的最大和最小值以及列表中所有元素之和。
2. 可以使用 random 模块中的 shuffle 函数将一个列表中的元素打乱。
3. 可以使用下标运算符 [] 来引用列表中的一个独立元素。
4. 程序员常常会错误地用下标 1 来引用列表中的第一个元素，但它应该是 0。这被称为下标出 1 错误。
5. 可以使用连接操作符 + 来连接两个列表，使用复制运算符 * 来复制元素，使用截取运算符 [:] 获取一个子列表，使用 in 和 not in 运算符来检查一个元素是否在列表中。
6. 可以使用 for 循环来遍历列表中的所有元素。
7. 可以使用比较运算符来比较两个列表中的元素。
8. 一个列表对象是可变的。可以使用方法 append、extend、insert、pop 和 remove 向一个列表添加元素和从一个列表删除元素。
9. 可以使用 index 方法获取列表中一个元素的下标，使用 count 方法来返回列表中元素的个数。

10. 可以使用 sort 和 reverse 方法来对一个列表中的元素进行排序和翻转。
11. 可以使用 split 方法来将一个字符串分离成列表。
12. 当调用一个带列表参数的函数时，列表的引用则被传递给这个函数。
13. 如果一个列表已经排好序，那么在列表中查找一个元素时二分查找比线性查找效率更高。
14. 选择排序将列表中的最小元素和第一个元素交换。然后找到剩余元素中最小的元素并与剩余元素的第一个交换，依此类推，直到只剩一个元素为止。
15. 插入排序算法重复地将一个新元素插入排好序的子列表中，直到整个表都排好序为止。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

注意：如果程序提示用户输入一个值列表，就输入一行由空格分隔的值。

第 10.2 ～ 10.3 节

*10.1 （定级）编写程序读取一个成绩列表，然后按照下面的方案对成绩分级：

如果成绩 >= best−10，那么级别为 A。
如果成绩 >= best−20，那么级别为 B。
如果成绩 >= best−30，那么级别为 C。
如果成绩 >= best−40，那么级别为 D。
否则成绩为 F。
下面是一个示例运行。

```
Enter scores: 40 55 70 58 ↵Enter
Student 0 score is 40 and grade is C
Student 1 score is 55 and grade is B
Student 2 score is 70 and grade is A
Student 3 score is 58 and grade is B
```

10.2 （逆序读取的数字）编写程序读取一个整数列表，然后以读取它们的逆序顺序显示。

**10.3 （统计数字个数）编写程序读取 1 到 100 之间的一些整数，并统计每个数字的个数。下面是这个程序的示例运行。

```
Enter integers between 1 and 100: 2 5 6 5 4 3 23 43 2 ↵Enter
2 occurs 2 times
3 occurs 1 time
4 occurs 1 time
5 occurs 2 times
6 occurs 1 time
23 occurs 1 time
43 occurs 1 time
```

注意：如果一个数字出现次数多过一次，在输出时使用 time 的复数形式 times。

10.4 （分析成绩）编写程序读取未指定个数的分数，然后决定多少个分数是大于等于平均分数，而多少个是低于平均分数的。假设输入数是在一行由空格分隔的。

**10.5 （打印不重复数字）编写程序读取一行由空格分隔开的数字，然后显示不重复数字（即如果一个数字出现多次，只显示它一次）。（提示：读取所有数字并将它们存储在 list1。创建一个新列表 list2。添加 list1 里的一个数字到 list2。如果这个数字已经在列表中，忽略它。）下面是这个程序的示例运行。

```
Enter ten numbers: 1 2 3 2 1 6 3 4 5 2 Enter
The distinct numbers are: 1 2 3 6 4 5
```

*10.6 （修改程序清单 5-13）程序清单 5-13 通过检查 2、3、4、5、6、…、$n/2$ 是否是 n 的除数来确定一个数字 n 是否是素数。如果找到除数，那么 n 不是素数。一个更有效的方法是检测任何一个小于或等于 $\sqrt{n}$ 的素数是否可以被 n 整数。如果不行，那么 n 是素数。使用这个方法改写程序清单 5-13 显示前 50 个素数。我们需要使用一个列表存储这些素数，随后使用它们检测它们是否是 n 的可能除数。

*10.7 （统计单个数字）编写程序产生 1000 个 0 到 9 之间的随机整数，然后显示每个数字的个数。（提示：使用 10 个整数组成的列表，即 counts，来存储数字 0、1、…、9 的个数。）

第 10.4 ～ 10.7 节

10.8 （找出最小元素的下标）编写函数返回整数列表最小元素的下标。如果这个元素的个数超过 1，那么返回最小的下标。使用下面的函数头：

```
def indexOfSmallestElement(lst):
```

编写一个测试程序，提示用户输入一个数字列表，调用这个函数返回最小元素的下标，显示这个下标。

*10.9 （统计：计算方差）编程题 5.46 计算数字的标准方差。本题使用不同但是等价的公式计算 n 个数字的标准方差。

$$mean = \frac{\sum_{i=1}^{n} x_i}{n} = \frac{x_1 + x_2 + ... + x_n}{n} \qquad deviation = \sqrt{\frac{\sum_{i=1}^{n} (x_i - mean)^2}{n-1}}$$

为了利用这个公式计算标准方差，必须使用列表存储各个数字，这样就可以在获取平均值 mean 之后使用它们。

程序应该包含下面的函数。

```
# Compute the standard deviation of values
def deviation(x):

# Compute the mean of a list of values
def mean(x):
```

编写测试程序提示用户输入一个数字列表，然后显示它们的平均值和标准方差，运行示例如下所示。

```
Enter numbers: 1.9 2.5 3.7 2 1 6 3 4 5 2 Enter
The mean is 3.11
The standard deviation is 1.55738
```

*10.10 （反转一个列表）第 10.8 节的 reverse 函数通过将它拷贝到一个新列表反转一个列表。改写这个函数，将这个列表作为参数传递给函数，并且返回这个函数。编写一个测试程序提示用户输入一个数字列表，调用这个函数反转这些数字，然后显示这些数字。

第 10.8 节

*10.11 （随机数字选择器）可以使用 random.shuffle(lst) 打乱一个列表。不使用 random.shuffle(lst) 编写函数来打乱一个列表并返回这个列表。使用下面的函数头：

```
def shuffle(lst):
```

编写一个测试程序，提示用户输入一个数字列表，调用这个函数打乱数字，然后显示这些数字。

10.12 （计算 GCD）编写函数返回列表中整数的最大公约数（GCD）。使用下面的方法头：

```
def gcd(numbers):
```

编写一个测试程序，提示用户输入五个数字，调用这个函数找出这些数字的 GCD，然后显示这个 GCD。

第 10.9 ～ 10.12 节

10.13 （消除重复）编写一个函数，消除列表中的重复值之后返回一个新列表。使用下面的函数头：

```
def eliminateDuplicates(lst):
```

编写一个测试程序读取一个整数列表，调用这个函数，然后显示这个结果。下面是这个程序的示例运行。

```
Enter ten numbers: 1 2 3 2 1 6 3 4 5 2 Enter
The distinct numbers are: 1 2 3 6 4 5
```

*10.14 （修改选择排序）在第 10.11.1 节，我们已经使用过选择排序对列表进行排序。选择排序函数重复找出当前列表的最小数，并将它和第一个进行互换。改写这个程序找出最大数，然后和最后一个进行互换。编写一个测试程序，读取 10 个数字，调用这个函数，然后显示排好序的数字。

**10.15 （有序吗？）编写下面的函数，如果列表已经以升序排列则返回 true：

```
def isSorted(lst):
```

编写一个测试程序，提示用户输入一个列表，然后显示这个列表是否排好序。下面是一个示例运行。

```
Enter list: 1 1 3 4 4 5 7 9 10 30 11 Enter
The list is not sorted
```

```
Enter list: 1 1 3 4 4 5 7 9 10 30 Enter
The list is already sorted
```

**10.16 （冒泡排序）编写一个使用冒泡排序算法的排序函数。这个冒泡排序算法要在列表来回穿梭。每次排序时，都对相邻的一对数进行比较，如果它们是降序，就进行互换；否则，保持值不变。这个技术被称为*冒泡排序*或*下沉排序*，因为较小值会不断“冒上来”到顶部，而较大值会不断“下沉”到底部。编写测试程序读取 10 个数，调用这个函数，然后显示排好序的列表。

**10.17 （相似词）编写函数检查两个单词是否是相似词。两个单词如果包含相同的字母，则它们是相似词。例如：silent 和 listen 是相似词。函数头是：

```
def isAnagram(s1, s2):
```

（提示：获取两个字符串的两个列表。对列表排序检测两个列表是否一致。）

编写一个测试程序提示用户输入两个字符串，如果它们是相似词，显示“is an anagram”；否则，显示“is not an anagram”。

***10.18 （游戏：八皇后）经典的八皇后谜题是将八个皇后放在棋盘上，而不让她们彼此攻击对方（即没有两个皇后在同一行、同一列或同一对角线上）。这里有很多可能的解决方案。编写程序显示一个这样的解决方案。一个示例输出如下所示。

```
|Q| | | | | | | |
| | | | |Q| | | |
| | | | | | | |Q|
| | | | | | |Q| |
| | |Q| | | | | |
| | | | | | |Q| |
| |Q| | | | | | |
| | | |Q| | | | |
```

***10.19 （游戏：豆机器）豆机器，也称为梅花或高尔顿盒子，它是一个统计实验的设备，这个实验是以英国科学家 Francis Galton 爵士命名的。它是由一个三角形直立板和均匀分布的钉子（或桩）构成，如图 10-15 所示。

球从板子的开口处落下。每次球碰到钉子，它就有 50% 的可能掉到左边或者右边。球就堆积在板子底部的槽内。

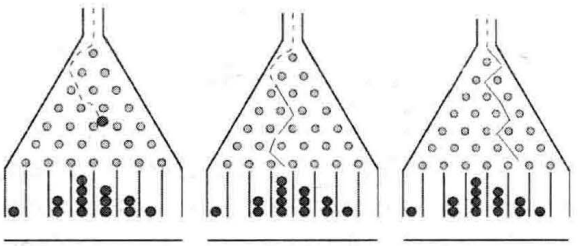


图 10-15　每个球都走随机路径落入槽内

编写程序模拟豆机器。程序应该提示用户输入球的个数以及机器的槽数。打印每个球的路径模拟它的下落。例如：图 10-15b 中球的路径是 LLRRLLR，而图 10-15c 中球的路径是 RLRRLRR。显示球在槽中的最终形状是直方图。下面是这个程序的一个示例运行。

```
Enter the number of balls to drop: 5 ←Enter
Enter the number of slots in the bean machine: 8 ←Enter

LRLRLRR
RRLLLRR
LLRLLRR
RRLLLLL
LRLRRLR

   0
   0
  000
```

（提示：创建一个名为 slots 的列表。slots 的每个元素存储槽中球的个数。每个球通过一条路径落进一个槽。路径中 R 的个数是球所落在那个槽的位置。例如：对路径 LRLRLRR 而言，求落进 slots[4]，对路径 RRLLLLL 而言，球落进 slots[2]。）

***10.20 （游戏：多个八皇后解决方案）编程题 10.18 是八皇后难题的一个解决方案。编写程序统计八皇后问题所有可能的解决方案，并显示所有的解决方案。

**10.21 （更衣室难题）一个学校有 100 个更衣室和 100 个学生。所有的更衣室在开学第一天都是锁着的。随着学生进入，第一个学生表示为 S1，打开每个更衣室。然后第二个学生 S2，从第二个更衣室开始，用 L2 表示，关闭所有其他更衣室。学生 S3 从第三个更衣室开始，改变每三个更衣室（如果打开则关闭，如果关闭则打开）。学生 S4 从更衣室 L4 开始，改变每四个更衣室。学生 S5 从更衣室 L5 开始，改变每五个更衣室，依此类推，直到学生 S100 改变 L100。

在所有学生都经过了房子并改变了更衣室之后，哪个更衣室是打开的？编写程序找出答案。

（提示：使用 100 个布尔元素的列表，每个都表示更衣室是开着的（True）还是关着的（False）。初始状态下，所有更衣室都是关闭的。）

**10.22 （模拟：优惠券收集问题）优惠券收集是一个经典的统计问题，它有很多实际的应用。这个问题是从一个对象集合里重复取出对象，然后找出要取出所有对象至少一次，需要取多少次。这个问题的变体是从一副打乱的 52 张牌中重复取牌，然后找出在看到每种花色一个之前需要取多少次。假设在选下一个之前取出的牌是要放回去的。编写程序模拟获取四张牌，每张都是不同花色需要取的次数，然后显示这四张取出的牌（可能牌被取出两次）。下面是程序的示例运行。

```
Queen of Spades
5 of Clubs
Queen of Hearts
4 of Diamonds
Number of picks: 12
```

10.23 （几何：解二次方程）使用下面的方法头编写函数解二次方程。

```
def solveQuadratic(eqn, roots):
```

一个二次方程 $ax^2 + bx + c = 0$ 的系数被传递给列表 eqn，它的非负根被存储在 roots。函数返回根的个数。参见编程题 4.1 中关于如何解二次方程的方法。

编写程序提示用户输入 a、b 和 c 的值，然后显示根的个数以及所有非负根。

*10.24 （数学：组合）编写程序提示用户输入 10 个整数，然后显示从这个 10 个数中选取两个数的所有组合。

*10.25 （游戏：选取四张牌）编写程序从一副 52 张牌中选取四张牌，计算它们的和。A、K、Q、J 分别表示 1、13、12 和 11。程序应该显示所选牌的数字和为 24。

**10.26 （合并两个有序列表）编写下面的函数合并两个有序列表构成一个新的有序列表。

```
def merge(list1, list2):
```

比照 len(list1) + len(list2) 实现这个函数。编写测试程序提示用户输入两个有序列表，然后显示合并后的列表。下面是一个示例运行。

```
Enter list1: 1 5 16 61 111 Enter
Enter list2: 2 4 5 6 Enter
The merged list is 1 2 4 5 5 6 16 61 111
```

*10.27 （模式识别：四个连续的相同的数）编写下面的函数测试列表是否具有同样值的四个连续数字。

```
def isConsecutiveFour(values):
```

编写测试程序提示用户输入一个整数序列，然后报告这个序列是否包含具有相同值的四个连续数字。

**10.28 （划分列表）编写下面的函数使用第一个元素，称为枢纽，来划分列表。

```
def partition(lst):
```

在划分之后，列表的元素被改编，所有在枢纽之前的元素都小于或等于枢纽，而枢纽之后的元素都大于枢纽。函数还返回枢纽在新列表中所处位置的下标值。例如：假设列表是 [5,2,9,3,6,8]。在划分之后，列表变成 [3,2,5,9,6,8]。将 len(lst) 作为比照实现这个函数。编写测试程序提示用户输入一个列表，然后显示划分之后的列表。下面是一个示例运行。

```
Enter a list: 10 1 5 16 61 9 11 1 Enter
After the partition, the list is 9 1 5 1 10 61 11 16
```

***10.29 （游戏：侩子手）编写侩子手游戏，随机产生一个单词然后提示用户一次猜一个字母，如样本示例所示。单词中的每个字母都显示为一个星号。当用户猜测正确时就会显示确切的字母。当用户完成一个单词时，显示失误的次数并询问用户是否继续玩游戏。创建一个列表存储这些单词如下所示。

```
# Use any words you wish
words = ["write", "that", "program", ...]
```

```
(Guess) Enter a letter in word ******* > p Enter
(Guess) Enter a letter in word p****** > r Enter
(Guess) Enter a letter in word pr**r** > p Enter
    p is already in the word
(Guess) Enter a letter in word pr**r** > o Enter
(Guess) Enter a letter in word pro*r** > g Enter
(Guess) Enter a letter in word progr** > n Enter
    n is not in the word
(Guess) Enter a letter in word progr** > m Enter
(Guess) Enter a letter in word progr*m > a Enter
The word is program. You missed 1 time

Do you want to guess another word? Enter y or n>
```

*10.30 （文化：中国生肖）使用字符串列表存储动物名字来简化程序清单 4-5。

10.31 （字符串中每个数字出现次数）使用下面的函数头编写函数统计字符串中每个数字的出现次数。

```
def count(s):
```

这个函数统记字符串中一个数字的出现次数。返回值是 10 个元素构成的列表，每个都表示一个数字的出现次数。例如：在执行完 counts=count("12203AB3") 之后，counts[0] 为 1，counts[1] 为 1，counts[2] 为 2，counts[3] 为 2。

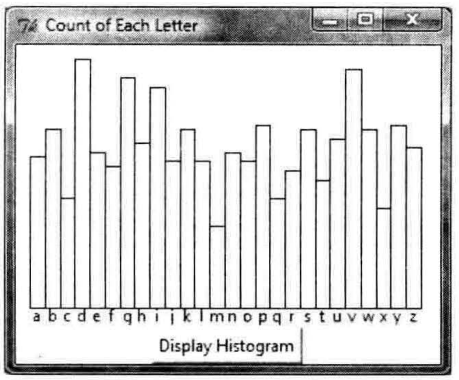
编写测试程序提示用户输入一个字符串，然后显示字符串中每个数字的出现次数。下面是这个程序的一个示例运行。

```
Enter a string: 232534312 Enter
1 occurs 1 time
2 occurs 3 times
3 occurs 3 times
4 occurs 1 time
5 occurs 1 time
```

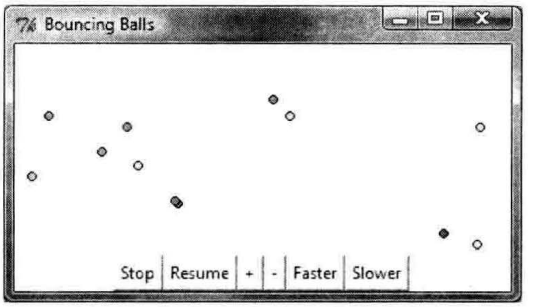
10.32 （Turtle：绘制线段）编写下面的函数绘制从点 p1（[x1,y1]）到点 p2（[x2,y2]）的线段。

```
# Draw a line
def drawLine(p1, p2):
```

10.33 （Tkinter：绘制直方图）编写程序随机产生 1000 个小写字母，统计每个字母的出现次数，然后显示直方图表示次数，如图 10-16a 所示。



a）绘制每个字母出现次数对应的直方图



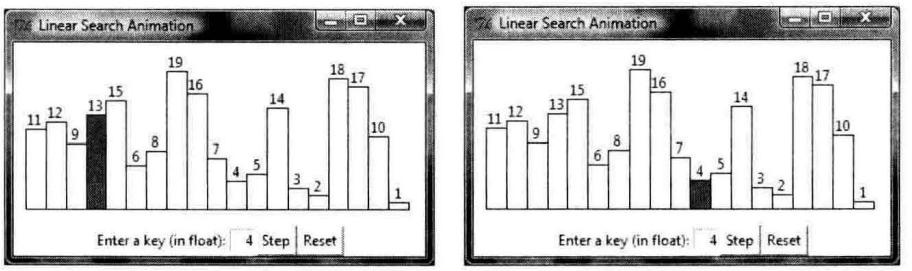
b）添加两个按钮来控制球的速度

图　10-16

10.34　（Turtle：绘制直方图）使用 Turtle 改写前一个程序。

*10.35　（Turtle：弹球）改写程序清单 10-13 添加两个按钮，“Faster”和“Slower”（如图 10-16b 所示），来加速或减速球的移动。

**10.36　（Tkinter：线性查找动画）编写程序实现线性查找算法的动画。创建一个由 20 个不同的从 1 到 20 随机顺序排列的数字构成的列表。元素以直方图显示，如图 10-17 所示。你需要在文本域输入查找的关键值。单击“Step”按钮可以让程序使用算法完成一次比较，然后用表示查找位置的条重绘直方图。当算法结束时，显示一个对话框来通知用户。单击“Reset”按钮为新的开始创建一个新的随机列表。

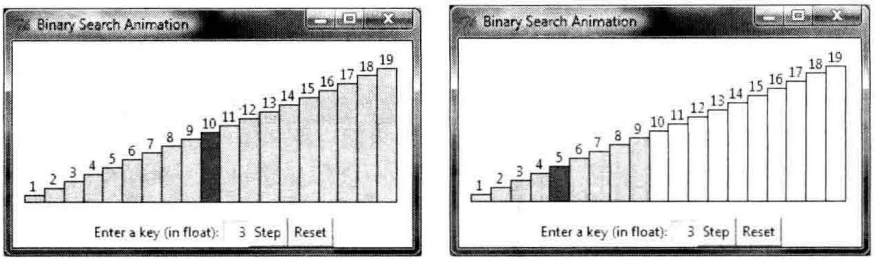


a)　　b)

图 10-17　程序实现线性查找的动画

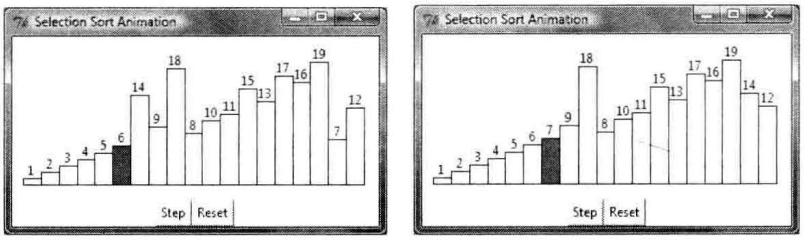
**10.37　（Tkinter：二分查找动画）编写程序实现二分查找算法的动画。创建一个以从 1 到 20 的顺序排列的数字构成的列表。元素以直方图显示，如图 10-18 所示。你需要在文本域输入查找的关键值。单击“Step”按钮可以让程序使用算法完成一次比较。使用亮灰色绘制当前查找范围的数字条，使用红色绘制表明当前查找范围的中间数字条。当算法结束时，显示一个对话框来通知用户。单击“Reset”按钮可以开始一次新的查找。这个按钮也可以使文本域可编辑。

*10.38　（Tkinter：选择排序动画）编写程序实现选择排序算法的动画。创建一个由 20 个不同的从 1 到 20 随机顺序排列的数字构成的列表。元素以直方图显示，如图 10-19 所示。单击“Step”按钮可以让程序使用算法完成外层循环的一次迭代，然后为新列表重绘直方图。给排好序的子列表中的最后一个条上色。当算法结束时，显示一个对话框来通知用户。单击“Reset”按钮为新的开始创建一个新的随机列表。



a)　　b)

图 10-18　程序实现二分查找的动画



a)　　b)

图 10-19　程序实现选择排序的动画

10.39　（Tkinter：24 点扑克牌游戏）24 点扑克牌游戏是关于从 52 张牌中选取任意四张，如图 10-20 所示。注意：大小王是被排除在外的。每张牌表示一个数字。A、K、Q、J 分别表示 1、13、12 和 11。输入一个使用四张所选牌对应数字的表达式。每个牌数都只能在每个表达式中使用一次，而且每张牌必须被使用。可以在表达式中使用运算符（+、-、 和 /）以及括号，表达式结果必须是 24。在输入表达式之后，单击“Verify”按钮来检查表达式中的数字是否是当前选择的，以及表达式的结果是否是正确的。在一个对话框中显示确认信息。可以单击“Refresh”按钮获得另四张牌。假设图像存储在以黑桃、红桃、方块、梅花为顺序且名为 1.gif、2.gif、…、52.gif 的文件内。所以前 13 个图像是黑桃 1、2、3、…、13。

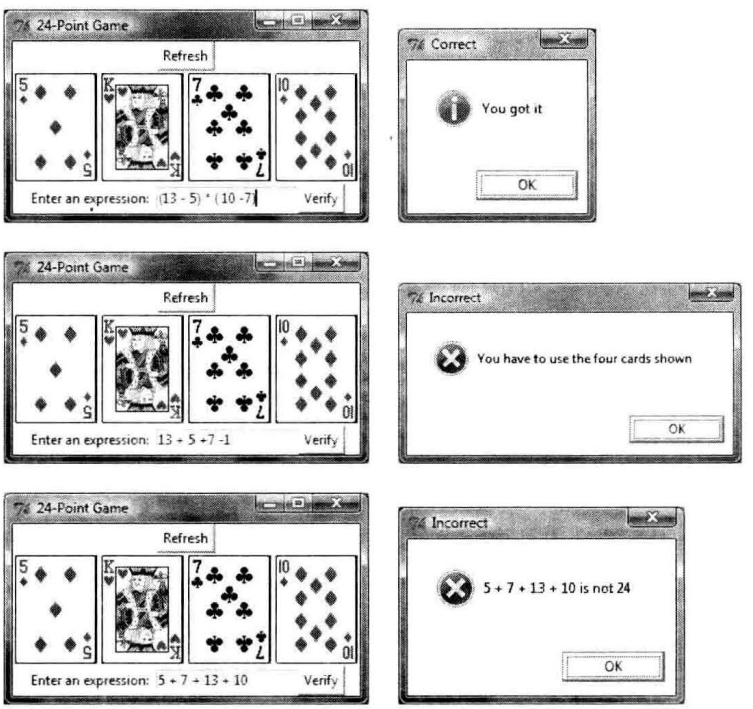


图 10-20　用户使用牌上的数字输入表达式

*10.40 （Tkinter：插入排序动画）编写程序实现插入排序算法的动画。创建一个由 20 个不同的从 1 到 20 随机顺序排列的数字构成的列表。元素以直方图显示，如图 10-21 所示。单击“Step”按钮可以让程序使用算法完成外层循环的一轮迭代，然后以新列表重绘直方图。给排好序的子列表的最后一个条上色。当算法结束时，显示一个对话框来通知用户。单击“Reset”按钮为新的开始创建一个新的随机列表。

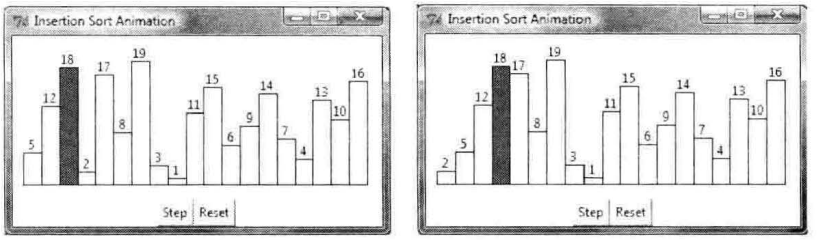
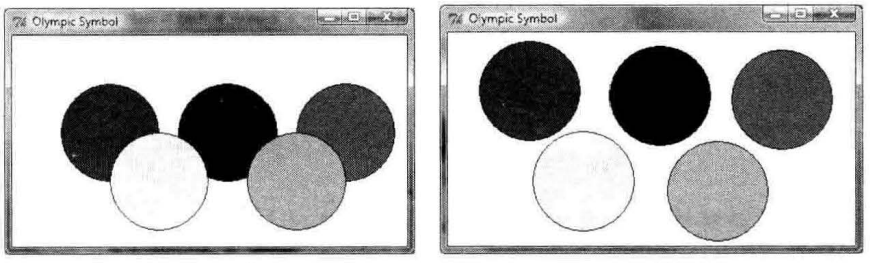


图 10-21　程序实现插入排序的动画

10.41 （显示五个圆）编写程序显示五个圆，如图 10-22a 所示。可以使用鼠标拖动每个圆，如图 10-22b 所示。



a)　　b)

图　10-22

第 11 章

Introduction to Programming Using Python

多维列表

学习目标

- 学习如何使用二维列表来表示二维数据（第 11.1 节）。
- 使用行下标和列下标来访问二维列表中的元素（第 11.2 节）。
- 为二维列表编写常用的操作（显示列表、求所有元素总和、找出 min 和 max 元素，随意打乱和排序）(第 11.2 节)。
- 向函数传递二维列表（第 11.3 节）。
- 使用二维列表编写对多选题评分的程序（第 11.4 节）。
- 使用二维列表来解决找出距离最近点对（第 11.5 ～ 11.6 节）。
- 使用二维列表来检查数独解决方案（第 11.7 ～ 11.8 节）。
- 使用多维列表（第 11.9 节）。

11.1 引言

关键点：一张表或矩阵中的数据可以存储在一个二维列表中。

二维列表是将其他列表作为它的元素的列表。前一章介绍了如何使用一个列表来存储线性的元素集合。可以使用列表来存储二维数据，例如：一个矩阵或者一张表等。例如，下表所示的提供了不同城市之间距离的表可以使用一个命名为 distances 的列表来存储。

距离表（公里）

	芝加哥	波士顿	纽约	亚特兰大	迈阿密	达拉斯	休斯敦
芝加哥	0	983	787	714	1375	967	1087
波士顿	983	0	214	1102	1505	1723	1842
纽约	787	214	0	888	1549	1548	1627
亚特兰大	714	1102	888	0	661	781	810
迈阿密	1375	1505	1549	661	0	1426	1187
达拉斯	967	1723	1548	781	1426	0	239
休斯敦	1087	1842	1627	810	1187	239	0

```
distances = [
    [0, 983, 787, 714, 1375, 967, 1087],
    [983, 0, 214, 1102, 1505, 1723, 1842],
    [787, 214, 0, 888, 1549, 1548, 1627],
    [714, 1102, 888, 0, 661, 781, 810],
    [1375, 1505, 1549, 661, 0, 1426, 1187],
    [967, 1723, 1548, 781, 1426, 0, 239],
    [1087, 1842, 1627, 810, 1187, 239, 0]
  ]
```

距离表中的每个元素又是另一个列表，因此它又被认为是一个嵌套列表。在这个例子中，二维列表被用于存储二维数据。

11.2　处理二维列表

关键点：二维列表中的值可以通过行下标和列下标来访问。

可以将二维列表理解为一个由行组成的列表。而每一行又是一个由值组成的列表。二维列表的每一行可以使用下标访问，为方便称为行下标。每一行中的值可以通过另一个下标访问，称为列下标。一个命名为 matrix 的二维列表如图 11-1 所示。

```
matrix = [
    [1, 2, 3, 4, 5],
    [6, 7, 0, 0, 0],
    [0, 1, 0, 0, 0],
    [1, 0, 0, 0, 8],
    [0, 0, 9, 0, 3],
]
```

	[0]	[1]	[2]	[3]	[4]
[0]	1	2	3	4	5
[1]	6	7	0	0	0
[2]	0	1	0	0	0
[3]	1	0	0	0	8
[4]	0	0	9	0	3

```
matrix[0] is [1, 2, 3, 4, 5]
matrix[1] is [6, 7, 0, 0, 0]
matrix[2] is [0, 1, 0, 0, 0]
matrix[3] is [1, 0, 0, 0, 8]
matrix[4] is [0, 0, 9, 0, 3]

matrix[0][0] is 1
matrix[4][4] is 3
```

图 11-1　二维列表中的值可以通过行下标和列下标来访问

矩阵中的每个值都可以用 matrix[i][j] 来访问，这里的 i 和 j 分别是行下标和列下标。下面的小节给出一些使用二维列表的例子。

11.2.1　使用输入值初始化列表

下面的循环使用用户输入值来初始化矩阵。

```
matrix = [] # Create an empty list

numberOfRows = eval(input("Enter the number of rows: "))
numberOfColumns = eval(input("Enter the number of columns: "))
for row in range(numberOfRows):
    matrix.append([]) # Add an empty new row
    for column in range(numberOfColumns):
        value = eval(input("Enter an element and press Enter: "))
        matrix[row].append(value)

print(matrix)
```

11.2.2　使用随机数初始化列表

下面的循环初始化一个存储 0 到 99 之间随机数的列表。

```
import random

matrix = [] # Create an empty list

numberOfRows = eval(input("Enter the number of rows: "))
numberOfColumns = eval(input("Enter the number of columns: "))
for row in range(numberOfRows):
    matrix.append([]) # Add an empty new row
    for column in range(numberOfColumns):
        matrix[row].append(random.randint(0, 99))

print(matrix)
```

11.2.3　打印列表

为了打印一个二维列表，必须通过使用下面的循环来打印列表中的每一个元素。

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # Assume a list is given

for row in range(len(matrix)):
    for column in range(len(matrix[row])):
        print(matrix[row][column], end = " ")
    print() # Print a new line
```

或者也可以写成：

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # Assume a list is given

for row in matrix:
    for value in row:
        print(value, end = " ")
    print() # Print a new line
```

11.2.4　对所有元素求和

使用一个名为 total 的变量来存储元素的总和。初始状态下，total 值为 0。利用如下循环对列表中的每一个元素相加，值赋给 total。

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # Assume a list is given

total = 0
for row in matrix:
    for value in row:
        total += value

print("Total is", total) # Print the total
```

11.2.5　按列求和

对于每一列，使用名为 total 的变量来存储每一列的总和。使用如下循环将每一列中元素相加，和赋值给 total。

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # Assume a list is given

for column in range(len(matrix[0])):
    total = 0
    for row in range(len(matrix)):
        total += matrix[row][column]
    print("Sum for column", column, "is", total)
```

11.2.6　找出和最大的行

为了找出和最大的行，可以使用变量 maxRow 和 indexOfMaxRow 来跟踪最大的和及对应的行下标。对每一行，计算它的和，如果新的和要大些时，更新 maxRow 和 indexOfMaxRow。

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # Assume a list is given

maxRow = sum(matrix[0]) # Get sum of the first row in maxRow
indexOfMaxRow = 0

for row in range(1, len(matrix)):
    if sum(matrix[row]) > maxRow:
        maxRow = sum(matrix[row])
        indexOfMaxRow = row

print("Row", indexOfMaxRow, "has the maximum sum of", maxRow)
```

11.2.7 随意打乱

第 10.2.3 小节介绍过可以使用函数 random.shuffle(list) 打乱一维列表中的元素。如何打乱二维列表中的所有元素？为了实现这个目的，对每一个元素 matrix[row][column]，随机生成下标 i 和 j 并且将 matrix[row][column] 和 matrix[i][j] 进行互换，如下所示：

```
import random

matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]] # Assume a list is given

for row in range(len(matrix)):
    for column in range(len(matrix[row])):
        i = random.randint(0, len(matrix) - 1)
        j = random.randint(0, len(matrix[row]) - 1)

        # Swap matrix[row][column] with matrix[i][j]
        matrix[row][column], matrix[i][j] = \
            matrix[i][j], matrix[row][column]

print(matrix)
```

11.2.8 排序

可以应用 sort 方法对一个二维列表排序。它通过每一行的第一个元素进行排序。对于第一个元素相同的行，则通过它们的第二个元素进行排序。如果行中的第一个和第二个元素都相同，则利用它们的第三个元素进行排序，依此类推。例如：

```
points = [[4, 2], [1, 7], [4, 5], [1, 2], [1, 1], [4, 1]]
points.sort()
print(points)
```

显示 [[1,1],[1,2],[1,7],[4,1],[4,2],[4,5]]。

检查点

11.1　如何创建数据集构成的一个 3 行 4 列的二维列表，列表中的元素初始化为 0？

11.2　可以创建一个每行元素个数不同的二维数据列表吗？

11.3　下面代码的输出是什么？

```
matrix = []
matrix.append(3 * [1])
matrix.append(3 * [1])
matrix.append(3 * [1])
matrix[0][0] = 2
print(matrix)
```

11.4　下面代码的输出是什么？

```
matrix = []
matrix.append([3 * [1]])
matrix.append([3 * [1]])
matrix.append([3 * [1]])
print(matrix)
matrix[0] = 3
print(matrix)
```

11.5　下面代码的输出是什么？

```
matrix = []
matrix.append([1, 2, 3])
matrix.append([4, 5])
matrix.append([6, 7, 8, 9])
print(matrix)
```

11.3 将二维列表传递给函数

关键点：当给函数传递二维列表时，是将这个列表的引用传递给函数。

可以像传递一维列表一样给函数传递一个二维列表。同样可以从函数中返回一个二维列表。清单 11-1 给出了一个含有两个函数的例子。第一个函数是 getMatrix()，它返回一个二维列表，而第二个函数是 accumulate(m)，它返回一个矩阵所有元素的总和。

程序清单 11-1 PassTwoDimensionalList.py

```
1  def getMatrix():
2      matrix = [] # Create an empty list
3
4      numberOfRows = eval(input("Enter the number of rows: "))
5      numberOfColumns = eval(input("Enter the number of columns: "))
6      for row in range(numberOfRows):
7          matrix.append([]) # Add an empty new row
8          for column in range(numberOfColumns):
9              value = eval(input("Enter a value and press Enter: "))
10             matrix[row].append(value)
11
12     return matrix
13
14 def accumulate(m):
15     total = 0
16     for row in m:
17         total += sum(row)
18
19     return total
20
21 def main():
22     m = getMatrix() # Get a list
23     print(m)
24
25     # Display sum of elements
26     print("\nSum of all elements is", accumulate(m))
27
28 main() # Invoke main function
```

```
Enter the number of rows: 2 Enter
Enter the number of columns: 2 Enter
Enter a value and press Enter: 1 Enter
Enter a value and press Enter: 2 Enter
Enter a value and press Enter: 3 Enter
Enter a value and press Enter: 4 Enter
[[1, 2], [3, 4]]
Sum of all elements is 10
```

函数 getMatrix（第 1 ～ 12 行）提示用户输入矩阵的元素值（第 9 行），然后返回这个列表（第 12 行）。

函数 accumulate（第 14 ～ 19 行）有一个二维列表参数。它返回这个列表所有元素的总和（第 26 行）。

检查点

11.6　给出下面代码的输出结果。

```
def f(m):
    for i in range(len(m)):
```

```
        for j in range(len(m[i])):
            m[i][j] += 1
def printM(m):
    for i in range(len(m)):
        for j in range(len(m[i])):
            print(m[i][j], end = "")
        print()

m = [[0, 0], [0, 1]]

printM(m)
f(m)
printM(m)
```

11.4　问题：给多选题评分

关键点：编写一个用于给多选题评分的程序。

假设有 8 名学生和 10 道选择题，他们的答案存储在一个二维列表当中。每一行记录了一位学生对这些问题的答案，如下图所示：

	学生对这些问题的答案									
	0	1	2	3	4	5	6	7	8	9
Student 0	A	B	A	C	C	D	E	E	A	D
Student 1	D	B	A	B	C	A	E	E	A	D
Student 2	E	D	D	A	C	B	E	E	A	D
Student 3	C	B	A	E	D	C	E	E	A	D
Student 4	A	B	D	C	C	D	E	E	A	D
Student 5	B	B	E	C	C	D	E	E	A	D
Student 6	B	B	A	C	C	D	E	E	A	D
Student 7	E	B	E	C	C	D	E	E	A	D

标准答案存储在一个一维列表中：

	这些问题的答案：									
	0	1	2	3	4	5	6	7	8	9
Key	D	B	D	C	C	D	A	E	A	D

程序对这个测试评分并且显示最后的结果。为了实现上述功能，本程序将每位学生的答案与标准答案进行比较，统计正确答案的个数并且显示它。程序清单 11-2 给出这个程序。

程序清单 11-2　GradeExam.py

```
1  def main():
2      # Students' answers to the questions
3      answers = [
4          ['A', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
5          ['D', 'B', 'A', 'B', 'C', 'A', 'E', 'E', 'A', 'D'],
6          ['E', 'D', 'D', 'A', 'C', 'B', 'E', 'E', 'A', 'D'],
7          ['C', 'B', 'A', 'E', 'D', 'C', 'E', 'E', 'A', 'D'],
8          ['A', 'B', 'D', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
9          ['B', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
10         ['B', 'B', 'A', 'C', 'C', 'D', 'E', 'E', 'A', 'D'],
11         ['E', 'B', 'E', 'C', 'C', 'D', 'E', 'E', 'A', 'D']]
12
13     # Key to the questions
14     keys = ['D', 'B', 'D', 'C', 'C', 'D', 'A', 'E', 'A', 'D']
15
16     # Grade all answers
```

```
18          # Grade one student
19          correctCount = 0
20          for j in range(len(answers[i])):
21              if answers[i][j] == keys[j]:
22                  correctCount += 1
23
24          print("Student", i, "'s correct count is", correctCount)
25
26  main() # Call the main function
```

```
Student 0's correct count is 7
Student 1's correct count is 6
Student 2's correct count is 5
Student 3's correct count is 4
Student 4's correct count is 8
Student 5's correct count is 7
Student 6's correct count is 7
Student 7's correct count is 7
```

第 3 ～ 11 行的语句创建了一个字符构成的二维列表并且将它的引用赋值给 answers。

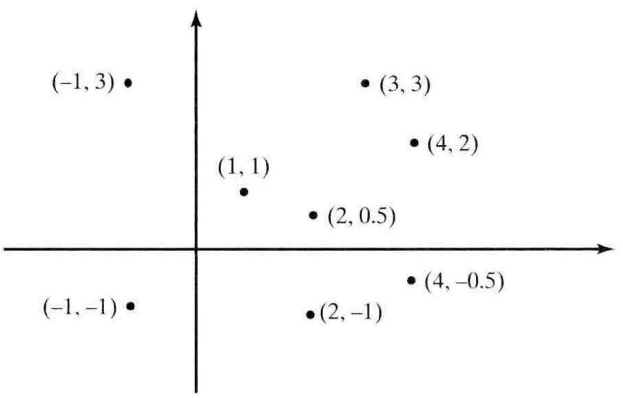
第 14 行的语句创建了一个标准答案的列表并且将它的引用赋值给 keys。

列表 answers 的每一行存储了一名学生的答案，并且通过与列表 keys 中的标准答案进行对比来对这名学生评分。在这名学生的答案评分结束后立刻显示结果（第 19 ～ 22 行）。

11.5　问题：找出距离最近的点对

关键点：本节提出了一个几何上关于寻找距离最近点对的问题。

假设给出一个集合的点，最近点对问题就是如何寻找这些点之中哪两个点距离最近。例如：在图 11-2 中，点 (1，1) 和点 (2，0.5) 距离最近。解决这个问题的方法有几种。一种直观的方法是计算每一对点之间的距离然后找出其中距离最小的一对，如程序清单 11-3 所示实现。



	x	y
0	–1	3
1	–1	–1
2	1	1
3	2	0.5
4	2	–1
5	3	3
6	4	2
7	4	–0.5

图 11-2　点可以表示为嵌套列表

程序清单 11-3　NearestPoints.py

```
1  # Compute the distance between two points (x1, y1) and (x2, y2)
2  def distance(x1, y1, x2, y2):
3      return ((x2 - x1) * (x2 - x1) + (y2 - y1) * (y2 - y1)) ** 0.5
4
5  def nearestPoints(points):
6      # p1 and p2 are the indexes in the points list
```

```
 7      p1, p2 = 0, 1  # Initial two points
 8
 9      shortestDistance = distance(points[p1][0], points[p1][1],
10          points[p2][0], points[p2][1]) # Initialize shortestDistance
11
12      # Compute distance between every two points
13      for i in range(len(points)):
14          for j in range(i + 1, len(points)):
15              d = distance(points[i][0], points[i][1],
16                           points[j][0], points[j][1])  # Find distance
17
18              if shortestDistance > d:
19                  p1, p2 = i, j # Update p1, p2
20                  shortestDistance = d # New shortestDistance
21
22      return p1, p2
```

模块定义了 nearestPoints(points) 函数，这个函数返回二维列表 points 中距离最近的两点的下标。程序使用变量 shortestDistance（第 9 行）来存储两个最近点的距离，points 列表中这两点的下标都存储在变量 p1 和 p2 中（第 19 行）。

对于下标为 i 的点 point[i]，程序计算它与所有 j>i 的点 point[j] 之间的距离（第 15 ～ 16 行）。当所计算距离更短时，更新变量 shortestDistance、p1 和 p2（第 19 ～ 20 行）。

使用公式 $\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$ 计算两点 (x1，y1) 和 (x2，y2) 之间距离（第 2 ～ 3 行）。

注意：很可能存在不止一对距离相同且均为最短的点对。这个程序只能找到其中的一对。可以在编程题 11.8 中修改上述程序使程序可以找出更多距离最短的点对。

程序清单 11-4 中的程序提示用户输入点的坐标然后显示距离最近的两点。

程序清单 11-4　FindNearestPoints.py

```
1  import NearestPoints
2
3  def main():
4      numberOfPoints = eval(input("Enter the number of points: "))
5
6      # Create a list to store points
7      points = []
8      print("Enter", numberOfPoints, "points:", end = '')
9      for i in range(numberOfPoints):
10         point = 2 * [0]
11         point[0], point[1] = \
12             eval(input("Enter coordinates separated by a comma: "))
13         points.append(point)
14
15     # p1 and p2 are the indexes in the points list
16     p1, p2 = NearestPoints.nearestPoints(points)
17
18     # Display result
19     print("The closest two points are (" +
20         str(points[p1][0]) + ", " + str(points[p1][1]) + ") and (" +
21         str(points[p2][0]) + ", " + str(points[p2][1]) + ")")
22
23 main() # Call the main function
```

```
Enter the number of points: 8 Enter
Enter coordinates separated by a comma: -1, 3 Enter
Enter coordinates separated by a comma: -1, -1 Enter
Enter coordinates separated by a comma: 1, 1 Enter
```

```
Enter coordinates separated by a comma: 2, 0.5 ↵Enter
Enter coordinates separated by a comma: 2, -1 ↵Enter
Enter coordinates separated by a comma: 3, 3 ↵Enter
Enter coordinates separated by a comma: 4, 2 ↵Enter
Enter coordinates separated by a comma: 4, -0.5 ↵Enter
The closest two points are (1, 1) and (2, 0.5)
```

这个程序要求用户输入点的个数（第 4 行）。这些点从控制台读取并且存储在一个名为 points 的二维列表中（第 11 行）。程序调用 nearestPoints(points) 函数来返回列表中距离最近的两点的下标（第 16 行）。

程序假设平面上至少有两个点。可以很容易修改程序来防止平面上只有一个或没有点。

提示：从键盘上输入所有点是一项繁重的工作。可以将它们存入一个名字类似 FindNearestPoints.txt 的文件当中，当在控制台运行程序时使用以下命令。

```
python FindNearestPoints < FindNearestPoints.txt
```

11.6　图形用户界面：找出距离最近的点对

关键点：本节在一个画布上显示一些点，找出其中距离最近的点对并且用一条直线连接这两个点。

上一节描述了一个提示用户输入点然后找出其中距离最近的点对的程序。本节给出一个图形用户界面程序（程序清单 11-5），它允许用户单击鼠标左键时在画布中创建一个点，然后程序将动态地在画布上找到距离最近的两点并且用一条直线将两点连接，如图 11-3 所示。

程序清单 11-5　NearestPointsGUI.py

```
1  import NearestPoints
2  from tkinter import * # Import all definitions from tkinter
3
4  RADIUS = 2 # Radius of the point
5
6  class NearestPointsGUI:
7      def __init__(self):
8          self.points = [] # Store self.points
9          window = Tk() # Create a window
10         window.title("Find Nearest Points") # Set title
11
12         self.canvas = Canvas(window, width = 400, height = 200)
13         self.canvas.pack()
14
15         self.canvas.bind("<Button-1>", self.addPoint)
16
17         window.mainloop() # Create an event loop
18
19     def addPoint(self, event):
20         if not self.isTooCloseToOtherPoints(event.x, event.y):
21             self.addThisPoint(event.x, event.y)
22
23     def addThisPoint(self, x, y):
24         # Display this point
25         self.canvas.create_oval(x - RADIUS, y - RADIUS,
26             x + RADIUS, y + RADIUS)
27         # Add this point to self.points list
28         self.points.append([x, y])
29         if len(self.points) > 2:
```

```
30              p1, p2 = NearestPoints.nearestPoints(self.points)
31              self.canvas.delete("line")
32              self.canvas.create_line(self.points[p1][0],
33                  self.points[p1][1], self.points[p2][0],
34                  self.points[p2][1], tags = "line")
35
36      def isTooCloseToOtherPoints(self, x, y):
37          for i in range(len(self.points)):
38              if NearestPoints.distance(x, y,
39                  self.points[i][0], self.points[i][1]) <= RADIUS + 2:
40                  return True
41
42          return False
43
44  NearestPointsGUI() # Create GUI
```

这个程序创建并显示一个画布（第 12-13 行），并且将鼠标左键单击事件与回调函数 addPoint 绑定（第 15 行）。当用户在画布中单击鼠标左键时，addPoint 处理程序被调用（第 19 ～ 21 行）。方法 isTooCloseToOtherPoints(x,y) 判断鼠标所在的点是否与已经存在的点距离太近了。如果不是，那么调用函数 addThisPoint(x,y) 将该点加入画布当中（第 21 行）。

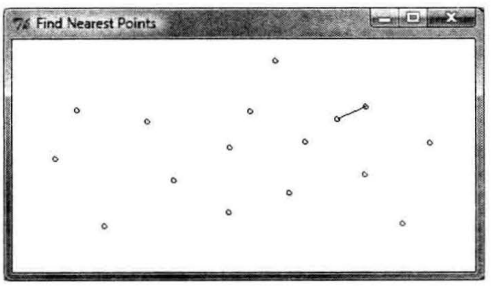


图 11-3　可以通过单击鼠标左键在画布上添加一点，距离最近的两点通过一条线相连

方法 isTooCloseToOtherPoints(x,y)（第 36 ～ 42 行）判断点（x，y）是否与画布中的其他点距离太近了。如果是，程序返回 True（第 40 行）；如果不是，程序返回 False（第 42 行）。

方法 addThisPoint(x,y)（第 23 ～ 34 行）显示画布上所有的点（第 25 ～ 26 行），将新点添加到点列表中（第 28 行），找出画布上新的距离最近的两点（第 30 行），并且绘制一条直线连接这两点（第 32 ～ 34 行）。

注意：每当有一个新点加入时，都会调用 nearestPoints 函数来寻找距离最近的点对。这个函数将计算画布上每两个点之间的距离。随着画布上点数的增加，这将是非常费时的。关于更高效的方法，参见编程题 11.50。

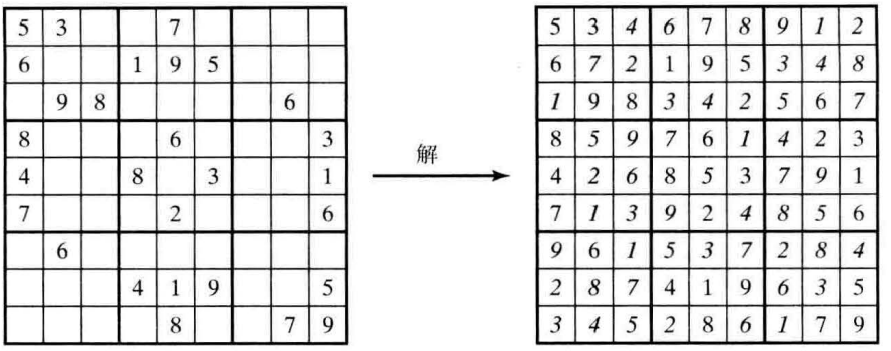
11.7　问题：数独

关键点：如何判定一个给定的数独解决方案是正确的。

本节给出一个每天报纸上都会出现的有趣问题：被大家称为数独的数字放置难题。这是一个非常有挑战性的问题。为了让初学编程者更容易入门，本节给出关于数独问题简化版本的一种解决方案，它验证一个解决方案是否正确。一个解数独问题的完整解决方案方法在附录Ⅲ .A 给出。

数独是一个可以分为更小的 3 × 3 盒子（也称为区域或块）的 9 × 9 网格，如图 11-4a 所示。一些单元被称为固定单元，已经填入从 1 到 9 的数字。目标是用 1 到 9 之间的数字填满其他空白单元，也被称为自由单元，使得每一行、每一列都含有从 1 到 9 之间的数字，如图 11-4b 所示。

为了方便起见，使用值 0 来表示一个空白单元，如图 11-5a 所示。很自然，这个网格可以用一个二维列表来表示，如图 11-5b 所示。



a）数独问题　　b）解

图　11-4

为了找到数独问题的解决方案，必须将网格中每一个 0 值用一个合适的 1 到 9 之间的数字代替。对图 11-4b 的解，列表 grid 应该如图 11-6 所示。

5	3	0	0	7	0	0	0	0
6	0	0	1	9	5	0	0	0
0	9	8	0	0	0	0	6	0
8	0	0	0	6	0	0	0	3
4	0	0	8	0	3	0	0	1
7	0	0	0	2	0	0	0	6
0	6	0	0	0	0	0	0	0
0	0	0	4	1	9	0	0	5
0	0	0	0	8	0	0	7	9

a）

```
grid =
  [[5, 3, 0, 0, 7, 0, 0, 0, 0],
   [6, 0, 0, 1, 9, 5, 0, 0, 0],
   [0, 9, 8, 0, 0, 0, 0, 6, 0],
   [8, 0, 0, 0, 6, 0, 0, 0, 3],
   [4, 0, 0, 8, 0, 3, 0, 0, 1],
   [7, 0, 0, 0, 2, 0, 0, 0, 6],
   [0, 6, 0, 0, 0, 0, 2, 8, 0],
   [0, 0, 0, 4, 1, 9, 0, 0, 5],
   [0, 0, 0, 0, 8, 0, 0, 7, 9]
  ]
```

b）

图 11-5　可以用一个二维列表表示一个网格

```
A solution grid is
  [[5, 3, 4, 6, 7, 8, 9, 1, 2],
   [6, 7, 2, 1, 9, 5, 3, 4, 8],
   [1, 9, 8, 3, 4, 2, 5, 6, 7],
   [8, 5, 9, 7, 6, 1, 4, 2, 3],
   [4, 2, 6, 8, 5, 3, 7, 9, 1],
   [7, 1, 3, 9, 2, 4, 8, 5, 6],
   [9, 6, 1, 5, 3, 7, 2, 8, 4],
   [2, 8, 7, 4, 1, 9, 6, 3, 5],
   [3, 4, 5, 2, 8, 6, 1, 7, 9]
  ]
```

图 11-6　解决方案被存在列表 grid 中

假设输入一个数独问题的解决方案。如何判断这个解决方案是否正确呢？这里有两种方法：

- 一种方法是验证每一行、每一列和盒子都有从 1 到 9 的数字。
- 另一种方法是验证每一个单元。每个单元都必须包含从 1 到 9 的数字，且每一行、每一列和每个盒子里的单元都必须不同。

程序清单 11-6 提示用户输入一个数独解决方案并且报告这个解决方案是否合法。使用第二种方法来判定解决方案是否正确。将 isValid 函数放在程序清单 11-7 中的独立模块中，这样，这个函数就能够被其他程序使用。

程序清单 11-6 TestCheckSudokuSolution.py

```
from CheckSudokuSolution import isValid

def main():
    # Read a Sudoku solution
    grid = readASolution()

    if isValid(grid):
        print("Valid solution")
    else:
        print("Invalid solution")

# Read a Sudoku solution from the console
def readASolution():
    print("Enter a Sudoku puzzle solution:")
    grid = []
    for i in range(9):
        line = input().strip().split()
        grid.append([eval(x) for x in line])

    return grid

main() # Call the main function
```

```
Enter a Sudoku puzzle solution:
9 6 3 1 7 4 2 5 8 Enter
1 7 8 3 2 5 6 4 9 Enter
2 5 4 6 8 9 7 3 1 Enter
8 2 1 4 3 7 5 9 6 Enter
4 9 6 8 5 2 3 1 7 Enter
7 3 5 9 6 1 8 2 4 Enter
5 8 9 7 1 3 4 6 2 Enter
3 1 7 2 4 6 9 8 5 Enter
6 4 2 5 9 8 1 7 3 Enter
Valid solution
```

程序清单 11-7 CheckSudokuSolution.py

```
# Check whether a solution is valid
def isValid(grid):
    for i in range(9):
        for j in range(9):
            if grid[i][j] < 1 or grid[i][j] > 9 \
               or not isValidAt(i, j, grid):
                return False
    return True # The fixed cells are valid

# Check whether grid[i][j] is valid in the grid
def isValidAt(i, j, grid):
    # Check whether grid[i][j] is valid in i's row
    for column in range(9):
        if column != j and grid[i][column] == grid[i][j]:
            return False

    # Check whether grid[i][j] is valid in j's column
    for row in range(9):
        if row != i and grid[row][j] == grid[i][j]:
            return False

    # Check whether grid[i][j] is valid in the 3-by-3 box
```

```
23      for row in range((i // 3) * 3, (i // 3) * 3 + 3):
24          for col in range((j // 3) * 3, (j // 3) * 3 + 3):
25              if row != i and col != j and \
26                      grid[row][col] == grid[i][j]:
27                  return False
28
29      return True # The current value at grid[i][j] is valid
```

在程序清单 11-6 中，程序调用 readASolution() 函数（第 5 行）来读取一个数独解决方案并且返回一个表示数独网格的二维列表。

函数 isValid(grid) 判断网格中的值是否为空。它检测网格中是否所有值都在 1 到 9 之间以及每个值是否合法（第 7 ～ 10 行）。

程序清单 11-7 中的 isValidAt(i,j,grid) 函数检测列表 grid 中的 grid[i][j] 值是否合法。它检测 grid[i][j] 是否在行 i 中出现了多次（第 18 ～ 20 行），是否在列 j 中出现了多次（第 13 ～ 15 行），是否在 3 × 3 的方块中出现了多次（第 23 ～ 27 行）。

如何定位一个盒子中的单元？对于任意的 grid[i][j]，包含 grid[i][j] 的 3 × 3 盒子中第一个单元是 grid[(i//3)*3][(j//3)*3]，如图 11-7 所示。

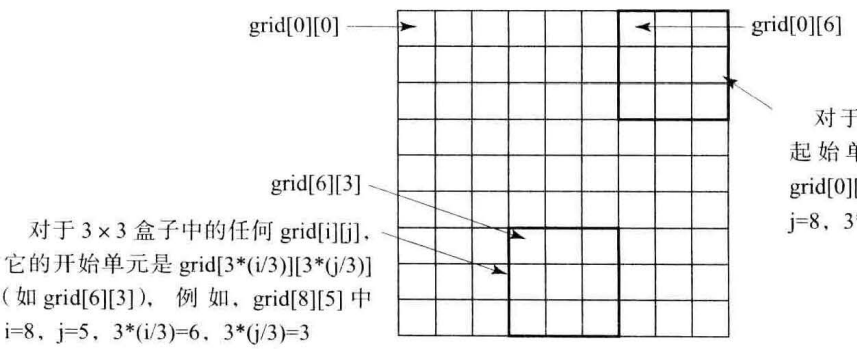


图 11-7　3 × 3 盒子中的第一个单元的位置决定这个盒子中其他单元的位置

通过上述说明，可以很容易判别一个盒子中的所有单元。例如，如果 grid[r][c] 是一个 3 × 3 盒子的起始单元，那么这个盒子中所有的单元可以通过下面的嵌套循环来遍历。

```
# Get all cells in a 3-by-3 box starting at grid[r][c]
for row in range(r, r + 3):
    for col in range(c, c + 3):
        # grid[row][col] is in the box
```

从控制台输入 81 个数字是非常繁重的。当你测试程序时，可以将输入存入一个名为 CheckSudokuSolution.txt 的文件中（参见 www.cs.armstrong.edu/liang/data/CheckSudokuSolution.txt），在命令行使用下面的命令来运行程序。

```
python TestCheckSudokuSolution.py < CheckSudokuSolution.txt
```

11.8　实例研究：数独图形用户界面

关键点：如何创建一个检测数独解决方案是否正确的图形用户界面程序。

上节的程序从控制台读取一个数独解决方案并且检测这个解决方案是否正确。本节给出一个图形用户界面程序，从 Entry 小构件输入解决方案，并且单击“Validate”按钮来检测这个解决方案是否正确，如图 11-8 所示。

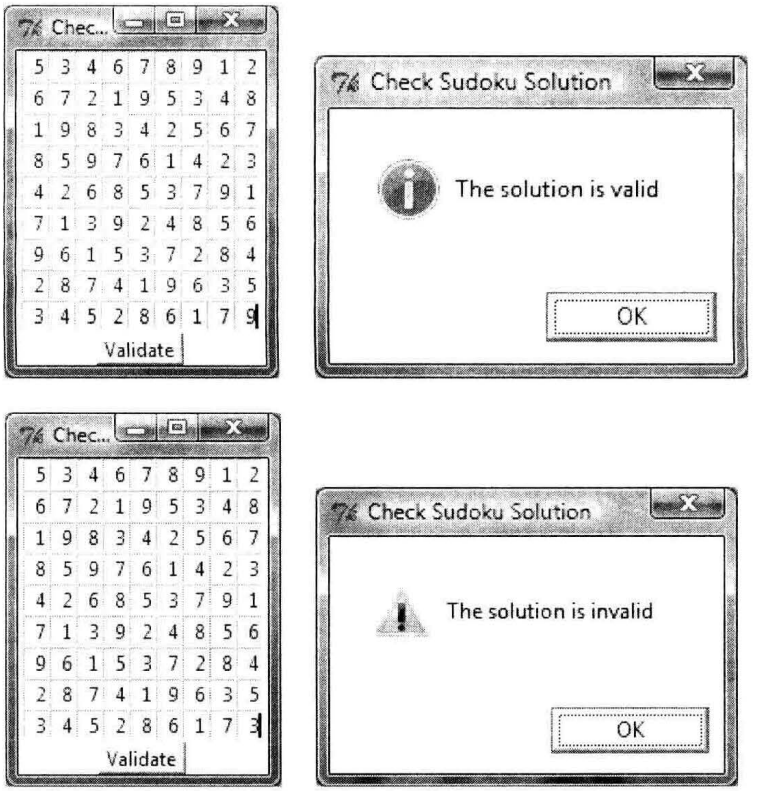


图 11-8　可以在 Entry 小构件中输入数字并单击 Validate 按钮来检测解决方案是否正确

完整的程序在清单 11-8 中给出。

程序清单 11-8　SudokuGUI.py

```
from tkinter import * # Import all definitions from tkinter
import tkinter.messagebox # Import tkinter.messagebox
from CheckSudokuSolution import isValid # Defined in Listing 11.7

class SudokuGUI:
    def __init__(self):
        window = Tk() # Create a window
        window.title("Check Sudoku Solution") # Set title

        frame = Frame(window) # Hold entries
        frame.pack()

        self.cells = [] # A list of variables tied to entries
        for i in range(9):
            self.cells.append([])
            for j in range(9):
                self.cells[i].append(StringVar())

        for i in range(9):
            for j in range(9):
                Entry(frame, width = 2, justify = RIGHT,
                    textvariable = self.cells[i][j]).grid(
                        row = i, column = j)

        Button(window, text = "Validate",
            command = self.validate ).pack()

        window.mainloop() # Create an event loop
```

```
29
30      # Check if the numbers entered are a valid solution
31      def validate(self):
32          # Get the numbers from the entries
33          values = [[eval(x.get())
34                     for x in self.cells[i]] for i in range(9)]
35
36          if isValid(values):
37              tkinter.messagebox.showinfo("Check Sudoku Solution",
38                                          "The solution is valid")
39          else:
40              tkinter.messagebox.showwarning("Check Sudoku Solution",
41                                          "The solution is invalid")
42
43  SudokuGUI() # Create GUI
```

程序创建一个名为 cells 的二维列表（第 13 ～ 17 行）。列表 cells 中每一个单元对应于相应输入框中的值（第 19 ～ 23 行）。这些输入框通过网格管理器在框架内创建和放置。一个按钮被创建并放置在框架之下（第 25 ～ 26 行）。当按钮被单击时，会调用回调处理程序 validate（第 31 ～ 41 行）。这个函数获取这些输入框中的值，并将它们放入二维列表 values 中（第 33 ～ 34 行），然后调用 isValid 函数（在程序清单 11-7 中定义）来检测这些来自输入框的数是否是一个有效的解决方案（第 36 行）。使用 Tkinter 的标准对话框来显示这个解决方案是否有效（第 36 ～ 41 行）。

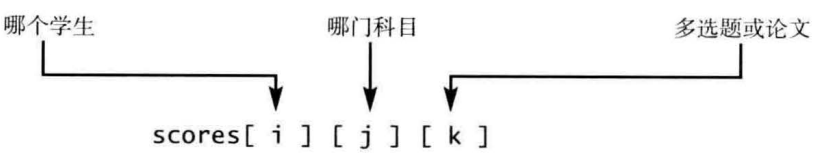
11.9　多维列表

关键点：二维列表是包含了一维列表的列表，而三维列表是包含了二维列表的列表。

在前面的小节中，使用过二维列表来表示矩阵或表格。偶尔，你需要表示 n 维数据。可以创建一个任意 n 维的列表。例如：可以使用一个三维列表来存储 6 名学生 5 门考试的成绩，其中每科考试成绩由两部分构成（多选题和论文）。下面的语法创建一个名为 scores 的三维列表。

```
scores = [
    [[11.5, 20.5], [11.0, 22.5], [15, 33.5], [13, 21.5], [15, 2.5]],
    [[4.5, 21.5], [11.0, 22.5], [15, 34.5], [12, 20.5], [14, 11.5]],
    [[6.5, 30.5], [11.4, 11.5], [11, 33.5], [11, 23.5], [10, 2.5]],
    [[6.5, 23.5], [11.4, 32.5], [13, 34.5], [11, 20.5], [16, 11.5]],
    [[8.5, 26.5], [11.4, 52.5], [13, 36.5], [13, 24.5], [16, 2.5]],
    [[11.5, 20.5], [11.4, 42.5], [13, 31.5], [12, 20.5], [16, 6.5]]]
```

scores[0][1][0] 是指第一位学生第二门课的多选题成绩，它是 11.0。scores[0][1][1] 是指第一位学生第二门课的论文成绩，它是 22.5。下图描绘了列表中每个值的含义。



一个多维列表是每个元素是其他列表的列表。具体地说，一个三维列表是由二维列表组成的列表，而一个二维列表是由一维列表组成的列表。例如：scores[0] 和 scores[1] 都是二维列表，而 scores[0][0]、scores[0][1]、scores[1][0] 和 scores[1][1] 都是一维列表并且每个列表都含有两个元素。len(scores) 是 6，len(scores[0]) 是 5 而 len(scores[0][0]) 是 2。

11.9.1　问题：每日温度和湿度

假设一个气象站记录下每天每个小时的温度和湿度并且将过去十天里的这些温度和湿度数据存储到一个文本文件 weather.tx t 当中（参见 www.cs.armstrong.edu/liang/data/weather.txt）。文件中的每一行含有 4 个数字，它们分别表示日期、时间、温度和湿度。文件中的内容如图 a 所示。

```
1 1 76.4 0.92
1 2 77.7 0.93
...
10 23 97.7 0.71
10 24 98.7 0.74
```

a)

```
10 24 98.7 0.74
1 2 77.7 0.93
...
10 23 97.7 0.71
1 1 76.4 0.92
```

b)

注意：文件中每一行并不一定要按顺序排列。例如：文件内容可能如图 b 所示。

编写一个程序计算这 10 天每天的平均温度和湿度。可以使用输入重定向从文件读取数据并将这些数据存入一个名为 data 的三维列表中。data 的第一个下标范围在 0 到 9 之间，它表示这 10 天中第几天；第二个下标范围在 0 到 23 之间，它表示 24 个小时周期；第三个下标取 0 或 1，分别表示温度和湿度。注意：文件中日期和时间分别取值从 1 到 10 和从 1 到 24。由于列表下标是从 0 开始的，所以 data[0][0][0] 存储的是第一天 1 点时的温度，而 data[9][23][1] 存储的是第 10 天 24 点的湿度。

程序在程序清单 11-9 中给出。

程序清单 11-9　Weather.py

```
def main():
    NUMBER_OF_DAYS = 10
    NUMBER_OF_HOURS = 24

    # Initialize data
    data = []
    for i in range(NUMBER_OF_DAYS):
        data.append([])
        for j in range(NUMBER_OF_HOURS):
            data[i].append([])
            data[i][j].append(0) # Temperature value
            data[i][j].append(0) # Humidity value

    # Read input using input redirection from a file
    for k in range(NUMBER_OF_DAYS * NUMBER_OF_HOURS):
        line = input().strip().split()
        day = eval(line[0])
        hour = eval(line[1])
        temperature = eval(line[2])
        humidity = eval(line[3])
        data[day - 1][hour - 1][0] = temperature
        data[day - 1][hour - 1][1] = humidity

    # Find the average daily temperature and humidity
    for i in range(NUMBER_OF_DAYS):
        dailyTemperatureTotal = 0
        dailyHumidityTotal = 0
        for j in range(NUMBER_OF_HOURS):
            dailyTemperatureTotal += data[i][j][0]
            dailyHumidityTotal += data[i][j][1]

```

```
32          # Display result
33          print("Day " + str(i) + "'s average temperature is "
34              + str(dailyTemperatureTotal / NUMBER_OF_HOURS))
35          print("Day " + str(i) + "'s average humidity is "
36              + str(dailyHumidityTotal / NUMBER_OF_HOURS))
37
38  main() # Call the main function
```

```
Day 0's average temperature is 77.7708
Day 0's average humidity is 0.929583
Day 1's average temperature is 77.3125
Day 1's average humidity is 0.929583
...
Day 9's average temperature is 79.3542
Day 9's average humidity is 0.9125
```

可以使用下面的命令来运行程序。

```
python Weather.py < Weather.txt
```

程序在第 6 ～ 12 行创建一个三维列表来存储温度和湿度数据，这个三维列表中的元素初始值为 0。第 15 ～ 22 行的循环将输入读取至列表中。可以从键盘上直接输入数据，但是这样做会非常笨拙。为了方便起见，将数据存储在一个文件当中，运行程序时通过输入重定向来从文件中读取这些数据。程序将每一行的输入作为字符串读入然后将字符串分隔成一个列表（第 16 行）以获得日期、时间、温度和湿度（第 17 ～ 20 行）。第 25 ～ 30 行的循环将一天中每个小时的温度都加起来存入 dailyTemperatureTotal 变量中，一天中每个小时的湿度加起来存入 dailyHumidityTotal 变量中。程序的第 33 ～ 36 行显示每天的平均温度和湿度。

11.9.2　问题：猜生日

程序清单 4-3 是一个猜生日的程序。这个程序能简化为将数字存入一个三维列表并使用循环提示用户输入答案，如程序清单 11-10 所示。这个程序的示例运行和程序清单 4-3 一样。

程序清单 11-10　GuessBirthdayUsingList.py

```
 1  def main():
 2      day = 0 # Day to be determined
 3
 4      dates = [
 5        [[ 1,  3,  5,  7],
 6         [ 9, 11, 13, 15],
 7         [17, 19, 21, 23],
 8         [25, 27, 29, 31]],
 9        [[ 2,  3,  6,  7],
10         [10, 11, 14, 15],
11         [18, 19, 22, 23],
12         [26, 27, 30, 31]],
13        [[ 4,  5,  6,  7],
14         [12, 13, 14, 15],
15         [20, 21, 22, 23],
16         [28, 29, 30, 31]],
17        [[ 8,  9, 10, 11],
18         [12, 13, 14, 15],
19         [24, 25, 26, 27],
20         [28, 29, 30, 31]],
21        [[16, 17, 18, 19],
22         [20, 21, 22, 23],
23         [24, 25, 26, 27],
```

```
24          [28, 29, 30, 31]]]
25
26      for i in range(5):
27          print("Is your birthday in Set" + str(i + 1) + "?")
28          for j in range(4):
29              for k in range(4):
30                  print(format(dates[i][j][k], "4d"), end = " ")
31              print()
32
33          answer = eval(input("Enter 0 for No and 1 for Yes: "))
34
35          if answer == 1:
36              day += dates[i][0][0]
37
38      print("Your birthday is " + str(day))
39
40  main() # Call the main function
```

第 4 ～ 24 行创建了一个三维列表 dates。这个列表存储了 5 个二维数字列表，每个二维列表都是一个 4 × 4 的二维列表。

开始于第 26 行的循环显示每个二维列表的数字并且提示用户回答生日是否在这个二维列表当中（第 33 行）。如果生日在这个列表中，那么列表中的第一个数字（dates[i][0][0]）将被加入变量 day 中（第 36 行）。这个程序与程序清单 4-3 中的程序是相同的，除了这个程序是将五组数据集合存在一个列表。这样组织数据是更好的，因为数据可以在循环中重用和处理。

检查点

11.7　给出下面代码的输出结果。

```
def f(m):
    for i in range(len(m)):
        for j in range(len(m[i])):
            for k in range(len(m[j])):
                m[i][j][k] += 1

def printM(m):
    for i in range(len(m)):
        for j in range(len(m[i])):
            for k in range(len(m[j])):
                print(m[i][j][k], end = "")
        print()

m = [[[0, 0], [0, 1]], [[0, 0], [0, 1]]]

printM(m)
f(m)
printM(m)
```

关键术语

column index（列下标）
multidimensional list（多维列表）
nested list（嵌套列表）
row index（行下标）
two-dimensional list（二维列表）

本章总结

1. 二维列表能用来存储二维数据，例如：一张表和一个矩阵。
2. 二维列表也是列表。二维列表中的元素是一个列表。

3. 二维列表中的元素可以使用下面的语法来访问。

```
listName[rowIndex][columnIndex].
```

4. 可以利用一个列表的列表来形成多维列表以存储多维数据。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

第 11.2 ～ 11.3 节

*11.1 （按列求和）编写一个函数，使用下面的函数头返回矩阵里特定某一列所有元素的和。

```
def sumColumn(m, columnIndex):
```

编写一个测试程序，程序读入一个 3 × 4 的矩阵，然后显示每一列的和值。下面是一个示例运行。

```
Enter a 3-by-4 matrix row for row 0: 1.5 2 3 4 Enter
Enter a 3-by-4 matrix row for row 1: 5.5 6 7 8 Enter
Enter a 3-by-4 matrix row for row 2: 9.5 1 3 1 Enter
Sum of the elements for column 0 is 16.5
Sum of the elements for column 1 is 9.0
Sum of the elements for column 2 is 13.0
Sum of the elements for column 3 is 13.0
```

*11.2 （矩阵的主对角线元素求和）使用下面的函数头编写一个函数对一个 n × n 的整数矩阵的主对角线上的所有元素求和。

```
def sumMajorDiagonal(m):
```

主对角线是一条从方阵左上角到右下角的对角线。编写一个测试程序，读入一个 4 × 4 矩阵然后显示矩阵主对角线所有元素的和。下面是一个示例运行。

```
Enter a 4-by-4 matrix row for row 1: 1 2 3 4 Enter
Enter a 4-by-4 matrix row for row 2: 5 6.5 7 8 Enter
Enter a 4-by-4 matrix row for row 3: 9 10 11 12 Enter
Enter a 4-by-4 matrix row for row 4: 13 14 15 16 Enter

Sum of the elements in the major diagonal is 34.5
```

*11.3 （按分数对学生排序）改写程序清单 11-2 按正确答案个数的升序显示学生。

**11.4 （计算员工的一周工作时间）假设所有员工每周的工作时间被存在一张表中。每一行有 7 列记录一位员工每周 7 天每天的工作时间。例如：下面这张表存储了 8 位员工每周的工作时间。编写一个程序显示员工和他们每周总的工作时间，按总工作时间降序排列员工。

	Su	M	T	W	Th	F	Sa
Employee 0	2	4	3	4	5	8	8
Employee 1	7	3	4	3	3	4	4
Employee 2	3	3	4	3	3	2	2
Employee 3	9	3	4	7	3	4	1
Employee 4	3	5	4	3	6	3	8
Employee 5	3	4	4	6	3	4	4
Employee 6	3	7	4	8	3	8	4
Employee 7	6	3	5	9	2	7	9

11.5 （代数：矩阵相加）编写一个函数，实现矩阵相加。函数头如下所示。

```
def addMatrix(a, b):
```

两个相加矩阵的维数必须相同且它们元素的类型必须一致。假设 c 是相加之后的矩阵。那么每个元素 C_{ij} 是 $a_{ij} + b_{ij}$。例如，对两个 3 × 3 矩阵 a 和 b 相加，c 是

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} + \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix} = \begin{pmatrix} a_{11}+b_{11} & a_{12}+b_{12} & a_{13}+b_{13} \\ a_{21}+b_{21} & a_{22}+b_{22} & a_{23}+b_{23} \\ a_{31}+b_{31} & a_{32}+b_{32} & a_{33}+b_{33} \end{pmatrix}$$

编写一个测试程序提示用户输入两个 3 × 3 矩阵，然后显示它们相加的结果。下面是一个示例运行。

```
Enter matrix1: 1 2 3 4 5 6 7 8 9 ↵Enter
Enter matrix2: 0 2 4 1 4.5 2.2 1.1 4.3 5.2 ↵Enter
The matrices are added as follows:
 1.0 2.0 3.0       0.0 2.0 4.0      1.0 4.0 11.0
 4.0 5.0 6.0   +   1.0 4.5 2.2  =   5.0 11.5 8.2
 11.0 8.0 11.0     1.1 4.3 5.2      8.1 12.3 14.2
```

**11.6 （代数：矩阵相乘）编写一个函数实现矩阵相乘。函数头如下所示。

```
def multiplyMatrix(a, b)
```

为了实现矩阵 a 乘以 b，矩阵 a 的列数必须等于矩阵 b 的行数且两个矩阵的元素类型必须相同或者兼容。假设 c 是矩阵相乘的结果，矩阵 a 的列数为 n。每个元素 C_{ij} 是 $a_{i1} \times b_{1j} + a_{i2} \times b_{2j} + \cdots + a_{in} \times b_{nj}$。例如，对于两个 3 × 3 矩阵 a 和 b，c 是

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{pmatrix} \times \begin{pmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{pmatrix} = \begin{pmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{pmatrix}$$

其中 $c_{ij} = a_{i1} \times b_{1j} + a_{i2} \times b_{2j} + a_{i3} \times b_{3j}$。

编写一个测试程序提示用户输入两个 3 × 3 矩阵，然后显示它们相乘的结果。下面是一个示例运行。

```
Enter matrix1: 1 2 3 4 5 6 7 8 9 ↵Enter
Enter matrix2: 0 2 4 1 4.5 2.2 1.1 4.3 5.2 ↵Enter
The multiplication of the matrices is
 1 2 3      0 2.0 4.0       5.3 23.9 24
 4 5 6   *  1 4.5 2.2   =   11.6 56.3 58.2
 7 8 9      1.1 4.3 5.2     111.9 88.7 92.4
```

*11.7 （距离最近的两点）程序清单 11-3 中的程序找出二维空间里距离最近的两点。修改这个程序使得它能找出三维空间里距离最近的两点。使用一个二维列表来表示这些点。使用下面的点来测试程序。

```
points = [[-1, 0, 3], [-1, -1, -1], [4, 1, 1],
  [2, 0.5, 9], [3.5, 2, -1], [3, 1.5, 3], [-1.5, 4, 2],
  [5.5, 4, -0.5]]
```

计算三维空间两点（x1,y1,z1）和（x2,y2,z2）之间距离的公式是 $\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$。

**11.8 （所有距离最近的对点）修改程序清单 11-4，找出所有距离都取最小值的点对。

***11.9 （游戏：井字游戏）在井字游戏中，两位玩家依次在一个 3×3 的网格中的单元格使用他们对应的符号（X 或 O）做出标记。当有一位玩家将 3 个他对应的标记在一条对角线、一行、一列中成功连接时，游戏以这名玩家获胜结束。当两位玩家都没有成功连接且网格中的单元格全部填满时，双方平局。编写一个井字游戏程序。

程序提示两位玩家依次输入 X 符号和 O 符号。每当符号被输入时，程序在控制台上重新显示棋盘并且判定游戏的状态（一方获取、平局或者继续游戏）。下面是一个示例运行。

```
-------------
|   |   |   |
-------------
|   |   |   |
-------------
|   |   |   |
-------------
Enter a row (0, 1, or 2) for player X: 1 Enter
Enter a column (0, 1, or 2) for player X: 1 Enter

-------------
|   |   |   |
-------------
|   | X |   |
-------------
|   |   |   |
-------------
Enter a row (0, 1, or 2) for player O: 1 Enter
Enter a column (0, 1, or 2) for player O: 2 Enter
-------------
|   |   |   |
-------------
|   | X | O |
-------------
|   |   |   |
-------------
Enter a row (0, 1, or 2) for player X:

...

-------------
| X |   |   |
-------------
| O | X | O |
-------------
|   |   | X |
-------------
X player won
```

*11.10 （最大的行和列）编写一个程序将 0 和 1 随机填入一个 4×4 的矩阵中，打印这个矩阵然后找出含有 1 最多的行和列。下面是这个程序的一个示例运行。

```
0011
0011
1101
1010
The largest row index: 2
The largest column index: 2, 3
```

**11.11 （游戏：九个正反面）9 个硬币放在 3×3 的矩阵中，这些硬币要么是正面向上要么就是反面向上。可以用值 0（正面朝上）和值 1（反面朝上）来反映硬币相应的状态。下面是一些例子。

```
0 0 0    1 0 1    1 1 0    1 0 1    1 0 0
0 1 0    0 0 1    1 0 0    1 1 0    1 1 1
0 0 0    1 0 0    0 0 1    1 0 0    1 1 0
```

每种状态也都可以用一个二进制数表示。例如，上面各个矩阵的状态对应的二进制数是：

0000 10000 1010 01100 1101 00001 1011 10100 1001 11110

这里会有 512 种可能性。因此，可以使用十进制数 0、1、2、3、…、511 来表示矩阵所有的状态。编写一个程序提示用户输入一个 0 到 511 之间的整数，然后用字符 H 和 T 来显示矩阵对应的状态。下面是一个示例运行。

```
Enter a number between 0 and 511: 7 Enter
H H H
H H H
T T T
```

用户输入 7，它对应的二进制数是 0000 00111。又因为 0 代表 H，1 代表 O，因此输出是正确的。

**11.12 （经济学应用：计算税款）使用列表重写程序清单 4-7。对于每种报税情况，都有 6 种相应税率。每种税率都对应相应的应纳税收入总额。例如：对应纳税收入总额为 400 000 的单身而言，8350 按 10% 纳税，（33 950-8350）按 15% 纳税，（82 250-33 950）按 25% 纳税，（171 550-82 250）按 28% 纳税，（372 950-171 550）按 33% 纳税，（400 000-372 950）按 35% 纳税。这六种税率对所有纳税人都是相同的，可以用下面的列表表示。

```
rates = [0.10, 0.15, 0.25, 0.28, 0.33, 0.35]
```

所有纳税人的不同税率的门限可以用一个如下的二维列表表示。

```
brackets = [
  [8350, 33950, 82250, 171550, 372950],   # Single filer
  [16700, 67900, 137050, 208850, 372950], # Married jointly
  [8350, 33950, 68525, 104425, 186475], # Married separately
  [11950, 45500, 117450, 190200, 372950] # Head of household
]
```

假设单身纳税人的应纳税收入总额为 400 000 美元。他的税率可以按如下方法计算。

```
tax = brackets[0][0] * rates[0] +
  (brackets[0][1] - brackets[0][0]) * rates[1] +
  (brackets[0][2] - brackets[0][1]) * rates[2] +
  (brackets[0][3] - brackets[0][2]) * rates[3] +
  (brackets[0][4] - brackets[0][3]) * rates[4] +
  (400000 - brackets[0][4]) * rates[5]
```

*11.13 （定位最大元素）编写下面的函数返回一个二维列表中最大元素的位置。

```
def locateLargest(a):
```

函数的返回值是一个含有两个元素的一维列表。这两个元素指出了二维列表中最大元素的行下标和列下标。编写一个测试程序，提示用户输入一个二维列表，并显示该列表中最大元素的位置。下面是一个示例运行。

```
Enter the number of rows in the list: 3 Enter
Enter a row: 23.5 35 2 10 Enter
Enter a row: 4.5 3 45 3.5 Enter
Enter a row: 35 44 5.5 11.6 Enter
The location of the largest element is at (1, 2)
```

**11.14 （探究矩阵）编写一个程序提示用户输入一个方阵的长度，将 0 和 1 随机填入方阵，打印这个矩阵，并找出由全 0 或全 1 组成的行、列和主对角线。下面是该程序的一个示例运行。

```
Enter the size for the matrix: 4 Enter
0111
```

```
0000
0100
1111
All 0s on row 1
All 1s on row 3
No same numbers in a column
No same numbers in the major diagonal
```

第 11.4 ～ 11.9 节

*11.15 （几何学：共线？）编程题 6.19 给出一个用于测试三点是否共线的函数。编写下面的函数来测试是否 points 列表中的所有点都共线。

```
def sameLine(points):
```

编写一个程序提示用户输入五个点，然后显示它们是否共线。下面是示例运行。

```
Enter five points: 3.4 2 6.5 11.5 2.3 2.3 5.5 5 -5 4 ↵Enter
The five points are not on the same line
```

```
Enter five points: 1 1 2 2 3 3 4 4 5 5 ↵Enter
The five points are on the same line
```

*11.16 （按 y 坐标对一个点列表排序）编写下面的函数来将列表中的点按 y 坐标排序。每个点都是由 x 和 y 坐标这两个值组成的列表。

```
# Returns a new list of points sorted on the y-coordinates
def sort(points):
```

例如：点集 [[4，2]，[1，7]，[4，5]，[1，2]，[1，1]，[4，1]] 将会被排序为 [[1，1]，[4，1]，[1，2]，[4，2]，[4，5]，[1，7]]。编写一个测试程序，使用 print(list) 显示点集 [[4，34]，[1，7.5]，[4，8.5]，[1，−4.5]，[1，4.5]，[4，6.6]] 的排序结果。

**11.17 （金融风暴）银行间会相互借贷。在经济艰难时期，如果一家银行宣告破产，它可能无法偿还贷款。一家银行的资产总额是它现有的余额加上它给其他银行的贷款。图 11-9 显示了五家银行之间的借贷关系和自身资产余额。各银行的现有余额分别是 25、125、175、75 和 181 百万美元。结点 1 到结点 2 的有向边表明银行 1 将 40 百万美元借给银行 2。

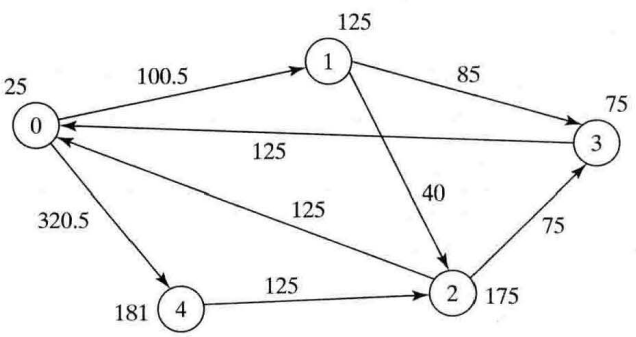


图 11-9　银行间的贷款关系

如果一家银行的资产总额低于一定的界限，这家银行就身处险境了。此时该银行将无法给其他银行偿还它的贷款，因此贷方银行无法将这些贷款计入它的资产总额。因此，该贷方银行的资产总额同样可能低于这个界限，它也就可能会变得危险。编写一个程序找出所有不安全的

银行。程序应读入如下所示的输入值。它首先读入两个整数 n 和 limit，其中 n 表明银行的数量，limit 是维持银行安全的最低资产总额。接着读入 n 行信息，它们以 0 到 n-1 标识这 n 家银行。行中的第一个数是银行的余额，第二个数指明从银行贷款的银行数量，其余数字都是由两个数组成的数对。每对数描述一个借方。每对数中的第一个数是借方银行的标识，第二个数是所借的金额。例如，图 11-9 中的五家银行的输入值如下所示（注意界限是 201）。

```
5 201
25 2 1 100.5 4 320.5
125 2 2 40 3 85
175 2 0 125 3 75
75 1 0 125
181 1 2 125
```

银行 3 的资产总额是（75 + 125），低于 201，因此银行 3 是不安全的。在银行 3 不再安全之后，银行 1 的资产总额下降到低于界限（125 + 40），因此银行 1 也变成不安全的。程序的输出应该是：

```
Unsafe banks are 3 1
```

（提示：使用二维列表 borrowers 来表示贷款，borrowers[i][j] 表示银行 i 贷款给银行 j 的总额。一旦银行 j 不再安全，borrowers[i][j] 应当置为 0。）

*11.18 （按行打乱）使用下面的函数头编写一个函数对一个二维列表按行打乱。

```
def shuffle(m):
```

编写一个测试程序打乱下面的矩阵。

```
m = [[1, 2], [3, 4], [5, 6], [7, 8], [9, 10]]
```

**11.19 （模式识别：四个连续的相同数字）编写下面的函数来检测一个二维列表中是否有四个连续的相同数字，不管这四个数字是在水平方向、垂直方向还是对角线方向。

```
def isConsecutiveFour(values):
```

编写一个测试程序提示用户输入二维列表的行数和列数，然后再输入二维列表的元素值。当列表中包含四个连续的相同数字时，程序显示 True；否则，程序显示 False。下面是一些具有连续相同数字的例子。

0 1 0 3 1 6 1	0 1 0 3 1 6 1	0 1 0 3 1 6 1	0 1 0 3 1 6 1
0 1 6 8 6 0 1	0 1 6 8 6 0 1	0 1 6 8 6 0 1	0 1 6 8 6 0 1
5 6 2 1 8 2 9	5 **5** 2 1 8 2 9	5 6 2 1 **6** 2 9	**9** 6 2 1 8 2 9
6 5 6 1 1 9 1	6 **5** 6 1 1 9 1	6 5 6 **6** 1 9 1	6 **9** 6 1 1 9 1
1 3 6 1 4 0 7	1 **5** 6 1 4 0 7	1 3 **6** 1 4 0 7	1 3 **9** 1 4 0 7
3 3 3 4 0 7	3 **5** 3 3 4 0 7	3 **6** 3 3 4 0 7	3 3 3 **9** 4 0 7

***11.20 （游戏：四点相连）四点相连游戏是一个双人游戏，两位玩家依次将不同颜色的棋子下在一个 7 列 6 行的垂直悬挂的网格上，如 cs.armstrong.edu/liang/ConnectFour/ConnectFour.html 所示。游戏的目标是在对手之前将 4 个相同颜色的棋子在一行、一列或一条斜线上连接起来。程序提示两位玩家依次下红色或黄色的棋子。每当一个棋子下完，程序就会在控制台上重新绘制棋盘并且判决当前游戏状态（一方获胜，平局或者游戏继续）。下面是一个示例运行。

```
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
---------------
Drop a red disk at column (0-6): 0 Enter

| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
|R| | | | | | |
---------------
Drop a yellow disk at column (0-6): 3 Enter
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |
|R| | |Y| | | |

...
...
...

Drop a yellow disk at column (0-6): 6 Enter
| | | | | | | |
| | | | | | | |
| | | |R| | | |
| | | |Y|R|Y| |
| | |R|Y|Y|Y|Y|
|R|Y|R|Y|R|R|R|
---------------
The yellow player won
```

***11.21 （游戏：多个数独解决方案）关于数独难题的完整解法在附录Ⅲ.A 中给出。一个数独问题可能有多个解决方案。修改附录Ⅲ.A 中的程序 Sudoku.py，使得它能够显示该数独问题所有解决方案的个数。如果存在多个解决方案则显示其中的两个数独解决方案。

**11.22 （偶数个 1）编写一个程序生成一个 6×6 的二维矩阵，其中的元素填入 0 或者 1。显示这个矩阵，并且检测每一行和每一列中是否都有偶数个 1。

*11.23 （游戏：翻转单元）假设有一个 6×6 的矩阵，矩阵中的元素不是取 0 就是取 1。为了使所有的行和列都有偶数个 1，翻转矩阵中的一个单元（即将 1 翻转为 0 将 0 翻转为 1）使得矩阵所有行和列有偶数个 1。编写一个程序找出应当翻转哪个单元。程序提示用户输入一个 6×6 的取 0 或 1 值的二位列表，并且找出第一个违反偶数个 1 准则的行号 r 和列号 c。翻转单元就在（r,c）位置上。

*11.24 （检测独解的解决方案）程序清单 11-7 通过检查网格中所有数字是否合法来检测一个数独的解决方案是否有效。重写这个程序检测是否网格的每一行、每一列和每个盒子的值都有从 1 到 9 的数字。

*11.25 （马尔可夫矩阵）如果一个 n×n 的方阵中所有元素都是正的且每一列的和都是 1，那么这个矩

阵就被称为一个正定马尔可夫矩阵。编写下面的函数来检测一个矩阵是否是马尔可夫矩阵。

```
def isMarkovMatrix(m):
```

编写一个测试程序提示用户输入一个 3 × 3 的数字矩阵并且测试它是否是马尔可夫矩阵。下面是示例运行。

```
Enter a 3-by-3 matrix row by row:
0.15 0.875 0.375 Enter
0.55 0.005 0.225 Enter
0.30 0.12 0.4 Enter
It is a Markov matrix
```

```
Enter a 3-by-3 matrix row by row:
0.95 -0.875 0.375 Enter
0.65 0.005 0.225 Enter
0.30 0.22 -0.4 Enter
It is not a Markov matrix
```

*11.26 （按行排序）实现下面函数来将一个二维列表按行排序。函数返回一个新列表而原始的列表保持不变。

```
def sortRows(m):
```

编写一个测试程序提示用户输入一个 3 × 3 的数字矩阵并且显示一个新的按行排序的矩阵。下面是一个示例运行。

```
Enter a 3-by-3 matrix row by row:
0.15 0.875 0.375 Enter
0.55 0.005 0.225 Enter
0.30 0.12 0.4 Enter

The row-sorted list is
0.15 0.375 0.875
0.005 0.225 0.55
0.12 0.30 0.4
```

*11.27 （按列排序）实现下面的函数将一个二维列表按列排序。函数返回一个新列表而原始列表保持不变。

```
def sortColumns(m):
```

编写一个测试程序提示用户输入一个 3 × 3 的数字矩阵并且显示一个新的按列排序的矩阵。下面是一个示例运行。

```
Enter a 3-by-3 matrix row by row:
0.15 0.875 0.375 Enter
0.55 0.005 0.225 Enter
0.30 0.12 0.4 Enter

The column-sorted list is
0.15 0.005 0.225
0.3  0.12  0.375
0.55 0.875 0.4
```

11.28 （严格相等列表）如果两个列表 m1 和 m2 中对应的元素是相等的，那么这两个列表被认为是严格相等的。使用下面的函数头编写一个函数，如果列表 m1 和 m2 是严格相等的，返回 True。

```
def equals(m1, m2):
```

编写一个测试程序，程序要求用户输入两个 3 × 3 整数列表并且显示着这两个列表是否是严格相等的。下面是示例运行。

```
Enter m1: 51 22 25 6 1 4 24 54 6 ↵Enter
Enter m2: 51 22 25 6 1 4 24 54 6 ↵Enter
The two lists are strictly identical
```

```
Enter m1: 51 25 22 6 1 4 24 54 6 ↵Enter
Enter m2: 51 22 25 6 1 4 24 54 6 ↵Enter
The two lists are not strictly identical
```

11.29 （相等列表）如果两个列表 m1 和 m2 含有相同的内容，那么认为这两个列表是相等的。使用下面的函数头编写一个函数，如果列表 m1 和 m2 是相等的，返回 True。

```
def equals(m1, m2):
```

编写一个测试程序提示用户输入两个 3 × 3 整数列表并且显示这两个列表是否相等。下面是示例运行。

```
Enter m1: 51 25 22 6 1 4 24 54 6 ↵Enter
Enter m2: 51 22 25 6 1 4 24 54 6 ↵Enter
The two lists are identical
```

```
Enter m1: 51 5 22 6 1 4 24 54 6 ↵Enter
Enter m2: 51 22 25 6 1 4 24 54 6 ↵Enter
The two lists are not identical
```

*11.30 （代数：解线性方程）编写一个函数解下面的二元一次方程组。

$$a_{00}x + a_{01}y = b_0$$
$$a_{10}x + a_{11}y = b_1$$

$$x = \frac{b_0a_{11} - b_1a_{01}}{a_{00}a_{11} - a_{01}a_{10}} \qquad y = \frac{b_1a_{00} - b_0a_{10}}{a_{00}a_{11} - a_{01}a_{10}}$$

函数头为：`def linearEquation(a, b):`

如果 $a_{00}a_{11}-a_{01}a_{10}$ 为 0，函数返回 None；否则，函数通过一个列表返回 x、y 的解。编写一个测试程序，程序提示用户输入方程组的系数 a_{00}、a_{01}、a_{10}、a_{11}、b_0 和 b_1 并且显示结果。如果 $a_{00}a_{11}-a_{01}a_{10}$ 为 0，程序输出“The equation has no solution”。下面是示例运行。

```
Enter a00, a01, a10, a11, b0, b1: 9, 4, 3, -5, -6, -21 ↵Enter
x is -2.0 and y is 3.0
```

```
Enter a00, a01, a10, a11, b0, b1: 1, 2, 2, 4, 40, 5 ↵Enter
The equation has no solution
```

*11.31 （几何：交点）编写一个函数返回两条直线的交点。使用编程题 4.25 中的公式可以找出两条直线的交点。假设（x1,y1）和（x2,y2）是直线 1 上的两个点，而（x3,y3）和（x4,y4）是直线 2 上的两个点。函数头如下所示。

```
def getIntersectingPoint(points):
```

这四个点被存储在一个 4 × 2 的二维列表 points 中，其中（points[0][0],points[0][1]）是（x1,y1）。函数通过一个列表返回交点，如果这两条直线平行则返回 None。编写一个程序提示用户输入四个点并且显示交点。示例运行参见编程题 4.25。

*11.32 （几何：三角形的面积）使用下面的函数头编写一个函数返回一个三角形的面积。

```
def getTriangleArea(points):
```

三角形的顶点被存在一个 3 × 2 的二维列表 points 中，其中（points[0][0],points[0][1]）是点（x1,y1）。三角形的面积可以使用编程题 2.14 中的公式来计算。如果这三个点是在一条直线上则函数返回 None。编写一个程序提示用户输入三个点并且显示这三个点构成的三角形的面积。下面是一个示例运行。

```
Enter x1, y1, x2, y2, x3, y3: 2.5 2 5 -1.0 4.0 2.0 Enter
The area of the triangle is 2.25
```

```
Enter x1, y1, x2, y2, x3, y3: 2 2 4.5 4.5 6 6 Enter
The three points are on the same line
```

*11.33 （几何：多边形的子区域）一个凸四边形能够划分成 4 个三角形，如图 11-10 所示。

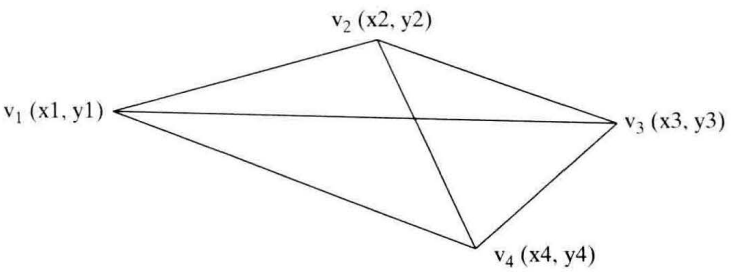


图 11-10　一个四边形通过 4 个顶点定义

编写一个程序提示用户输入四边形对应顶点的坐标并且按升序显示划分的四个三角形的面积。下面是一个示例运行。

```
Enter x1, y1, x2, y2, x3, y3, x4, y4: -2.5 2 4 4 3 -2 -2 -3.5 Enter
The areas are 6.17 7.96 8.08 10.42
```

*11.34 （几何：最右最低点）在计算几何学中，通常需要在一个点集中寻找最右最低的点。编写下面的函数返回一个点集中最右最低点的点。

```
# Return a list of two values for a point
def getRightmostLowestPoint(points):
```

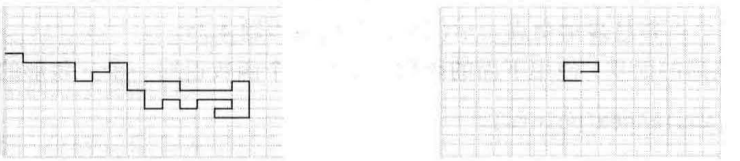
编写一个测试程序提示用户输入 6 个点的坐标然后显示其中最右最低点。下面是一个示例运行。

```
Enter 6 points: 1.5 2.5 -3 4.5 5.6 -7 6.5 -7 8 1 10 2.5 Enter
The rightmost lowest point is (6.5, -7)
```

*11.35 （中心城市）给定一个城市的集合，中心城市是指到其他城市的距离之和最小的城市。编写一个程序提示用户输入城市的个数和所有城市的位置坐标，并且找出中心城市。

**11.36 （使用 Turtle 仿真：自我规避随机游走）在一个网格中的自我规避游走是一条从一点到另一点的路径，并且这条路径不会经过同一个点两次。自我规避游走在物理、化学和数学有很多的应

用。它可以用来模拟链状实体，例如溶剂和高分子聚合物。编写一个 Turtle 程序显示一条随机路径，该路径从中心点开始在边界上的某点结束，如图 11-11a 所示，或者在死点处结束（即该点被其他已经经过的 4 个点包围），如图 11-11b 所示。假设这个网格的大小是 16 × 16。



a）在边界点结束的一条路径　　b）在死点结束的一条路径

图　11-11

**11.37 （仿真：自我规避随机游走）编写一个仿真程序来展示随着网格大小的扩大，路径在死点结束的概率将会提高。程序模拟网格大小从 10 变化到 80。对于每一种网格大小，仿真 10 000 次自我规避随机游走然后显示在死点结束的概率，输出如下所示。

```
For a lattice of size 10, the probability of dead-end paths is 11.6%
For a lattice of size 11, the probability of dead-end paths is 14.0%
...
For a lattice of size 80, the probability of dead-end paths is 99.5%
```

**11.38 （Turtle：绘制多边形 / 折线）编写下面的函数绘制一个多边形或折线来连接列表中的所有点。列表中的每个元素是有两个坐标的列表。

```
# Draw a polyline to connect all the points in the list
def drawPolyline(points):

# Draw a polygon to connect all the points in the list and
# close the polygon by connecting the first point with the last point
def drawPolygon(points):

# Fill a polygon by connecting all the points in the list
def fillPolygon(points):
```

**11.39 （Tkinter：四个连续的相同数字）为编程题 11.19 编写一个图形用户界面程序，如图 11-12 所示。通过 6 行 7 列的网格的文本域输入数字。当单击“ Solve”按钮时，如果网格中存在 4 个连续相同数字，将标记出这 4 个连续数字。

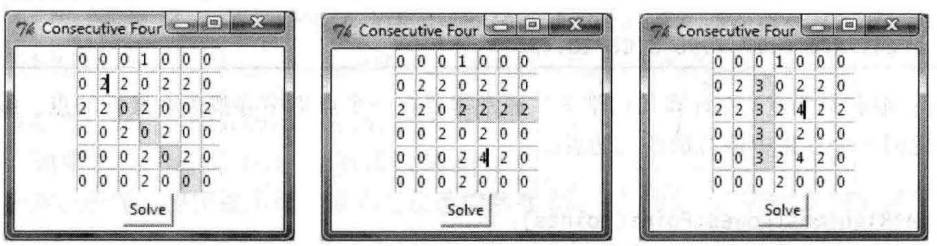


图 11-12　单击“Solve”按钮来标记出这四个在对角、行或列上连续的数字

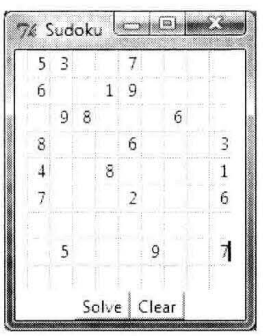
**11.40 （猜首都）编写一个程序重复提示用户输入一个国家的首都。一旦收到用户的输入，程序报告用户输入答案是否正确。假设将 50 个国家和它们的首都存在一个二维列表当中，如图 11-13 所示。程序提示用户回答所有国家的首都并且显示用户答对的总个数。用户答案是不分大小写的。使用列表表示下表中的数据来实现这个程序。下面是一个示例运行。

Alabama	Montgomery
Alaska	Juneau
Arizona	Phoenix
...	...
...	...

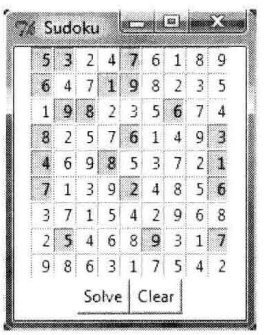
图 11-13　国家和首都名称都存在一个二维列表中

```
What is the capital of Alabama? Montogomery ↵Enter
The correct answer should be Montgomery
What is the capital of Alaska? Juneau ↵Enter
Your answer is correct
What is the capital of Arizona? ...
...
The correct count is 35
```

***11.41 （Tkinter：数独的解决方案）数独的完整解决方案在附录Ⅲ.A 中给出。编写一个 GUI 程序让用户输入一个数独难题，然后单击“Solve”按钮时将显示一个解决方案，如图 11-14 所示。



a）用户输入一个数独难题



b）单击“Solve”按钮显示这个数独问题的解决方案

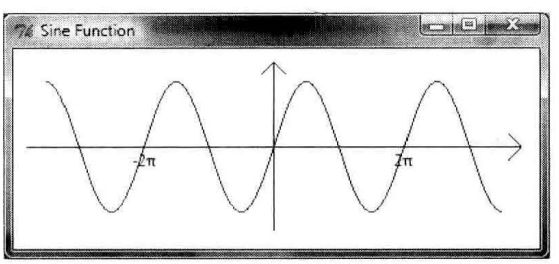
图　11-14

*11.42 （Tkinter：绘制正弦函数）编程题 5.52 使用 Turtle 绘制一条正弦函数曲线。使用 Tkinter 重写这个程序来绘制正弦函数曲线，如图 11-15a 所示。

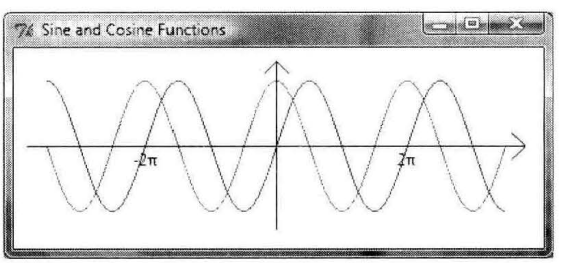
提示：统一码中 π 的编码是 \u03c0。为了显示 -2π，使用语句 turtle.write("−2\u03c0")。对于像 sin(x) 的三角函数，x 是用弧度表示。使用下面循环将点添加到多边形 p 中。

```
p = []
for x in range(-175, 176):
    p.append([x, -50 * math.sin((x / 100.0) * 2 *
    math.pi)])
```

−2π 在坐标（−100，−15）处显示，坐标系的中心是（0，0），2π 在坐标（100，−15）处显示。



a）绘制一条正弦函数曲线



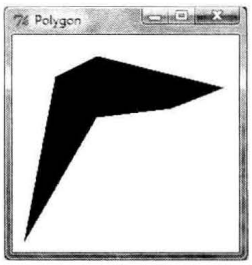
b）绘制一条蓝色正弦曲线和一条红色余弦曲线

图　11-15

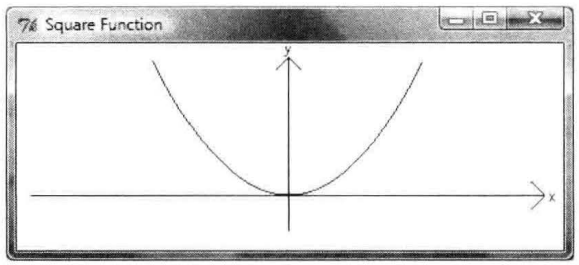
*11.43 （Tkinter：绘制正弦函数和余弦函数）编程题 5.53 中使用 Turtle 绘制正弦和余弦函数。利用 Tkinter 重写绘制正弦和余弦函数的程序，如图 11-15b 所示。

11.44 （Tkinter：绘制多边形）编写一个程序提示用户输入六个点的坐标并且将这些点构成填充的多边形，如图 11-16a 所示。注意：可以使用语句 canvas.create_polygon(points) 绘制多边形，其中 points 为存储点的 x、y 坐标的二维列表。

*11.45 （Tkinter：绘制平方函数）编程题 5.54 绘制一条平方函数曲线。使用 Tkinter 重写这个程序，程序运行结果如图 11-16b 所示。



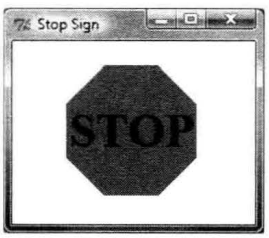
a）通过点的列表绘制一个多边形



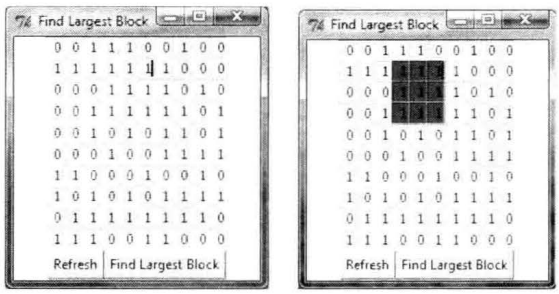
b）程序绘制函数 $f(x) = x^2$ 的曲线

图　11-16

*11.46 （Tkinter：显示一个 STOP 标志）编写程序显示一个 STOP 标志，如图 11-17a 所示。这个正六边形是红色的，而文本是黑色的。



a）程序显示一个 STOP 标志



b）～c）单击“Refresh”按钮时程序显示一个随机取 0 或 1 的矩阵

图　11-17

*11.47 （Tkinter：最大方块）编写一个程序显示一个 10 × 10 的矩阵，如图 11-17b 所示。矩阵的每个元素不是 0 就是 1，单击“Refresh”按钮时程序给矩阵中的元素随机赋值为 0 或 1。在一个文本框内居中显示矩阵的元素值。允许用户修改矩阵的元素值。单击“Find Largest Block”按钮来找出矩阵中全为 1 的最大子方阵。标记出这个子方阵的数字，如图 11-17c 所示。

**11.48 （几何：寻找边界矩形）编写一个程序让用户动态增添或删除一个二维面板上的点，如图 11-18 所示。当有点被加入或删除时，能够包含这些点的最小边界矩形会自动更新。假设这些点的半径都是 10 个像素。

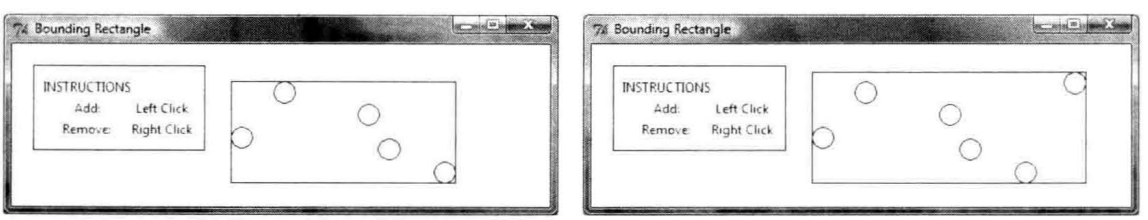


图 11-18　程序允许用户动态实时地添加和删除点并且显示边界矩形

11.49 （游戏：井字游戏）重写编程题 9.6 来显示一个下方有“Refresh”按钮的井字游戏板，如图 11-19 所示。

11.50 （几何：找出最近点）每当有一个新点加入面板时，程序清单 11-5 通过重新计算每个点对之间的距离找出距离最短的两点。这个方法虽然正确的但效率很低。一个效率更高的算法如下所示。

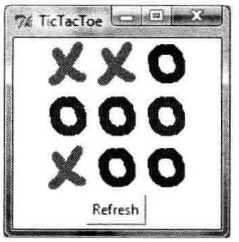


图 11-19　程序显示一个下方有“Refresh”按钮的井字游戏板

```
Let d be the current shortest distance between two
       nearest points p1 and p2
Let p be the new point added to the plane
For each existing point t:
    if distance(p, t) < d:
       d = distance(p, t)
       p1, p2 = p, t
```

使用这个新算法重写程序清单 11-5。

**11.51 （对学生排序）编写一个程序提示用户在同一行输入学生的名字和他们的成绩，程序按他们成绩的升序打印学生姓名。（提示：创建一个列表。列表中的每个元素含有两个元素：姓名和成绩的子列表。用 sort 方法依据成绩对列表排序。）

```
Enter students' names and scores: John 34 Jim 45 Peter 59
  Tim 45 Enter
John    34
Jim     45
Tim     45
Peter   59
```

**11.52 （拉丁方阵）拉丁方阵是指一个含有 n 个不同拉丁字母的 n × n 的方阵，每个字母在方阵的每一行和每一列都只出现一次。编写一个程序，程序提示用户输入方阵的大小 n，如实例输出所示并且检测输入的列表是否是一个拉丁方阵。方阵的字母是从 A 开始的 n 个字母。

```
Enter number n: 4 Enter
Enter 4 rows of letters separated by spaces:
A B C D Enter
B A D C Enter
C D B A Enter
D C A B Enter
The input list is a Latin square
```

```
Enter number n: 3 Enter
Enter 3 rows of letters separated by spaces:
A F D Enter
Wrong input. The letters must be from A to C.
```

第 12 章

Introduction to Programming Using Python

继承和多态

学习目标

- 通过继承由超类定义子类（第 12.2 节）。
- 覆盖子类中的方法（第 12.3 节）。
- 探究 Object 类和它的方法（第 12.4 节）。
- 理解多态和动态绑定（第 12.5 节）。
- 使用 isinstance 函数来判定一个对象是否是类的一个实例（第 12.6 节）。
- 设计一个 GUI 类显示一个可重用的时钟（第 12.7 节）。
- 探索类之间的关系（第 12.8 节）。
- 使用组合和继承关系设计类（第 12.9 ～ 12.11 节）。

12.1 引言

关键点：面向对象程序设计（OOP）可以从现有类定义新类，这被称为继承。

如本书之前所讨论的，面向过程范型的重点在函数的设计上，而面向对象范型的重点放在将数据和方法封装到对象上。使用面向对象范型的软件设计将重点放在对象和对对象的操作上。面向对象方法是在面向过程泛型上增加一个维度，这个维度将数据和操作都集成到对象里。

继承是面向对象泛型软件重用方面一个重要且功能强大的特征。假设要定义类来对圆、矩形和三角形建模。这些类都有很多共同的特性。设计这些类来避免冗余并使系统更易理解和更易维护的最好方式是什么？答案就是使用继承。

12.2 父类和子类

关键点：继承可以定义一个通用类（父类），随后将它扩展为更多特定的类（子类）。

使用类来对同一类型的对象建模。不同类可能会有一些共同的特性和行为，这些共同的特性和行动都囊括在一个类中，然后它们就可以被其他类所共享。继承可以定义一个通用类，随后将它扩展为定义更多特定的类。这些特定的类继承了通用类中的特征和方法。

考虑一下几何对象。假设要设计类建模像圆和矩形这样的集合对象。几何对象有许多共同的属性和行为；例如，它们是可以用某种特定颜色画出来，可以是填充的或者不填充的。这样，一个通用类 GeometricObject 可以被用来建模所有的几何对象。这个类包括属性 color 和 filled，以及适用于这些属性的 get 和 set 方法。假设该类还包括 dateCreated 属性以及 getDateCreated() 和 __srt__() 方法。__srt__() 方法返回代表该对象的字符串。

由于圆是一个特殊类型的几何对象，所以它和其他几何对象共享相同的属性和方法。因此，扩展 GeometricObject 类来定义 Circle 类是很有意义的。同样的，也可以定义 Rectangle 为 GeometricObject 的子类。图 12-1 显示这些类之间的关系。指向父类的三角箭头用来表示

相关的两个类之间的继承关系。

注意：在面向对象的术语中，如果类 C1 扩展自另一个类 C2，那么就将 C1 称为派生类、孩子类或子类，而将 C2 称为超类基类、父类或超类。

子类除了从它的父类中继承可访问的数据域和方法，还可以有其他数据域和方法。在我们的例子中：

- Circle 类继承了 GeometricObject 类所有的可访问数据域和方法。除此之外，它还有一个新的数据域 radius，以及与 radius 相关的 get 和 set 方法。它还包括 getArea()、getPerimeter() 和 getDiameter() 方法来返回圆的面积、周长和直径。

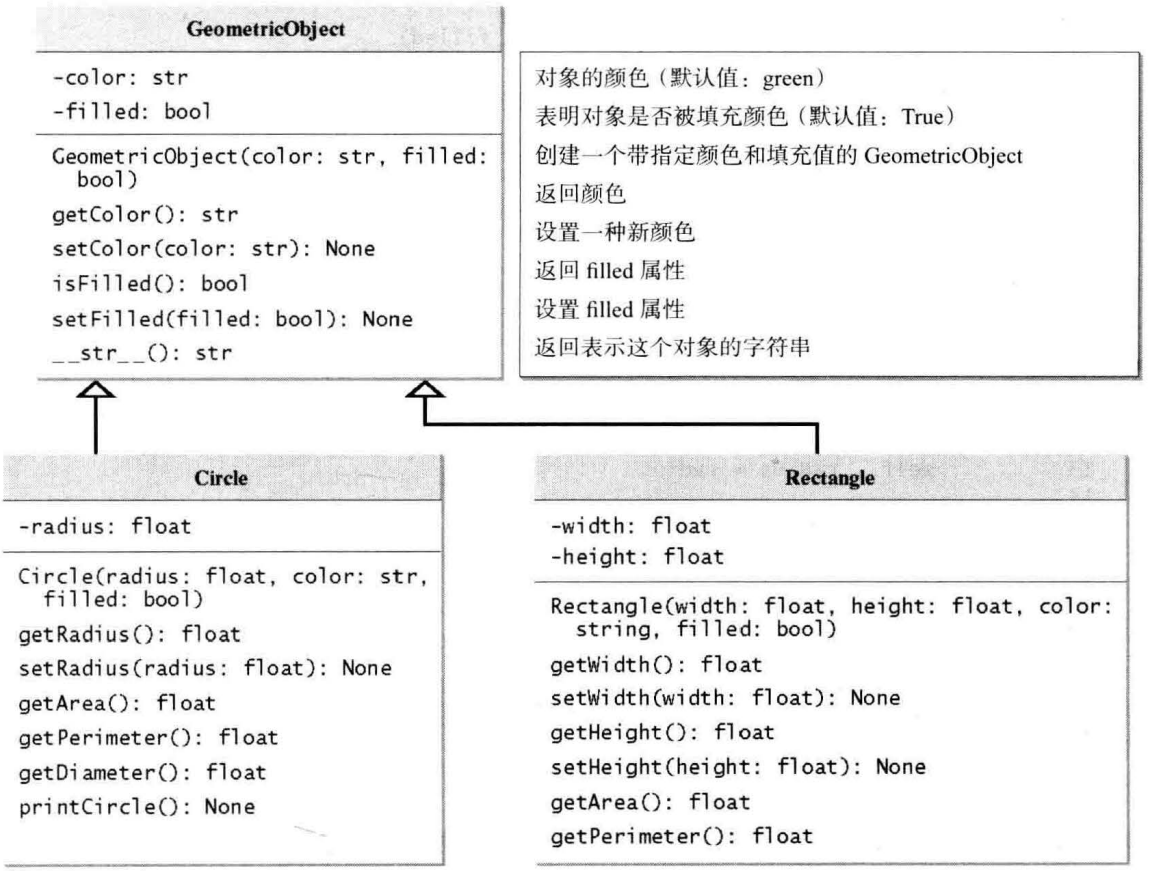


图 12-1　GeometricObject 类是 Circle 和 Rectangle 的超类

- Rectangle 类从 GeometricObject 类继承所有可访问的数据域和方法。除此之外，它还有数据域 width 和 height，以及和它们相关的 get 和 set 方法。它还包括 getArea() 和 getPerimeter() 方法来返回矩形的面积和周长。

GeometricObject 类、Circle 类和 Rectangle 类分别在程序清单 12-1、程序清单 12-2 和程序清单 12-3 中给出。

程序清单 12-1　GeometricObject.py

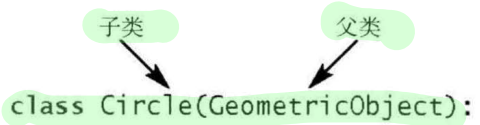
```
1  class GeometricObject:
2      def __init__(self, color = "green", filled = True):
3          self.__color = color
4          self.__filled = filled
5
```

```
6       def getColor(self):
7           return self.__color
8
9       def setColor(self, color):
10          self.__color = color
11
12      def isFilled(self):
13          return self.__filled
14
15      def setFilled(self, filled):
16          self.__filled = filled
17
18      def __str__(self):
19          return "color: " + self.__color + \
20              " and filled: " + str(self.__filled)
```

程序清单 12-2 CircleFromGeometricObject.py

```
1  from GeometricObject import GeometricObject
2  import math # math.pi is used in the class
3
4  class Circle(GeometricObject):
5      def __init__(self, radius):
6          super().__init__()
7          self.__radius = radius
8
9      def getRadius(self):
10         return self.__radius
11
12     def setRadius(self, radius):
13         self.__radius = radius
14
15     def getArea(self):
16         return self.__radius * self.__radius * math.pi
17
18     def getDiameter(self):
19         return 2 * self.__radius
20
21     def getPerimeter(self):
22         return 2 * self.__radius * math.pi
23
24     def printCircle(self):
25         print(self.__str__() + " radius: " + str(self.__radius))
```

Circle 类使用下面的语法派生自 GeometricObject 类（程序清单 12-1）。



这就告诉 Python：Circle 类继承了 GeometricObject 类。这样，它就继承了 getcolor、setColor、isFilled、setFilled 和 __str__ 方法。printCircle 方法调用定义在超类中的 __str__() 方法以获得属性（第 25 行）。

super().__init__() 调用父类的 __init__() 方法（第 6 行）。这在创建定义在父类中的数据域是很重要的。

注意：也可以选择使用下面的语句调用父类 __init__() 方法：

```
GeometricObject.__init__(self)
```

这是一种 Python 现在仍然支持的较老语法，但是并不推荐这种风格。super() 指向父类。

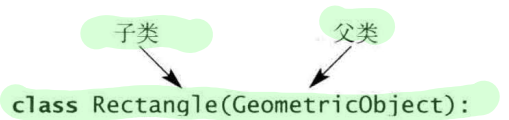
使用 super() 来避免显式地指向父类。每当使用 super() 来调用一个方法时，不需要传递 self 参数。例如，你应该使用：

```
super().__init__()
```

而不是使用

```
super().__init__(self)
```

Rectangle 类派生自 GeometricObject 类（程序清单 12-1），它在定义程序清单 12-3 中定义。



程序清单 12-3 RectangleFromGeometricObject.py

```
1  from GeometricObject import GeometricObject
2
3  class Rectangle(GeometricObject):
4      def __init__(self, width = 1, height = 1):
5          super().__init__()
6          self.__width = width
7          self.__height = height
8
9      def getWidth(self):
10         return self.__width
11
12     def setWidth(self, width):
13         self.__width = width
14
15     def getHeight(self):
16         return self.__height
17
18     def setHeight(self, height):
19         self.__height = self.__height
20
21     def getArea(self):
22         return self.__width * self.__height
23
24     def getPerimeter(self):
25         return 2 * (self.__width + self.__height)
```

程序清单 12-4 中的代码创建了 Circle 和 Rectangle 的对象，并调用这些对象上的方法 getArea() 和 getPerimeter()。__str__() 方法继承自 GeometricObject 类，并且从 Circle 对象（第 5 行）和 Rectangle 对象（第 11 行）上调用。

程序清单 12-4 TestCircleRectangle.py

```
1  from CircleFromGeometricObject import Circle
2  from RectangleFromGeometricObject import Rectangle
3
4  def main():
5      circle = Circle(1.5)
6      print("A circle", circle)
7      print("The radius is", circle.getRadius())
8      print("The area is", circle.getArea())
9      print("The diameter is", circle.getDiameter())
10
11     rectangle = Rectangle(2, 4)
12     print("\nA rectangle", rectangle)
13     print("The area is", rectangle.getArea())
```

```
14      print("The perimeter is", rectangle.getPerimeter())
15
16  main() # Call the main function
```

```
A circle color: green and filled: True
The radius is 1.5
The area is 7.06858347058
The diameter is 3.0

A rectangle color: green and filled: True
The area is 8
The perimeter is 12
```

第 6 行调用 print 函数来打印一个圆。回顾第 8.5 节，它与下面语句一样。

```
print("A circle", circle.__str__())
```

__str__() 方法并没有在 Circle 类中定义，但是它在 GeometricObject 类中定义。因为 Circle 类是 GeometricObject 类的子类，所以 Circle 对象可以调用 __str__() 方法。

__str__() 方法显示一个 GeometricObject 对象的 color 和 filled 属性（程序清单 12-1 的第 18 ～ 20 行）。GeometricObject 对象的 color 属性默认为 green，filled 属性默认为 True（程序清单 12-1 的第 2 行）。因为 Circle 类继承自 GeometricObject 类，那么 Circle 对象的 color 属性默认值也为 green，filled 属性默认值也为 True。

下面是关于继承的值得注意的几点。

- 和传统的理解相反，子类并不是父类的一个子集。实际上，一个子类通常比它的父类包含更多的信息和方法。
- 继承被建模为是关系（is-a），但并不是所有的是关系都应该使用继承建模。例如：正方形是一个矩形，但是不应该从 Rectangle 类来扩展一个 Square 类，因为 width 和 height 属性并不适用于正方形。而是应该定义一个扩展自 GeometricObject 类的 Square 类，并为正方形的边定义 side 属性。
- 不要仅仅为了重用方法这个原因而盲目地扩展一个类。例如：尽管 Person 类和 Tree 类可以共享类似高度和重量这样的通用特性，但是从 Person 类扩展出 Tree 类是毫无意义的。一个父类和它的子类之间必须存在是关系。
- Python 允许从几个类派生出一个子类。这种能力被称为多重继承。使用下面的语法来定义一个从多个类派生出的类。

```
class Subclass(SuperClass1, SuperClass2, ...):
    initializer
    methods
```

检查点

12.1　如何定义一个扩展自父类的类？ super() 是什么？如何调用父类的初始化函数？

12.2　运行下面的程序时有何问题？如何修改它？

```
class A:
    def __init__(self, i = 0):
        self.i = i

class B(A):
    def __init__(self, j = 0):
        self.j = j
```

```
def main():
    b = B()
    print(b.i)
    print(b.j)

main() # Call the main function
```

12.3　对或错？子类是父类的一个子集。

12.4　Python 支持多重继承吗？如何定义一个扩展自多个类的类？

12.3　覆盖方法

关键点：为了覆盖父类的方法，子类中的方法必须使用与父类方法一样的方法头。

子类从父类继承方法。有时，子类需要修改定义在父类中的方法的实现。这称作方法覆盖。

GeometricObject 类中的 __str__() 方法返回表示几何对象的字符串。这个方法可以被覆盖返回表示圆的字符串。为了覆盖它，在程序清单 12-2，CircleFromGeometricObject.py 中加入下面的新方法。

```
1  class Circle(GeometricObject):
2      # Other methods are omitted
3
4      # Override the __str__ method defined in GeometricObject
5      def __str__(self):
6          return super().__str__() + " radius: " + str(radius)
```

__str__() 方法在 GeometricObject 类中定义，在 Circle 类中修改。在这两个类中定义的该方法都可以在 Circle 类中使用。要在 Circle 类中调用定义在 GeometricObject 中的 __str__() 方法，使用 super.__str__()（第 6 行）。

同样的，可以覆盖 Rectangle 类中 __str__() 方法，如下所示。

```
def __str__(self):
    return super().__str__() + " width: " + \
        str(self.__width) + " height: " + str(self.__height)
```

在本书其余内容中，我们假设 GeometricObject 类中的 __str__() 方法在 Circle 类和 Rectangle 类中已经被覆盖。

注意：在 Python 语言中，可以通过在一个方法名前加两条下划线来定义一个私有方法（参见第 7 章）。私有方法不能被覆盖。如果子类中的方法在父类中是私有的，那么这两个方法是完全不相关的，即使这两个方法有同样的方法名。

检查点

12.5　真或假？

（a）父类中非私有的方法能够被子类覆盖。

（b）子类能够覆盖父类的私有方法。

（c）子类能够覆盖父类的初始化方法。

（d）当创建一个类的对象时，这个类的父类的初始化方法被自动调用。

12.6　给出下面程序的输出。

```
class A:
    def __init__(self, i = 0):
        self.i = i
```

```
    def m1(self):
        self.i += 1

class B(A):
    def __init__(self, j = 0):
        super().__init__(3)
        self.j = j

    def m1(self):
        self.i += 1

def main():
    b = B()
    b.m1()
    print(b.i)
    print(b.j)

main() # Call the main function
```

12.4 object 类

关键点：Python 中的所有类都派生自 object 类。

object 类在 Python 库中定义。如果一个类定义时没有指定它的父类，那么它的父类默认是 object 类。例如，下面两个类的定义是一样的。

```
class ClassName:
    ...
```

Equivalent

```
class ClassName(object):
    ...
```

Circle 类和 Rectangle 类都是由 GeometricObject 类派生出来的。GeometricObject 类实际上是由 object 类派生出来的。熟悉 object 类中提供的方法对在类中使用它们是十分重要的。object 类中定义的所有方法都有两条前导下划线和两条后置下划线。本节主要讨论四个方法，即 __new__()、__init__()、__str__() 和 __eq__(other)。

当创建一个对象时，__new__() 方法被自动调用。这个方法随后调用 __init__() 方法来初始化这个对象。一般你只应该覆盖 __init__() 方法来初始化新类中定义的数据域。

__str__() 方法会返回一个描述该对象的字符串。默认情况下，它返回一个由该对象所属的类名以及该对象十六进制形式的内存地址组成的字符串。例如，考虑下面这些在程序清单 7-8 中定义的 Loan 类的代码。

```
loan = Loan(1, 1, 1, "Smith")
print(loan) # Same as print(loan.__str__())
```

这些代码会显示像 <Loan.Loan object at 0x01B99C10> 这样的字符串。这个信息不是很有用，或者说没有什么信息量。通常，应该覆盖这个 __str__() 方法，这样，它可以返回一个代表该对象的描述性字符串。例如，object 类中的 __str__() 方法在 GeometricObject 类中被覆盖，如程序清单 12-1 中第 18 ～ 20 行所示。

```
def __str__(self):
    return "color: " + self.__color + \
        " and filled: " + str(self.__filled)
```

如果两个对象相等，那么 __eq__(other) 方法返回 True。因此，x.__eq__(x) 返回 True，但是 x.__eq__(y) 返回 False，虽然 x 和 y 有相同的内容，但它们还是两个不同的对

象。回顾一下，x.__eq__(y) 与 x==y 是等价的（参见第 8.5 节）。

可以覆盖这个方法使两个对象内容相同时返回 True。Python 的许多像 int、float、bool、string 和 list 这样的内置类中，__eq__ 方法被覆盖使两个对象内容相同时返回 True。

检查点

12.7　对或错？

（a）所有对象都是 object 类的一个实例。

（b）如果一个类没有显式地扩展自某父类，那么它默认扩展自 object 类。

12.8　给出下面代码的输出。

```
class A:
    def __init__(self, i = 0):
        self.i = i

    def m1(self):
        self.i += 1

    def __str__(self):
        return str(self.i)

x = A(8)
print(x)
```

12.9　给出下面代码的输出。

```
class A:
    def __new__(self):
        print("A's __new__() invoked")

    def __init__(self):
        print("A's __init__() invoked")
class B(A):
    def __new__(self):
        print("B's __new__() invoked")

    def __init__(self):
        print("B's __init__() invoked")

def main():
    b = B()
    a = A()

main() # Call the main function
```

12.10　给出下面代码的输出。

```
class A:
    def __new__(self):
        self.__init__(self)
        print("A's __new__() invoked")

    def __init__(self):
        print("A's __init__() invoked")

class B(A):
    def __new__(self):
        self.__init__(self)
        print("B's __new__() invoked")
```

```
    def __init__(self):
        print("B's __init__() invoked")

def main():
    b = B()
    a = A()

main() # Call the main function
```

12.11　给出下面代码的输出。

```
class A:
    def __init__(self):
        print("A's __init__() invoked")

class B(A):
    def __init__(self):
        print("B's __init__() invoked")

def main():
    b = B()
    a = A()

main() # Call the main function
```

12.12　给出下面代码的输出。

```
class A:
    def __init__(self, i):
        self.i = i
    def __str__(self):
        return "A"

class B(A):
    def __init__(self, i, j):
        super().__init__(i)
        self.j = j

def main():
    b = B(1, 2)
    a = A(1)
    print(a)
    print(b)

main() # Call the main function
```

12.13　给出下面代码的输出。

```
class A:
    def __init__(self, i):
        self.i = i

    def __str__(self):
        return "A"

    def __eq__(self, other):
        return self.i == other.i

def main():
    x = A(1)
    y = A(1)
    print(x == y)

main() # Call the main function
```

12.5　多态和动态绑定

关键点：多态是指子类的对象可以传递给需要父类类型的参数。一个方法可以被沿着继承链的几个类执行。运行时由 Python 决定调用哪个方法，这被称为动态绑定。

面向对象程序设计的三个特点是封装、继承和多态。前面已经学习了前两个特点，本节将介绍多态性。

继承关系使一个子类继承父类的特征，并且附加一些新特征。子类是它的父类的特殊化；每个子类的实例都是其父类的实例，但是反过来就不成立。例如：每个圆都是一个几何对象，但并非每个几何对象都是圆。因此，总可以将子类的实例传给需要父类类型的参数。考虑程序清单 12-5 中的代码。

程序清单 12-5　PolymorphismDemo.py

```
1  from CircleFromGeometricObject import Circle
2  from RectangleFromGeometricObject import Rectangle
3
4  def main():
5      # Display circle and rectangle properties
6      c = Circle(4)
7      r = Rectangle(1, 3)
8      displayObject(c)
9      displayObject(r)
10     print("Are the circle and rectangle the same size?",
11         isSameArea(c, r))
12
13 # Display geometric object properties
14 def displayObject(g) :
15     print(g.__str__())
16
17 # Compare the areas of two geometric objects
18 def isSameArea(g1, g2) :
19     return g1.getArea() == g2.getArea()
20
21 main() # Call the main function
```

```
color: green and filled: True radius: 4
color: green and filled: True width: 1 height: 3
Are the circle and rectangle the same size? False
```

方法 displayObject（第 14 行）采用 GeometricObject 类型的参数。可以通过传递任何一个 GeometricObject 的实例（例如：第 8 ～ 9 行的 Circle(4) 和 Rectangle(1,3)）来调用 displayObject。使用父类对象的地方都可以使用子类的对象。这就是通常所说的多态（它源于希腊文字，意思是“多种形式”）。

在这个例子中可以看到，c 是 Circle 类的一个对象。Circle 类是 GeometricObject 类的子类。__str__() 方法在这两个类中都有定义。因此，在 displayObject 方法中 g 应当调用哪个 __str__() 方法（第 15 行）？ g 应当调用哪个 __str__() 方法由动态绑定决定。

动态绑定工作机制如下：假设对象 o 是类 C_1、C_2、…、C_{n-1}、C_n 的实例，这里的 C_1 是 C_2 的子类，C_2 是 C_3 的子类，…，C_{n-1} 是 C_n 的子类，如图 12-2 所示。这也就是说，C_n 是最通用的类，C_1 是最特定的类。在 Python 中，C_n 是 object 类。如果对象 o 调用一个方法 p，那么 Python 会依次在类 C_1、C_2、…、C_{n-1}、C_n 中查找方法 p 的实现，直到找到为止。一旦找到一个实现，就停止查找然后调用这个第一次找到的实现。

图 12-2　哪个方法将被调用是在运行时动态决定的

程序清单 12-6　DynamicBindingDemo.py

```
1  class Student:
2      def __str__(self):
3          return "Student"
4
5      def printStudent(self):
6          print(self.__str__())
7
8  class GraduateStudent(Student):
9      def __str__(self):
10         return "Graduate Student"
11
12 a = Student()
13 b = GraduateStudent()
14 a.printStudent()
15 b.printStudent()
```

```
Student
Graduate Student
```

由于a是Student的一个实例，所以在Student类中的printStudent方法采用a.printStudent()（第14行）调用，这个方法将调用Student类中的__str__()方法来返回Student。

GraduateStudent类中没有定义printStudent方法。然而，因为该方法在Student类中定义而GraduateStudent是Student的一个子类，Student类中的printStudent方法可以通过b.printStudent()来调用（第15行）。printStudent方法调用GraduateStudent的__str__()方法来显示Graduate Student，因为调用printStudent的对象b是GraduateStudent（第6和10行）。

12.6　isinstance 函数

关键点：*isinstance 函数能够用来判断一个对象是否是一个类的实例。*

假设需要修改程序清单12-5中的displayObject函数来完成以下任务：

- 显示GeometricObject实例的面积和周长。
- 如果实例是Circle，显示它的周长；如果是Rectangle，显示它的长和宽。

如何可以做到这点？可以尝试如下方式编写程序。

```
def displayObject(g):
    print("Area is", g.getArea())
    print("Perimeter is", g.getPerimeter())
    print("Diameter is", g.getDiameter())
    print("Width is", g.getWidth())
    print("Height is", g.getHeight())
```

但是，这样并不会完成上述功能，因为不是所有的GeometricObject实例都有getDiameter()、getWidth()或getHeigth()方法。例如：调用display(Circle(5))将会产生一个运行

时错误，因为 Circle 类并没有 getWidth() 和 getHeight() 方法，同样，调用 display(Rectangle (2,3)) 也会产生一个运行时错误，因为 Rectangle 类也没有 getDiameter() 方法。

可以使用 Python 内置的 isinstance 函数来解决这个问题。这个函数使用下面的语法来判断一个对象是否是一个类的实例。

```
isinstance(object, ClassName)
```

例如：isinstance("abc", str) 将返回 True，因为 "abc" 是 str 类的一个实例，但是，isinstance(12,str) 返回 False，因为 12 并不是 str 类的实例。

使用 isinstance 函数，可以如程序清单 12-7 所示来实现 displayObject 函数。

程序清单 12-7 IsinstanceDemo.py

```
1  from CircleFromGeometricObject import Circle
2  from RectangleFromGeometricObject import Rectangle
3
4  def main():
5      # Display circle and rectangle properties
6      c = Circle(4)
7      r = Rectangle(1, 3)
8      print("Circle...")
9      displayObject(c)
10     print("Rectangle...")
11     displayObject(r)
12
13 # Display geometric object properties
14 def displayObject(g):
15     print("Area is", g.getArea())
16     print("Perimeter is", g.getPerimeter())
17
18     if isinstance(g, Circle):
19         print("Diameter is", g.getDiameter())
20     elif isinstance(g, Rectangle):
21         print("Width is", g.getWidth())
22         print("Height is", g.getHeight())
23
24 main() # Call the main function
```

```
Circle...
Area is 50.26548245743669
Perimeter is 25.132741228718345
Diameter is 8
Rectangle...
Area is 3
Perimeter is 8
Width is 1
Height is 3
```

调用 displayObject(c) 来将 c 传递给 g（第 9 行）。g 现在是 Circle 类的一个实例（第 18 行）。程序显示圆的周长（第 19 行）。

调用 displayObject(r) 来将 r 传递给 g（第 11 行）。g 现在是 Rectangle 类的一个实例（第 20 行）。程序显示矩形的长和宽（第 21 ～ 22 行）。

检查点

12.14　解释封装、继承和多态。

12.15　给出下面代码的输出。

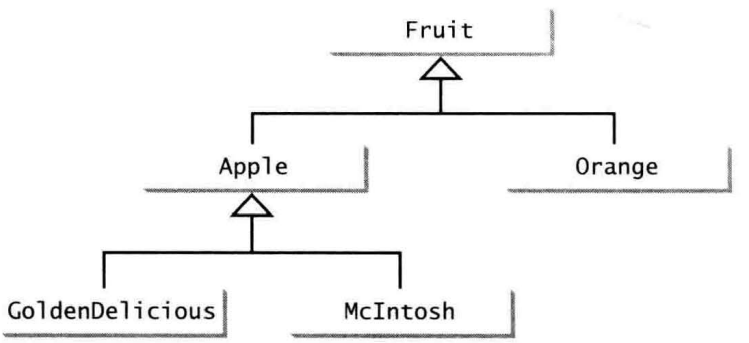
```
class Person:
    def getInfo(self):
        return "Person"

    def printPerson(self):
        print(self.getInfo())

class Student(Person):
    def getInfo(self):
        return "Student"

Person().printPerson()
Student().printPerson()
```

a)

```
class Person:
    def __getInfo(self):
        return "Person"

    def printPerson(self):
        print(self.__getInfo())

class Student(Person):
    def __getInfo(self):
        return "Student"

Person().printPerson()
Student().printPerson()
```

b)

12.16　假设 Fruit、Apple、Orange、GoldenDelicious 和 McIntosh 是按如下继承关系定义。



假设已经给出下面的语句：

```
goldenDelicious = GoldenDelicious()
orange = Orange()
```

回答下面的问题：

（a）goldenDelicious 是 Fruit 类的实例吗？

（b）goldenDelicious 是 Orange 类的实例吗？

（c）goldenDelicious 是 Apple 类的实例吗？

（d）goldenDelicious 是 GoldenDelicious 类的实例吗？

（e）goldenDelicious 是 McIntosh 类的实例吗？

（f）orange 是 Orange 类的实例吗？

（g）orange 是 Fruit 类的实例吗？

（h）orange 是 Apple 类的实例吗？

（i）假设 Apple 类中定义了 makeAppleCider 方法。goldenDelicious 能否调用这个方法？orange 能否调用这个方法？

（j）假设 Orange 类中定义了 makeOrangeJuice 方法。orange 能否调用这个方法？goldenDelicious 能否调用这个方法？

12.7　实例研究：可重用时钟

关键点：设计一个 GUI 类来显示一个时钟。

假设想要在一个画布内显示一个时钟而且能够在其他程序中重用这个时钟。需要定义一

个时钟类来实现时钟的重用。进一步说，为了在图形上显示这个时钟，需要将它定义为一个 widget 小构件。最好的选择是定义时钟类扩展 Canvas 类，使时钟对象能够像 Canvas 对象一样使用。

这个类的合约如图 12-3 所示。

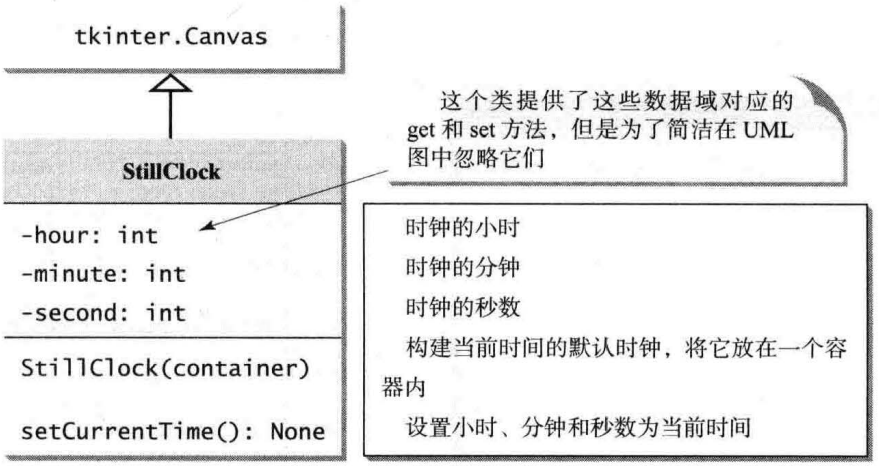


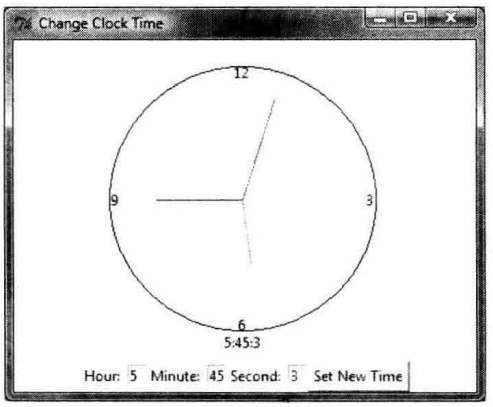
图 12-3　StillClock 显示一个模拟时钟

程序清单 12-8 是一个使用 StillClock 类来显示模拟时钟的测试程序。程序允许用户从 Entry 域中输入新的小时、分钟和秒数，如图 12-4a 所示。

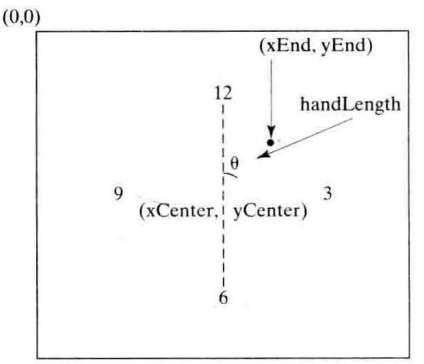
程序清单 12-8　DisplayClock.py

```
1  from tkinter import * # Import all definitions from tkinter
2  from StillClock import StillClock
3
4  class DisplayClock:
5      def __init__(self):
6          window = Tk() # Create a window
7          window.title("Change Clock Time") # Set title
8
9          self.clock = StillClock(window) # Create a clock
10         self.clock.pack()
11
12         frame = Frame(window)
13         frame.pack()
14         Label(frame, text = "Hour: ").pack(side = LEFT)
15         self.hour = IntVar()
16         self.hour.set(self.clock.getHour())
17         Entry(frame, textvariable = self.hour,
18             width = 2).pack(side = LEFT)
19         Label(frame, text = "Minute: ").pack(side = LEFT)
20         self.minute = IntVar()
21         self.minute.set(self.clock.getMinute())
22         Entry(frame, textvariable = self.minute,
23             width = 2).pack(side = LEFT)
24         Label(frame, text = "Second: ").pack(side = LEFT)
25         self.second = IntVar()
26         self.second.set(self.clock.getMinute())
27         Entry(frame, textvariable = self.second,
28             width = 2).pack(side = LEFT)
29         Button(frame, text = "Set New Time",
30             command = self.setNewTime).pack(side = LEFT)
```

```
31
32          window.mainloop() # Create an event loop
33
34      def setNewTime(self):
35          self.clock.setHour(self.hour.get())
36          self.clock.setMinute(self.minute.get())
37          self.clock.setSecond(self.second.get())
38
39  DisplayClock() # Create GUI
```



a）DisplayClock 程序显示一个时钟并且允许用户更改时间



b）利用指针转过角度、指针长度以及中心点来确定时钟指针的末端点

图　12-4

本节剩余部分介绍如何实现 StillClock 类。因为可以使用这个类而不需要了解它是如何实现的，所以如果希望只是使用它，那么你可以跳过这一段实现部分。

如何获取当前时间？ Python 提供 datetime 类，它可以用来获取计算机的当前时间。你可以使用 now() 函数来返回当前时间 datetime 的一个实例，并使用数据域 year、month、day、hour、minute 和 second 从对象提取出日期和时间信息，代码如下所示。

```
from datetime import datetime

d = datetime.now()
print("Current year is", d.year)
print("Current month is", d.month)
print("Current day of month is", d.day)
print("Current hour is", d.hour)
print("Current minute is", d.minute)
print("Current second is", d.second)
```

为了绘制一个时钟，需要绘制一个圆和三条时钟指针代表秒、分和时。为了绘制一条指针，需要指定指针的两个端点。如图 12-4b 所示，它的一个端点是在时钟的中心位置 (xCenter, yCenter)，而另一个端点在 (xEnd, yEnd)，它们是由下面的公式决定。

```
xEnd = xCenter + handLength × sin(θ)
yEnd = yCenter - handLength × cos(θ)
```

因为一分钟有 60 秒，所以秒指针的角度 θ（见图 12-4b）是：

```
θ = second × (2π/60)
```

分针的位置是由分钟和秒数决定的。确切的分钟值为 minute+second/60。例如，如果时间是 3 分 30 秒，那么总的分钟数是 3.5。因为一个小时有 60 分钟，那么分针的角度是：

$\theta = (minute + second/60) \times (2\pi/60)$

因为一个圆被划分为 12 个小时，所以时指针的角度是：

$\theta = (hour + minute/60 + second/(60 \times 60)) \times (2\pi/12)$

为了简化分针和时针角度的计算，可以忽略秒针，因为它们基本上可以忽略不计。因此，一个秒针、分针和时针的末端点可以按如下方式计算。

$xSecond = xCenter + secondHandLength \times \sin(second \times (2\pi/60))$
$ySecond = yCenter - secondHandLength \times \cos(second \times (2\pi/60))$
$xMinute = xCenter + minuteHandLength \times \sin(minute \times (2\pi/60))$
$yMinute = yCenter - minuteHandLength \times \cos(minute \times (2\pi/60))$
$xHour = xCenter + hourHandLength \times \sin((hour + minute/60) \times (2\pi/12))$
$yHour = yCenter - hourHandLength \times \cos((hour + minute/60) \times (2\pi/12))$

程序清单 12-9 中实现了 StillClock 类。

程序清单 12-9 StillClock.py

```
1  from tkinter import * # Import all definitions from tkinter
2  import math
3  from datetime import datetime
4
5  class StillClock(Canvas):
6      def __init__(self, container):
7          super().__init__(container)
8          self.setCurrentTime()
9
10     def getHour(self):
11         return self.__hour
12
13     def setHour(self, hour):
14         self.__hour = hour
15         self.delete("clock")
16         self.drawClock()
17
18     def getMinute(self):
19         return self.__minute
20
21     def setMinute(self, minute):
22         self.__minute = minute
23         self.delete("clock")
24         self.drawClock()
25
26     def getSecond(self):
27         return self.__second
28
29     def setSecond(self, second):
30         self.__second = second
31         self.delete("clock")
32         self.drawClock()
33
34     def setCurrentTime(self):
35         d = datetime.now()
36         self.__hour = d.hour
37         self.__minute = d.minute
38         self.__second = d.second
39         self.delete("clock")
40         self.drawClock()
41
42     def drawClock(self):
43         width = float(self["width"])
```

```
44          height = float(self["height"])
45          radius = min(width, height) / 2.4
46          secondHandLength = radius * 0.8
47          minuteHandLength = radius * 0.65
48          hourHandLength = radius * 0.5
49
50          self.create_oval(width / 2 - radius, height / 2 - radius,
51              width / 2 + radius, height / 2 + radius, tags = "clock")
52          self.create_text(width / 2 - radius + 5, height / 2,
53                           text = "9", tags = "clock")
54          self.create_text(width / 2 + radius - 5, height / 2,
55                           text = "3", tags = "clock")
56          self.create_text(width / 2, height / 2 - radius + 5,
57                           text = "12", tags = "clock")
58          self.create_text(width / 2, height / 2 + radius - 5,
59                           text = "6", tags = "clock")
60
61          xCenter = width / 2
62          yCenter = height / 2
63          second = self.__second
64          xSecond = xCenter + secondHandLength \
65              * math.sin(second * (2 * math.pi / 60))
66          ySecond = yCenter - secondHandLength \
67              * math.cos(second * (2 * math.pi / 60))
68          self.create_line(xCenter, yCenter, xSecond, ySecond,
69                           fill = "red", tags = "clock")
70
71          minute = self.__minute
72          xMinute = xCenter + \
73              minuteHandLength * math.sin(minute * (2 * math.pi / 60))
74          yMinute = yCenter - \
75              minuteHandLength * math.cos(minute * (2 * math.pi / 60))
76          self.create_line(xCenter, yCenter, xMinute, yMinute,
77                           fill = "blue", tags = "clock")
78
79          hour = self.__hour % 12
80          xHour = xCenter + hourHandLength * \
81              math.sin((hour + minute / 60) * (2 * math.pi / 12))
82          yHour = yCenter - hourHandLength * \
83              math.cos((hour + minute / 60) * (2 * math.pi / 12))
84          self.create_line(xCenter, yCenter, xHour, yHour,
85                           fill = "green", tags = "clock")
86
87          timestr = str(hour) + ":" + str(minute) + ":" + str(second)
88          self.create_text(width / 2, height / 2 + radius + 10,
89                           text = timestr, tags = "clock")
```

StillClock 类扩展自 Canvas 小构件（第 5 行），因此，StillClock 也是一个 Canvas。可以像使用 Canvas 一样使用 StillClock。

StillClock 类的初始化方法调用 Canvas 类的初始化方法（第 7 行），随后通过调用 setCurrentTime 方法利用当前时间设置数据域 hour、minute 和 second（第 8 行）。

通过 get 和 set 方法来获取和设置数据域 hour、minute 和 second（第 10 ～ 32 行）。当给小时、分钟和秒数设置一个新的时间值时，drawClock 方法被调用来重新绘制时钟（第 16、24 和 32 行）。

setCurrentTime 方法通过调用 datetime.now() 来获取当前时间（第 35 行）的小时、分钟和秒数（第 36 ～ 38 行），然后调用 drawClock 方法来重新绘制时钟（第 40 行）。

drawClock 方法获取画布的宽度和高度（第 43 ～ 44 行）并且设置时针、分针和秒针的准确大小（45 ～ 48 行）。随后使用 Canvas 的绘制方法来绘制一个圆、线和文本字符串来显

示一个时钟（第 50 ～ 89 行）。

12.8　类之间的关系

关键点：为了设计类，我们需要探究不同类之间的关系。类之间的常见关系有关联、聚合和继承。

前面已经使用过继承来对 is-a 关系建模，现在我们来探究其他的关系。

12.8.1　关联

关联是一种常用的二进制关系，可以描述两个类之间的一个动作。例如：一个学生选一门课程就是 Student 类和 Course 类之间的一种关联，而一个教员教授一门课程就是 Faculty 类和 Course 类之间的一种关联。这些类之间的关联性可以用 UML 图形来表示，如图 12-5 所示。

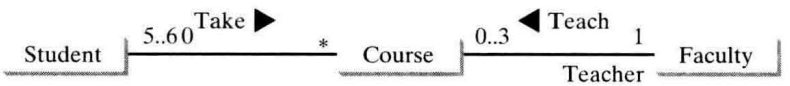


图 12-5　这个 UML 图显示一名学生可以选修任意多门课程，一名教员最多可以教授三门课，一门课程可以有 5 到 60 名学生，且只能有一名教员教授

两个类之间的一条带可选标签的实线可以用来描述类之间的这种关系。在图 12-5 中，标签是 Take 和 Teach。每个关系可能都会有一个小的三角形，表明类之间关系的方向。在这张图上，方向表明一名学生选修一门课程（而不是一门课程选择一名学生）。

在这种关系中的每一个类都有一个角色名来描述它在这个关系所扮演的角色。在图 12-5 中，Teacher 是 Faculty 类的角色名。

在一个关联关系中的每个类可能会指定一个多样性。多样性可以是一个数字或者一个区间，它指定这个关系涉及多少个类的对象。字符 * 意味着无限个对象数量，而区间 m..n 表明对象的数目是在 m 到 n 之间，且包括 m 和 n。在图 12-5 中，每个学生可以选修任意多门课程，而每门课程有 5 到 60 名学生。每门课程只能由一名教员教授，而一名教员每学期只能教授 0 到 3 门课程。

在 Python 代码中，可以使用数据域和方法实现关联性。例如：图 12-5 的关系可以使用图 12-6 中的类来实现。关系“一名学生选修一门课程”能够使用 Student 类中的 addCourse 方法和 Course 类中的 addStudent 方法来实现。关系“一名教员教授一门课程”能够使用 Faculty 类中的 addCourse 方法和 Course 类中的 setFaculty 方法来实现。Student 类可以使用一个列表来存储该学生选修的课程，Faculty 类可以使用一个列表来存储教员教授的课程，而 Course 类可以使用一个存储登记选修该课程的学生列表和一个存储带这门课程的教员的数据域。

```
class Student:
    # Add course to a list
    def addCourse(self,
        course):
```

```
class Course:
    # Add student to a list
    def addStudent(self,
        student):
    def setFaculty(self, faculty):
```

```
class Faculty:
    # Add course to a list
    def addCourse(self,
        course):
```

图 12-6　使用类中的数据域和方法来实现类之间的关联

12.8.2　聚合和组合

聚合是关联的一种特殊形式，它反映了两个对象之间的归属关系。聚合对 has-a 关系建模。所有者对象被称为聚合对象，它的类被称为聚合类。主体对象被称为被聚合对象，它的类被称为被聚合类。

一个对象能够被几个其他聚合对象所拥有。如果一个对象是某个聚合对象专门拥有，那么这个对象和它的聚合对象之间的关系被认为是一个组件。例如，“一名学生有一个名字”是 Student 类和 Name 类的一种组合关系，但是，“一名学生有一个地址”是 Student 类和 Address 类的一种聚合关系，因为一个地址可以被多个学生所共有。在 UML 图中，一个附属到聚合类（在这个例子中是指 Student）的实心菱形表示和被聚合类（在这个例子中是指 Name）的一种组合关系，而一个附属到聚合类（在这个例子中是指 Student）的空心菱形表示和被聚合类（在这个例子中是指 Address）的一种聚合关系，如图 12-7 所示。

图 12-7　每名学生都有一个名字和一个地址

在图 12-7 中，每名学生都只有一种多样性（地址），每个地址能够被 3 人以上的学生共享。每名学生只能有一个名字，且一个名字只对应一名学生。

聚合关系通常在聚集类中表示为一个数据域。例如，图 12-7 中的关系可以使用图 12-8 中的类来实现。关系“一名学生有一个名字”和“一名学生有一个地址”在 Student 类中分别用数据域 Name 和 Address 实现。

```
class Name:
    ...
```

Aggregated class

```
class Student:
    def __init__(self, name, addresses)
        self.name = name
        self.address = address

    ...
```

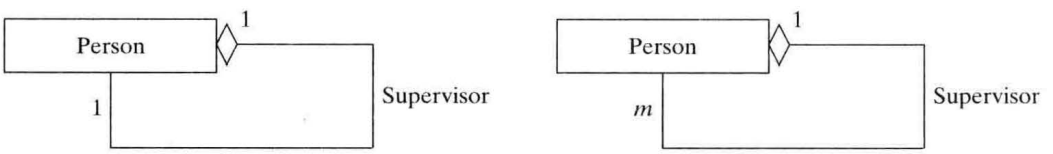
Aggregating class

```
class Address:

    ...
```

Aggregated class

图 12-8　使用类中数据域实现聚合关系

聚合可以存在于同一类的实例之间。例如：图 12-9a 表示一个人可以有一个监督人。



a）一个人可以有一名监督人　　b）一个人可以有多名监督人

图　12-9

关系“一个人有一名监督人”可以用 Person 类的一个数据域表示，如下所示。

```
class Person:
    # The type for the data is the class itself
    def __init__(self, supervisor)
```

```
        self.supervisor = supervisor
    ...
```

如果一个人有多名监督人，如图 12-9b 所示，可以使用一个列表存储监督人。

注意：因为聚合和组合关系在类中使用同样的方式实现，为了简化，我们都将它们归为组合。

检查点

12.17　类之间通常的关系类型是什么？描述对类之间关系建模的 UML 图形符号。

12.18　什么关系适用于下面的类？使用 UML 图画出它们的关系。

- 公司和雇员
- 课程和教员
- 学生和人
- 房屋和窗户
- 账户和存钱

12.9　实例研究：设计 Course 类

关键点：设计一个类来对课程建模。

假设需要处理课程信息。每门课程都有一个名称和选修的学生。你希望能够向课程添加学生，也可以从课程删除学生。可以使用一个类对课程进行建模，如图 12-10 所示。

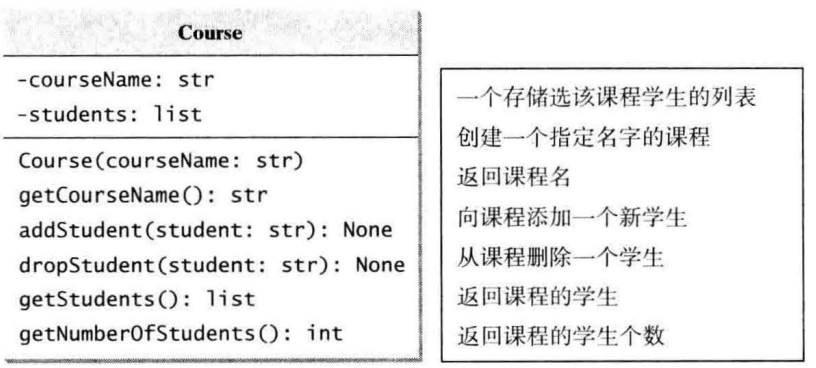


图 12-10　Course 类对课程建模

通过传递课程名使用构造函数 Course(name) 来创建一个 Course 对象。可以使用方法 addStudent(student) 向课程添加一名学生，使用方法 dropStudent(student) 从课程中删除一名学生，使用方法 getStudents() 返回选修这门课程所有学生的名字。假设 Course 类可用。程序清单 12-10 给出一个测试程序来创建两门课程并将学生添加进去。

程序清单 12-10　TestCourse.py

```
1  from Course import Course
2
3  def main():
4      course1 = Course("Data Structures")
5      course2 = Course("Database Systems")
6
7      course1.addStudent("Peter Jones")
```

```
 8      course1.addStudent("Brian Smith")
 9      course1.addStudent("Anne Kennedy")
10
11      course2.addStudent("Peter Jones")
12      course2.addStudent("Steve Smith")
13
14      print("Number of students in course1:",
15          course1.getNumberOfStudents())
16      students = course1.getStudents()
17      for student in students:
18          print(student, end = ", ")
19
20      print("\nNumber of students in course2:",
21          course2.getNumberOfStudents())
22
23  main() # Call the main function
```

```
Number of students in course1: 3
Peter Jones, Brian Smith, Anne Kennedy,
Number of students in course2: 2
```

Course 类在程序清单 12-11 中实现。它在第 4 行使用一个列表存储选修这门课程的学生。方法 addStudent()（第 6 行）将一个学生添加到列表中。方法 getStudents 返回这个列表（第 9 行）。方法 dropStudent 方法（第 18 行）被留下来作为一个练习题。

程序清单 12-11 Course.py

```
 1  class Course:
 2      def __init__(self, courseName):
 3          self.__courseName = courseName
 4          self.__students = []
 5
 6      def addStudent(self, student):
 7          self.__students.append(student)
 8
 9      def getStudents(self):
10          return self.__students
11
12      def getNumberOfStudents(self):
13          return len(self.__students)
14
15      def getCourseName(self):
16          return self.__courseName
17
18      def dropStudent(student):
19          print("Left as an exercise")
```

当创建一个 Course 对象时，就会创建一个列表对象。一个 Course 对象包含一个列表的引用。为了简化起见，可以认为 Course 对象包含这个列表。

用户可以创建一个 Course 对象并且通过公有方法 addStudent、dropStudent、getNumberOfStudents 和 getStudents 来操作。但是，用户不需要知道如何实现这些方法。Course 类封装内部实现。这个例子使用一个列表来存储学生的姓名。可以使用不同的数据类型来存储学生的姓名。只要公共方法的合约保持不变，这个使用 Course 的程序就不需要改变。

12.10　为栈设计类

关键点：设计一个类来建模栈。

回顾栈（第 6 章）是以后进先出的方式保存数据的，如图 12-11 所示。

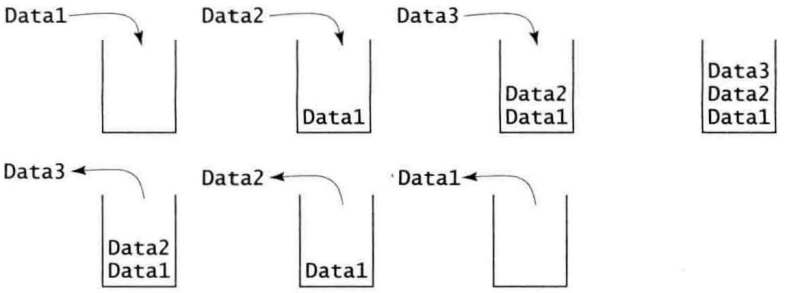


图 12-11　以 Data1、Data2 和 Data3 的顺序依次入栈，出栈顺序相反

栈有很多应用。例如：一台计算机使用栈来处理函数调用。当调用一个函数时，一个存储这个函数的参数和局部变量的活动记录被压入栈中。当一个函数调用另一个函数时，这个新函数的活动记录被压入栈。当一个函数结束它的工作返回它的调用者时，它的活动记录被从栈中删除。

可以定义一个类来对堆栈建模，并且使用列表来存储栈中的元素。下面有两种方法来设计一个堆栈类：

- 使用继承，可以定义扩展 list 类的栈类，如图 12-12a 所示。
- 使用组合，可以在 Stack 类中创建一个列表，如图 12-12b 所示。

a）使用继承　　b）使用组合

图 12-12　可以使用继承或组合来实现 Stack 类

两种方法都很好，但是使用组合会更好，因为这样可以定义一个完整的新栈类而不需要继承 list 类中不需要的和不适用的方法。本节使用组合方法，在后面编程题 12.16 中实现继承方法设计栈类。Stack 类的 UML 图如图 12-13 所示。

假设 Stack 类是可用的。程序清单 12-12 中的测试程序使用它来创建一个堆栈（第 3 行），存储 10 个整数 0、1、2、…、和 9（第 5 ～ 6 行），并将它们倒序显示。

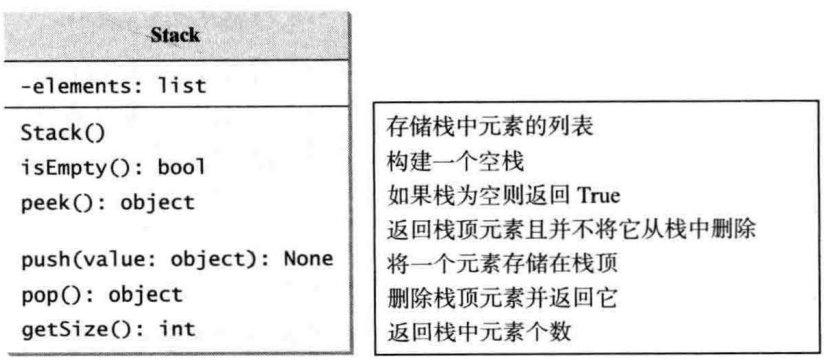


图 12-13　Stack 类封装栈存储空间并提供操作栈的方法

程序清单 12-12　TestStack.py

```
1  from Stack import Stack
2
3  stack = Stack()
```

```
4
5  for i in range(10):
6      stack.push(i)
7
8  while not stack.isEmpty():
9      print(stack.pop(), end = " ")
```

```
9 8 7 6 5 4 3 2 1 0
```

如何实现 Stack 类？可以使用一个列表来存储堆栈中的元素，如程序清单 12-13 所示。

程序清单 12-13 Stack.py

```
1  class Stack:
2      def __init__(self):
3          self.__elements = []
4
5      # Return True if the stack is empty
6      def isEmpty(self):
7          return len(self.__elements) == 0
8
9      # Return the element at the top of the stack
10     # without removing it from the stack.
11     def peek(self):
12         if self.isEmpty():
13             return None
14         else:
15             return self.__elements[len(elements) - 1]
16
17     # Store an element at the top of the stack
18     def push(self, value):
19         self.__elements.append(value)
20
21     # Remove the element at the top of the stack and return it
22     def pop(self):
23         if self.isEmpty():
24             return None
25         else:
26             return self.__elements.pop()
27
28     # Return the size of the stack
29     def getSize(self):
30         return len(self.__elements)
```

在第 3 行中，数据域 elements 被定义为带有两个前导下划线的私有数据域。elements 是一个列表，但是用户并没有意识到元素是存储在一个列表中。用户通过方法 isEmpty()、peek()、push(element)、pop() 和 getSize() 方法来对列表进行访问。

12.11　实例研究：FigureCanvas 类

关键点：开发显示各种图形的 FigureCanvas 类。

FigureCanvas 类允许用户设置图形类型、确定是否填充该图形以及是否在画布上显示这个图形。该类的 UML 图如图 12-14 所示，它可以显示直线、矩形、椭圆和弧。显示哪个图形是由 figuretype 属性决定的。如果 filled 属性为 True，那么矩形、椭圆和弧在画布上都是填充颜色的。

这个 UML 图就是 FigureCanvas 类的合约。用户可以在不知道这个类是如何实现的情况下使用它。我们从编写程序清单 12-14 中的程序开始，它使用这个类在一个面板上显示 7 个

图形面板，如图 12-15 所示。

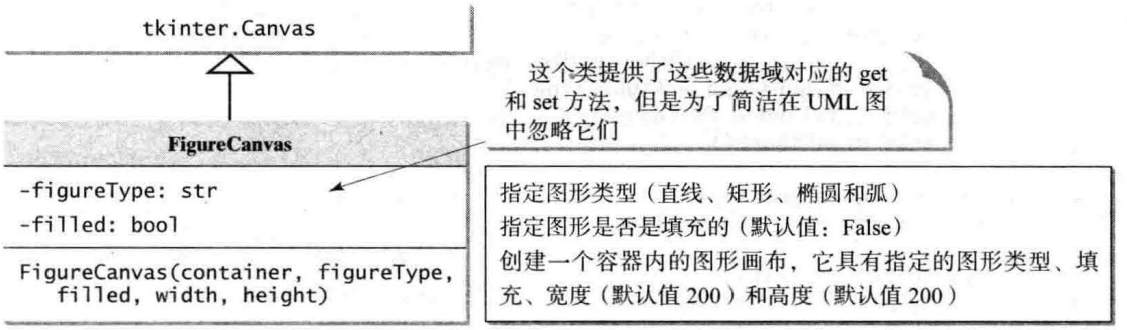


图 12-14　FigureCanvas 在面板上显示各种类型的图形

程序清单 12-14　DisplayFigures.py

```
1  from tkinter import * # Import all definitions from tkinter
2  from FigureCanvas import FigureCanvas
3
4  class DisplayFigures:
5      def __init__(self):
6          window = Tk() # Create a window
7          window.title("Display Figures") # Set title
8
9          figure1 = FigureCanvas(window, "line", width = 100, height = 100)
10         figure1.grid(row = 1, column = 1)
11         figure2 = FigureCanvas(window, "rectangle", False, 100, 100)
12         figure2.grid(row = 1, column = 2)
13         figure3 = FigureCanvas(window, "oval", False, 100, 100)
14         figure3.grid(row = 1, column = 3)
15         figure4 = FigureCanvas(window, "arc", False, 100, 100)
16         figure4.grid(row = 1, column = 4)
17         figure5 = FigureCanvas(window, "rectangle", True, 100, 100)
18         figure5.grid(row = 1, column = 5)
19         figure6 = FigureCanvas(window, "oval", True, 100, 100)
20         figure6.grid(row = 1, column = 6)
21         figure7 = FigureCanvas(window, "arc", True, 100, 100)
22         figure7.grid(row = 1, column = 7)
23
24         window.mainloop() # Create an event loop
25
26 DisplayFigures() # Create GUI
```

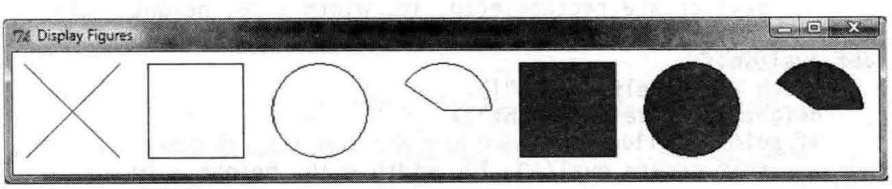


图 12-15　创建 7 个 FigureCanvas 对象显示 7 个图形

程序清单 12-15 实现 FigureCanvas 类。四种图形都是依照 figureType 属性绘制的（第 26 ～ 34 行）。

程序清单 12-15　FigureCanvas.py

```
1  from tkinter import * # Import all definitions from tkinter
2
```

```
class FigureCanvas(Canvas):
    def __init__(self, container, figureType, filled = False,
                 width = 100, height = 100):
        super().__init__(container,
                         width = width, height = height)
        self.__figureType = figureType
        self.__filled = filled
        self.drawFigure()

    def getFigureType(self):
        return self.__figureType

    def getFilled(self):
        return self.__filled

    def setFigureType(self, figureType):
        self.__figureType = figureType
        self.drawFigure()

    def setFilled(self, filled):
        self.__filled = filled
        self.drawFigure()

    def drawFigure(self):
        if self.__figureType == "line":
            self.line()
        elif self.__figureType == "rectangle":
            self.rectangle()
        elif self.__figureType == "oval":
            self.oval()
        elif self.__figureType == "arc":
            self.arc()

    def line(self):
        width = int(self["width"])
        height = int(self["height"])
        self.create_line(10, 10, width - 10, height - 10)
        self.create_line(width - 10, 10, 10, height - 10)

    def rectangle(self):
        width = int(self["width"])
        height = int(self["height"])
        if self.__filled:
            self.create_rectangle(10, 10, width - 10, height - 10,
                                  fill = "red")
        else:
            self.create_rectangle(10, 10, width - 10, height - 10)

    def oval(self):
        width = int(self["width"])
        height = int(self["height"])
        if self.__filled:
            self.create_oval(10, 10, width - 10, height - 10,
                             fill = "red")
        else:
            self.create_oval(10, 10, width - 10, height - 10)

    def arc(self):
        width = int(self["width"])
        height = int(self["height"])
        if self.__filled:
            self.create_arc(10, 10, width - 10, height - 10,
                            start = 0, extent = 145, fill = "red")
        else:
```

```
67              self.create_arc(10, 10, width - 10, height - 10,
68                              start = 0, extent = 145)
```

FigureCanvas 类扩展自 Canvas 小构件（第 3 行）。这样，一个 FigureCanvas 也是一个画布，可以像使用画布一样使用 FigureCanvas。可以通过指定容器、图形类型、图形是否被填充以及画布的长和宽来构建一个 FigureCanvas（第 4 ～ 5 行）。

FigureCanvas 类的初始化方法调用 Canvas 的初始化方法（第 6 ～ 7 行），设置数据域的 figureType 和 filled 属性（第 8 ～ 9 行），然后调用 drawFigure 方法（第 10 行）来绘制图形。

方法 drawFigure 利用 figureType 和 filled 属性来绘制一个图形（第 26 ～ 34 行）。

方法 line、rectangle、oval 和 arc 分别绘制直线、矩形、椭圆和弧（第 36 ～ 68 行）。

关键术语

aggregation（聚合）
association（关联）
composition（组合）
dynamic binding（动态绑定）
inheritance（继承）
is-a relationships（是关系）
multiple inheritance（多重继承）
override（覆盖）
polymorphism（多态）

本章总结

1. 可以从现有的类派生出新类。这被称为*类的继承*。新类被称为*次类*、*子类*或*扩展类*。现有类被称为*超类*、*父类*或*基类*。
2. 为了*覆盖*一个方法，必须使用与它的父类中的方法相同的方法名来定义子类中的方法。
3. object 类是所有 Python 类的基类。在 object 类中定义了 __str__() 和 __eq__() 方法。
4. *多态*意味着一个子类对象可以传递给一个需要父类类型的参数。一个方法可能在一条继承链中不同的类中使用。Python 决定运行时调用哪个方法。这被称为*动态绑定*。
5. 可以使用 isinstance 函数测试一个对象是否是一个类的实例。
6. 类之间常见的关系是*关联*、*聚合*、*组合*和*继承*。

测试题

本章的在线测试题位于 www.cs.armstrong.edu/liang/py/test.html。

编程题

第 12.2 ～ 12.6 节

12.1 （Triangle 类）设计一个名为 Triangle 的类来扩展 GeometricObject 类。该类包括：

- 三个名为 side1、side2 和 side3 的浮点数据域分别表示这个三角形的三条边。
- 一个构造方法构建默认一个三角形，指定它的三条边 side1、side2 和 side3 的默认值是 1.0。
- 三个数据域的访问器方法。
- 一个名为 getArea() 的方法返回这个三角形的面积。
- 一个名为 getPerimeter() 的方法返回这个三角形的周长。
- 一个名为 __str__() 的方法返回对这个三角形的字符串描述。

计算三角形面积的公式参见编程题 2.14。__str__() 方法的实现如下所示。

```
return "Triangle: side1 = " + str(side1) + " side2 = " +
    str(side2) + " side3 = " + str(side3)
```

绘制 Triangle 类和 GeometricObject 类的 UML 图。实现 Triangle 类。编写一个测试程序，程序提示用户输入三角形的三个边长、颜色以及表明三角形填充属性的 1 或 0。程序应该创建创建一个 Triangle 对象，这个三角形使用输入给它的边赋值并设置颜色和填充属性。程序应该显示这个三角形的面积、周长、颜色以及表示这个三角形是否被填充的 True 或 False。

**12.2 （Location 类）设计一个名为 Location 的类来定位一个二维列表中最大值及其位置。这个类包含有公共数据域 row、column 和 maxValue 来存储最大值和它在二维列表的下标值，其中 row 和 column 是 int 型的而 maxValue 为 float 型的。

编写下面的方法返回二维列表最大元素的位置。

```
def Location locateLargest(a):
```

返回值是 Location 的一个对象。编写一个测试程序提示用户输入一个二维列表并显示列表中最大值和它的位置。下面是一个示例运行。

```
Enter the number of rows and columns in the list: 3, 4 Enter
Enter row 0: 23.5 35 2 10 Enter
Enter row 1: 4.5 3 45 3.5 Enter
Enter row 2: 35 44 5.5 12.6 Enter
The location of the largest element is 45 at (1, 2)
```

12.3 （游戏：ATM 机）使用编程题 7.3 中创建的 Account 类来模拟一台 ATM 机。创建一个有 10 个账号的列表，其 id 为 0、1、…、9，并初始化收支为 100 美元。系统提示用户输入一个 id 号。如果输入的 id 不正确，就要求用户输入一个正确的 id。一旦接受一个 id，就显示如运行实例所示的主菜单。可以输入选择 1 来查看当前的收支，选择 2 表示取钱，选择 3 表示存钱，选择 4 表示退出主菜单。一旦退出，系统就会提示再次输入 id。所以，系统一旦启动，就不会停止。

```
Enter an account id: 4 Enter

Main menu
1: check balance
2: withdraw
3: deposit
4: exit
```

```
Enter a choice: 1 Enter
The balance is 100.00

Main menu
1: check balance
2: withdraw
3: deposit
4: exit
Enter a choice: 2 Enter
Enter an amount to withdraw: 3 Enter

Main menu
1: check balance
2: withdraw
3: deposit
4: exit
Enter a choice: 1 Enter
The balance is 97.00

Main menu
1: check balance
```

```
2: withdraw
3: deposit
4: exit
Enter a choice: 3 Enter
Enter an amount to deposit: 10 Enter

Main menu
1: check balance
2: withdraw
3: deposit
4: exit
Enter a choice: 1 Enter
The balance is 107.00

Main menu
1: check balance
2: withdraw
3: deposit
4: exit
Enter a choice: 4 Enter

Enter an account id:
```

*12.4 （几何：找出边界矩形）边界矩形是指能够包括一个二维平面上点集中所有点的最小矩形，如图 12-16 所示。编写一个方法返回二维平面上点集的边界矩形，如下所示。

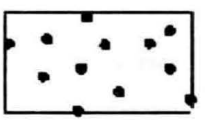


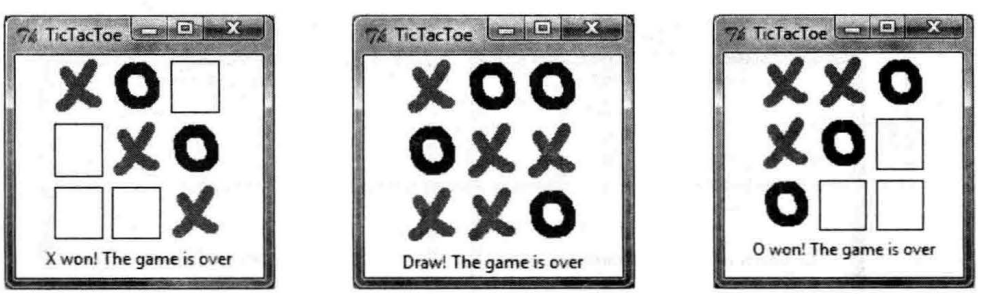
图 12-16　点都被包围在一个矩形中

```
def getRectangle(points):
```

可以使用在编程题 8.19 中定义的 Rectangle2D 类。编写一个测试程序提示用户在同一行输入这些点的坐标，例如：x1 y1 x2 y2 x3 y3 …，然后显示这个边界矩形的中心、宽和长。下面是一个示例运行。

第 12.7 ～ 12.11 节

**12.5 （游戏：井字游戏）编写一个井字游戏的程序。两名玩家依次单击 3 × 3 的网格中的一个可用单元，将该单元标记为玩家对应的标志（X 或 O）。当一位玩家将 3 个标志放在同一行或同一列或一条对角线上时，游戏结束，这名玩家获胜。当没有玩家将 3 个标志连成一线且网格上的单元全都标记完时，此时平局（没有赢家）。图 12-17 给出这个例子的一个代表性的示例运行。



a）X 标志的玩家获胜　　b）平局　　c）O 标志的玩家获胜

图　12-17

假设初始状态时网格中的所有单元都未被标记，第一位玩家是 X 标志而第二位玩家是 O 标志。为了对网格中的单元标记，玩家将鼠标移至要标记单元然后单击该单元。如果该单元未被标记，则显示标志（X 或 O）。如果单元已被标记，玩家的动作被忽略。

定义一个名为 Cell 的用户类，该类扩展自 Label 用以显示一个标志并对鼠标单击事件进行响

应。该类包含有一个标记数据域，该数据域有三个可能取值' '、X 和 O，用来表示这个单元是否已被占用以及如果它被占用那么使用的是哪个标记。

这三个图像文件 x.gif、o.gif 和 empty.gif 可以从网站 cs.armstrong.edu/liang/py/book.zip 获取。使用这三个图像来显示 X、O 和空白单元。

**12.6 （Tkinter：两圆相交？）使用在编程题 8.18 中定义的 Circle2D 类，编写一个程序允许用户通过将鼠标置于圆内来拖动它。当圆被拖动后，标签显示两圆是否相交，如图 12-18 所示。

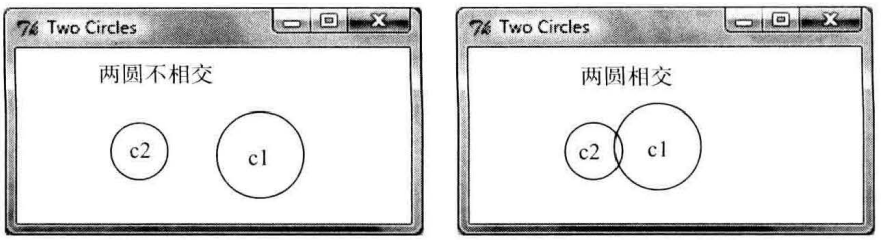


图 12-18　检测两圆是否相交

**12.7 （Tkinter：矩形相交？）使用在编程题 8.19 中定义的 Rectangle2D 类，编写一个程序允许用户通过将鼠标置于矩形内来拖动它。当矩形被拖动后，标签显示两个矩形是否相交，如图 12-19 所示。

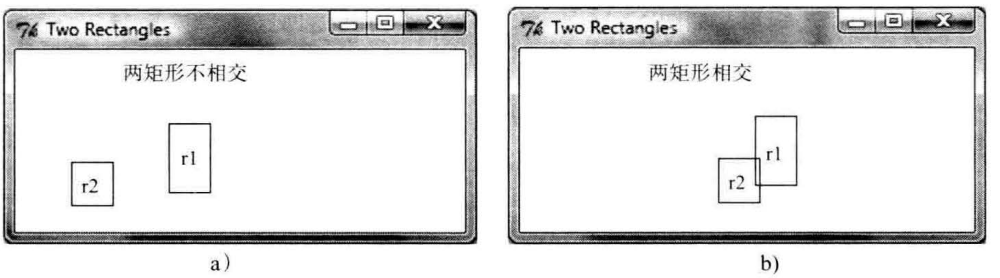


图 12-19　检测矩形是否相交

**12.8 （Tkinter：两圆相交？）使用编程题 8.18 中定义的 Circle2D 类，编写一个程序允许用户指定两个圆的位置和大小并且显示两圆是否相交，如图 12-20 所示。允许用户通过将鼠标置于圆内来拖动它。当圆被拖动后，程序将更新文本域中圆的中心坐标和它的半径。

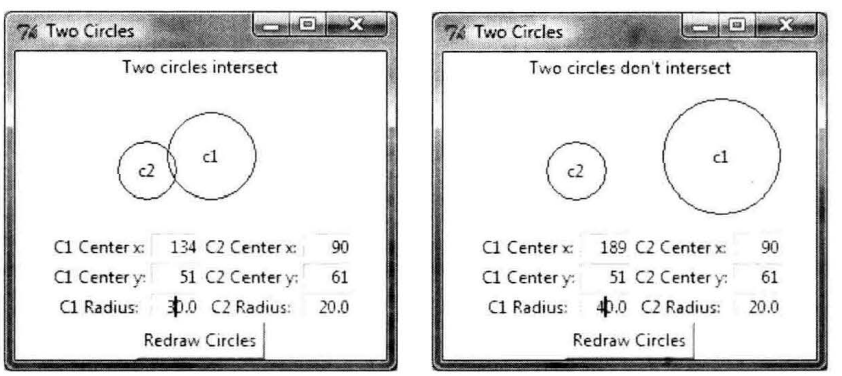


图 12-20　检测两圆是否重叠

**12.9 （TkinterL 矩形相交？）使用编程题 8.19 中定义的 Rectangle2D 类，编写一个程序允许用户指定两个矩形的位置和大小并且显示两矩形是否相交，如图 12-21 所示。允许用户通过将鼠标置于矩形内来拖动它。当矩形被拖动后，程序将更新文本域中矩形的中心坐标和它的长和宽。

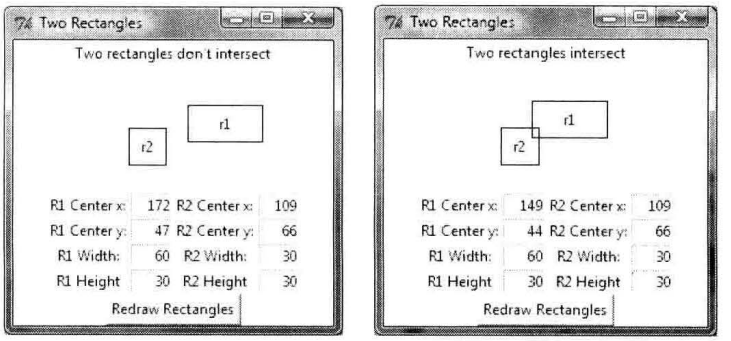


图 12-21　检测两个矩形是否重叠

**12.10 （Tkinter：四辆汽车）编写一个程序来模拟 4 辆汽车比赛，如图 12-22 所示。可以定义一个 Canvas 的子类来显示一辆汽车。

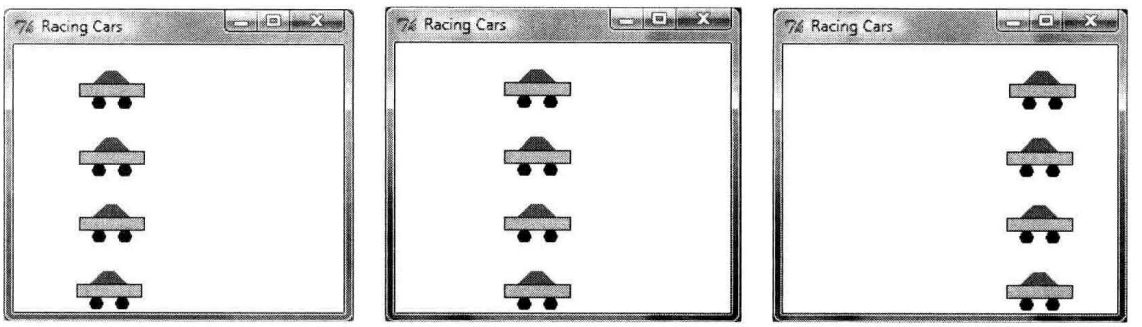


图 12-22　程序模拟四辆汽车比赛

**12.11 （Tkinter：猜生日）程序清单 4-3 给出了一个猜生日的程序。编写一个如图 12-23 所示的猜生日程序。这个程序提示用户检查生日日期是否在五个集合中某几个内。单击“Guess Birthday”按钮后，猜测的生日将在一个消息框中显示。

*12.12 （Tkinter：时钟组）编写一个程序显示四个时钟，如图 12-24 所示。

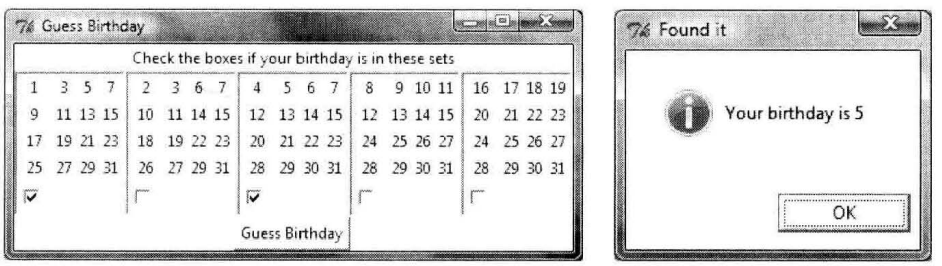


图 12-23　这个程序猜测生日日期

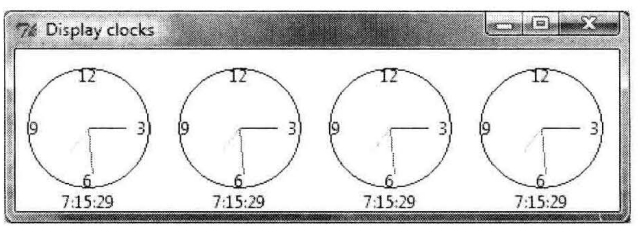


图 12-24　程序显示四个时钟

***12.13 （Tkinter：四点相连游戏）编程题 11.20 编写了一个控制台双人对战的四点相连游戏。使用 GUI 程序重写这个程序，如图 12-25 所示。这个程序允许两名玩家依次在棋盘上放置红色和黄色棋子。玩家通过单击一个可用单元来放置棋子。如果一个玩家获胜，程序将突出加亮 4 个获胜相连的棋子，如果棋盘下满了棋子却没有玩家获胜则报告平局。

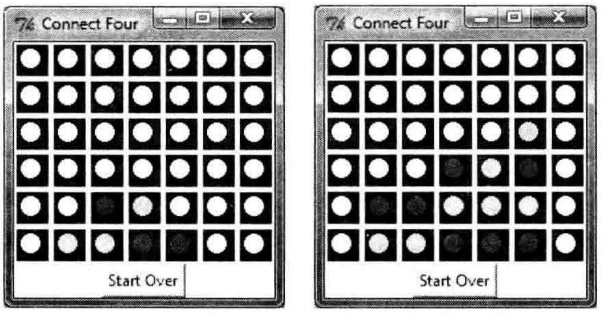


图 12-25　双人对战的四点相连游戏

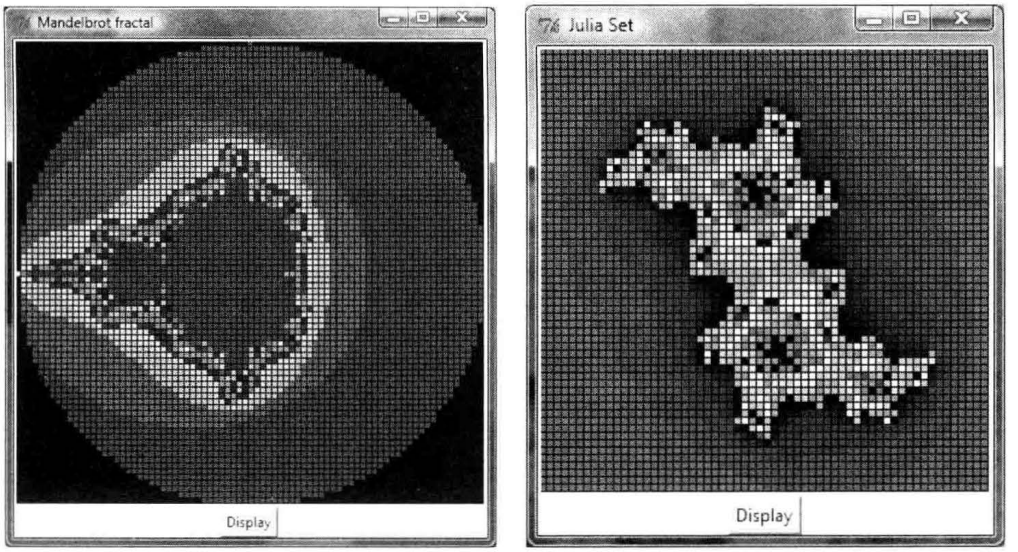
**12.14 （Tkinter：曼德布洛特分形）曼德布洛特分形是由从曼德布洛特集合创造的一个非常著名的图形（参见图 12-26a）。一个曼德布洛特集合如下定义。

$$z_{n+1} = z_n^2 + c$$

c 是一个复数，迭代的起始点为 $z_0 = 0$。（有关复数更多的信息参见编程题 8.21）对于一个给定的 c，这个迭代将会生成一个复数序列：$[z_0, z_1, \cdots, z_n, \cdots]$。可以看出，这个序列要么趋于无穷，要么在一定范围内，这取决于 c 的取值。例如：如果 c 取值为 i，那么序列是 $[0, i, -1+i, i, \cdots]$，它是会收敛在一定范围内。如果 c 取值为 1+i，那么序列为 $[0, 1+i, +1+3i, \cdots]$，这个序列就是趋于无穷的。易知当序列中有绝对值大于 2 的复数时，这个序列将趋于无穷。曼德布洛特集合包括 c 值，它是收敛在一定范围内的。例如，0 和 i 就在曼德布洛特集合内。使用下面的代码可以创建一幅曼德布洛特图形。

```
COUNT_LIMIT = 60

# Paint a Mandelbrot image in the canvas
def paint():
    x = -2.0
    while x < 2.0:
        y = -2.0
        while y < 2.0:
            c = count(complex(x, y))
            if c == COUNT_LIMIT:
                color = "red" # c is in a Mandelbrot set
            else:
                # get hex value RRGGBB that is dependent on c
                color = "#RRGGBB"

            # Fill a tiny rectangle with the specified color
            canvas.create_rectangle(x * 100 + 200, y * 100 + 200,
                x * 100 + 200 + 5, y * 100 + 200 + 5, fill = color)
            y += 0.05
        x += 0.05

# Return the iteration count
def count(c):
    z = complex(0, 0) # z0

    for i in range(COUNT_LIMIT):
        z = z * z + c # Get z1, z2, ...
```

```
28            if abs(z) > 2: return i # The sequence is unbounded
29
30        return COUNT_LIMIT # Indicate a bounded sequence
```



a）曼德布洛特图形　　b）茱莉亚集合图形

图　12-26

count(c) 函数（第 23 ～ 28 行）计算序列 z_1, z_2, …, z_{60}。如果它们的模均小于 2，我们假设 c 属于曼德布洛特集合。当然，这可能会出错，但是 60 次迭代（COUNT_LIMIT）一般是足够的。一旦我们发现这个序列是无界的，方法就返回迭代次数（第 28 行）。如果这个序列是有界的，那么方法返回 COUNT_LIMIT 常量（第 30 行）。

第 6 ～ 20 行的循环检查对于范围 $-2<x<2$、$-2<y<2$ 中的每一点（x，y）每隔 0.01 所对应的 $c = x+yi$ 是否属于曼德布洛特集合（第 9 行）。如果属于，将这些点用红色标记（第 11 行）。如果不是，设置一个依赖于迭代次数的颜色（第 14 行）。注意将这些点绘制为一个 5 × 5 像素的方阵。所有的点被重新排列组合为一个 400 × 400 像素的网格（第 17 ～ 18 行）。

**12.15 （Tkinter：朱丽亚集合）上一题描述了曼德布洛特集合。曼德布洛特集合指复数 c 的集合，其中 c 满足序列 $z_{n+1} = z_n^2 + c$ 在 z_0 固定而 c 变化时有界这一条件。如果 c 固定，改变 z_0（$= x + yi$）的值，如果对于一个固定的复数 c，使函数 z_0 保持有界，那么点（x，y）就被称为在朱丽亚集合里。编写一个程序绘制一个朱丽亚集合，如图 12-26b 所示。注意只需要使用一个固定的 c 值 (−0.3+0.6i) 修改编程题 12.14 中的 count 方法即可。

*12.16 （使用继承实现 Stack）在程序清单 12-13 中，Stack 类是用组合实现的。使用继承扩展 list 创建一个新的 Stack 类。

绘制这个新类的 UML 图。实现这个类。编写一个测试程序，提示用户输入五个字符串，然后以逆序显示这些字符串。

***12.17 （Tkinter：24 点扑克牌游戏）增强编程题 10.37，如果存在一个 24 点游戏的解决方案，那么计算机能够显示这个解的表达式，如图 12-27 所示；否则，报告解决方案不存在。

**12.18 （Tkinter：BarChart 类）开发一个名为 BarChart 的类扩展 Canvas 以显示一个条形图。

```
BarChart(parent, data, width = 400, height = 300)
```

这里的 data 是一个列表，该列表中的每个元素都是一个包含一个值、值标题和条形图中条形颜色的嵌套列表。例如，对 `data = [[40, "CS", "red"], [30, "IS", "blue"], [50, "IT",`

"yellow"]] 而言，条形图如图 12-28 的左图所示。对 data = [[140, "Freshman", "red"], [130, "Sophomore", "blue"], [150, "Junior", "yellow"], [80, "Senior", 'green"]] 而言，条形图如图 12-28 的右图所示。编写一个测试程序来显示这两幅条形图，如图 12-28 所示。

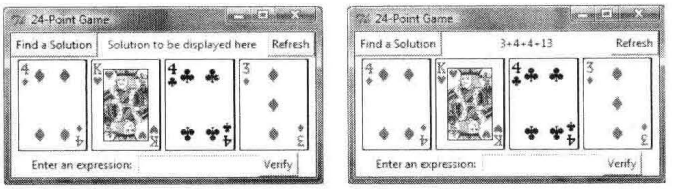


图 12-27　如果存在一个解决方案，那么程序自动寻找它

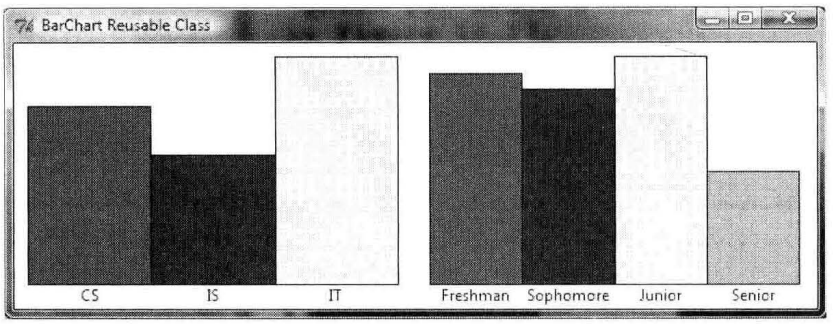


图 12-28　程序使用 BarChart 类来显示条形图

**12.19 （Tkinter：PieChart 类）使用下面的构造方法开发一个名为 PieChart 的类扩展自 Canvas 以显示一个饼状图。

这里的 data 是一个列表，该列表中的每个元素都是一个包含一个值、值标题和饼状图中小构件的颜色构成的嵌套列表。例如，对 ple, for data = [[40, "CS", "red"], [30, "IS", "blue"], [50, "IT", "yellow"]]而言，饼状图如图 12-29 的左图所示。对 For data = [[140, "Freshman", "red"], [130, "Sophomore", "blue"], [150, "Junior", "yellow"], [80, "Senior", "green"]]而言，饼状图如图 12-29 的右图所示。编写一个测试程序来显示这两幅饼状图，如图 12-29 所示。

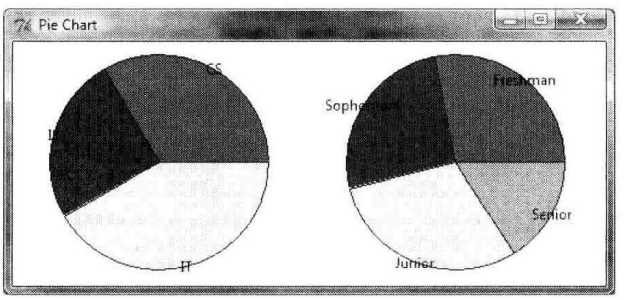


图 12-29　程序使用 PieChart 类来显示饼状图

12.20 （Tkinter：RegularPolygonCanva 类）定义一个名为 RegularPolygonCanva 的子类 Canvas，来绘制一个 n 条边的正多边形。这个类包括名为 numberOfSides 的属性，它表示多边形的边数。多边形被放在画布的中心位置。多边形大小和面布的大小成正比。从 RegularPolygonCanvas 创建一个五边形、六边形、七边形、八边形、九边形和十边形，然后显示它们，如图 12-30 所示。

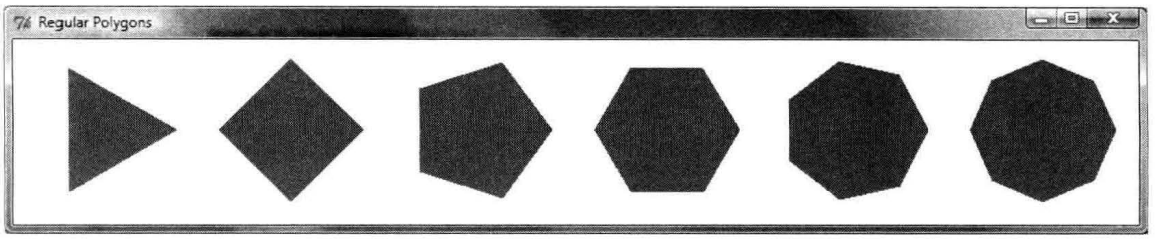
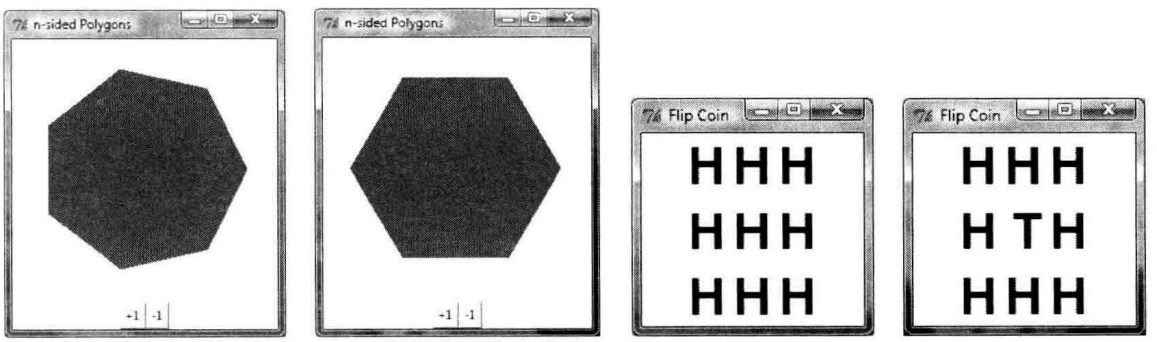


图 12-30　程序显示几个 n 边形

*12.21 （Tkinter：显示一个正 n 边形）在编程题 12.20 中创建了一个 RegularPolygonCanvas 子类来显示一个正 n 边形。编写一个程序来显示一个正多边形且使用两个名为“+1”和“-1”的按钮来增加和减少多边形的边数，如图 12-31a、12-31b 所示。同样允许用户通过单击鼠标左键和右键以及单击 UP 箭头键和 DOWN 箭头键来增加和减少多边形的边数。



a）～b）单击“+1”或“-1”按钮来增加或减少多边形的边数

c）～d）程序允许用户单击格子来翻硬币

图　12-31

*12.22 （翻硬币）编写一个程序，显示九个硬币的正面（H）或反面（T），如图 12-31c、12-31d 所示。当单击一个格子时，硬币就被翻面。编写一个自定制的格子类，该类扩展自 Label。在该类的初始化方法中，将 <Button-1> 事件与翻硬币的方法绑定。当程序启动时，所有的格子都被初始化为 H。

第 13 章

Introduction to Programming Using Python

文件和异常处理

学习目标

- 为了读写一个文件需要使用 open 函数打开该文件（第 13.2.1 节）。
- 在一个文件对象中使用 write 方法来写入数据（第 13.2.2 节）。
- 使用 os.path.isfile 函数来检测一个文件是否存在（第 13.2.3 节）。
- 在一个文件对象中使用 read、readline 和 readlines 方法来从一个文件读取数据（第 13.2.4 ～ 13.2.5 节）。
- 为了给一个文件追加数据，以追加模式打开这个文件（第 13.2.6 节）。
- 读写数值数据（第 13.2.7 节）。
- 显示打开和保存对话框以获取文件名对数据进行读写（第 13.3 节）。
- 利用文件开发应用程序（第 13.4 节）。
- 从网站资源上读取数据（第 13.5 节）。
- 使用 try、except 和 finally 子句来处理异常（第 13.6 节）。
- 使用 raise 语句抛出异常（第 13.7 节）。
- 熟悉 Python 的内置异常类（第 13.8 节）。
- 访问句柄中的一个异常对象（第 13.8 节）。
- 定义自定制的异常类。（第 13.9 节）。
- 使用 pickle 模型中的 load 和 dump 函数来实现二进制 IO（第 13.10 节）。
- 使用二进制 IO 创建一个地址簿（第 13.11 节）。

13.1 引言

关键点：可以使用一个文件来永久保存数据；可以使用异常处理使编写的程序安全可靠且鲁棒性强。

程序中使用的数据都是暂时的，当程序终止时它们就会丢失，除非这些数据被特别地保存起来。为了能够永久地保存程序中创建的数据，需要将它们存储到磁盘或光盘上的文件中。这些文件可以被传送，可以随后被其他程序读取。在本章中，我们来学习如何从（向）一个文件读（写）数据。

如果程序试图从一个并不存在的文件读取数据，将会发生什么？程序将会意外终止。下面我们将学习如何编写程序来处理这个异常以使程序继续执行。

13.2 文本输入和输出

关键点：为了从文件读数据或向文件写数据，需要使用 open 函数创建一个文件对象并使用这个对象的 read 和 write 方法来读写数据。

在文件系统中，每个文件都存放在一个目录下。绝对文件名是由文件名和它的完整路

径以及驱动器字母组成。例如：c:\pybook\Scores.txt 是文件 Scores.txt 在 Windows 操作系统上的绝对文件名。这里的 c:\pybook 是指该文件的目录路径。绝对文件名是依赖机器的。在 UNIX 平台上，绝对文件名可能会是 /home/liang/pybook/Scores.txt，其中 /home/liang/pybook 是文件 Scores.txt 的目录路径。

相对文件名是相对于文件当前的工作路径而言的。一个相对文件名的完整路径被忽略。例如，Scores.py 是一个相对文件名。如果它当前的工作路径是 c:\pybook，那么绝对文件名应该是 c:\pybook\Scores.py。

文件可以分为文本文件和二进制文件两类。在 Windows 系统中能够使用文本编辑器或 Notepad 处理（读写和创建）或者在 UNIX 系统中能够使用 vi 处理的文件被称为文本文件（text file）。所有其他文件都被称为二进制文件。例如，Python 源程序都被存在文本文件中且可以被文本编辑器处理，但是微软的 Word 文件是被存储在二进制文件且是用 Microsoft Word 程序处理的。

尽管在技术上不够严谨正确，但是可以认为一个文本文件是由一系列的字符组成而一个二进制文件是由一系列的比特组成。文本文件中的字符使用像 ASCII 和 Unicode 这样的字符编码表来编码。例如：十进制整数 199 在文本文件中被存为三个字符 1、9 和 9，而同一个整数在二进制文件中就被存为一个字节类型 C7，因为十进制 199 等于十六进制 C7（$199=12\times16^1+7$）。二进制文件的优势就是它们比文本文件的处理效率更高。

注意：计算机并不会区分二进制文件和文本文件。所有的文件都以二进制格式存储，因此实际上所有的文件都是二进制文件。文本 IO（输入和输出）是建立在二进制 IO 的基础上提供一定程度上抽象的字符编码和解码。

本节介绍如何从一个文本文件读取字符串和向一个文本文件写入字符串。第 13.10 节将介绍二进制文件。

13.2.1　打开一个文件

如何向（从）一个文件写（读）数据？需要创建一个和物理文件相关的文件对象。这被称为打开一个文件。打开一个文件的语法如下。

```
fileVariable = open(filename, mode)
```

open 函数为 filename 返回一个文件对象。参数 mode 是一个指定这个文件将被如何使用（只读或只写）的字符串，如表 13-1 所示。

表 13-1　文件模式

模式	描述
"r"	为了读取打开一个文件
"w"	为了写入打开一个文件，如果文件已经存在，它的就内容就被销毁
"a"	打开一个文件从文件末尾追加数据
"rb"	为读取二进制数据打开文件
"wb"	为写入二进制数据打开文件

例如，下面的语句打开当前目录下一个名为 Scores.txt 的文件来进行读操作。

```
input = open("Scores.txt", "r")
```

也可以使用绝对文件名来打开 Windows 下的文件，如下所示。

```
input = open(r"c:\pybook\Scores.txt", "r")
```

上述语句打开 c:\pybook 路径下的 Scores.txt 文件来进行读操作。绝对文件名前的 r 前缀表明这个字符串是一个行字符串，这会使 Python 解释器将文件名中的反斜线理解为字面意义上的反斜线。如果没有 r 前缀，需要使用转义序列将上述语句改写为：

```
input = open("c:\\pybook\\Scores.txt", "r")
```

13.2.2　写入数据

open 函数创建了一个文件对象，这是 _io.TextIOWrapper 类的一个实例。这个类包含了读写数据和关闭文件的方法，如图 13-1 所示。

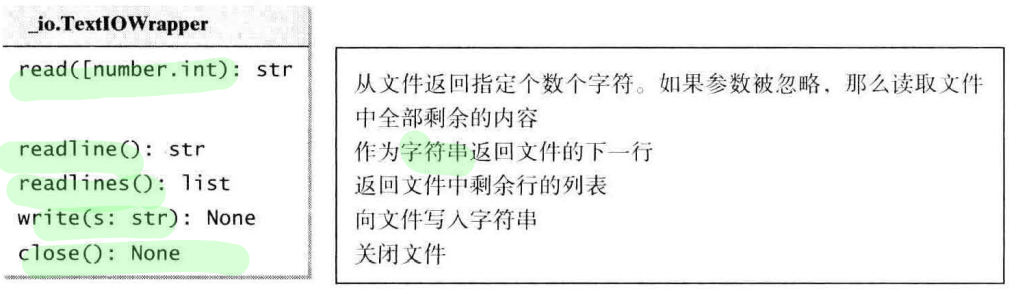


图 13-1　文件对象包含读写数据的方法

当一个文件被打开来进行写数据的操作后，可以使用 write 方法来将一个字符串写入文件。在程序清单 13-1 中，程序将三个字符串写入文件 Presidents.txt。

程序清单 13-1　WriteDemo.py

```
1  def main():
2      # Open file for output
3      outfile = open("Presidents.txt", "w")
4
5      # Write data to the file
6      outfile.write("Bill Clinton\n")
7      outfile.write("George Bush\n")
8      outfile.write("Barack Obama")
9
10     outfile.close() # Close the output file
11
12 main() # Call the main function
```

这个程序使用 w 模式打开一个名为 Presidents.txt 的文件来写入数据（第 3 行）。如果这个文件不存在，那么 open 函数就会创建一个新文件。如果该文件已经存在，那么这个文件中的内容将会被新的数据覆盖重写。现在，可以向文件中写入数据。

当一个文件被打开来进行写操作或读操作时，一个被称为文件指针的特殊标记将会被放在文件内部。读或写操作在指针位置发生。当一个文件被打开时，文件指针被放在文件的起始位置。当对文件进行读或写操作时，文件指针将会前移。

程序调用文件对象上的 write 方法写入三个字符串（第 6 ～ 8 行）。图 13-2 给出每次写操作之后文件指针的位置。

程序最后关闭文件以保证数据被写入文件（第 10 行）。当这个程序被执行后，三个姓名被写入文件。可以利用文本编辑器来查看该文件，如图 13-3 所示。

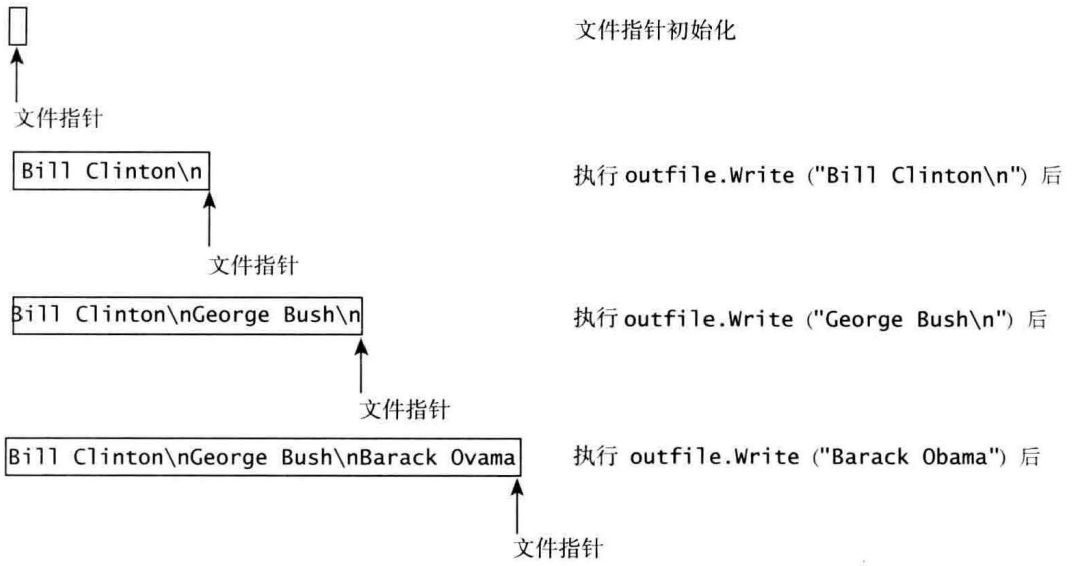


图 13-2　三个字符串被写入文件

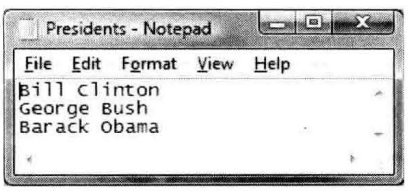


图 13-3　一个名为 Presidents.txt 的文件包含三个姓名

注意：当调用 print(str) 时，函数在显示字符串后将自动插入一个换行字符 \n。但是，write 函数不会自动插入一个新行字符。你必须显式地给文件写入一个换行字符。

警告：如果打开一个已经存在的文件来进行写操作，那么该文件的原始内容将会被新的文本覆盖或销毁。

13.2.3　测试文件的存在性

为了防止已存在文件中的数据被意外消除，应该在打开一个文件进行写操作前检测该文件是否已经存在。os.path 模块中的 isfile 函数可以用来判断一个文件是否存在。例如：

```
import os.path
if os.path.isfile("Presidents.txt"):
    print("Presidents.txt exists")
```

在这里，如果文件 Presidents.txt 文件在当前目录下存在，那么 isfile（"Presidents.txt"）返回 True。

13.2.4　读数据

当一个文件被打开来进行读操作时，可以使用 read 方法从该文件中读取特定数目的字符或全部字符并将它们作为字符串返回，readline() 方法读取下一行，而 readlines() 方法读取所有行并放入一个字符串列表中。

假设 Presidents.txt 文件包含三行，如图 13-3 所示。程序清单 13-2 中的程序从文件读取数据。

程序清单 13-2 ReadDemo.py

```
1  def main():
2      # Open file for input
3      infile = open("Presidents.txt", "r")
4      print("(1) Using read(): ")
5      print(infile.read())
6      infile.close() # Close the input file
7
8      # Open file for input
9      infile = open("Presidents.txt", "r")
10     print("\n(2) Using read(number): ")
11     s1 = infile.read(4)
12     print(s1)
13     s2 = infile.read(10)
14     print(repr(s2))
15     infile.close() # Close the input file
16
17     # Open file for input
18     infile = open("Presidents.txt", "r")
19     print("\n(3) Using readline(): ")
20     line1 = infile.readline()
21     line2 = infile.readline()
22     line3 = infile.readline()
23     line4 = infile.readline()
24     print(repr(line1))
25     print(repr(line2))
26     print(repr(line3))
27     print(repr(line4))
28     infile.close() # Close the input file
29
30     # Open file for input
31     infile = open("Presidents.txt", "r")
32     print("\n(4) Using readlines(): ")
33     print(infile.readlines())
34     infile.close() # Close the input file
35
36 main() # Call the main function
```

```
(1) Using read():
Bill Clinton
George Bush
Barack Obama

(2) Using read(number):
Bill
' Clinton\nG'

(3) Using readline():
'Bill Clinton\n'
'George Bush\n'
'Barack Obama'
''

(4) Using readlines():
['Bill Clinton\n', 'George Bush\n', 'Barack Obama']
```

程序首先使用 r 模式打开 Presidents.txt 文件来从文件对象 infile 读取数据（第 3 行）。调用 infile.read() 方法从文件读取所有字符并将它们作为字符串返回（第 5 行）。关闭文件（第 6 行）。

该文件被重新打开进行读操作（第 9 行）。程序使用 read(number) 方法从文件中读取特

定数目的字符。调用 infile.read(4) 读取 4 个字符（第 11 行），而调用 infile.read(10) 读取 10 个字符（第 13 行）。repr(s) 函数返回 s 的原始字符串，这样使得转义字符也作为字面意义上的字符显示，如输出所示。

图 13-4 显示了每次读取之后文件指针的位置。

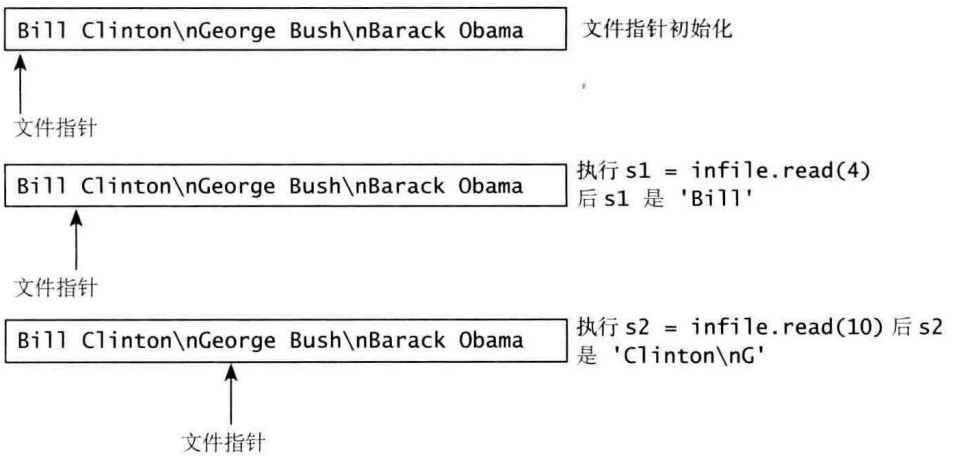


图 13-4　文件指针位置随着文件读取数据向前移动

文件关闭（第 15 行）并重新打开读取（第 18 行）。程序使用 readline() 方法来读取一行（第 20 行）。调用 infile.readline() 来读取以 \n 结束的一行。每一行的所有字符都被读取，包括 \n。当文件指针到达文件末尾时，调用 readline() 或 read() 将返回一个空字符串 ''。

图 13-5 给出每次调用完 readline() 之后文件指针的位置。

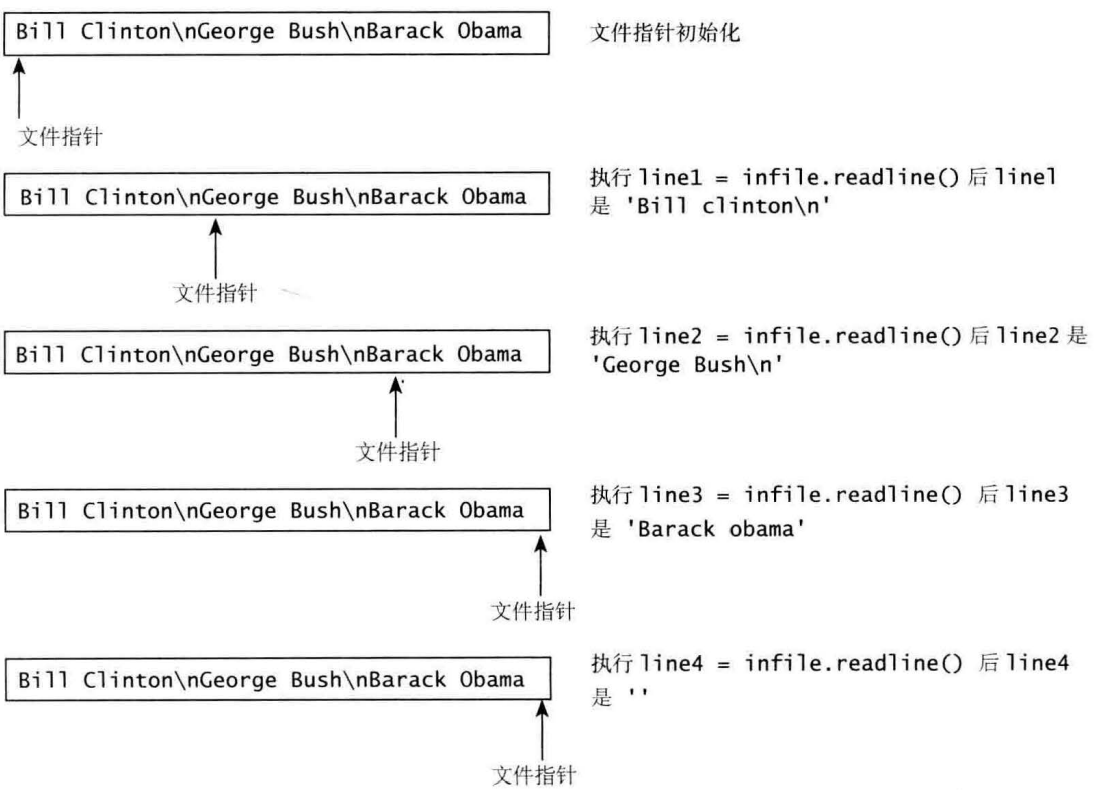


图 13-5　readline() 方法读取文件的一行

文件被关闭（第 28 行）并且重新打开读取数据（第 31 行）。程序使用 readlines() 方法来读取所有行并返回一个字符串列表。每一个字符串对应文件中的一行。

13.2.5　从文件读取所有数据

程序经常需要从一个文件中读取全部数据。这里有两种常用的方法来完成这个任务：

1）使用 read() 方法来从文件读取所有数据，然后将它作为一个字符串返回。

2）使用 readlines() 方法从文件中读取所有数据，然后将它作为一个字符串列表返回。

这两个方法对于小文件而言是简单且有效的，但是如果文件大到它的内容无法全部存在存储器中时该怎么办？可以编写下面循环每次读取文件的一行，并且持续读取下一行直到文件末端：

```
line = infile.readline() # Read a line
while line != '':
    # Process the line here ...
    # Read next line
    line = infile.readline()
```

注意：当程序达到文件的末尾时，readline() 返回 ''。

Python 同样允许你使用 for 循环来读取文件所有行，如下所示。

```
for line in infile:
    # Process the line here ...
```

这比使用 while 循环要简单多了。

程序清单 13-3 编写了一个程序，该程序将一个源文件的数据复制到目标文件，并统计文件的行数和字符数。

程序清单 13-3　CopyFile.py

```
1  import os.path
2  import sys
3
4  def main():
5      # Prompt the user to enter filenames
6      f1 = input("Enter a source file: ").strip()
7      f2 = input("Enter a target file: ").strip()
8
9      # Check if target file exists
10     if os.path.isfile(f2):
11         print(f2 + " already exists")
12         sys.exit()
13
14     # Open files for input and output
15     infile = open(f1, "r")
16     outfile = open(f2, "w")
17
18     # Copy from input file to output file
19     countLines = countChars = 0
20     for line in infile:
21         countLines += 1
22         countChars += len(line)
23         outfile.write(line)
24     print(countLines, "lines and", countChars, "chars copied")
25
26     infile.close()  # Close the input file
27     outfile.close() # Close the output file
28
29 main() # Call the main function
```

```
Enter a source file: input.txt Enter
Enter a target file: output1.txt Enter
output1.txt already exists
```

```
Enter a source file: input.txt Enter
Enter a target file: output2.txt Enter
3 lines and 73 characters copied
```

这个程序提示用户输入一个源文件 f1 和一个目标文件 f2（第 6 ～ 7 行），然后判断 f2 文件是否已经存在（第 10 ～ 12 行）。如果存在，程序将显示一条文件已存在的消息（第 11 行）并退出（第 12 行）。如果文件不存在，程序打开文件 f1 作为输入、打开文件 f2 作为输出（第 15 ～ 16 行）。随后使用一个 for 循环来读取文件 f1 的每一行并将每一行写入文件 f2（第 20 ～ 23 行）。程序跟踪从文件读取的行数和字符数（第 21 ～ 22 行）。为了确保文件能够被正确处理，需要在处理文件之后关闭文件（第 26 ～ 27 行）。

13.2.6　追加数据

可以使用 a 模式打开一个文件来在一个已经存在的文件末尾添加数据。程序清单 13-4 给出了一个给名为 Info.txt 的文件追加两行的例子。

程序清单 13-4　AppendDemo.py

```
1  def main():
2      # Open file for appending data
3      outfile = open("Info.txt", "a")
4      outfile.write("\nPython is interpreted\n")
5      outfile.close() # Close the file
6
7  main() # Call the main function
```

程序使用 a 模式打开一个名为 Info.txt 文件来通过文件对象 outfile 向文件添加数据（第 3 行）。假设已存在的文件中包含有文本数据“Programming is fun”。图 13-6 给出当文件被打开时和每次写入后文件指针的位置。当文件被打开时，文件指针被置于文件末尾。

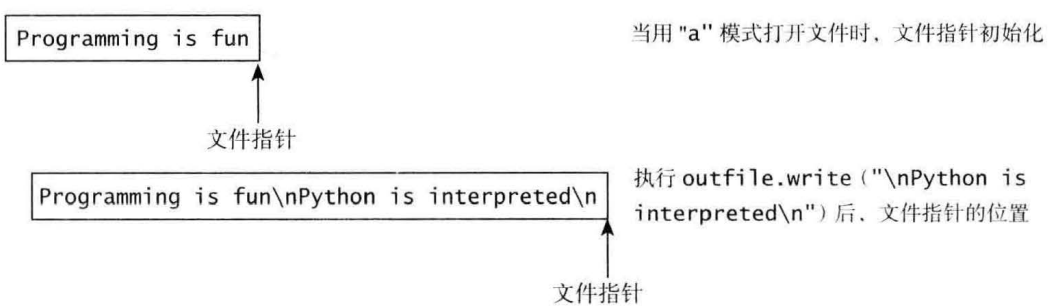


图 13-6　数据被追加到文件中

程序最后关闭文件以确保数据被正确写入文件当中（第 5 行）。

13.2.7　读写数值数据

为了向一个文件写入数字，必须首先将它们转换成字符串，然后使用 write 方法将它们写入文件。为了正确地读出文件中的数字，利用像 " " 或 \n 的空白符来分隔数字。

在程序清单 13-5 中，程序向一个文件写入 10 个随机单独的数字并且从文件读取这些数字。

程序清单 13-5 WriteReadNumbers.py

```
1  from random import randint
2
3  def main():
4      # Open file for writing data
5      outfile = open("Numbers.txt", "w")
6      for i in range(10):
7          outfile.write(str(randint(0, 9)) + " ")
8      outfile.close()  # Close the file
9
10     # Open file for reading data
11     infile = open("Numbers.txt", "r")
12     s = infile.read()
13     numbers = [eval(x) for x in s.split()]
14     for number in numbers:
15         print(number, end = " ")
16     infile.close() # Close the file
17
18 main() # Call the main function
```

```
8 1 4 1 2 5 5 1 3 2
```

程序使用 w 模式打开一个名为 Numbers.txt 的文件来向使用文件对象 outfile 的文件写入数据（第 5 行）。for 循环向文件写入 10 个数字，这些数字之间用空白符分隔（第 6 ～ 7 行）。注意：将数字转换为字符串才能将它们写入文件。

这个程序关闭了输出文件（第 8 行）并使用 r 模式重新打开文件通过文件对象 infile 来读取数据（第 11 行）。read() 方法将所有数据作为一个字符串读取（第 12 行）。因为数字间被空白符分隔，所以字符串的 split 方法能够将这个字符串分成一个列表（第 13 行）。可以从该列表中获取数字并显示出来（第 14 ～ 15 行）。

检查点

13.1　如何打开一个文件分别进行数据读取、写入和追加？

13.2　使用下面的语句创建一个文件对象会有什么问题？

```
infile = open("c:\book\test.txt", "r")
```

13.3　当打开一个文件进行读取时，如果文件不存在会出现什么情况？当打开一个文件进行写入时，文件不存在会出现什么情况？

13.4　如何判断一个文件是否存在？

13.5　使用什么方法可以从一个文件中读取 30 个字符？

13.6　使用什么方法可以将文件所有数据读取到一个字符串？

13.7　使用什么方法可以读取一行？

13.8　使用什么方法可以将文件的所有行读入一个列表中？

13.9　如果在文件末尾调用 read() 和 readline()，那么程序是否会出现运行时错误？

13.10　读取数据时，如何判断到了文件末尾？

13.11　使用什么函数来将数据写入文件？

13.12　如何在程序中表示一个字面意义上的原始字符串？

13.13　如何读写数值数据？

13.3　文件对话框

关键点：tkinter.filedialog 模块中包含有 askopenfilename 和 asksaveasfilename 函数来显示文件打开和保存为对话框。

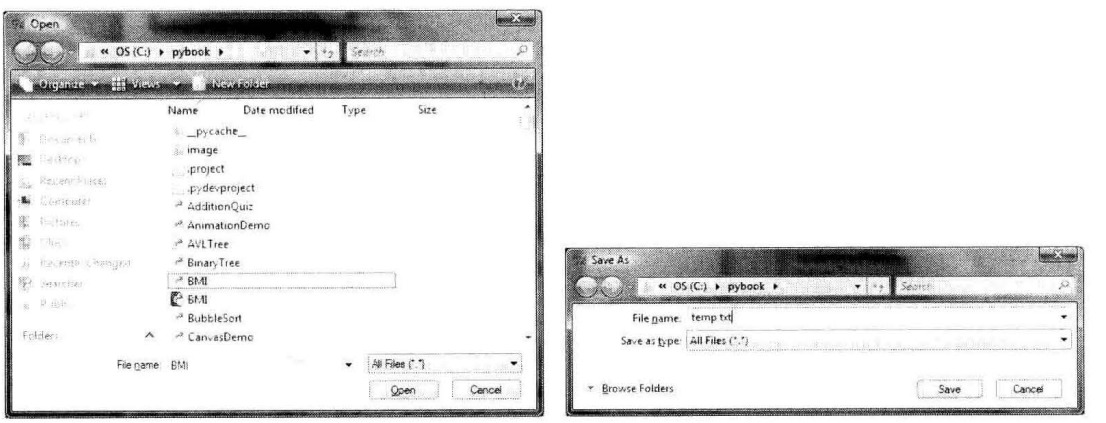
Tkinter 提供了带以下两个函数的 tkinter.filedialog 模块。

```
# Display a file dialog box for opening an existing file
filename = askopenfilename()
# Display a file dialog box for specifying a file for saving data
filename = asksaveasfilename()
```

这两个函数都会返回一个文件名。如果对话框被用户取消，那么该函数返回 None。下面是使用这两个函数的一个示例。

```
1  from tkinter.filedialog import askopenfilename
2  from tkinter.filedialog import asksaveasfilename
3
4  filenameforReading = askopenfilename()
5  print("You can read from " + filenameforReading)
6
7  filenameforWriting = asksaveasfilename()
8  print("You can write data to " + filenameforWriting)
```

当运行这段代码时，askopenfilename() 函数将显示打开对话框以指明要打开的文件，如图 13-7a 所示。asksaveasfilename() 函数将显示保存为对话框来指明要保存的文件的名字和路径，如图 13-7b 所示。



a）askopenfilename() 函数显示打开对话框　　b）asksaveasfilename() 函数显示保存为对话框

图　13-7

现在创建一个简单的文本编辑器，它包括菜单栏、工具栏和文件对话框，如图 13-8 所示。这个编辑器允许用户打开并保存文本文件。程序清单 13-8 给出这个程序。

程序清单 13-6　FileEditor.py

```
1  from tkinter import *
2  from tkinter.filedialog import askopenfilename
3  from tkinter.filedialog import asksaveasfilename
4
5  class FileEditor:
6      def __init__(self):
7          window = Tk()
8          window.title("Simple Text Editor")
```

```
 9
10          # Create a menu bar
11          menubar = Menu(window)
12          window.config(menu = menubar) # Display the menu bar
13
14          # Create a pull-down menu and add it to the menu bar
15          operationMenu = Menu(menubar, tearoff = 0)
16          menubar.add_cascade(label = "File", menu = operationMenu)
17          operationMenu.add_command(label = "Open",
18              command = self.openFile )
19          operationMenu.add_command(label = "Save",
20              command = self.saveFile )
21
22          # Add a tool bar frame
23          frame0 = Frame(window) # Create and add a frame to window
24          frame0.grid(row = 1, column = 1, sticky = W)
25
26          # Create images
27          openImage = PhotoImage(file = "image/open.gif")
28          saveImage = PhotoImage(file = "image/save.gif")
29
30          Button(frame0, image = openImage, command =
31              self.openFile).grid(row = 1, column = 1, sticky = W)
32          Button(frame0, image = saveImage,
33              command = self.saveFile).grid(row = 1, column = 2)
34
35          frame1 = Frame(window) # Hold editor pane
36          frame1.grid(row = 2, column = 1)
37
38          scrollbar = Scrollbar(frame1)
39          scrollbar.pack(side = RIGHT, fill = Y)
40          self.text = Text(frame1, width = 40, height = 20,
41              wrap = WORD, yscrollcommand = scrollbar.set)
42          self.text.pack()
43          scrollbar.config(command = self.text.yview)
44
45          window.mainloop() # Create an event loop
46
47      def openFile(self):
48          filenameforReading = askopenfilename()
49          infile = open(filenameforReading, "r")
50          self.text.insert(END, infile.read()) # Read all from the file
51          infile.close()  # Close the input file
52
53      def saveFile(self):
54          filenameforWriting = asksaveasfilename()
55          outfile = open(filenameforWriting, "w")
56          # Write to the file
57          outfile.write(self.text.get(1.0, END))
58          outfile.close() # Close the output file
59
60  FileEditor() # Create GUI
```

程序创建 File 菜单（第 15 ～ 20 行）。File 菜单栏包括菜单命令“ Open”，它是来加载一个文件的（第 18 行），而菜单命令“ Save”是来保存一个文件的（第 20 行）。当单击“ Open”菜单时，就会调用 openFile 方法（第 47 ～ 51 行）来显示打开对话框以便使用 askopenfilename 函数打开一个文件（第 48 行）。当用户选择一个文件之后，该文件的文件名被返回并被用来打开这个文件以读取数据（第 49 行）。这个程序从文件读取数据并将这个数据插入 Text 小构件中（第 50 行）。

当单击“ Save”菜单时，saveFile 方法（第 53 ～ 58 ）被调用以显示保存为对话框来使

用 asksavesfilename 函数保存一个文件（第 54 行）。当用户输入或选择一个文件之后，这个文件的文件名被返回来打开该文件以写入数据（第 55 行）。这个程序从 Text 小构件中读取数据并将数据写入文件中（第 57 行）。

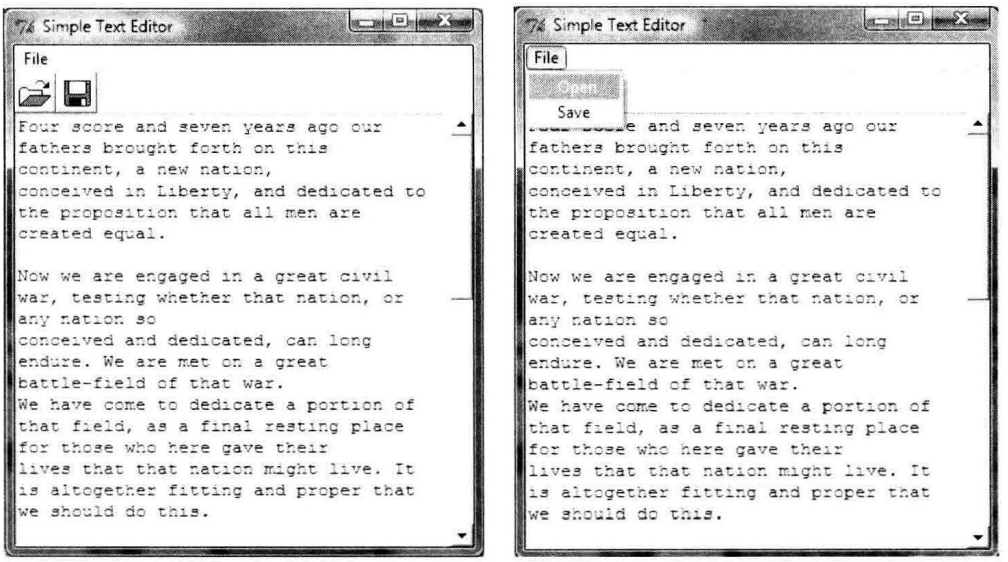


图 13-8　这个编辑器允许用户从 File 菜单或从工具栏打开和保存文件

这个程序同样也创建了工具栏按钮（第 30 ～ 33 行）并将它们放在一个框架里。这些工具栏按钮都是一些含有图标的按钮。当单击“Open”工具栏按钮时，调用 openFile 回调方法（第 31 行）。当单击“Save”工具栏按钮时，调用 save 回调 File 方法（第 33 行）。

这个程序使用有滚动栏的 Text 小构件创建一个文本域（第 38 ～ 43 行）。Text 小构件和滚动条都被放在 framel 框架里。

检查点

13.14　如何显示一个打开文件的对话框？

13.15　如何显示一个保存文件的对话框？

13.4　实例研究：统计文件中的字符个数

关键点：编写一个程序，提示用户输入一个文件名然后统计在不计大小写的情况下每个字符的出现次数。

下面是解决这个问题的步骤：

1）将文件中的每行作为一个字符串读取。

2）使用字符串的 lower() 方法来将字符串中的所有大写字母转换成小写字母。

3）创建一个含有 26 个整型值的名为 counts 的列表，每一个值是对每个字母出现次数的统计。也就是说，counts[0] 统计字母 a 的出现次数，counts[1] 统计字母 b 的出现次数，依此类推。

4）对于字符串中的每个字符，判断它是否是一个小写字母。如果是，将列表中相应的计数器加 1。

5）最后，显示统计结果。

程序清单 13-7 给出了完整的程序。

程序清单 13-7 CountEachLetter.py

```
1  def main():
2      filename = input("Enter a filename: ").strip()
3      infile = open(filename, "r") # Open the file
4
5      counts = 26 * [0] # Create and initialize counts
6      for line in infile:
7          # Invoke the countLetters function to count each letter
8          countLetters(line.lower(), counts)
9
10     # Display results
11     for i in range(len(counts)):
12         if counts[i] != 0:
13             print(chr(ord('a') + i) + " appears " + str(counts[i])
14                 + (" time" if counts[i] == 1 else " times"))
15
16     infile.close() # Close file
17
18 # Count each letter in the string
19 def countLetters(line, counts):
20     for ch in line:
21         if ch.isalpha() :
22             counts[ord(ch) - ord('a')] += 1
23
24 main() # Call the main function
```

```
Enter a filename: input.txt ↵Enter
a appears 3 times
b appears 3 times
x appears 1 time
```

这个主函数提示用户输入一个文件名（第 2 行）并打开这个文件（第 3 行）。程序创建一个含有 26 个元素的列表且初始化为 0(第 5 行)。for 循环（第 6 ～ 8 行）从文件中读取每一行，将字母全转换成小写，然后将它们传递来调用 countLetters。

countLetters(line，counts) 函数检查 line 中的每个字符。如果它是一个小写字母，程序将列表 counts 对应的元素加 1（第 21 ～ 22 行）。

当所有行都被处理之后，如果次数大于 0，那么程序显示文件中包含的每个字母和它们的出现次数（第 11 ～ 14 行）。

13.5　从网站上获取数据

关键点：使用 urlopen 函数打开一个统一资源定位器（URL）并从网站上读取数据。

可以使用 Python 编写简单的代码来从 Web 网站上读取数据。所需要做的就是通过使用 urlopen 函数打开一个 URL，如下所示。

```
infile = urllib.request.urlopen("http://www.yahoo.com")
```

urlopen 函数（在 urllib.request 模块中定义）像打开一个文件一样打开一个 URL 资源。下面是一个从给定的 URL 读取和显示网站内容的示例。

```
import urllib.request
infile = urllib.request.urlopen("http://www.yahoo.com/index.html")
print(infile.read().decode())
```

使用 infile.read() 从 URL 上读取的数据是比特形式的原始数据。调用 decode() 方法将原

始数据转换为一个字符串。

让我们重写程序清单 13-7 中的程序，提示用户输入一个来自因特网上的 URL 上的文件而不是来自本地系统上的文件。这个程序在程序清单 13-8 中给出。

程序清单 13-8 CountEachLetterURL.py

```
1  import urllib.request
2
3  def main():
4      url = input("Enter a URL for a file: ").strip()
5      infile = urllib.request.urlopen(url)
6      s = infile.read().decode() # Read the content as string
7
8      counts = countLetters(s.lower())
9
10     # Display results
11     for i in range(len(counts)):
12         if counts[i] != 0:
13             print(chr(ord('a') + i) + " appears " + str(counts[i])
14                 + (" time" if counts[i] == 1 else " times"))
15
16 # Count each letter in the string
17 def countLetters(s):
18     counts = 26 * [0] # Create and initialize counts
19     for ch in s:
20         if ch.isalpha():
21             counts[ord(ch) - ord('a')] += 1
22     return counts
23
24 main() # Call the main function
```

```
Enter a filename: http://cs.armstrong.edu/liang/data/Lincoln.txt Enter
a appears 102 times
b appears 14 times
c appears 31 times
d appears 58 times
e appears 165 times
f appears 27 times
g appears 28 times
h appears 80 times
i appears 68 times
k appears 3 times
l appears 42 times
m appears 13 times
n appears 77 times
o appears 92 times
p appears 15 times
q appears 1 time
r appears 79 times
s appears 43 times
t appears 126 times
u appears 21 times
v appears 24 times
w appears 28 times
y appears 10 times
```

主函数提示用户输入一个 URL 地址（第 4 行），打开这个 URL（第 5 行），并从这个 URL 读取数据放入一个字符串中（第 6 行）。这个程序将这个字符串转换为小写字母并且调用 countLetters 函数来对字符串中出现的每个字母进行计数（第 8 行）。函数返回一个显示每个字母出现次数的列表。

countLetters(s) 函数创建一个初始值为 0 的 26 个元素构成的列表（第 18 行）。函数检测 s 中的每个字符。如果它是小写字母，程序就给相应的 counts 加 1（第 20 ～ 21 行）。

注意：为了使 urlopen 函数识别一个有效的 URL，URL 地址需要有 http:// 前缀，如果像下面输入一个 URL 是错误的。

```
cs.armstrong.edu/liang/data/Lincoln.txt
```

检查点

13.16　如何在 Python 程序中打开一个 Web 网页？

13.17　可以使用什么函数来从一个正常的字符串返回一个原始字符串？

13.6　异常处理

关键点：异常处理使程序能够处理异常然后继续它的正常执行。

当运行上一节的程序时，如果用户输入一个不存在的文件或 URL 时将会怎样？这个程序将会中断并抛出一个错误。例如，如果运行程序清单 13-7 的程序时输入一个不存在的文件名，那么程序将会报告这个 IOError。

```
c:\pybook\python CountEachLetter.py
Enter a filename: NonexistentOrIncorrectFile.txt ↵Enter
Traceback (most recent call last):
  File "C:\pybook\CountEachLetter.py", line 23, in <module>
    main()
  File "C:\pybook\CountEachLetter.py", line 4, in main
    infile = open(filename, "r") # Open the file
IOError: [Errno 22] Invalid argument: 'NonexistentOrIncorrectFile.txt\r'
```

这些冗长的错误信息被称为堆栈回溯或回溯。回溯通过追溯到导致这条语句的函数调用来给出导致错误的这条语句的信息。在错误信息中显示函数调用的行号以便跟踪这个错误。

在运行时出现的错误被称为异常。如何处理一个异常以使程序能够捕获这个错误并提示用户输入一个正确的文件名？可以使用 Python 的异常处理语法来实现。

异常处理语法是将可能产生（抛出）异常的代码包裹在 try 子句中，如下所示。

```
try:
    <body>
except <ExceptionType>:
    <handler>
```

在这里，<body> 包含了可能抛出异常的代码。当一个异常出现时，<body> 中剩余代码被跳过。如果该异常匹配一个异常类型，那么该类型下的处理代码将会执行。<handler> 是处理异常的代码。现在，可以将异常处理的新代码插入到程序清单 13-7 的第 2 行和第 3 行，以便用户在输入不正确时重新输入文件名，如程序清单 13-9 所示。

程序清单 13-9　CountEachLetterWithExceptionHandling.py

```
1  def main():
2      while True:
3          try:
4              filename = input("Enter a filename: ").strip()
5              infile = open(filename, "r") # Open the file
6              break
7          except IOError:
8              print("File " + filename + " does not exist. Try again")
9
```

如果发生异常

```
10      counts = 26 * [0] # Create and initialize counts
11      for line in infile:
12          # Invoke the countLetters function to count each letter
13          countLetters(line.lower(), counts)
14
15      # Display results
16      for i in range(len(counts)):
17          if counts[i] != 0:
18              print(chr(ord('a') + i) + " appears " + str(counts[i])
19                + (" time" if counts[i] == 1 else " times"))
20
21      infile.close() # Close file
22
23  # Count each letter in the string
24  def countLetters(line, counts):
25      for ch in line:
26          if ch.isalpha():
27              counts[ord(ch) - ord('a')] += 1
28
29  main()
```

```
Enter a filename: NonexistentOrIncorrectFile ↵Enter
File NonexistentOrIncorrectFile does not exist. Try again
Enter a filename: Lincoln.dat ↵Enter
File Lincoln.dat does not exist. Try again
Enter a filename: Lincoln.txt ↵Enter
a appears 102 times
b appears 14 times
...
...
w appears 28 times
y appears 10 times
```

这个程序使用一个 while 循环来重复提示用户输入一个文件名（第 2 ～ 8 行）。如果输入的名字正确，那么程序将退出循环（第 6 行）。如果调用 open 函数时抛出一个 IOError 异常（第 5 行），那么 except 子句被执行来处理这个异常（第 7 ～ 8 行），随后循环继续。

try/except 块按如下方式工作：

- 首先，try 和 except 之间的语句被执行。
- 如果没有异常出现，跳过 except 子句。在这种情况下，执行 break 语句退出 while 循环。
- 如果在执行 try 子句时出现异常，子句的剩余部分将会被跳过。在这种情况下，如果文件不存在，那么 open 函数将会抛出一个异常，break 语句被跳过。
- 当一个异常出现时，如果异常类型匹配关键字 except 之后的异常名，那么这个 except 子句被执行，然后继续执行 try 语句之后的语句。
- 如果一个异常出现但是异常类型不匹配 except 子句中的异常名，那么这个异常被传递给这个函数的调用者；如果没有找到处理该异常的处理器，那么这是一个未处理异常且终止程序显示错误信息。

一个 try 语句可以有多个 except 子句来处理不同的异常。这个语句也可以选择 else 或 finally 语句，语法如下所示。

```
1  try:
2      <body>
3  except <ExceptionType1>:
4      <handler1>
```

```
5  ...
6  except <ExceptionTypeN>:
7      <handlerN>
8  except:
9      <handlerExcept>
10 else:
11     <process_else>
12 finally:
13     <process_finally>
```

多个 except 语句与 elif 语句类似。当一个异常出现时，它会被顺序检查是否匹配 try 子句后的 except 子句中的异常。如果找到一个匹配，那么匹配该异常的处理器将被执行，而 except 子句的其他部分将会忽略。注意：在 except 子句最后的 <ExceptionType> 可能会被忽略。如果异常在最后一个 except 子句之前不匹配任何一个异常类型（第 8 行），那么执行最后一个 except 子句的 <handlerExcept>（第 9 行）。

一个 try 语句可以有一个可选择的 else 子句，如果 try 块中没有异常抛出，将会执行 else 块。

一个 try 语句可以有一个可选择的 finally 块，这用来定义收尾动作，无论何种情况都会执行这个块。程序清单 13-10 给出一个使用异常处理的例子。

程序清单 13-10　TestException.py

```
1  def main():
2      try:
3          number1, number2 = eval(
4              input("Enter two numbers, separated by a comma: "))
5          result = number1 / number2
6          print("Result is", result)
7      except ZeroDivisionError:
8          print("Division by zero!")
9      except SyntaxError:
10         print("A comma may be missing in the input")
11     except:
12         print("Something wrong in the input")
13     else:
14         print("No exceptions")
15     finally:
16         print("The finally clause is executed")
17
18 main() # Call the main function
```

```
Enter two numbers, separated by a comma: 3, 4  ↵Enter
Result is 0.75
No exceptions
The finally clause is executed
```

```
Enter two numbers, separated by a comma: 2, 0  ↵Enter
Division by zero!
The finally clause is executed
```

```
Enter two numbers, separated by a comma: 2 3  ↵Enter
A comma may be missing in the input
The finally clause is executed
```

```
Enter two numbers, separated by a comma: a, v  ↵Enter
Something wrong in the input
The finally clause is executed
```

当输入 3、4 时，程序会计算这个除法并显示结果，然后执行 else 子句，并且最后执行 finally 块。

当输入 2、0 时，执行除法时会抛出一个 ZeroDivisionError 异常（第 5 行）。第 7 行的 except 子句将会捕获这个异常并处理它，然后执行 finally 块。

当输入 2 3 时，就会抛出一个 SyntaxError 异常。第 9 行的 except 子句将会捕获这个异常并处理它，之后执行 finally 块。

当输入 a、v 时，会抛出一个异常。这个异常被第 11 行的 except 子句处理，然后执行 finally 块。

13.7　抛出异常

关键点：异常被包裹在对象中，而对象由类创建。一个函数抛出一个异常。

在之前的小节中已经学习了如何编写处理异常的代码。那么异常是来自哪？一个异常是如何产生的？附属在异常上的信息包裹在一个对象中。异常产生自一个函数。当函数检测到一个错误时，它将使用下面语法从一个正确的异常类创建一个对象并把这个异常抛给这个函数的调用者。

```
raise ExceptionClass("Something is wrong")
```

下面介绍异常是如何工作的。假设程序检测到传递给函数的一个参数与这个函数的合约冲突；例如，这个参数必须是非负的，但是传递的参数是一个负数。这个程序将创建一个 RuntimeError 类的实例并将它抛出，如下所示。

```
ex = RuntimeError("Wrong argument")
raise ex
```

或者，如果你更喜欢，可以将前面两条语句合并成一条语句。

```
raise RuntimeError("Wrong argument")
```

现在，可以修改程序清单 12-2 中 Circle 类的 setRadius 方法，当半径是负数时抛出一个 RuntimeError 异常。修改后的 Circle 类在程序清单 13-11 中给出。

程序清单 13-11　CircleWithException.py

```
1  from GeometricObject import GeometricObject
2  import math
3
4  class Circle(GeometricObject):
5      def __init__(self, radius):
6          super().__init__()
7          self.setRadius(radius)
8
9      def getRadius(self):
10         return self.__radius
11
12     def setRadius(self, radius):
13         if radius < 0:
14             raise RuntimeError("Negative radius")
15         else:
16             self.__radius = radius
17
18     def getArea(self):
19         return self.__radius * self.__radius * math.pi
20
```

```
21      def getDiameter(self):
22          return 2 * self.__radius
23
24      def getPerimeter(self):
25          return 2 * self.__radius * math.pi
26
27      def printCircle(self):
28          print(self.__str__() + " radius: " + str(self.__radius))
```

程序清单 13-12 中的测试程序是使用程序清单 13-11 中的新 Circle 类来创建一个圆对象的。

程序清单 13-12 TestCircleWithException.py

```
1  from CircleWithException import Circle
2
3  try:
4      c1 = Circle(5)
5      print("c1's area is", c1.getArea())
6      c2 = Circle(-5)
7      print("c2's area is", c2.getArea())
8      c3 = Circle(0)
9      print("c3's area is", c3.getArea())
10 except RuntimeException:
11     print("Invalid radius")
```

```
c1's area is 78.53981633974483
Invalid radius
```

当试图用一个负数半径来创建一个 Circle 对象时（第 6 行），一个 RuntimeError 异常被抛出。这个异常被第 10 ～ 11 行的 except 子句捕获。

现在你知道了如何抛出和处理异常。那么使用异常处理的好处是什么？使用异常处理能够使函数给它的调用者抛出一个异常。调用者能够处理这个异常。如果没有这种能力，被调用函数必须自己处理这个异常或者终止这个程序。通常被调用函数不知道如何处理一个错误。这对库函数而言是典型情况。库函数可以检测到错误，但是只有调用者知道在错误出现时如何处理它。异常处理的最重要优势就是将错误检测（在被调用函数中完成）和错误处理（在调用函数中完成）分隔开来。

许多库函数能抛出多种异常，像 ZeroDivisionError、TypeError 和 IndexError 异常。可以使用 try-except 语法来捕获和处理这些异常。

函数可能会在一个函数调用链上调用其他函数。考虑多个函数调用的例子。假设 main 函数调用函数 function1，函数 function1 调用函数 function2，函数 function2 调用函数 function3，而函数 function3 抛出一个异常，如图 13-9 所示。考虑下面的方案：

- 如果异常类型是 Exception3，那么这个异常将被函数 function2 中处理这个异常的 except 块所捕获。statement5 将会被跳过，而 statement6 将被执行。
- 如果异常类型是 Exception2，函数 function2 被中止，控制权返回给函数 function1，而这个异常将被函数 function1 中的处理 Exception2 的 except 块所捕获。statement3 将会被跳过，而 statement4 将被执行。
- 如果异常类型是 Exception1，函数 function1 被中止，控制权返回给 main 函数，这个异常将被主函数 main 中处理 Exception1 的 except 块所捕获。statement1 将会被跳过，而 statement2 将被执行。

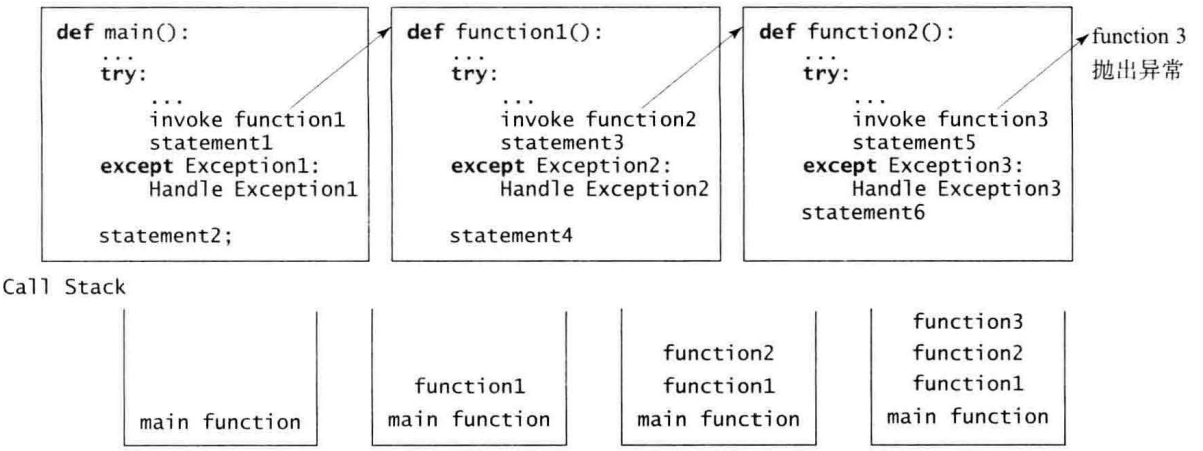


图 13-9　如果当前函数没有捕获一个异常，这个异常将被传递给它的调用者，这个过程一直重复直到这个异常被捕获或传递给主函数 main

13.8　使用对象处理异常

关键点：在 except 子句中访问一个异常对象。

如前所述，一个异常被包裹在一个对象中。为了抛出一个异常，需要首先创建一个异常对象，然后使用 raise 关键字将它抛出。这个异常对象能够从 except 子句访问吗？答案是肯定的。可以使用下面的语法将 exception 对象赋给一个变量。

```
try
    <body>
except ExceptionType as ex:
    <handler>
```

有了这个语法，当 except 子句捕获到异常时，这个异常对象就被赋给一个名为 ex 的变量。现在，可以在处理器中使用这个对象。

程序清单 13-13 给出了一个例子，它提示用户输入一个数字，如果输入正确时显示这个数字。否则，这个程序显示一条错误信息。

程序清单 13-13　ProcessExceptionObject.py

```
1  try:
2      number = eval(input("Enter a number: "))
3      print("The number entered is", number)
4  except NameError as ex:
5      print("Exception:", ex)
```

```
Enter a number: 34 Enter
The number entered is 34
```

```
Enter a number: one Enter
Exception: name 'one' is not defined
```

当输入一个非数字值时，就会从第 2 行抛出一个 NameError 对象。这个对象被赋给变量 ex。因此，可以访问它去处理这个异常。ex 中的 __str__() 方法被调用来返回一个描述该异常的字符串。在这种情况下，字符串是“name 'one' is not defined”。

13.9　定义自定制异常类

关键点：可以通过扩展 BaseException 类或 BaseException 类的子类来定义一个自定制异常类。

目前为止，我们已经在本章中使用了像 ZeroDivisionError、SyntaxError、RuntimeError 和 NameError 这样的 Python 内置异常类。还有其他异常类型可以供我们使用吗？答案是肯定的，Python 有许多内置异常。图 13-10 给出其中一些异常。

注意：类名 Exception、StandardError 和 RuntimeError 有点让人迷惑。所有这三种类都是异常，且这些错误都在运行时出现。

BaseException 类是所有异常类的父类。所有的 Python 异常类都直接或间接地继承自 BaseException 类。正如你所看到的，Python 提供了许多的异常类。也可以定义自己的异常类，它们都继承自 BaseException 类或 BaseException 类的子类，例如：RuntimeError。

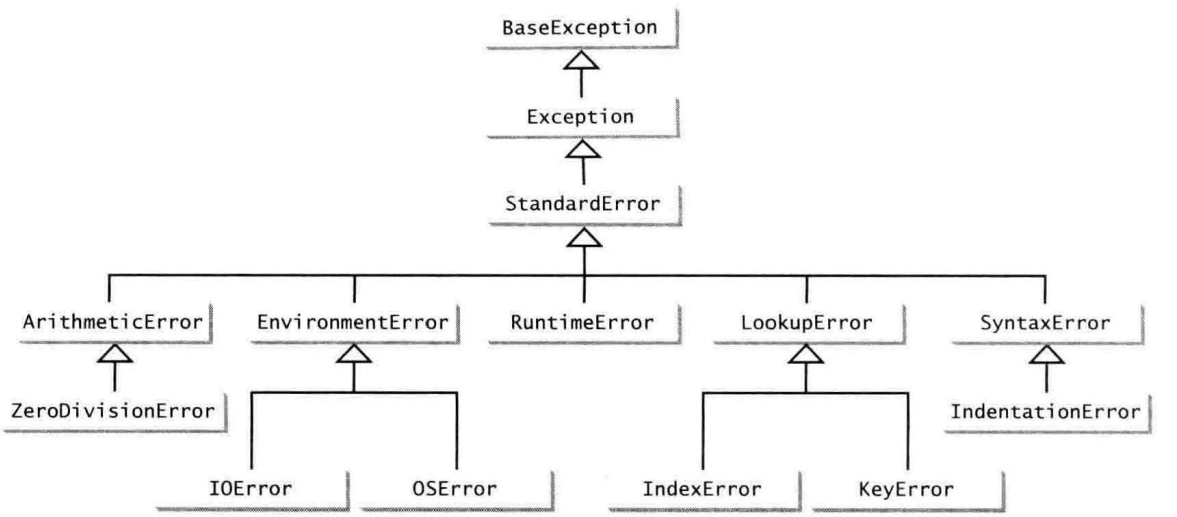


图 13-10　抛出的异常都是该图表所示的类或者这些类的子类的实例

程序清单 13-11 中的 Circle 类的 setRadius 方法在半径是负数的情况下抛出一个 RuntimeError 异常。调用者可以捕获这个异常，但是调用者并不知道什么半径会导致这个异常。为了修复这个问题，可以定义一个自定制异常类来存储半径，如程序清单 13-14 所示。

程序清单 13-14　InvalidRadiusException.py

```
1  class InvalidRadiusException(RuntimeError):
2      def __init__(self, radius):
3          super().__init__()
4          self.radius = radius
```

这个自定制异常类扩展自 RuntimeError（第 1 行）。初始化方法只是调用父类的初始化方法（第 3 行）并设置数据域中的半径（第 4 行）。

现在，修改 Circle 类中的 setRadius(radius) 方法在半径是负数的情况下抛出一个 InvalidRadiusException 异常，如程序清单 13-15 所示。

程序清单 13-15　CircleWithCustomException.py

```
1  from GeometricObject import GeometricObject
2  from InvalidRadiusException import InvalidRadiusException
```